```
.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
                """SELECT failed_attempts, blocked_until
                   FROM auth_login_throttle
                   WHERE scope_type = 'account' AND key_hash = %s""",
                (key,),
            )
            row = cur.fetchone()

        assert blocked_until is not None
        assert blocked_until > now
        assert row[0] == 1
        assert row[1] is not None and row[1] > now
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                "DELETE FROM auth_login_throttle WHERE scope_type = 'account' AND key_hash
= %s",
                (key,),
            )
'''

test_path.write_text(tests, encoding="utf-8")
print(f"Added threshold regression test to {test_path}")
PY

git diff --check

PYTHONPATH=apps/api \
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test' \
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' \
.venv/bin/python -m pytest \
  apps/api/tests/test_throttle.py \

apps/api/tests/test_fin14_auth_security.py::test_throttle_expires_and_correct_login_succeeds_a
fter_retry \
  -q
Patched initial-bucket threshold handling in apps/api/app/throttle.py
Added threshold regression test to apps/api/tests/test_throttle.py
.....................
[100%]
==============================================================================
========= warnings summary
```

```
============================================================================
==========
.venv/lib/python3.14/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is
deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

.venv/lib/python3.14/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is
deprecated, use anyio.from_thread.BlockingPortal instead.
    _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
16 passed, 2 warnings in 1.67s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % printf '\n===== AUTH
ROUTES AND SESSION HANDLING =====\n'
grep -nE
'@router\.(get|post|delete|patch)|auth_sessions|revoke_reason|revoked_at|get_current_user|mf
a|MFA' \
  apps/api/app/main.py apps/api/app/*.py

printf '\n===== SESSION SCHEMA AND EXISTING MIGRATIONS =====\n'
grep -nE 'CREATE TABLE.*auth_sessions|CREATE TABLE.*users|auth_sessions|CREATE
TABLE.*audit_events|mfa|MFA' \
  apps/api/migrations/*.sql

printf '\n===== MFA-RELATED DEPENDENCIES =====\n'
grep -inE 'totp|pyotp|oath|mfa|cryptography|fernet|qrcode' \
  apps/api/requirements*.txt pyproject.toml requirements*.txt 2>/dev/null

printf '\n===== AUTHENTICATION TESTS =====\n'
grep -nE 'session|logout|login|MFA|mfa|audit|throttle' \
  apps/api/tests/test_auth.py \
  apps/api/tests/test_fin14_auth_security.py \
  apps/api/tests/test_audit_service.py

printf '\n===== CURRENT WORKTREE SUMMARY =====\n'
git status --short
```

```
===== AUTH ROUTES AND SESSION HANDLING =====
apps/api/app/main.py:128:def get_current_user(
apps/api/app/main.py:139:                FROM auth_sessions s JOIN users u ON u.user_id =
s.user_id
apps/api/app/main.py:143:                WHERE s.refresh_token_hash = %s AND s.revoked_at
IS NULL
apps/api/app/main.py:154:            "UPDATE auth_sessions SET last_seen_at = %s WHERE
refresh_token_hash = %s",
apps/api/app/main.py:162:CurrentUser = Annotated[dict[str, Any], Depends(get_current_user)]
apps/api/app/main.py:271:                    """INSERT INTO auth_sessions
apps/api/app/main.py:335:                """UPDATE auth_sessions
apps/api/app/main.py:336:                   SET revoked_at = %s, revoke_reason = 'logout'
apps/api/app/main.py:338:                     AND revoked_at IS NULL
apps/api/app/main.py:128:def get_current_user(
apps/api/app/main.py:139:                FROM auth_sessions s JOIN users u ON u.user_id =
s.user_id
apps/api/app/main.py:143:                WHERE s.refresh_token_hash = %s AND s.revoked_at
IS NULL
apps/api/app/main.py:154:            "UPDATE auth_sessions SET last_seen_at = %s WHERE
refresh_token_hash = %s",
apps/api/app/main.py:162:CurrentUser = Annotated[dict[str, Any], Depends(get_current_user)]
apps/api/app/main.py:271:                    """INSERT INTO auth_sessions
apps/api/app/main.py:335:                """UPDATE auth_sessions
apps/api/app/main.py:336:                   SET revoked_at = %s, revoke_reason = 'logout'
apps/api/app/main.py:338:                     AND revoked_at IS NULL

===== SESSION SCHEMA AND EXISTING MIGRATIONS =====
apps/api/migrations/0001_auth_identity.sql:14:CREATE TABLE users (
apps/api/migrations/0001_auth_identity.sql:23:    mfa_required BOOLEAN NOT NULL
DEFAULT false,
apps/api/migrations/0001_auth_identity.sql:31:CREATE TABLE auth_sessions (
apps/api/migrations/0001_auth_identity.sql:43:CREATE INDEX ix_auth_sessions_active ON
auth_sessions(user_id, expires_at) WHERE revoked_at IS NULL;
apps/api/migrations/0003_auth_audit_and_login_throttling.sql:6:CREATE TABLE audit_events (

===== MFA-RELATED DEPENDENCIES =====
zsh: no matches found: requirements*.txt

===== AUTHENTICATION TESTS =====
apps/api/tests/test_auth.py:11:from app.throttle import make_throttle_key
apps/api/tests/test_auth.py:33:                "UPDATE auth_sessions SET revoked_at =
COALESCE(revoked_at, now()), "
apps/api/tests/test_auth.py:50:    throttle_secret = "test-only-secret-material-at-least-32-bytes"
```

```
apps/api/tests/test_auth.py:51:    monkeypatch.setenv("AUTH_THROTTLE_SECRET",
throttle_secret)
apps/api/tests/test_auth.py:52:    ip_key = make_throttle_key(
apps/api/tests/test_auth.py:53:        throttle_secret.encode("utf-8"), "ip", "testclient"
apps/api/tests/test_auth.py:58:            "DELETE FROM auth_login_throttle WHERE scope_type
= 'ip' AND key_hash = %s",
apps/api/tests/test_auth.py:68:                "DELETE FROM auth_login_throttle WHERE
scope_type = 'ip' AND key_hash = %s",
apps/api/tests/test_auth.py:81:def
test_login_creates_protected_hashed_expiring_session(client, identity, caplog, capsys):
apps/api/tests/test_auth.py:83:    response = client.post("/auth/login", json={"email": email,
"password": TEST_PASSWORD})
apps/api/tests/test_auth.py:92:        cur.execute("SELECT refresh_token_hash, expires_at,
revoked_at FROM auth_sessions WHERE user_id = %s", (user_id,))
apps/api/tests/test_auth.py:106:    known = client.post("/auth/login", json={"email": email,
"password": "wrong-password"})
apps/api/tests/test_auth.py:107:    absent = client.post("/auth/login", json={"email":
"absent@example.com", "password": "wrong-password"})
apps/api/tests/test_auth.py:113:def test_logout_revokes_session_and_replay_fails(client,
identity):
apps/api/tests/test_auth.py:115:    login = client.post("/auth/login", json={"email": email,
"password": TEST_PASSWORD})
apps/api/tests/test_auth.py:116:    cookie = login.cookies.get(SESSION_COOKIE)
apps/api/tests/test_auth.py:118:    logout = client.post("/auth/logout",
cookies={SESSION_COOKIE: cookie})
apps/api/tests/test_auth.py:119:    assert logout.status_code == 204
apps/api/tests/test_auth.py:123:        cur.execute("SELECT revoked_at FROM auth_sessions
WHERE refresh_token_hash = %s", (token_digest(cookie),))
apps/api/tests/test_auth.py:127:def test_logout_without_session_is_idempotent(client):
apps/api/tests/test_auth.py:128:    response = client.post("/auth/logout")
apps/api/tests/test_auth.py:134:def test_expired_session_is_rejected(client, identity):
apps/api/tests/test_auth.py:136:    login = client.post("/auth/login", json={"email": email,
"password": TEST_PASSWORD})
apps/api/tests/test_auth.py:137:    cookie = login.cookies.get(SESSION_COOKIE)
apps/api/tests/test_auth.py:140:        cur.execute("UPDATE auth_sessions SET last_seen_at =
now() - interval '61 minutes' WHERE user_id = %s", (user_id,))
apps/api/tests/test_auth.py:151:        "/auth/logout", headers={"Origin": "https://attacker.invalid"},
apps/api/tests/test_auth.py:159:    response = client.post("/auth/login", json={"email": email,
"password": TEST_PASSWORD})
apps/api/tests/test_fin14_auth_security.py:10:from app.throttle import make_throttle_key
apps/api/tests/test_fin14_auth_security.py:15:def
test_successful_login_is_audited_without_secrets(monkeypatch):
apps/api/tests/test_fin14_auth_security.py:32:                "/auth/login", json={"email": email,
"password": password}
```

```
apps/api/tests/test_fin14_auth_security.py:39:                FROM audit_events
apps/api/tests/test_fin14_auth_security.py:40:                WHERE action_key = 'auth.login' AND
actor_user_id = %s
apps/api/tests/test_fin14_auth_security.py:47:        assert event[0] == "auth.login"
apps/api/tests/test_fin14_auth_security.py:51:        assert
response.cookies.get("finsight_session") not in event[3]
apps/api/tests/test_fin14_auth_security.py:55:            "UPDATE auth_sessions SET
revoked_at = COALESCE(revoked_at, now()), "
apps/api/tests/test_fin14_auth_security.py:67:def
test_failed_login_is_audited_and_repeated_failures_are_blocked(monkeypatch):
apps/api/tests/test_fin14_auth_security.py:76:    ip_key = make_throttle_key(
apps/api/tests/test_fin14_auth_security.py:84:            "DELETE FROM auth_login_throttle
WHERE scope_type = 'ip' AND key_hash = %s",
apps/api/tests/test_fin14_auth_security.py:97:            "/auth/login", json={"email": email,
"password": "wrong-password"}
apps/api/tests/test_fin14_auth_security.py:100:            "/auth/login", json={"email": email,
"password": "wrong-password"}
apps/api/tests/test_fin14_auth_security.py:103:            "/auth/login", json={"email": email,
"password": password}
apps/api/tests/test_fin14_auth_security.py:114:                FROM audit_events
apps/api/tests/test_fin14_auth_security.py:115:                WHERE action_key = 'auth.login'
apps/api/tests/test_fin14_auth_security.py:117:                    'invalid_credentials', 'throttled'
apps/api/tests/test_fin14_auth_security.py:134:            "DELETE FROM auth_login_throttle
WHERE scope_type = 'ip' AND key_hash = %s",
apps/api/tests/test_fin14_auth_security.py:138:            "UPDATE auth_sessions SET
revoked_at = COALESCE(revoked_at, now()), "
apps/api/tests/test_fin14_auth_security.py:150:def
test_logout_is_audited_without_recording_session_token(monkeypatch):
apps/api/tests/test_fin14_auth_security.py:168:        login = client.post(
apps/api/tests/test_fin14_auth_security.py:169:            "/auth/login",
apps/api/tests/test_fin14_auth_security.py:172:        assert login.status_code == 200
apps/api/tests/test_fin14_auth_security.py:173:        session_token =
login.cookies.get("finsight_session")
apps/api/tests/test_fin14_auth_security.py:174:        assert session_token
apps/api/tests/test_fin14_auth_security.py:176:        logout = client.post(
apps/api/tests/test_fin14_auth_security.py:177:            "/auth/logout",
apps/api/tests/test_fin14_auth_security.py:178:            cookies={"finsight_session":
session_token},
apps/api/tests/test_fin14_auth_security.py:180:        assert logout.status_code == 204
apps/api/tests/test_fin14_auth_security.py:185:                FROM audit_events
apps/api/tests/test_fin14_auth_security.py:186:                WHERE action_key = 'auth.logout'
apps/api/tests/test_fin14_auth_security.py:196:        assert session_token not in event[2]
apps/api/tests/test_fin14_auth_security.py:201:            "UPDATE auth_sessions SET
revoked_at = COALESCE(revoked_at, now()), "
```

apps/api/tests/test_fin14_auth_security.py:213:def
test_auth_correlation_id_is_returned_and_audited(monkeypatch):
apps/api/tests/test_fin14_auth_security.py:233:   "/auth/login",
apps/api/tests/test_fin14_auth_security.py:244:   FROM audit_events
apps/api/tests/test_fin14_auth_security.py:245:   WHERE action_key = 'auth.login'
apps/api/tests/test_fin14_auth_security.py:258:   "UPDATE auth_sessions SET
revoked_at = COALESCE(revoked_at, now()), "
apps/api/tests/test_fin14_auth_security.py:278:   "/auth/login",
apps/api/tests/test_fin14_auth_security.py:287:def
test_failed_login_returns_the_audited_correlation_id(monkeypatch):
apps/api/tests/test_fin14_auth_security.py:296:   "/auth/login",
apps/api/tests/test_fin14_auth_security.py:310:   FROM audit_events
apps/api/tests/test_fin14_auth_security.py:311:   WHERE action_key = 'auth.login'
apps/api/tests/test_fin14_auth_security.py:323:def
test_generated_auth_correlation_id_matches_audit_record(monkeypatch):
apps/api/tests/test_fin14_auth_security.py:332:   "/auth/login",
apps/api/tests/test_fin14_auth_security.py:344:   FROM audit_events
apps/api/tests/test_fin14_auth_security.py:345:   WHERE action_key = 'auth.login'
apps/api/tests/test_fin14_auth_security.py:356:def
test_throttle_expires_and_correct_login_succeeds_after_retry(monkeypatch):
apps/api/tests/test_fin14_auth_security.py:368:   account_key =
make_throttle_key(secret.encode("utf-8"), "account", email.lower())
apps/api/tests/test_fin14_auth_security.py:369:   ip_key =
make_throttle_key(secret.encode("utf-8"), "ip", "testclient")
apps/api/tests/test_fin14_auth_security.py:373:   """DELETE FROM auth_login_throttle
apps/api/tests/test_fin14_auth_security.py:387:   "/auth/login",
apps/api/tests/test_fin14_auth_security.py:394:   "/auth/login",
apps/api/tests/test_fin14_auth_security.py:404:   "/auth/login",
apps/api/tests/test_fin14_auth_security.py:415:   FROM audit_events
apps/api/tests/test_fin14_auth_security.py:428:   "DELETE FROM auth_login_throttle
WHERE key_hash IN (%s, %s)",
apps/api/tests/test_fin14_auth_security.py:432:   """UPDATE auth_sessions
apps/api/tests/test_audit_service.py:8:from app.audit import record_auth_event
apps/api/tests/test_audit_service.py:33:def
test_auth_audit_uses_canonical_columns_and_parameterized_sql():
apps/api/tests/test_audit_service.py:40:   action_key="auth.login",
apps/api/tests/test_audit_service.py:50:   assert "INSERT INTO audit_events" in query
apps/api/tests/test_audit_service.py:65:   assert "auth.login" in params
apps/api/tests/test_audit_service.py:71:def
test_unknown_account_failure_can_be_audited_without_an_actor():
apps/api/tests/test_audit_service.py:76:   action_key="auth.login",
apps/api/tests/test_audit_service.py:91:   "raw-session-token-value",
apps/api/tests/test_audit_service.py:96:def test_auth_audit_redacts_secret_values(secret):
apps/api/tests/test_audit_service.py:101:   action_key="auth.login",

```
apps/api/tests/test_audit_service.py:107:            "session_token": secret,
apps/api/tests/test_audit_service.py:120:def
test_auth_audit_rejects_unknown_outcome_before_database_write():
apps/api/tests/test_audit_service.py:126:            action_key="auth.login",

===== CURRENT WORKTREE SUMMARY =====
 M .env.example
 M FUNCTION_TEST_TRACEABILITY.txt
 M IMPLEMENTATION_STATUS.txt
 M SPRINT_1_TEST_EXECUTION_STATUS.txt
 M apps/api/app/main.py
 M apps/api/tests/test_auth.py
 M conversation.txt
 M docs/release/RELEASE_README.md
 M scripts/package_release.py
?? apps/api/app/audit.py
?? apps/api/app/throttle.py
?? apps/api/migrations/0003_auth_audit_and_login_throttling.sql
?? apps/api/migrations/0004_immutable_auth_audit.sql
?? apps/api/tests/test_audit_immutability.py
?? apps/api/tests/test_audit_service.py
?? apps/api/tests/test_fin14_auth_security.py
?? apps/api/tests/test_throttle.py
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % printf '\n=====
AUTHENTICATION DEPENDENCY =====\n'
sed -n '110,175p' apps/api/app/main.py

printf '\n===== LOGIN AND LOGOUT HANDLERS =====\n'
sed -n '220,355p' apps/api/app/main.py

printf '\n===== USERS AND SESSION TABLES =====\n'
sed -n '1,65p' apps/api/migrations/0001_auth_identity.sql

printf '\n===== AUTH AUDIT SCHEMA =====\n'
sed -n '1,100p' apps/api/migrations/0003_auth_audit_and_login_throttling.sql

printf '\n===== DEPENDENCY FILES =====\n'
find apps/api -maxdepth 2 -type f -name'requirements *.txt' -o -name'pyproject.toml'
-print

printf '\n===== MFA AND SESSION CONFIGURATION =====\n'
```

```
grep -RniE
'mfa_required|MFA_|SESSION_|SESSION_COOKIE|SESSION_TTL|token_digest|record_auth_
event' \
  apps/api/app .env.example apps/api/requirements.txt apps/api/requirements-dev.txt 2>/dev/null

===== AUTHENTICATION DEPENDENCY =====
        return existing

    supplied = request.headers.get("X-Correlation-ID", "").strip()
    if supplied:
        try:
            correlation_id = UUID(supplied)
        except ValueError as exc:
            raise HTTPException(
                status_code=400,
                detail="Invalid X-Correlation-ID",
            ) from exc
    else:
        correlation_id = uuid4()

    request.state.auth_correlation_id = correlation_id
    return correlation_id


def get_current_user(
    session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None,
) -> dict[str, Any]:
    if not session_token:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
detail="Authentication required")
    try:
        with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
            cur.execute(
                """SELECT u.user_id, u.email, u.display_name, r.role_key,
                        COALESCE(array_agg(DISTINCT p.permission_key)
                                FILTER (WHERE p.permission_key IS NOT NULL), ARRAY[]::varchar[])
AS permissions
                    FROM auth_sessions s JOIN users u ON u.user_id = s.user_id
                    LEFT JOIN roles r ON r.role_id = u.platform_role_id AND r.scope = 'platform'
                    LEFT JOIN role_permissions rp ON rp.role_id = r.role_id
                    LEFT JOIN permissions p ON p.permission_id = rp.permission_id
                    WHERE s.refresh_token_hash = %s AND s.revoked_at IS NULL
                      AND s.expires_at > %s AND u.account_status = 'active'
                      AND s.last_seen_at > %s - make_interval(mins => %s)
```

```
                 AND u.deleted_at IS NULL
               GROUP BY u.user_id, u.email, u.display_name, r.role_key""",
            (token_digest(session_token), datetime.now(UTC), datetime.now(UTC),
_ttl_minutes()),
        )
        user = cur.fetchone()
        if not user:
            raise HTTPException(status_code=401, detail="Authentication required")
        cur.execute(
            "UPDATE auth_sessions SET last_seen_at = %s WHERE refresh_token_hash = %s",
            (datetime.now(UTC), token_digest(session_token)),
        )
        return user
    except psycopg.Error as exc:
        raise HTTPException(status_code=503, detail="Authentication service unavailable") from
exc


CurrentUser = Annotated[dict[str, Any], Depends(get_current_user)]


def require_permission(permission_key: str):
    """Build a deny-by-default backend permission dependency."""
    def dependency(user: CurrentUser) -> dict[str, Any]:
        if permission_key not in user.get("permissions", []):
            raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Permission
denied")
        return user

    return dependency


@app.post("/auth/login", response_model=UserView)

===== LOGIN AND LOGOUT HANDLERS =====
                        AND deleted_at IS NULL""",
                    (email,),
                )
                user = cur.fetchone()

            password_hash = (
                user["password_hash"]
                if user and user["password_hash"]
                else DUMMY_PASSWORD_HASH
```

```
)
password_valid = verify_password(
    password_hash, payload.password
)
login_succeeded = bool(
    user and user["password_hash"] and password_valid
)

if not login_succeeded:
    record_failure(
        conn,
        scope_type="account",
        key_hash=account_key,
        config=config,
    )
    record_failure(
        conn,
        scope_type="ip",
        key_hash=ip_key,
        config=config,
    )
    record_auth_event(
        conn,
        action_key="auth.login",
        outcome="failure",
        actor_user_id=user["user_id"] if user else None,
        details={"reason": "invalid_credentials"},
        correlation_id=correlation_id,
    )
else:
    # Clear account failures only. Preserve IP-wide history.
    reset_bucket(
        conn,
        scope_type="account",
        key_hash=account_key,
    )

    token, digest = new_session_token()
    expires_at = session_expiry(_ttl_minutes())

    with conn.cursor() as cur:
        cur.execute(
            """INSERT INTO auth_sessions
                  (user_id, refresh_token_hash, expires_at)
```

```
                    VALUES (%s, %s, %s)""",
                (user["user_id"], digest, expires_at),
            )
            now = datetime.now(UTC)
            cur.execute(
                """UPDATE users
                    SET last_login_at = %s, updated_at = %s
                    WHERE user_id = %s""",
                (now, now, user["user_id"]),
            )

        record_auth_event(
            conn,
            action_key="auth.login",
            outcome="success",
            actor_user_id=user["user_id"],
            details={"method": "password"},
            correlation_id=correlation_id,
        )

except psycopg.Error as exc:
    raise HTTPException(
        status_code=503,
        detail="Authentication service unavailable",
    ) from exc

if not login_succeeded:
    raise HTTPException(status_code=401, detail=GENERIC_LOGIN_ERROR)

response.set_cookie(
    SESSION_COOKIE,
    token,
    httponly=True,
    secure=_secure_cookie(),
    samesite="lax",
    max_age=_ttl_minutes() * 60,
    path="/",
)
return UserView(
    user_id=user["user_id"],
    email=user["email"],
    display_name=user["display_name"],
)
```

```
@app.post("/auth/logout", status_code=204)
def logout(
    request: Request,
    response: Response,
    session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None,
) -> Response:
    _check_origin(request)
    correlation_id = _auth_correlation_id(request)
    response.headers['X-Correlation-ID'] = str(correlation_id)

    try:
        with connect() as conn:
            actor_user_id = None

            if session_token:
                with conn.cursor() as cur:
                    cur.execute(
                        """UPDATE auth_sessions
                           SET revoked_at = %s, revoke_reason = 'logout'
                           WHERE refresh_token_hash = %s
                             AND revoked_at IS NULL
                           RETURNING user_id""",
                        (datetime.now(UTC), token_digest(session_token)),
                    )
                    row = cur.fetchone()
                    if row:
                        actor_user_id = row[0]

            record_auth_event(
                conn,
                action_key="auth.logout",
                outcome="success" if actor_user_id else "denied",
                actor_user_id=actor_user_id,
                details={"reason": "logout"},
                correlation_id=correlation_id,
            )
    except psycopg.Error as exc:
        raise HTTPException(
```

```
===== USERS AND SESSION TABLES =====
-- FIN-13 identity/session subset. Derived from Database/schema (1).sql.
-- Apply only to a new development/test database; this is not the full target schema.
BEGIN;
```

```
CREATE TABLE roles (
   role_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
   role_key VARCHAR(50) NOT NULL UNIQUE,
   display_name VARCHAR(100) NOT NULL,
   scope VARCHAR(20) NOT NULL CHECK (scope IN ('platform','workspace')),
   description TEXT,
   created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE users (
   user_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
   email VARCHAR(320) NOT NULL,
   password_hash TEXT,
   display_name VARCHAR(160) NOT NULL,
   account_status VARCHAR(20) NOT NULL DEFAULT 'active'
      CHECK (account_status IN ('invited','active','locked','disabled','deleted')),
   platform_role_id UUID REFERENCES roles(role_id) ON DELETE SET NULL,
   email_verified_at TIMESTAMPTZ,
   mfa_required BOOLEAN NOT NULL DEFAULT false,
   last_login_at TIMESTAMPTZ,
   created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
   updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
   deleted_at TIMESTAMPTZ
);
CREATE UNIQUE INDEX uq_users_email_lower ON users (lower(email));

CREATE TABLE auth_sessions (
   session_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
   user_id UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
   refresh_token_hash TEXT UNIQUE,
   ip_address INET,
   user_agent TEXT,
   created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
   last_seen_at TIMESTAMPTZ NOT NULL DEFAULT now(),
   expires_at TIMESTAMPTZ NOT NULL,
   revoked_at TIMESTAMPTZ,
   revoke_reason VARCHAR(80)
);
CREATE INDEX ix_auth_sessions_active ON auth_sessions(user_id, expires_at) WHERE
revoked_at IS NULL;

COMMIT;
```

```
===== AUTH AUDIT SCHEMA =====
BEGIN;

-- Authentication/security audit trail.
-- workspace_id remains nullable because the current application schema
-- does not yet contain the workspaces table.
CREATE TABLE audit_events (
    audit_event_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id UUID NULL,
    actor_user_id UUID NULL REFERENCES users(user_id) ON DELETE SET NULL,
    action_key VARCHAR(120) NOT NULL,
    resource_type VARCHAR(80) NOT NULL,
    resource_id UUID NULL,
    outcome VARCHAR(20) NOT NULL
        CHECK (outcome IN ('success', 'failure', 'denied')),
    correlation_id UUID NULL,
    details_redacted JSONB NOT NULL DEFAULT '{}'::jsonb,
    occurred_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_audit_events_occurred_at
    ON audit_events (occurred_at DESC);

CREATE INDEX idx_audit_events_actor_occurred
    ON audit_events (actor_user_id, occurred_at DESC);

CREATE INDEX idx_audit_events_action_occurred
    ON audit_events (action_key, occurred_at DESC);

-- Persistent, shared login-failure counters.
-- key_hash must be a keyed digest generated by the application;
-- raw email addresses, IP addresses, passwords and tokens do not belong here.
CREATE TABLE auth_login_throttle (
    scope_type VARCHAR(16) NOT NULL
        CHECK (scope_type IN ('account', 'ip')),
    key_hash CHAR(64) NOT NULL,
    failed_attempts INTEGER NOT NULL DEFAULT 0
        CHECK (failed_attempts >= 0),
    window_started_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    blocked_until TIMESTAMPTZ NULL,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (scope_type, key_hash)
);
```

```
CREATE INDEX idx_auth_login_throttle_blocked_until
   ON auth_login_throttle (blocked_until)
   WHERE blocked_until IS NOT NULL;

COMMIT;

===== DEPENDENCY FILES =====
apps/api/requirements.txt
apps/api/requirements-dev.txt

===== MFA AND SESSION CONFIGURATION =====
apps/api/app/auth.py:28:def new_session_token() -> tuple[str, str]:
apps/api/app/auth.py:33:def session_expiry(ttl_minutes: int, now: datetime | None = None) ->
datetime:
apps/api/app/auth.py:35:        raise ValueError("SESSION_TTL_MINUTES must be between 1
and 10080")
apps/api/app/auth.py:39:def token_digest(token: str) -> str:
apps/api/app/audit.py:38:def record_auth_event(
Binary file apps/api/app/__pycache__/main.cpython-312.pyc matches
Binary file apps/api/app/__pycache__/audit.cpython-314.pyc matches
Binary file apps/api/app/__pycache__/main.cpython-314.pyc matches
Binary file apps/api/app/__pycache__/audit.cpython-312.pyc matches
Binary file apps/api/app/__pycache__/auth.cpython-314.pyc matches
Binary file apps/api/app/__pycache__/auth.cpython-312.pyc matches
apps/api/app/main.py:12:from app.audit import record_auth_event
apps/api/app/main.py:28:from app.auth import hash_password, new_session_token,
session_expiry, token_digest, verify_password
apps/api/app/main.py:68:SESSION_COOKIE = "finsight_session"
apps/api/app/main.py:86:        value = int(os.environ.get("SESSION_TTL_MINUTES", "60"))
apps/api/app/main.py:87:        session_expiry(value)
apps/api/app/main.py:129:    session_token: Annotated[str | None,
Cookie(alias=SESSION_COOKIE)] = None,
apps/api/app/main.py:131:    if not session_token:
apps/api/app/main.py:148:                (token_digest(session_token), datetime.now(UTC),
datetime.now(UTC), _ttl_minutes()),
apps/api/app/main.py:155:                (datetime.now(UTC), token_digest(session_token)),
apps/api/app/main.py:205:                record_auth_event(
apps/api/app/main.py:250:                    record_auth_event(
apps/api/app/main.py:266:                    token, digest = new_session_token()
apps/api/app/main.py:267:                    expires_at = session_expiry(_ttl_minutes())
apps/api/app/main.py:284:                    record_auth_event(
apps/api/app/main.py:303:        SESSION_COOKIE,
apps/api/app/main.py:322:    session_token: Annotated[str | None,
Cookie(alias=SESSION_COOKIE)] = None,
```

```
apps/api/app/main.py:332:            if session_token:
apps/api/app/main.py:340:                    (datetime.now(UTC), token_digest(session_token)),
apps/api/app/main.py:346:            record_auth_event(
apps/api/app/main.py:361:        SESSION_COOKIE,
.env.example:4:SESSION_TTL_MINUTES=60
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
env.example:4:SESSION_TTL_MINUTES=60
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % printf '\n===== MAIN
MODULE IMPORTS AND CONFIGURATION =====\n'
sed -n '1,115p' apps/api/app/main.py

printf '\n===== COMPLETE SESSION ENDPOINTS =====\n'
sed -n '180,375p' apps/api/app/main.py

printf '\n===== API DEPENDENCIES =====\n'
cat apps/api/requirements.txt
cat apps/api/requirements-dev.txt

printf '\n===== CURRENT TEST CLIENT FIXTURES =====\n'
sed -n '1,100p' apps/api/tests/test_auth.py

printf '\n===== MFA POLICY IN REQUIREMENTS =====\n'
grep -RniE 'FR-P1-11|FR-P1-12|TC_AUTHSEC_003|MFA|multi.factor|active.session' \
  FUNCTION_TEST_TRACEABILITY.txt SPRINT_1_TEST_EXECUTION_STATUS.txt \
  IMPLEMENTATION_STATUS.txt conversation.txt

===== MAIN MODULE IMPORTS AND CONFIGURATION =====
"""FinSight AI API with PostgreSQL-backed browser sessions."""

from __future__ import annotations

import os
import secrets
from datetime import UTC, datetime
from pathlib import Path
from typing import Annotated, Any
from uuid import UUID, uuid4

from app.audit import record_auth_event
from app.throttle import (
    is_blocked,
    load_throttle_config,
    make_throttle_key,
    record_failure,
```

```
    reset_bucket,
)

import psycopg
from dotenv import load_dotenv
from fastapi import Cookie, Depends, FastAPI, HTTPException, Request, Response, status
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel, EmailStr, Field
from psycopg.rows import dict_row

from app.auth import hash_password, new_session_token, session_expiry, token_digest,
verify_password
from app.database import connect

load_dotenv(Path(__file__).resolve().parents[3] / ".env")

app = FastAPI(title="FinSight AI API", version="0.2.0")


@app.middleware("http")
async def auth_correlation_middleware(request: Request, call_next):
    """Return a correlation ID on authentication responses, including errors."""
    if request.url.path not in {"/auth/login", "/auth/logout"}:
        return await call_next(request)

    supplied = request.headers.get("X-Correlation-ID", "").strip()
    try:
        correlation_id = UUID(supplied) if supplied else uuid4()
    except ValueError:
        from starlette.responses import JSONResponse

        return JSONResponse(
            status_code=400,
            content={"detail": "Invalid X-Correlation-ID"},
        )

    request.state.auth_correlation_id = correlation_id
    response = await call_next(request)
    response.headers["X-Correlation-ID"] = str(correlation_id)
    return response


web_origin = os.environ.get("WEB_ORIGIN", "http://localhost:3000")
app.add_middleware(
```

```
    CORSMiddleware,
    allow_origins=[web_origin],
    allow_credentials=True,
    allow_methods=["GET", "POST"],
    allow_headers=["Content-Type", "X-Correlation-ID"],
)

SESSION_COOKIE = "finsight_session"
GENERIC_LOGIN_ERROR = "Email or password is incorrect."
DUMMY_PASSWORD_HASH = hash_password(secrets.token_urlsafe(32))


class LoginRequest(BaseModel):
    email: EmailStr
    password: str = Field(min_length=1, max_length=1024)


class UserView(BaseModel):
    user_id: UUID
    email: str
    display_name: str


def _ttl_minutes() -> int:
    try:
        value = int(os.environ.get("SESSION_TTL_MINUTES", "60"))
        session_expiry(value)
        return value
    except ValueError as exc:
        raise HTTPException(status_code=500, detail="Session configuration is invalid") from exc


def _secure_cookie() -> bool:
    configured = os.environ.get("COOKIE_SECURE")
    if configured is not None:
        return configured.lower() in {"1", "true", "yes"}
    return os.environ.get("APP_ENV", "development").lower() == "production"


def _check_origin(request: Request) -> None:
    origin = request.headers.get("origin")
    if origin and origin != web_origin:
        raise HTTPException(status_code=403, detail="Request origin is not allowed")
```

```
def _auth_correlation_id(request: Request) -> UUID:
    """Reuse the middleware's correlation ID, or create one if needed."""
    existing = getattr(request.state, "auth_correlation_id", None)
    if existing is not None:
        return existing

    supplied = request.headers.get("X-Correlation-ID", "").strip()
    if supplied:
        try:
            correlation_id = UUID(supplied)

===== COMPLETE SESSION ENDPOINTS =====
    email = payload.email.strip().lower()
    try:
        config = load_throttle_config()
    except ValueError as exc:
        raise HTTPException(
            status_code=503,
            detail="Authentication service unavailable",
        ) from exc

    client_ip = request.client.host if request.client else "unknown"
    account_key = make_throttle_key(config.secret, "account", email)
    ip_key = make_throttle_key(config.secret, "ip", client_ip)
    login_succeeded = False
    user = None

    try:
        with connect() as conn:
            account_blocked = is_blocked(
                conn, scope_type="account", key_hash=account_key
            )
            ip_blocked = is_blocked(
                conn, scope_type="ip", key_hash=ip_key
            )

            if account_blocked or ip_blocked:
                record_auth_event(
                    conn,
                    action_key="auth.login",
                    outcome="denied",
                    actor_user_id=None,
                    details={"reason": "throttled"},
```

```
            correlation_id=correlation_id,
        )
    else:
        with conn.cursor(row_factory=dict_row) as cur:
            cur.execute(
                """SELECT user_id, email, display_name, password_hash
                   FROM users
                   WHERE lower(email) = %s
                     AND account_status = 'active'
                     AND deleted_at IS NULL""",
                (email,),
            )
            user = cur.fetchone()

        password_hash = (
            user["password_hash"]
            if user and user["password_hash"]
            else DUMMY_PASSWORD_HASH
        )
        password_valid = verify_password(
            password_hash, payload.password
        )
        login_succeeded = bool(
            user and user["password_hash"] and password_valid
        )

        if not login_succeeded:
            record_failure(
                conn,
                scope_type="account",
                key_hash=account_key,
                config=config,
            )
            record_failure(
                conn,
                scope_type="ip",
                key_hash=ip_key,
                config=config,
            )
            record_auth_event(
                conn,
                action_key="auth.login",
                outcome="failure",
                actor_user_id=user["user_id"] if user else None,
```

```
                details={"reason": "invalid_credentials"},
                correlation_id=correlation_id,
            )
        else:
            # Clear account failures only. Preserve IP-wide history.
            reset_bucket(
                conn,
                scope_type="account",
                key_hash=account_key,
            )

            token, digest = new_session_token()
            expires_at = session_expiry(_ttl_minutes())

            with conn.cursor() as cur:
                cur.execute(
                    """INSERT INTO auth_sessions
                           (user_id, refresh_token_hash, expires_at)
                       VALUES (%s, %s, %s)""",
                    (user["user_id"], digest, expires_at),
                )
                now = datetime.now(UTC)
                cur.execute(
                    """UPDATE users
                       SET last_login_at = %s, updated_at = %s
                       WHERE user_id = %s""",
                    (now, now, user["user_id"]),
                )

            record_auth_event(
                conn,
                action_key="auth.login",
                outcome="success",
                actor_user_id=user["user_id"],
                details={"method": "password"},
                correlation_id=correlation_id,
            )

    except psycopg.Error as exc:
        raise HTTPException(
            status_code=503,
            detail="Authentication service unavailable",
        ) from exc
```

```
    if not login_succeeded:
        raise HTTPException(status_code=401, detail=GENERIC_LOGIN_ERROR)

    response.set_cookie(
        SESSION_COOKIE,
        token,
        httponly=True,
        secure=_secure_cookie(),
        samesite="lax",
        max_age=_ttl_minutes() * 60,
        path="/",
    )
    return UserView(
        user_id=user["user_id"],
        email=user["email"],
        display_name=user["display_name"],
    )


@app.post("/auth/logout", status_code=204)
def logout(
    request: Request,
    response: Response,
    session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None,
) -> Response:
    _check_origin(request)
    correlation_id = _auth_correlation_id(request)
    response.headers['X-Correlation-ID'] = str(correlation_id)

    try:
        with connect() as conn:
            actor_user_id = None

            if session_token:
                with conn.cursor() as cur:
                    cur.execute(
                        """UPDATE auth_sessions
                           SET revoked_at = %s, revoke_reason = 'logout'
                           WHERE refresh_token_hash = %s
                             AND revoked_at IS NULL
                           RETURNING user_id""",
                        (datetime.now(UTC), token_digest(session_token)),
                    )
                    row = cur.fetchone()
```

```
                if row:
                    actor_user_id = row[0]

            record_auth_event(
                conn,
                action_key="auth.logout",
                outcome="success" if actor_user_id else "denied",
                actor_user_id=actor_user_id,
                details={"reason": "logout"},
                correlation_id=correlation_id,
            )
    except psycopg.Error as exc:
        raise HTTPException(
            status_code=503,
            detail="Authentication service unavailable",
        ) from exc

    response.delete_cookie(
        SESSION_COOKIE,
        path="/",
        httponly=True,
        secure=_secure_cookie(),
        samesite="lax",
    )
    response.status_code = status.HTTP_204_NO_CONTENT
    return response


@app.get("/auth/me", response_model=UserView)
def who_am_i(user: CurrentUser) -> UserView:
    return UserView(**user)



===== API DEPENDENCIES =====
fastapi==0.133.0
starlette==1.3.1
uvicorn==0.40.0
argon2-cffi==25.1.0
psycopg[binary]==3.2.13
python-dotenv==1.2.2
email-validator==2.3.0
-r requirements.txt
pytest==9.0.3
```

```
httpx==0.28.1
pytest-cov==6.2.1

===== CURRENT TEST CLIENT FIXTURES =====
from __future__ import annotations

import secrets
from uuid import uuid4

import pytest
from fastapi.testclient import TestClient
from psycopg.rows import dict_row

from app.auth import hash_password, token_digest, verify_password
from app.throttle import make_throttle_key
from app.database import connect
from app.main import SESSION_COOKIE, app

TEST_PASSWORD = secrets.token_urlsafe(32)


@pytest.fixture
def identity():
    email = f"{uuid4().hex}@example.com"
    encoded = hash_password(TEST_PASSWORD)
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "INSERT INTO users (email, password_hash, display_name, account_status) VALUES
(%s, %s, %s, 'active') RETURNING user_id",
            (email, encoded, "Synthetic Analyst"),
        )
        user_id = cur.fetchone()[0]
    try:
        yield user_id, email, encoded
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                "UPDATE auth_sessions SET revoked_at = COALESCE(revoked_at, now()), "
                "revoke_reason = COALESCE(revoke_reason, 'test_cleanup') "
                "WHERE user_id = %s",
                (user_id,),
            )
            cur.execute(
```

```
            "UPDATE users SET account_status = 'deleted', deleted_at =
COALESCE(deleted_at, now()) "
            "WHERE user_id = %s",
            (user_id,),
        )


@pytest.fixture
def client(monkeypatch):
    monkeypatch.setenv("SESSION_TTL_MINUTES", "60")
    monkeypatch.setenv("COOKIE_SECURE", "false")
    monkeypatch.setenv("WEB_ORIGIN", "http://localhost:3000")
    throttle_secret = "test-only-secret-material-at-least-32-bytes"
    monkeypatch.setenv("AUTH_THROTTLE_SECRET", throttle_secret)
    ip_key = make_throttle_key(
        throttle_secret.encode("utf-8"), "ip", "testclient"
    )

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "DELETE FROM auth_login_throttle WHERE scope_type = 'ip' AND key_hash = %s",
            (ip_key,),
        )

    try:
        with TestClient(app) as test_client:
            yield test_client
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                "DELETE FROM auth_login_throttle WHERE scope_type = 'ip' AND key_hash = %s",
                (ip_key,),
            )


def test_password_hash_is_argon2_and_never_plaintext():
    encoded = hash_password(TEST_PASSWORD)
    assert encoded.startswith("argon2id")
    assert encoded != TEST_PASSWORD
    assert verify_password(encoded, TEST_PASSWORD)
    assert not verify_password(encoded, "wrong")


def test_login_creates_protected_hashed_expiring_session(client, identity, caplog, capsys):
```

```
    user_id, email, encoded = identity
    response = client.post("/auth/login", json={"email": email, "password": TEST_PASSWORD})
    assert response.status_code == 200
    assert response.json() == {"user_id": str(user_id), "email": email, "display_name": "Synthetic
Analyst"}
    assert TEST_PASSWORD not in response.text and encoded not in response.text
    cookie = response.cookies.get(SESSION_COOKIE)
    assert cookie
    set_cookie = response.headers["set-cookie"].lower()
    assert "httponly" in set_cookie and "samesite=lax" in set_cookie and "path=/" in set_cookie
    with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
        cur.execute("SELECT refresh_token_hash, expires_at, revoked_at FROM auth_sessions
WHERE user_id = %s", (user_id,))
        stored = cur.fetchone()
    assert stored["refresh_token_hash"] == token_digest(cookie)
    assert stored["refresh_token_hash"] != cookie
    assert stored["expires_at"] is not None and stored["revoked_at"] is None
    protected = client.get("/auth/me", cookies={SESSION_COOKIE: cookie})
    assert protected.status_code == 200 and protected.json()["user_id"] == str(user_id)
    captured = capsys.readouterr()
    assert TEST_PASSWORD not in caplog.text + captured.out + captured.err

===== MFA POLICY IN REQUIREMENTS =====
FUNCTION_TEST_TRACEABILITY.txt:5:Authorization rule: requires a valid active session and
database-resolved `platform.admin` permission; deny by default.
FUNCTION_TEST_TRACEABILITY.txt:111:- TC_AUTHSEC_003 — BLOCKED.
FUNCTION_TEST_TRACEABILITY.txt:113:  Current-session logout/revocation is implemented;
MFA and inspection/
FUNCTION_TEST_TRACEABILITY.txt:114:  termination of other active sessions are not
implemented.
FUNCTION_TEST_TRACEABILITY.txt:127:- TC_AUTHSEC_003 remains BLOCKED: MFA and
cross-session controls are not implemented.
SPRINT_1_TEST_EXECUTION_STATUS.txt:51:TC_AUTHZ_003 — Expire inactive sessions
according to configured policy
SPRINT_1_TEST_EXECUTION_STATUS.txt:99:TC_AUTHSEC_003 | Status: BLOCKED |
Execution date: 2026-10-21 | Actual result: MFA challenge enforcement and
inspection/termination of other active sessions are not implemented; current-session logout is
not equivalent | Tester/owner: Codex | Evidence: apps/api/app/main.py;
apps/api/tests/test_auth.py | Linked defect: MFApolicy/provider decision and cross-session
management
SPRINT_1_TEST_EXECUTION_STATUS.txt:230:- TC_AUTHSEC_003: BLOCKED. MFA
enforcement and listing/terminating other
SPRINT_1_TEST_EXECUTION_STATUS.txt:231:  active sessions are not implemented.
Existing logout only revokes the
```

SPRINT_1_TEST_EXECUTION_STATUS.txt:245:- TC_AUTHSEC_003 remains BLOCKED: MFA and cross-session inspection/termination are not implemented.
IMPLEMENTATION_STATUS.txt:96:- Deployed HTTPS verification is blocked pending an HTTPS deployment/endpoint; local secure-cookie behavior alone is insufficient. Role/permission administration, authentication event/audit recording, MFA, rate limiting, and broader auth cases remain unverified against SRS-wide requirements.
IMPLEMENTATION_STATUS.txt:496: into the login flow; logout revokes the presented active session and
IMPLEMENTATION_STATUS.txt:517:- TC_AUTHSEC_003 remains BLOCKED: MFA policy/provider and challenge flow are
IMPLEMENTATION_STATUS.txt:518: not implemented; inspection and termination of other active sessions are
IMPLEMENTATION_STATUS.txt:533:- Open: database-level append-only audit enforcement; complete login-level throttle expiry and successful retry; MFA policy/provider and challenge enforcement; cross-session inspection and termination.
conversation.txt:67:Status: PARTIAL. Enabled backend branch coverage in the existing CI test commands and retained XML and HTML reports as a 14-day GitHub artifact (workflow configured; remote upload not yet observed). Updated README commands. Added focused tests for missing session access, login/session database failures, required DATABASE_URL, logout without an active session, provisioning validation/hash/upsert behavior, and the provisioning module CLI entry point. CI-equivalent local suites passed: unit 37 tests; PostgreSQL integration 10 tests; combined 47 passed. Final cumulative backend coverage: line 156/156 (100%), branch 28/28 (100%). Evidence: ignored local `coverage.xml`, `htmlcov/index.html`, and `/tmp/finsight-coverage.json`. No overall threshold was invented. Frontend coverage is not measured: existing Playwright tests cover interactions but do not instrument source code. A minimal future configuration would instrument client source modules and connect coverage reporting to the existing interaction suite; decide whether that additional frontend metric isrequired for acceptance.
conversation.txt:130:Q-012 — FIN-14 MFA policy/provider and active-session management
conversation.txt:135:FIN-14 / FS-01-03 includes conditional MFA and active-session inspection/
conversation.txt:137:an MFA challenge/provider flow or a capability to inspect and terminate other
conversation.txt:138:active sessions. Logging out the current session is not equivalent to
conversation.txt:142:Approve the MFA policy (when MFA is required, the challenge method/provider,
conversation.txt:144:placeholder MFA flow. Implement session listing and revocation for authorized
conversation.txt:147:Until the policy and functionality are resolved, TC_AUTHSEC_003 remains
conversation.txt:156:- FIN-14 remains partial; append-only enforcement, complete throttle expiry/retry acceptance, MFA, and cross-session controls remain open.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro NVDIA_Play_Book % cd FinSight_AI
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear

```
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % printf '\n===== AUDIT
HELPER =====\n'
sed -n '1,180p' apps/api/app/audit.py

printf '\n===== REMAINING API MODULE =====\n'
sed -n '320,390p' apps/api/app/main.py

printf '\n===== DATABASE CONNECTION HELPER =====\n'
sed -n '1,120p' apps/api/app/database.py

===== AUDIT HELPER =====
"""Reusable, redacted audit persistence for authentication security events."""

from __future__ import annotations

import json
import re
from datetime import UTC, datetime
from typing import Any
from uuid import UUID

_ALLOWED_OUTCOMES = frozenset({"success", "failure", "denied"})
_SECRET_KEY = re.compile(
    r"(password|passwd|token|cookie|authorization|api[_-]?key|secret|credential)",
    re.IGNORECASE,
)


def _redact(value: Any) -> Any:
    """Return JSON-safe audit details without values under secret-like keys."""
    if isinstance(value, dict):
        return {
            str(key): (
                "[REDACTED]" if _SECRET_KEY.search(str(key)) else _redact(item)
            )
            for key, item in value.items()
        }
    if isinstance(value, (list, tuple)):
        return [_redact(item) for item in value]
    if isinstance(value, UUID):
        return str(value)
    if isinstance(value, datetime):
        return value.astimezone(UTC).isoformat()
    if value is None or isinstance(value, (str, int, float, bool)):
```

```
        return value
    return str(value)


def record_auth_event(
    conn: Any,
    *,
    action_key: str,
    outcome: str,
    actor_user_id: UUID | None,
    details: dict[str, Any] | None = None,
    correlation_id: UUID | None = None,
    occurred_at: datetime | None = None,
) -> None:
    """Persist one authentication audit event on the caller's transaction.

    Persistence errors intentionally propagate to the caller. The caller must
    roll back and apply the documented fail-closed policy; events are never
    reported as recorded when this insert fails.
    """
    if outcome not in _ALLOWED_OUTCOMES:
        raise ValueError("Unsupported audit outcome")
    if not action_key or len(action_key) > 120:
        raise ValueError("action_key must contain 1 to 120 characters")

    redacted_details = _redact(details or {})
    payload = json.dumps(redacted_details, separators=(",", ":"), sort_keys=True)

    with conn.cursor() as cur:
        cur.execute(
            """
            INSERT INTO audit_events (
                workspace_id,
                actor_user_id,
                action_key,
                resource_type,
                resource_id,
                outcome,
                correlation_id,
                details_redacted,
                occurred_at
            )
            VALUES (NULL, %s, %s, 'authentication', NULL, %s, %s, %s::jsonb, %s)
            """,
```

```
            (
                actor_user_id,
                action_key,
                outcome,
                correlation_id,
                payload,
                occurred_at or datetime.now(UTC),
            ),
        )

===== REMAINING API MODULE =====
    request: Request,
    response: Response,
    session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None,
) -> Response:
    _check_origin(request)
    correlation_id = _auth_correlation_id(request)
    response.headers['X-Correlation-ID'] = str(correlation_id)

    try:
        with connect() as conn:
            actor_user_id = None

            if session_token:
                with conn.cursor() as cur:
                    cur.execute(
                        """UPDATE auth_sessions
                           SET revoked_at = %s, revoke_reason = 'logout'
                           WHERE refresh_token_hash = %s
                             AND revoked_at IS NULL
                           RETURNING user_id""",
                        (datetime.now(UTC), token_digest(session_token)),
                    )
                    row = cur.fetchone()
                    if row:
                        actor_user_id = row[0]

            record_auth_event(
                conn,
                action_key="auth.logout",
                outcome="success" if actor_user_id else "denied",
                actor_user_id=actor_user_id,
                details={"reason": "logout"},
                correlation_id=correlation_id,
```

```
        )
    except psycopg.Error as exc:
        raise HTTPException(
            status_code=503,
            detail="Authentication service unavailable",
        ) from exc

    response.delete_cookie(
        SESSION_COOKIE,
        path="/",
        httponly=True,
        secure=_secure_cookie(),
        samesite="lax",
    )
    response.status_code = status.HTTP_204_NO_CONTENT
    return response


@app.get("/auth/me", response_model=UserView)
def who_am_i(user: CurrentUser) -> UserView:
    return UserView(**user)


@app.get("/api/v1/admin/authorization/capabilities")
def platform_admin_capabilities(
    request: Request,
    _authorized: Annotated[dict[str, Any], Depends(require_permission("platform.admin"))],
) -> dict[str, Any]:
    """Return the authenticated caller's database-backed platform capabilities."""

    supplied_request_id = request.headers.get("request_id", "").strip()
    if supplied_request_id and (
        len(supplied_request_id) > 128
        or any(ord(char) < 32 or ord(char) == 127 for char in supplied_request_id)
    ):
        raise HTTPException(status_code=400, detail="Invalid request_id")

    # Always issue a server-generated canonical request ID.

===== DATABASE CONNECTION HELPER =====
"""Small synchronous PostgreSQL boundary for the auth endpoints."""

from __future__ import annotations
```

```
import os
from pathlib import Path

import psycopg
from dotenv import load_dotenv

load_dotenv(Path(__file__).resolve().parents[3] / ".env")


def connect() -> psycopg.Connection:
    database_url = os.environ.get("DATABASE_URL")
    if not database_url:
        raise RuntimeError("DATABASE_URL is required")
    return psycopg.connect(database_url, connect_timeout=3)
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....

    if not terminated:
        raise HTTPException(status_code=404, detail="Active session not found")

    response.status_code = status.HTTP_204_NO_CONTENT
    return response


''' + marker

if source.count(old_methods) != 1:
    raise SystemExit("STOP: expected CORS allow_methods line not found exactly once.")
if source.count(old_middleware) != 1:
    raise SystemExit("STOP: expected correlation middleware guard not found exactly once.")
if source.count(marker) != 1:
    raise SystemExit("STOP: expected /auth/me insertion point not found exactly once.")
if '@app.get("/auth/sessions"' in source or '@app.delete("/auth/sessions/' in source:
    raise SystemExit("STOP: session routes already exist; no changes made.")

backup = path.with_name(
    f"main.py.backup-{datetime.now().strftime('%Y%m%d-%H%M%S')}"
)
shutil.copy2(path, backup)

updated = source.replace(old_methods, new_methods)
updated = updated.replace(old_middleware, new_middleware)
updated = updated.replace(marker, routes)
```

```
path.write_text(updated, encoding="utf-8")
print(f"Updated {path}")
print(f"Backup: {backup}")
PY
Updated apps/api/app/main.py
Backup: apps/api/app/main.py.backup-20261022-080642
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....

    if not terminated:
        raise HTTPException(status_code=404, detail="Active session not found")

    response.status_code = status.HTTP_204_NO_CONTENT
    return response


''' + marker

if source.count(old_methods) != 1:
    raise SystemExit("STOP: expected CORS allow_methods line not found exactly once.")
if source.count(old_middleware) != 1:
    raise SystemExit("STOP: expected correlation middleware guard not found exactly once.")
if source.count(marker) != 1:
    raise SystemExit("STOP: expected /auth/me insertion point not found exactly once.")
if '@app.get("/auth/sessions"' in source or '@app.delete("/auth/sessions/' in source:
    raise SystemExit("STOP: session routes already exist; no changes made.")

backup = path.with_name(
    f"main.py.backup-{datetime.now().strftime('%Y%m%d-%H%M%S')}"
)
shutil.copy2(path, backup)

updated = source.replace(old_methods, new_methods)
updated = updated.replace(old_middleware, new_middleware)
updated = updated.replace(marker, routes)

path.write_text(updated, encoding="utf-8")
print(f"Updated {path}")
print(f"Backup: {backup}")
PY
Updated apps/api/app/main.py
Backup: apps/api/app/main.py.backup-20261022-080642
```

```
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % git diff --check
git diff -- apps/api/app/main.py
diff --git a/apps/api/app/main.py b/apps/api/app/main.py
index 5e0e618..6e64241 100644
--- a/apps/api/app/main.py
+++ b/apps/api/app/main.py
@@ -7,7 +7,16 @@ import secrets
 from datetime import UTC, datetime
 from pathlib import Path
 from typing import Annotated, Any
-from uuid import UUID
+from uuid import UUID, uuid4
+
+from app.audit import record_auth_event
+from app.throttle import (
+    is_blocked,
+    load_throttle_config,
+    make_throttle_key,
+    record_failure,
+    reset_bucket,
+)

 import psycopg
 from dotenv import load_dotenv
@@ -22,13 +31,41 @@ from app.database import connect
 load_dotenv(Path(__file__).resolve().parents[3] / ".env")

 app = FastAPI(title="FinSight AI API", version="0.2.0")
+
+
+@app.middleware("http")
+async def auth_correlation_middleware(request: Request, call_next):
+    """Return a correlation ID on authentication responses, including errors."""
+    if (
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % .venv/bin/python -m
py_compile apps/api/app/main.py
git diff --check
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % grep -nE 'class
SessionView|def list_active_sessions|def terminate_session|auth/sessions'
apps/api/app/main.py
41:        and not request.url.path.startswith("/auth/sessions")
374:class SessionView(BaseModel):
384:@app.get("/auth/sessions", response_model=list[SessionView])
```

```
385:def list_active_sessions(
418:@app.delete("/auth/sessions/{session_id}", status_code=204)
419:def terminate_session(
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % PYTHONPATH=apps/api
.venv/bin/python - <<'PY'
from app.main import app

for route in app.routes:
    path = getattr(route, "path", "")
    methods = getattr(route, "methods", set()) or set()
    if path.startswith("/auth/sessions"):
        print(f"{','.join(sorted(methods))} {path}")

print("FastAPI app import: OK")
PY
GET /auth/sessions
DELETE /auth/sessions/{session_id}
FastAPI app import: OK
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % PYTHONPATH=apps/api \
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test' \
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' \
.venv/bin/python -m pytest \
  apps/api/tests/test_auth.py \
  apps/api/tests/test_fin14_auth_security.py \
  apps/api/tests/test_audit_immutability.py \
  -q
.....................
[100%]
=============================================================================
========= warnings summary
=============================================================================
==========
.venv/lib/python3.14/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is
deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

.venv/lib/python3.14/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
```

```
s/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is
deprecated, use anyio.from_thread.BlockingPortal instead.
   _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

apps/api/tests/test_auth.py::test_login_creates_protected_hashed_expiring_session
apps/api/tests/test_auth.py::test_logout_revokes_session_and_replay_fails
apps/api/tests/test_auth.py::test_logout_revokes_session_and_replay_fails
apps/api/tests/test_auth.py::test_expired_session_is_rejected
apps/api/tests/test_fin14_auth_security.py::test_logout_is_audited_without_recording_session_t
oken

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being
deprecated, because the expected behaviour on cookie persistence is ambiguous. Set cookies
directly on the client instance instead.
   return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
18 passed, 7 warnings in 2.97s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
18 passed, 7 warnings in 2.97s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % printf '\n===== AUTH TEST
FIXTURES AND HELPERS =====\n'
grep -nE '^(def |@pytest.fixture|class Test)' apps/api/tests/test_auth.py

printf '\n===== AUTH TEST CLIENT AND LOGIN TESTS =====\n'
sed -n '1,220p' apps/api/tests/test_auth.py

printf '\n===== SESSION ENDPOINTS =====\n'
sed -n '365,465p' apps/api/app/main.py

===== AUTH TEST FIXTURES AND HELPERS =====
18:@pytest.fixture
19:def identity():
45:@pytest.fixture
46:def client(monkeypatch):
73:def test_password_hash_is_argon2_and_never_plaintext():
81:def test_login_creates_protected_hashed_expiring_session(client, identity, caplog, capsys):
104:def test_invalid_credentials_have_same_generic_response(client, identity):
113:def test_logout_revokes_session_and_replay_fails(client, identity):
127:def test_logout_without_session_is_idempotent(client):
134:def test_expired_session_is_rejected(client, identity):
144:def test_public_registration_is_not_available(client):
```

149:def test_auth_mutation_rejects_foreign_origin(client):
156:def test_secure_cookie_can_be_required_by_deployment(client, identity, monkeypatch):

===== AUTH TEST CLIENT AND LOGIN TESTS =====
```
from __future__ import annotations

import secrets
from uuid import uuid4

import pytest
from fastapi.testclient import TestClient
from psycopg.rows import dict_row

from app.auth import hash_password, token_digest, verify_password
from app.throttle import make_throttle_key
from app.database import connect
from app.main import SESSION_COOKIE, app

TEST_PASSWORD = secrets.token_urlsafe(32)


@pytest.fixture
def identity():
    email = f"{uuid4().hex}@example.com"
    encoded = hash_password(TEST_PASSWORD)
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "INSERT INTO users (email, password_hash, display_name, account_status) VALUES
(%s, %s, %s, 'active') RETURNING user_id",
            (email, encoded, "Synthetic Analyst"),
        )
        user_id = cur.fetchone()[0]
    try:
        yield user_id, email, encoded
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                "UPDATE auth_sessions SET revoked_at = COALESCE(revoked_at, now()), "
                "revoke_reason = COALESCE(revoke_reason, 'test_cleanup') "
                "WHERE user_id = %s",
                (user_id,),
            )
            cur.execute(
```

```
            "UPDATE users SET account_status = 'deleted', deleted_at =
COALESCE(deleted_at, now()) "
            "WHERE user_id = %s",
            (user_id,),
        )


@pytest.fixture
def client(monkeypatch):
    monkeypatch.setenv("SESSION_TTL_MINUTES", "60")
    monkeypatch.setenv("COOKIE_SECURE", "false")
    monkeypatch.setenv("WEB_ORIGIN", "http://localhost:3000")
    throttle_secret = "test-only-secret-material-at-least-32-bytes"
    monkeypatch.setenv("AUTH_THROTTLE_SECRET", throttle_secret)
    ip_key = make_throttle_key(
        throttle_secret.encode("utf-8"), "ip", "testclient"
    )

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "DELETE FROM auth_login_throttle WHERE scope_type = 'ip' AND key_hash = %s",
            (ip_key,),
        )

    try:
        with TestClient(app) as test_client:
            yield test_client
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                "DELETE FROM auth_login_throttle WHERE scope_type = 'ip' AND key_hash = %s",
                (ip_key,),
            )


def test_password_hash_is_argon2_and_never_plaintext():
    encoded = hash_password(TEST_PASSWORD)
    assert encoded.startswith("argon2id")
    assert encoded != TEST_PASSWORD
    assert verify_password(encoded, TEST_PASSWORD)
    assert not verify_password(encoded, "wrong")


def test_login_creates_protected_hashed_expiring_session(client, identity, caplog, capsys):
```

```
    user_id, email, encoded = identity
    response = client.post("/auth/login", json={"email": email, "password": TEST_PASSWORD})
    assert response.status_code == 200
    assert response.json() == {"user_id": str(user_id), "email": email, "display_name": "Synthetic
Analyst"}
    assert TEST_PASSWORD not in response.text and encoded not in response.text
    cookie = response.cookies.get(SESSION_COOKIE)
    assert cookie
    set_cookie = response.headers["set-cookie"].lower()
    assert "httponly" in set_cookie and "samesite=lax" in set_cookie and "path=/" in set_cookie
    with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
        cur.execute("SELECT refresh_token_hash, expires_at, revoked_at FROM auth_sessions
WHERE user_id = %s", (user_id,))
        stored = cur.fetchone()
    assert stored["refresh_token_hash"] == token_digest(cookie)
    assert stored["refresh_token_hash"] != cookie
    assert stored["expires_at"] is not None and stored["revoked_at"] is None
    protected = client.get("/auth/me", cookies={SESSION_COOKIE: cookie})
    assert protected.status_code == 200 and protected.json()["user_id"] == str(user_id)
    captured = capsys.readouterr()
    assert TEST_PASSWORD not in caplog.text + captured.out + captured.err
    assert cookie not in caplog.text + captured.out + captured.err


def test_invalid_credentials_have_same_generic_response(client, identity):
    _, email, _ = identity
    known = client.post("/auth/login", json={"email": email, "password": "wrong-password"})
    absent = client.post("/auth/login", json={"email": "absent@example.com", "password":
"wrong-password"})
    assert (known.status_code, known.json()) == (absent.status_code, absent.json())
    assert known.status_code == 401 and known.json() == {"detail": "Email or password is
incorrect."}
    assert SESSION_COOKIE not in known.cookies and SESSION_COOKIE not in
absent.cookies


def test_logout_revokes_session_and_replay_fails(client, identity):
    _, email, _ = identity
    login = client.post("/auth/login", json={"email": email, "password": TEST_PASSWORD})
    cookie = login.cookies.get(SESSION_COOKIE)
    assert cookie
    logout = client.post("/auth/logout", cookies={SESSION_COOKIE: cookie})
    assert logout.status_code == 204
    replay = client.get("/auth/me", cookies={SESSION_COOKIE: cookie})
```

```
    assert replay.status_code == 401
    with connect() as conn, conn.cursor() as cur:
        cur.execute("SELECT revoked_at FROM auth_sessions WHERE refresh_token_hash =
%s", (token_digest(cookie),))
        assert cur.fetchone()[0] is not None


def test_logout_without_session_is_idempotent(client):
    response = client.post("/auth/logout")

    assert response.status_code == 204
    assert response.headers["set-cookie"].startswith(f"{SESSION_COOKIE}=")


def test_expired_session_is_rejected(client, identity):
    user_id, email, _ = identity
    login = client.post("/auth/login", json={"email": email, "password": TEST_PASSWORD})
    cookie = login.cookies.get(SESSION_COOKIE)
    assert cookie
    with connect() as conn, conn.cursor() as cur:
        cur.execute("UPDATE auth_sessions SET last_seen_at = now() - interval '61 minutes'
WHERE user_id = %s", (user_id,))
    assert client.get("/auth/me", cookies={SESSION_COOKIE: cookie}).status_code == 401


def test_public_registration_is_not_available(client):
    response = client.post("/auth/register", json={"email": "public@example.com", "password":
TEST_PASSWORD})
    assert response.status_code == 404


def test_auth_mutation_rejects_foreign_origin(client):
    response = client.post(
        "/auth/logout", headers={"Origin": "https://attacker.invalid"},
    )
    assert response.status_code == 403


def test_secure_cookie_can_be_required_by_deployment(client, identity, monkeypatch):
    _, email, _ = identity
    monkeypatch.setenv("COOKIE_SECURE", "true")
    response = client.post("/auth/login", json={"email": email, "password": TEST_PASSWORD})
    assert response.status_code == 200
    assert "secure" in response.headers["set-cookie"].lower()
```

```
===== SESSION ENDPOINTS =====
        path="/",
        httponly=True,
        secure=_secure_cookie(),
        samesite="lax",
    )
    response.status_code = status.HTTP_204_NO_CONTENT
    return response


class SessionView(BaseModel):
    session_id: UUID
    created_at: datetime
    last_seen_at: datetime
    expires_at: datetime
    ip_address: str | None
    user_agent: str | None
    is_current: bool


@app.get("/auth/sessions", response_model=list[SessionView])
def list_active_sessions(
    request: Request,
    user: CurrentUser,
    session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None,
) -> list[SessionView]:
    correlation_id = _auth_correlation_id(request)
    token_hash = token_digest(session_token) if session_token else None
    now = datetime.now(UTC)

    try:
        with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
            cur.execute(
                """SELECT session_id, created_at, last_seen_at, expires_at,
                        host(ip_address) AS ip_address, user_agent,
                        (refresh_token_hash = %s) AS is_current
                   FROM auth_sessions
                   WHERE user_id = %s
                     AND revoked_at IS NULL
                     AND expires_at > %s
                     AND last_seen_at > %s - make_interval(mins => %s)
                   ORDER BY created_at DESC""",
                (token_hash, user["user_id"], now, now, _ttl_minutes()),
```

```
            )
            rows = cur.fetchall()
    except psycopg.Error as exc:
        raise HTTPException(
            status_code=503,
            detail="Authentication service unavailable",
        ) from exc

    return [SessionView(**row) for row in rows]


@app.delete("/auth/sessions/{session_id}", status_code=204)
def terminate_session(
    session_id: UUID,
    request: Request,
    response: Response,
    user: CurrentUser,
) -> Response:
    _check_origin(request)
    correlation_id = _auth_correlation_id(request)
    response.headers["X-Correlation-ID"] = str(correlation_id)
    now = datetime.now(UTC)
    terminated = False

    try:
        with connect() as conn:
            with conn.cursor() as cur:
                cur.execute(
                    """UPDATE auth_sessions
                       SET revoked_at = %s, revoke_reason = 'user_terminated'
                       WHERE session_id = %s
                         AND user_id = %s
                         AND revoked_at IS NULL
                         AND expires_at > %s
                       RETURNING session_id""",
                    (now, session_id, user["user_id"], now),
                )
                terminated = cur.fetchone() is not None

            record_auth_event(
                conn,
                action_key="auth.session.terminate",
                outcome="success" if terminated else "denied",
                actor_user_id=user["user_id"],
```

```
            details={
                "session_id": str(session_id),
                "reason": "user_terminated" if terminated else "not_found_or_inactive",
            },
            correlation_id=correlation_id,
        )
    except psycopg.Error as exc:
        raise HTTPException(
            status_code=503,
            detail="Authentication service unavailable",
        ) from exc

    if not terminated:
        raise HTTPException(status_code=404, detail="Active session not found")

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cd
"$HOME/Documents/NVDIA_Play_Book/FinSight_AI"
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % .venv/bin/python - <<'PY'
from pathlib import Path

p = Path("apps/api/app/main.py")
s = p.read_text()

old = '''def list_active_sessions(
    request: Request,
    session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None,
    user: CurrentUser = None,
) -> list[SessionView]:'''

new = '''def list_active_sessions(
    request: Request,
    user: CurrentUser,
    session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None,
) -> list[SessionView]:'''

if old in s:
    p.write_text(s.replace(old, new, 1))
    print("Corrected list_active_sessions parameter order.")
elif new in s:
    print("Parameter order already corrected; no changes made.")
else:
```

```
    raise SystemExit("STOP: expected function signature not found; file unchanged.")
PY
Parameter order already corrected; no changes made.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % .venv/bin/python -m
py_compile apps/api/app/main.py

printf '\n===== SESSION ROUTES =====\n'
grep -nE 'allow_methods|class SessionView|def list_active_sessions|def
terminate_session|auth/sessions' apps/api/app/main.py

printf '\n===== SESSION IMPLEMENTATION =====\n'
sed -n '325,445p' apps/api/app/main.py

printf '\n===== WHITESPACE CHECK =====\n'
git diff --check

===== SESSION ROUTES =====
41:        and not request.url.path.startswith("/auth/sessions")
67:    allow_methods=["GET", "POST", "DELETE"],
374:class SessionView(BaseModel):
384:@app.get("/auth/sessions", response_model=list[SessionView])
385:def list_active_sessions(
418:@app.delete("/auth/sessions/{session_id}", status_code=204)
419:def terminate_session(

===== SESSION IMPLEMENTATION =====
    session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None,
) -> Response:
    _check_origin(request)
    correlation_id = _auth_correlation_id(request)
    response.headers['X-Correlation-ID'] = str(correlation_id)

    try:
        with connect() as conn:
            actor_user_id = None

            if session_token:
                with conn.cursor() as cur:
                    cur.execute(
                        """UPDATE auth_sessions
                           SET revoked_at = %s, revoke_reason = 'logout'
                           WHERE refresh_token_hash = %s
                             AND revoked_at IS NULL
                           RETURNING user_id""",
```

```
                    (datetime.now(UTC), token_digest(session_token)),
                )
                row = cur.fetchone()
                if row:
                    actor_user_id = row[0]

        record_auth_event(
            conn,
            action_key="auth.logout",
            outcome="success" if actor_user_id else "denied",
            actor_user_id=actor_user_id,
            details={"reason": "logout"},
            correlation_id=correlation_id,
        )
    except psycopg.Error as exc:
        raise HTTPException(
            status_code=503,
            detail="Authentication service unavailable",
        ) from exc

    response.delete_cookie(
        SESSION_COOKIE,
        path="/",
        httponly=True,
        secure=_secure_cookie(),
        samesite="lax",
    )
    response.status_code = status.HTTP_204_NO_CONTENT
    return response


class SessionView(BaseModel):
    session_id: UUID
    created_at: datetime
    last_seen_at: datetime
    expires_at: datetime
    ip_address: str | None
    user_agent: str | None
    is_current: bool


@app.get("/auth/sessions", response_model=list[SessionView])
def list_active_sessions(
    request: Request,
```

```
    user: CurrentUser,
    session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None,
) -> list[SessionView]:
    correlation_id = _auth_correlation_id(request)
    token_hash = token_digest(session_token) if session_token else None
    now = datetime.now(UTC)

    try:
        with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
            cur.execute(
                """SELECT session_id, created_at, last_seen_at, expires_at,
                          host(ip_address) AS ip_address, user_agent,
                          (refresh_token_hash = %s) AS is_current
                   FROM auth_sessions
                   WHERE user_id = %s
                     AND revoked_at IS NULL
                     AND expires_at > %s
                     AND last_seen_at > %s - make_interval(mins => %s)
                   ORDER BY created_at DESC""",
                (token_hash, user["user_id"], now, now, _ttl_minutes()),
            )
            rows = cur.fetchall()
    except psycopg.Error as exc:
        raise HTTPException(
            status_code=503,
            detail="Authentication service unavailable",
        ) from exc

    return [SessionView(**row) for row in rows]


@app.delete("/auth/sessions/{session_id}", status_code=204)
def terminate_session(
    session_id: UUID,
    request: Request,
    response: Response,
    user: CurrentUser,
) -> Response:
    _check_origin(request)
    correlation_id = _auth_correlation_id(request)
    response.headers["X-Correlation-ID"] = str(correlation_id)
    now = datetime.now(UTC)
    terminated = False
```

```
    try:
        with connect() as conn:
            with conn.cursor() as cur:
                cur.execute(
                    """UPDATE auth_sessions
                       SET revoked_at = %s, revoke_reason = 'user_terminated'
                       WHERE session_id = %s
                         AND user_id = %s
                         AND revoked_at IS NULL
                         AND expires_at > %s
                       RETURNING session_id""",
                    (now, session_id, user["user_id"], now),
                )
                terminated = cur.fetchone() is not None


===== WHITESPACE CHECK =====
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
printf '\n===== SESSION TERMINATION ROUTE =====\n'
sed -n '418,490p' apps/api/app/main.py

printf '\n===== EXISTING AUTH TEST PATTERNS =====\n'
grep -nE 'def test_|def client|def identity|auth/sessions|auth/me|login' \
  apps/api/tests/test_auth.py \
  apps/api/tests/test_fin14_auth_security.py

printf '\n===== CURRENT API TEST SUITE =====\n'
PYTHONPATH=apps/api \
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test' \
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' \
.venv/bin/python -m pytest apps/api/tests -q


===== SESSION TERMINATION ROUTE =====
@app.delete("/auth/sessions/{session_id}", status_code=204)
def terminate_session(
    session_id: UUID,
    request: Request,
    response: Response,
    user: CurrentUser,
) -> Response:
    _check_origin(request)
```

```
correlation_id = _auth_correlation_id(request)
response.headers["X-Correlation-ID"] = str(correlation_id)
now = datetime.now(UTC)
terminated = False

try:
    with connect() as conn:
        with conn.cursor() as cur:
            cur.execute(
                """UPDATE auth_sessions
                   SET revoked_at = %s, revoke_reason = 'user_terminated'
                   WHERE session_id = %s
                     AND user_id = %s
                     AND revoked_at IS NULL
                     AND expires_at > %s
                   RETURNING session_id""",
                (now, session_id, user["user_id"], now),
            )
            terminated = cur.fetchone() is not None

        record_auth_event(
            conn,
            action_key="auth.session.terminate",
            outcome="success" if terminated else "denied",
            actor_user_id=user["user_id"],
            details={
                "session_id": str(session_id),
                "reason": "user_terminated" if terminated else "not_found_or_inactive",
            },
            correlation_id=correlation_id,
        )
except psycopg.Error as exc:
    raise HTTPException(
        status_code=503,
        detail="Authentication service unavailable",
    ) from exc

if not terminated:
    raise HTTPException(status_code=404, detail="Active session not found")

response.status_code = status.HTTP_204_NO_CONTENT
return response
```

```
@app.get("/auth/me", response_model=UserView)
def who_am_i(user: CurrentUser) -> UserView:
    return UserView(**user)


@app.get("/api/v1/admin/authorization/capabilities")
def platform_admin_capabilities(
    request: Request,
    _authorized: Annotated[dict[str, Any], Depends(require_permission("platform.admin"))],
) -> dict[str, Any]:
    """Return the authenticated caller's database-backed platform capabilities."""

    supplied_request_id = request.headers.get("request_id", "").strip()
    if supplied_request_id and (
        len(supplied_request_id) > 128
        or any(ord(char) < 32 or ord(char) == 127 for char in supplied_request_id)
    ):
        raise HTTPException(status_code=400, detail="Invalid request_id")

    # Always issue a server-generated canonical request ID.
    request_id = f"req_{secrets.token_urlsafe(12)}"

===== EXISTING AUTH TEST PATTERNS =====
apps/api/tests/test_auth.py:19:def identity():
apps/api/tests/test_auth.py:46:def client(monkeypatch):
apps/api/tests/test_auth.py:58:            "DELETE FROM auth_login_throttle WHERE scope_type
= 'ip' AND key_hash = %s",
apps/api/tests/test_auth.py:68:                "DELETE FROM auth_login_throttle WHERE
scope_type = 'ip' AND key_hash = %s",
apps/api/tests/test_auth.py:73:def test_password_hash_is_argon2_and_never_plaintext():
apps/api/tests/test_auth.py:81:def
test_login_creates_protected_hashed_expiring_session(client, identity, caplog, capsys):
apps/api/tests/test_auth.py:83:    response = client.post("/auth/login", json={"email": email,
"password": TEST_PASSWORD})
apps/api/tests/test_auth.py:97:    protected = client.get("/auth/me",
cookies={SESSION_COOKIE: cookie})
apps/api/tests/test_auth.py:104:def
test_invalid_credentials_have_same_generic_response(client, identity):
apps/api/tests/test_auth.py:106:    known = client.post("/auth/login", json={"email": email,
"password": "wrong-password"})
apps/api/tests/test_auth.py:107:    absent = client.post("/auth/login", json={"email":
"absent@example.com", "password": "wrong-password"})
apps/api/tests/test_auth.py:113:def test_logout_revokes_session_and_replay_fails(client,
identity):
```

```
apps/api/tests/test_auth.py:115:    login = client.post("/auth/login", json={"email": email,
"password": TEST_PASSWORD})
apps/api/tests/test_auth.py:116:    cookie = login.cookies.get(SESSION_COOKIE)
apps/api/tests/test_auth.py:120:    replay = client.get("/auth/me", cookies={SESSION_COOKIE:
cookie})
apps/api/tests/test_auth.py:127:def test_logout_without_session_is_idempotent(client):
apps/api/tests/test_auth.py:134:def test_expired_session_is_rejected(client, identity):
apps/api/tests/test_auth.py:136:    login = client.post("/auth/login", json={"email": email,
"password": TEST_PASSWORD})
apps/api/tests/test_auth.py:137:    cookie = login.cookies.get(SESSION_COOKIE)
apps/api/tests/test_auth.py:141:    assert client.get("/auth/me", cookies={SESSION_COOKIE:
cookie}).status_code == 401
apps/api/tests/test_auth.py:144:def test_public_registration_is_not_available(client):
apps/api/tests/test_auth.py:149:def test_auth_mutation_rejects_foreign_origin(client):
apps/api/tests/test_auth.py:156:def
test_secure_cookie_can_be_required_by_deployment(client, identity, monkeypatch):
apps/api/tests/test_auth.py:159:    response = client.post("/auth/login", json={"email": email,
"password": TEST_PASSWORD})
apps/api/tests/test_fin14_auth_security.py:15:def
test_successful_login_is_audited_without_secrets(monkeypatch):
apps/api/tests/test_fin14_auth_security.py:32:                "/auth/login", json={"email": email,
"password": password}
apps/api/tests/test_fin14_auth_security.py:40:                  WHERE action_key = 'auth.login' AND
actor_user_id = %s
apps/api/tests/test_fin14_auth_security.py:47:        assert event[0] == "auth.login"
apps/api/tests/test_fin14_auth_security.py:67:def
test_failed_login_is_audited_and_repeated_failures_are_blocked(monkeypatch):
apps/api/tests/test_fin14_auth_security.py:84:            "DELETE FROM auth_login_throttle
WHERE scope_type = 'ip' AND key_hash = %s",
apps/api/tests/test_fin14_auth_security.py:97:                "/auth/login", json={"email": email,
"password": "wrong-password"}
apps/api/tests/test_fin14_auth_security.py:100:                "/auth/login", json={"email": email,
"password": "wrong-password"}
apps/api/tests/test_fin14_auth_security.py:103:                "/auth/login", json={"email": email,
"password": password}
apps/api/tests/test_fin14_auth_security.py:115:                  WHERE action_key = 'auth.login'
apps/api/tests/test_fin14_auth_security.py:134:                "DELETE FROM auth_login_throttle
WHERE scope_type = 'ip' AND key_hash = %s",
apps/api/tests/test_fin14_auth_security.py:150:def
test_logout_is_audited_without_recording_session_token(monkeypatch):
apps/api/tests/test_fin14_auth_security.py:168:            login = client.post(
apps/api/tests/test_fin14_auth_security.py:169:                "/auth/login",
apps/api/tests/test_fin14_auth_security.py:172:            assert login.status_code == 200
```

```
apps/api/tests/test_fin14_auth_security.py:173:           session_token =
login.cookies.get("finsight_session")
apps/api/tests/test_fin14_auth_security.py:213:def
test_auth_correlation_id_is_returned_and_audited(monkeypatch):
apps/api/tests/test_fin14_auth_security.py:233:               "/auth/login",
apps/api/tests/test_fin14_auth_security.py:245:                 WHERE action_key = 'auth.login'
apps/api/tests/test_fin14_auth_security.py:270:def
test_invalid_auth_correlation_id_is_rejected(monkeypatch):
apps/api/tests/test_fin14_auth_security.py:278:           "/auth/login",
apps/api/tests/test_fin14_auth_security.py:287:def
test_failed_login_returns_the_audited_correlation_id(monkeypatch):
apps/api/tests/test_fin14_auth_security.py:296:           "/auth/login",
apps/api/tests/test_fin14_auth_security.py:311:             WHERE action_key = 'auth.login'
apps/api/tests/test_fin14_auth_security.py:323:def
test_generated_auth_correlation_id_matches_audit_record(monkeypatch):
apps/api/tests/test_fin14_auth_security.py:332:           "/auth/login",
apps/api/tests/test_fin14_auth_security.py:345:             WHERE action_key = 'auth.login'
apps/api/tests/test_fin14_auth_security.py:356:def
test_throttle_expires_and_correct_login_succeeds_after_retry(monkeypatch):
apps/api/tests/test_fin14_auth_security.py:373:           """DELETE FROM auth_login_throttle
apps/api/tests/test_fin14_auth_security.py:387:               "/auth/login",
apps/api/tests/test_fin14_auth_security.py:394:               "/auth/login",
apps/api/tests/test_fin14_auth_security.py:404:               "/auth/login",
apps/api/tests/test_fin14_auth_security.py:428:               "DELETE FROM auth_login_throttle
WHERE key_hash IN (%s, %s)",

===== CURRENT API TEST SUITE =====
.....................................................................................
[100%]
=============================================================================
========= warnings summary
=============================================================================
==========
.venv/lib/python3.14/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is
deprecated; install `httpx2` instead.
  from starlette.testclient import TestClient as TestClient  # noqa

.venv/lib/python3.14/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
```

```
s/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is
deprecated, use anyio.from_thread.BlockingPortal instead.
    _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

apps/api/tests/test_auth.py: 4 warnings
apps/api/tests/test_auth_unit.py: 3 warnings
apps/api/tests/test_fin14_auth_security.py: 1 warning
apps/api/tests/test_identity_constraints.py: 9 warnings

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being
deprecated, because the expected behaviour on cookie persistence is ambiguous. Set cookies
directly on the client instance instead.
    return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
89 passed, 19 warnings in 3.36s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
printf '\n===== AUTH TEST FIXTURES =====\n'
sed -n '1,75p' apps/api/tests/test_auth.py

printf '\n===== SESSION TABLE SCHEMA =====\n'
grep -n -A28 -B4 'CREATE TABLE auth_sessions' \
  apps/api/migrations/0001_auth_identity.sql

printf '\n===== AUTH SESSION CREATION =====\n'
grep -n -A35 -B12 'INSERT INTO auth_sessions' apps/api/app/main.py

printf '\n===== SESSION ROUTE IMPLEMENTATION =====\n'
sed -n '374,470p' apps/api/app/main.py


===== AUTH TEST FIXTURES =====
from __future__ import annotations

import secrets
from uuid import uuid4

import pytest
from fastapi.testclient import TestClient
from psycopg.rows import dict_row
```

```
from app.auth import hash_password, token_digest, verify_password
from app.throttle import make_throttle_key
from app.database import connect
from app.main import SESSION_COOKIE, app

TEST_PASSWORD = secrets.token_urlsafe(32)


@pytest.fixture
def identity():
    email = f"{uuid4().hex}@example.com"
    encoded = hash_password(TEST_PASSWORD)
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "INSERT INTO users (email, password_hash, display_name, account_status) VALUES
(%s, %s, %s, 'active') RETURNING user_id",
            (email, encoded, "Synthetic Analyst"),
        )
        user_id = cur.fetchone()[0]
    try:
        yield user_id, email, encoded
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                "UPDATE auth_sessions SET revoked_at = COALESCE(revoked_at, now()), "
                "revoke_reason = COALESCE(revoke_reason, 'test_cleanup') "
                "WHERE user_id = %s",
                (user_id,),
            )
            cur.execute(
                "UPDATE users SET account_status = 'deleted', deleted_at =
COALESCE(deleted_at, now()) "
                "WHERE user_id = %s",
                (user_id,),
            )


@pytest.fixture
def client(monkeypatch):
    monkeypatch.setenv("SESSION_TTL_MINUTES", "60")
    monkeypatch.setenv("COOKIE_SECURE", "false")
    monkeypatch.setenv("WEB_ORIGIN", "http://localhost:3000")
    throttle_secret = "test-only-secret-material-at-least-32-bytes"
    monkeypatch.setenv("AUTH_THROTTLE_SECRET", throttle_secret)
```

```
    ip_key = make_throttle_key(
        throttle_secret.encode("utf-8"), "ip", "testclient"
    )

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "DELETE FROM auth_login_throttle WHERE scope_type = 'ip' AND key_hash = %s",
            (ip_key,),
        )

    try:
        with TestClient(app) as test_client:
            yield test_client
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                "DELETE FROM auth_login_throttle WHERE scope_type = 'ip' AND key_hash = %s",
                (ip_key,),
            )


def test_password_hash_is_argon2_and_never_plaintext():
    encoded = hash_password(TEST_PASSWORD)
    assert encoded.startswith("argon2id")

===== SESSION TABLE SCHEMA =====
27-    deleted_at TIMESTAMPTZ
28-);
29-CREATE UNIQUE INDEX uq_users_email_lower ON users (lower(email));
30-
31:CREATE TABLE auth_sessions (
32-    session_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
33-    user_id UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
34-    refresh_token_hash TEXT UNIQUE,
35-    ip_address INET,
36-    user_agent TEXT,
37-    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
38-    last_seen_at TIMESTAMPTZ NOT NULL DEFAULT now(),
39-    expires_at TIMESTAMPTZ NOT NULL,
40-    revoked_at TIMESTAMPTZ,
41-    revoke_reason VARCHAR(80)
42-);
43-CREATE INDEX ix_auth_sessions_active ON auth_sessions(user_id, expires_at) WHERE
revoked_at IS NULL;
```

```
44-
45-COMMIT;

===== AUTH SESSION CREATION =====
262-                # Clear account failures only. Preserve IP-wide history.
263-                reset_bucket(
264-                    conn,
265-                    scope_type="account",
266-                    key_hash=account_key,
267-                )
268-
269-                token, digest = new_session_token()
270-                expires_at = session_expiry(_ttl_minutes())
271-
272-                with conn.cursor() as cur:
273-                    cur.execute(
274:                        """INSERT INTO auth_sessions
275-                               (user_id, refresh_token_hash, expires_at)
276-                           VALUES (%s, %s, %s)""",
277-                        (user["user_id"], digest, expires_at),
278-                    )
279-                    now = datetime.now(UTC)
280-                    cur.execute(
281-                        """UPDATE users
282-                           SET last_login_at = %s, updated_at = %s
283-                           WHERE user_id = %s""",
284-                        (now, now, user["user_id"]),
285-                    )
286-
287-                record_auth_event(
288-                    conn,
289-                    action_key="auth.login",
290-                    outcome="success",
291-                    actor_user_id=user["user_id"],
292-                    details={"method": "password"},
293-                    correlation_id=correlation_id,
294-                )
295-
296-    except psycopg.Error as exc:
297-        raise HTTPException(
298-            status_code=503,
299-            detail="Authentication service unavailable",
300-        ) from exc
301-
```

```
302-    if not login_succeeded:
303-        raise HTTPException(status_code=401, detail=GENERIC_LOGIN_ERROR)
304-
305-    response.set_cookie(
306-        SESSION_COOKIE,
307-        token,
308-        httponly=True,
309-        secure=_secure_cookie(),

===== SESSION ROUTE IMPLEMENTATION =====
class SessionView(BaseModel):
    session_id: UUID
    created_at: datetime
    last_seen_at: datetime
    expires_at: datetime
    ip_address: str | None
    user_agent: str | None
    is_current: bool


@app.get("/auth/sessions", response_model=list[SessionView])
def list_active_sessions(
    request: Request,
    user: CurrentUser,
    session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None,
) -> list[SessionView]:
    correlation_id = _auth_correlation_id(request)
    token_hash = token_digest(session_token) if session_token else None
    now = datetime.now(UTC)

    try:
        with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
            cur.execute(
                """SELECT session_id, created_at, last_seen_at, expires_at,
                          host(ip_address) AS ip_address, user_agent,
                          (refresh_token_hash = %s) AS is_current
                   FROM auth_sessions
                   WHERE user_id = %s
                     AND revoked_at IS NULL
                     AND expires_at > %s
                     AND last_seen_at > %s - make_interval(mins => %s)
                   ORDER BY created_at DESC""",
                (token_hash, user["user_id"], now, now, _ttl_minutes()),
            )
```

```
            rows = cur.fetchall()
    except psycopg.Error as exc:
        raise HTTPException(
            status_code=503,
            detail="Authentication service unavailable",
        ) from exc

    return [SessionView(**row) for row in rows]


@app.delete("/auth/sessions/{session_id}", status_code=204)
def terminate_session(
    session_id: UUID,
    request: Request,
    response: Response,
    user: CurrentUser,
) -> Response:
    _check_origin(request)
    correlation_id = _auth_correlation_id(request)
    response.headers["X-Correlation-ID"] = str(correlation_id)
    now = datetime.now(UTC)
    terminated = False

    try:
        with connect() as conn:
            with conn.cursor() as cur:
                cur.execute(
                    """UPDATE auth_sessions
                       SET revoked_at = %s, revoke_reason = 'user_terminated'
                       WHERE session_id = %s
                         AND user_id = %s
                         AND revoked_at IS NULL
                         AND expires_at > %s
                       RETURNING session_id""",
                    (now, session_id, user["user_id"], now),
                )
                terminated = cur.fetchone() is not None

            record_auth_event(
                conn,
                action_key="auth.session.terminate",
                outcome="success" if terminated else "denied",
                actor_user_id=user["user_id"],
                details={
```

```
                "session_id": str(session_id),
                "reason": "user_terminated" if terminated else "not_found_or_inactive",
            },
            correlation_id=correlation_id,
        )
    except psycopg.Error as exc:
        raise HTTPException(
            status_code=503,
            detail="Authentication service unavailable",
        ) from exc

    if not terminated:
        raise HTTPException(status_code=404, detail="Active session not found")

    response.status_code = status.HTTP_204_NO_CONTENT
    return response


@app.get("/auth/me", response_model=UserView)
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %

@app.get("/auth/me", response_model=UserView)
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
        assert owner_login.status_code == 200
        assert other_login.status_code == 200

        owner_cookie = owner_login.cookies.get(SESSION_COOKIE)
        other_cookie = other_login.cookies.get(SESSION_COOKIE)
        assert owner_cookie and other_cookie

        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                "SELECT session_id FROM auth_sessions WHERE refresh_token_hash = %s",
                (token_digest(other_cookie),),
            )
            row = cur.fetchone()
        assert row is not None
        other_session_id = row[0]

        response = client.delete(
            f"/auth/sessions/{other_session_id}",
            cookies={SESSION_COOKIE: owner_cookie},
            headers={"Origin": "http://localhost:3000"},
```

```
        )
        assert response.status_code == 404

        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                "SELECT revoked_at FROM auth_sessions WHERE session_id = %s",
                (other_session_id,),
            )
            assert cur.fetchone()[0] is None
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                """UPDATE auth_sessions
                   SET revoked_at = COALESCE(revoked_at, now()),
                       revoke_reason = COALESCE(revoke_reason, 'test_cleanup')
                   WHERE user_id = %s""",
                (other_user_id,),
            )
            cur.execute(
                """UPDATE users
                   SET account_status = 'deleted',
                       deleted_at = COALESCE(deleted_at, now())
                   WHERE user_id = %s""",
                (other_user_id,),
            )
'''

p.write_text(s.rstrip() + "\n\n" + tests.lstrip())
print("Added four active-session regression tests to", p)
PY

PYTHONPATH=apps/api \
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test' \
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' \
.venv/bin/python -m pytest \
  apps/api/tests/test_auth.py -q

Added four active-session regression tests to apps/api/tests/test_auth.py
.....
[100%]
=============================================================================
========= warnings summary
=============================================================================
==========
```

```
.venv/lib/python3.14/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-packages/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

.venv/lib/python3.14/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-packages/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is deprecated, use anyio.from_thread.BlockingPortal instead.
    _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

apps/api/tests/test_auth.py::test_login_creates_protected_hashed_expiring_session
apps/api/tests/test_auth.py::test_logout_revokes_session_and_replay_fails
apps/api/tests/test_auth.py::test_logout_revokes_session_and_replay_fails
apps/api/tests/test_auth.py::test_expired_session_is_rejected
apps/api/tests/test_auth.py::test_list_active_sessions_shows_current_session
apps/api/tests/test_auth.py::test_terminate_own_session_revokes_it
apps/api/tests/test_auth.py::test_terminate_own_session_revokes_it
apps/api/tests/test_auth.py::test_user_cannot_terminate_another_users_session

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-packages/starlette/testclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being deprecated, because the expected behaviour on cookie persistence is ambiguous. Set cookies directly on the client instance instead.
    return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
13 passed, 10 warnings in 1.41s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
PYTHONPATH=apps/api \
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test' \
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' \
.venv/bin/python -m pytest apps/api/tests -q

printf '\n===== MFA REFERENCES IN MIGRATIONS =====\n'
grep -n -i -E 'mfa_required|mfa_factors|secret_ref|factor_status' \
  apps/api/migrations/*.sql

printf '\n===== MFA REFERENCES IN APPLICATION =====\n'
grep -n -i -E 'mfa_required|mfa_factors|secret_ref|totp|mfa_code' \
```

```
  apps/api/app/*.py apps/api/requirements*.txt

.....................................................................................
[100%]
=============================================================================
========= warnings summary
=============================================================================
==========
.venv/lib/python3.14/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is
deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

.venv/lib/python3.14/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is
deprecated, use anyio.from_thread.BlockingPortal instead.
    _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

apps/api/tests/test_auth.py: 8 warnings
apps/api/tests/test_auth_unit.py: 3 warnings
apps/api/tests/test_fin14_auth_security.py: 1 warning
apps/api/tests/test_identity_constraints.py: 9 warnings

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being
deprecated, because the expected behaviour on cookie persistence is ambiguous. Set cookies
directly on the client instance instead.
    return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
93 passed, 23 warnings in 3.79s

===== MFA REFERENCES IN MIGRATIONS =====
apps/api/migrations/0001_auth_identity.sql:23:    mfa_required BOOLEAN NOT NULL
DEFAULT false,

===== MFA REFERENCES IN APPLICATION =====
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
```

```
printf '\n===== USERS TABLE =====\n'
sed -n '1,45p' apps/api/migrations/0001_auth_identity.sql

printf '\n===== LOGIN REQUEST MODEL AND VALIDATION =====\n'
grep -n -A35 -B12 -E 'class Login|def login|@app.post"/auth/login"|LoginRequest'
apps/api/app/main.py

printf '\n===== MIGRATION PACKAGING =====\n'
grep -n -A20 -B8 -E
'0001_auth_identity|0002_platform_authorization|0003_auth_audit|0004_immutable'
scripts/package_release.py

printf '\n===== MFA ENVIRONMENT CONFIGURATION =====\n'
grep -n -i -E 'MFA|AUTH_|SESSION_|COOKIE_' .env.example

printf '\n===== MIGRATION STATUS =====\n'
ls apps/api/migrations


===== USERS TABLE =====
-- FIN-13 identity/session subset. Derived from Database/schema (1).sql.
-- Apply only to a new development/test database; this is not the full target schema.
BEGIN;

CREATE TABLE roles (
    role_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    role_key VARCHAR(50) NOT NULL UNIQUE,
    display_name VARCHAR(100) NOT NULL,
    scope VARCHAR(20) NOT NULL CHECK (scope IN ('platform','workspace')),
    description TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE users (
    user_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(320) NOT NULL,
    password_hash TEXT,
    display_name VARCHAR(160) NOT NULL,
    account_status VARCHAR(20) NOT NULL DEFAULT 'active'
        CHECK (account_status IN ('invited','active','locked','disabled','deleted')),
    platform_role_id UUID REFERENCES roles(role_id) ON DELETE SET NULL,
    email_verified_at TIMESTAMPTZ,
    mfa_required BOOLEAN NOT NULL DEFAULT false,
    last_login_at TIMESTAMPTZ,
```

```
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    deleted_at TIMESTAMPTZ
);
CREATE UNIQUE INDEX uq_users_email_lower ON users (lower(email));

CREATE TABLE auth_sessions (
    session_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    refresh_token_hash TEXT UNIQUE,
    ip_address INET,
    user_agent TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    last_seen_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    expires_at TIMESTAMPTZ NOT NULL,
    revoked_at TIMESTAMPTZ,
    revoke_reason VARCHAR(80)
);
CREATE INDEX ix_auth_sessions_active ON auth_sessions(user_id, expires_at) WHERE
revoked_at IS NULL;

COMMIT;

===== LOGIN REQUEST MODEL AND VALIDATION =====
64-    CORSMiddleware,
65-    allow_origins=[web_origin],
66-    allow_credentials=True,
67-    allow_methods=["GET", "POST", "DELETE"],
68-    allow_headers=["Content-Type", "X-Correlation-ID"],
69-)
70-
71-SESSION_COOKIE = "finsight_session"
72-GENERIC_LOGIN_ERROR = "Email or password is incorrect."
73-DUMMY_PASSWORD_HASH = hash_password(secrets.token_urlsafe(32))
74-
75-
76:class LoginRequest(BaseModel):
77-    email: EmailStr
78-    password: str = Field(min_length=1, max_length=1024)
79-
80-
81-class UserView(BaseModel):
82-    user_id: UUID
83-    email: str
```

```
84-    display_name: str
85-
86-
87-def _ttl_minutes() -> int:
88-    try:
89-        value = int(os.environ.get("SESSION_TTL_MINUTES", "60"))
90-        session_expiry(value)
91-        return value
92-    except ValueError as exc:
93-        raise HTTPException(status_code=500, detail="Session configuration is invalid") from
exc
94-
95-
96-def _secure_cookie() -> bool:
97-    configured = os.environ.get("COOKIE_SECURE")
98-    if configured is not None:
99-        return configured.lower() in {"1", "true", "yes"}
100-    return os.environ.get("APP_ENV", "development").lower() == "production"
101-
102-
103-def _check_origin(request: Request) -> None:
104-    origin = request.headers.get("origin")
105-    if origin and origin != web_origin:
106-        raise HTTPException(status_code=403, detail="Request origin is not allowed")
107-
108-
109-def _auth_correlation_id(request: Request) -> UUID:
110-    """Reuse the middleware's correlation ID, or create one if needed."""
111-    existing = getattr(request.state, "auth_correlation_id", None)
--
167-
168-def require_permission(permission_key: str):
169-    """Build a deny-by-default backend permission dependency."""
170-    def dependency(user: CurrentUser) -> dict[str, Any]:
171-        if permission_key not in user.get("permissions", []):
172-            raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,
detail="Permission denied")
173-        return user
174-
175-    return dependency
176-
177-
178-@app.post("/auth/login", response_model=UserView)
179:def login(payload: LoginRequest, request: Request, response: Response) -> UserView:
```

```
180-    _check_origin(request)
181-    correlation_id = _auth_correlation_id(request)
182-    response.headers['X-Correlation-ID'] = str(correlation_id)
183-    email = payload.email.strip().lower()
184-    try:
185-        config = load_throttle_config()
186-    except ValueError as exc:
187-        raise HTTPException(
188-            status_code=503,
189-            detail="Authentication service unavailable",
190-        ) from exc
191-
192-    client_ip = request.client.host if request.client else "unknown"
193-    account_key = make_throttle_key(config.secret, "account", email)
194-    ip_key = make_throttle_key(config.secret, "ip", client_ip)
195-    login_succeeded = False
196-    user = None
197-
198-    try:
199-        with connect() as conn:
200-            account_blocked = is_blocked(
201-                conn, scope_type="account", key_hash=account_key
202-            )
203-            ip_blocked = is_blocked(
204-                conn, scope_type="ip", key_hash=ip_key
205-            )
206-
207-            if account_blocked or ip_blocked:
208-                record_auth_event(
209-                    conn,
210-                    action_key="auth.login",
211-                    outcome="denied",
212-                    actor_user_id=None,
213-                    details={"reason": "throttled"},
214-                    correlation_id=correlation_id,

===== MIGRATION PACKAGING =====
46-        shutil.copy2(ROOT / "apps/api/requirements.lock", backend / "requirements.lock")
47-
48-        metadata = {
49-            "artifact": artifact_base,
50-            "version": version,
51-            "commit_sha": commit_sha,
52-            "frontend": {"runtime": "Node.js 22", "entrypoint": "node frontend/server.js"},
```

```
53-            "backend": {"runtime": "Python 3.12", "entrypoint": "python -m uvicorn
app.main:app"},
54:            "database_migrations": ["0001_auth_identity.sql", "0002_platform_authorization.sql",
"0003_auth_audit_and_login_throttling.sql", "0004_immutable_auth_audit.sql"],
55-            "configuration": "Runtime values are supplied through environment variables; see
RELEASE_README.md.",
56-        }
57-        (payload / "RELEASE-METADATA.json").write_text(json.dumps(metadata, indent=2) +
"\n", encoding="utf-8")
58-        release_readme = ROOT / "docs/release/RELEASE_README.md"
59-        shutil.copy2(release_readme, payload / "RELEASE_README.md")
60-
61-        artifact_path = output_dir / f"{artifact_base}.tar.gz"
62-        def normalized_member(member: tarfile.TarInfo) -> tarfile.TarInfo:
63-            member.mtime = 0
64-            member.uid = 0
65-            member.gid = 0
66-            member.uname = ""
67-            member.gname = ""
68-            return member
69-
70-        with artifact_path.open("wb") as raw:
71-            import gzip
72-
73-            with gzip.GzipFile(fileobj=raw, mode="wb", mtime=0, filename="") as compressed:
74-                with tarfile.open(fileobj=compressed, mode="w", format=tarfile.PAX_FORMAT) as
archive:

===== MFA ENVIRONMENT CONFIGURATION =====
4:SESSION_TTL_MINUTES=60
5:COOKIE_SECURE=false
9:AUTH_THROTTLE_SECRET=replace-with-a-random-secret-of-at-least-32-bytes
10:AUTH_LOGIN_MAX_FAILURES=5
11:AUTH_LOGIN_WINDOW_SECONDS=900
12:AUTH_LOGIN_BLOCK_SECONDS=900

===== MIGRATION STATUS =====
0001_auth_identity.sql                  0003_auth_audit_and_login_throttling.sql
0002_platform_authorization.sql         0004_immutable_auth_audit.sql
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
cd "$HOME/Documents/NVDIA_Play_Book/FinSight_AI"

printf '\n===== GIT IGNORE RULES =====\n'
```

```
sed -n '1,220p' .gitignore

printf '\n===== PYTHON DEPENDENCIES =====\n'
grep -n -i -E 'cryptography|pyotp|pytest|psycopg|fastapi' \
  apps/api/requirements.txt \
  apps/api/requirements-dev.txt \
  apps/api/requirements.lock

printf '\n===== CRYPTOGRAPHY AVAILABILITY =====\n'
.venv/bin/python - <<'PY'
from importlib.metadata import PackageNotFoundError, version

for package in ("cryptography", "pyotp", "pytest", "psycopg"):
    try:
        print(f"{package}: {version(package)}")
    except PackageNotFoundError:
        print(f"{package}: NOT INSTALLED")
PY

printf '\n===== MIGRATION CONVENTIONS =====\n'
sed -n '1,200p' apps/api/migrations/0003_auth_audit_and_login_throttling.sql

printf '\n===== LOGIN MODEL AND SESSION CREATION =====\n'
sed -n '70,90p' apps/api/app/main.py
sed -n '178,310p' apps/api/app/main.py

printf '\n===== LOCAL ENVIRONMENT IGNORE STATUS =====\n'
git check-ignore .env .env.local apps/api/.env apps/api/.env.local


===== GIT IGNORE RULES =====

# ==============================
# Operating system files
# ==============================
.DS_Store
.AppleDouble
.LSOverride
Thumbs.db
Desktop.ini

# ==============================
# Python
# ==============================
```

```
__pycache__/
*.py[cod]
*$py.class
*.pyo
*.pyd
.python-version

# Virtual environments
.venv/
venv/
env/
ENV/

# Python packaging and test artifacts
build/
dist/
*.egg-info/
.pytest_cache/
.mypy_cache/
.ruff_cache/
.coverage
htmlcov/

# ==============================
# Environment variables and secrets
# ==============================
.env
.env.*
!.env.example
!.env.sample
!.env.template

# Private keys and credentials
*.pem
*.key
credentials.json
service-account*.json

# ==============================
# Logs and temporary files
# ==============================
*.log
logs/
tmp/
```

```
temp/
*.tmp
*.swp
*.swo
*~

# ==============================
# IDE and editor files
# ==============================
.vscode/
.idea/

# ==============================
# Local databases and generated data
# ==============================
*.sqlite
*.sqlite3
*.db

# ==============================
# Machine-learning artifacts
# ==============================
__pycache__/
.ipynb_checkpoints/
wandb/
mlruns/
lightning_logs/

# Node.js / Next.js
node_modules/
.next/
playwright-report/
test-results/
out/
.vercel/
npm-debug.log*
yarn-debug.log*
yarn-error.log*
pnpm-debug.log*


coverage.xml

===== PYTHON DEPENDENCIES =====
```

```
apps/api/requirements.txt:1:fastapi==0.133.0
apps/api/requirements.txt:5:psycopg[binary]==3.2.13
apps/api/requirements-dev.txt:2:pytest==9.0.3
apps/api/requirements-dev.txt:4:pytest-cov==6.2.1
apps/api/requirements.lock:10:fastapi==0.133.0
apps/api/requirements.lock:13:psycopg==3.2.13
apps/api/requirements.lock:14:psycopg-binary==3.2.13

===== CRYPTOGRAPHY AVAILABILITY =====
cryptography: NOT INSTALLED
pyotp: NOT INSTALLED
pytest: 9.0.3
psycopg: 3.2.13

===== MIGRATION CONVENTIONS =====
BEGIN;

-- Authentication/security audit trail.
-- workspace_id remains nullable because the current application schema
-- does not yet contain the workspaces table.
CREATE TABLE audit_events (
    audit_event_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id UUID NULL,
    actor_user_id UUID NULL REFERENCES users(user_id) ON DELETE SET NULL,
    action_key VARCHAR(120) NOT NULL,
    resource_type VARCHAR(80) NOT NULL,
    resource_id UUID NULL,
    outcome VARCHAR(20) NOT NULL
        CHECK (outcome IN ('success', 'failure', 'denied')),
    correlation_id UUID NULL,
    details_redacted JSONB NOT NULL DEFAULT '{}'::jsonb,
    occurred_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_audit_events_occurred_at
    ON audit_events (occurred_at DESC);

CREATE INDEX idx_audit_events_actor_occurred
    ON audit_events (actor_user_id, occurred_at DESC);

CREATE INDEX idx_audit_events_action_occurred
    ON audit_events (action_key, occurred_at DESC);

-- Persistent, shared login-failure counters.
```

```
-- key_hash must be a keyed digest generated by the application;
-- raw email addresses, IP addresses, passwords and tokens do not belong here.
CREATE TABLE auth_login_throttle (
    scope_type VARCHAR(16) NOT NULL
        CHECK (scope_type IN ('account', 'ip')),
    key_hash CHAR(64) NOT NULL,
    failed_attempts INTEGER NOT NULL DEFAULT 0
        CHECK (failed_attempts >= 0),
    window_started_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    blocked_until TIMESTAMPTZ NULL,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (scope_type, key_hash)
);

CREATE INDEX idx_auth_login_throttle_blocked_until
    ON auth_login_throttle (blocked_until)
    WHERE blocked_until IS NOT NULL;

COMMIT;

===== LOGIN MODEL AND SESSION CREATION =====

SESSION_COOKIE = "finsight_session"
GENERIC_LOGIN_ERROR = "Email or password is incorrect."
DUMMY_PASSWORD_HASH = hash_password(secrets.token_urlsafe(32))


class LoginRequest(BaseModel):
    email: EmailStr
    password: str = Field(min_length=1, max_length=1024)


class UserView(BaseModel):
    user_id: UUID
    email: str
    display_name: str


def _ttl_minutes() -> int:
    try:
        value = int(os.environ.get("SESSION_TTL_MINUTES", "60"))
        session_expiry(value)
@app.post("/auth/login", response_model=UserView)
def login(payload: LoginRequest, request: Request, response: Response) -> UserView:
```

```
_check_origin(request)
correlation_id = _auth_correlation_id(request)
response.headers['X-Correlation-ID'] = str(correlation_id)
email = payload.email.strip().lower()
try:
    config = load_throttle_config()
except ValueError as exc:
    raise HTTPException(
        status_code=503,
        detail="Authentication service unavailable",
    ) from exc

client_ip = request.client.host if request.client else "unknown"
account_key = make_throttle_key(config.secret, "account", email)
ip_key = make_throttle_key(config.secret, "ip", client_ip)
login_succeeded = False
user = None

try:
    with connect() as conn:
        account_blocked = is_blocked(
            conn, scope_type="account", key_hash=account_key
        )
        ip_blocked = is_blocked(
            conn, scope_type="ip", key_hash=ip_key
        )

        if account_blocked or ip_blocked:
            record_auth_event(
                conn,
                action_key="auth.login",
                outcome="denied",
                actor_user_id=None,
                details={"reason": "throttled"},
                correlation_id=correlation_id,
            )
        else:
            with conn.cursor(row_factory=dict_row) as cur:
                cur.execute(
                    """SELECT user_id, email, display_name, password_hash
                       FROM users
                       WHERE lower(email) = %s
                         AND account_status = 'active'
                         AND deleted_at IS NULL""",
```

```
        (email,),
    )
    user = cur.fetchone()

password_hash = (
    user["password_hash"]
    if user and user["password_hash"]
    else DUMMY_PASSWORD_HASH
)
password_valid = verify_password(
    password_hash, payload.password
)
login_succeeded = bool(
    user and user["password_hash"] and password_valid
)

if not login_succeeded:
    record_failure(
        conn,
        scope_type="account",
        key_hash=account_key,
        config=config,
    )
    record_failure(
        conn,
        scope_type="ip",
        key_hash=ip_key,
        config=config,
    )
    record_auth_event(
        conn,
        action_key="auth.login",
        outcome="failure",
        actor_user_id=user["user_id"] if user else None,
        details={"reason": "invalid_credentials"},
        correlation_id=correlation_id,
    )
else:
    # Clear account failures only. Preserve IP-wide history.
    reset_bucket(
        conn,
        scope_type="account",
        key_hash=account_key,
    )
```

```
            token, digest = new_session_token()
            expires_at = session_expiry(_ttl_minutes())

            with conn.cursor() as cur:
                cur.execute(
                    """INSERT INTO auth_sessions
                            (user_id, refresh_token_hash, expires_at)
                       VALUES (%s, %s, %s)""",
                    (user["user_id"], digest, expires_at),
                )
                now = datetime.now(UTC)
                cur.execute(
                    """UPDATE users
                       SET last_login_at = %s, updated_at = %s
                       WHERE user_id = %s""",
                    (now, now, user["user_id"]),
                )

            record_auth_event(
                conn,
                action_key="auth.login",
                outcome="success",
                actor_user_id=user["user_id"],
                details={"method": "password"},
                correlation_id=correlation_id,
            )

    except psycopg.Error as exc:
        raise HTTPException(
            status_code=503,
            detail="Authentication service unavailable",
        ) from exc

    if not login_succeeded:
        raise HTTPException(status_code=401, detail=GENERIC_LOGIN_ERROR)

    response.set_cookie(
        SESSION_COOKIE,
        token,
        httponly=True,
        secure=_secure_cookie(),
        samesite="lax",
```

```
===== LOCAL ENVIRONMENT IGNORE STATUS =====
.env
.env.local
apps/api/.env
apps/api/.env.local
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
cd "$HOME/Documents/NVDIA_Play_Book/FinSight_AI"

printf '\n===== RUNTIME REQUIREMENTS =====\n'
cat apps/api/requirements.txt

printf '\n===== LOCK FILE HEADER =====\n'
sed -n '1,100p' apps/api/requirements.lock

printf '\n===== DEVELOPMENT REQUIREMENTS =====\n'
cat apps/api/requirements-dev.txt

printf '\n===== RELEASE DEPENDENCY INSTALLATION =====\n'
grep -n -A12 -B8 -E 'requirements.lock|requirements.txt|pip install' \
  scripts/package_release.py docs/release/RELEASE_README.md


===== RUNTIME REQUIREMENTS =====
fastapi==0.133.0
starlette==1.3.1
uvicorn==0.40.0
argon2-cffi==25.1.0
psycopg[binary]==3.2.13
python-dotenv==1.2.2
email-validator==2.3.0

===== LOCK FILE HEADER =====
annotated-doc==0.0.5
annotated-types==0.8.0
anyio==4.15.1
argon2-cffi==25.1.0
argon2-cffi-bindings==26.1.0
cffi==2.1.1
click==8.5.0
dnspython==2.9.0
email-validator==2.3.0
```

```
fastapi==0.133.0
h11==0.16.0
idna==3.20
psycopg==3.2.13
psycopg-binary==3.2.13
pycparser==3.11
pydantic==2.14.1
pydantic_core==2.50.1
python-dotenv==1.2.2
starlette==1.3.1
typing-inspection==0.4.4
typing_extensions==4.16.0
uvicorn==0.40.0

===== DEVELOPMENT REQUIREMENTS =====
-r requirements.txt
pytest==9.0.3
httpx==0.28.1
pytest-cov==6.2.1

===== RELEASE DEPENDENCY INSTALLATION =====
scripts/package_release.py-38-        payload = Path(temp_dir) / "finsight"
scripts/package_release.py-39-        frontend = payload / "frontend"
scripts/package_release.py-40-        shutil.copytree(frontend_dir, frontend, symlinks=True)
scripts/package_release.py-41-        shutil.copytree(web_static, frontend / ".next/static",
symlinks=True)
scripts/package_release.py-42-
scripts/package_release.py-43-        backend = payload / "backend"
scripts/package_release.py-44-        shutil.copytree(ROOT / "apps/api/app", backend / "app",
ignore=shutil.ignore_patterns("__pycache__", "*.pyc"))
scripts/package_release.py-45-        shutil.copytree(ROOT / "apps/api/migrations", backend /
"migrations")
scripts/package_release.py:46:        shutil.copy2(ROOT / "apps/api/requirements.lock", backend
/ "requirements.lock")
scripts/package_release.py-47-
scripts/package_release.py-48-        metadata = {
scripts/package_release.py-49-            "artifact": artifact_base,
scripts/package_release.py-50-            "version": version,
scripts/package_release.py-51-            "commit_sha": commit_sha,
scripts/package_release.py-52-            "frontend": {"runtime": "Node.js 22", "entrypoint": "node
frontend/server.js"},
scripts/package_release.py-53-            "backend": {"runtime": "Python 3.12", "entrypoint":
"python -m uvicorn app.main:app"},
```

```
scripts/package_release.py-54-            "database_migrations": ["0001_auth_identity.sql",
"0002_platform_authorization.sql", "0003_auth_audit_and_login_throttling.sql",
"0004_immutable_auth_audit.sql"],
scripts/package_release.py-55-            "configuration": "Runtime values are supplied through
environment variables; see RELEASE_README.md.",
scripts/package_release.py-56-        }
scripts/package_release.py-57-        (payload /
"RELEASE-METADATA.json").write_text(json.dumps(metadata, indent=2) + "\n",
encoding="utf-8")
scripts/package_release.py-58-        release_readme = ROOT /
"docs/release/RELEASE_README.md"
--
docs/release/RELEASE_README.md-25-```
docs/release/RELEASE_README.md-26-
docs/release/RELEASE_README.md-27-For the backend, from the extracted bundle root,
create a virtual environment and install the bundled pinned runtime requirements:
docs/release/RELEASE_README.md-28-
docs/release/RELEASE_README.md-29-```sh
docs/release/RELEASE_README.md-30-cd backend
docs/release/RELEASE_README.md-31-python3.12 -m venv .venv
docs/release/RELEASE_README.md-32-. .venv/bin/activate
docs/release/RELEASE_README.md:33:python -m pip install -r requirements.lock
docs/release/RELEASE_README.md-34-python -m uvicorn app.main:app --host 0.0.0.0 --port
8000
docs/release/RELEASE_README.md-35-```
docs/release/RELEASE_README.md-36-
docs/release/RELEASE_README.md-37-Run database migrations as a separate, explicitly
approved deployment step with a migration-owner role. The bundle includes the current
versioned migration under `backend/migrations`; it does not include or apply the supplied target
schema wholesale. Use a separate least-privileged application database role at runtime.
docs/release/RELEASE_README.md-38-
docs/release/RELEASE_README.md-39-## Promotion and rollback
docs/release/RELEASE_README.md-40-
docs/release/RELEASE_README.md-41-Retain each verified archive, checksum, metadata,
migration result, smoke-test report, and deployment record. Promote the same versioned
bundle through environments; configure `API_BASE_URL` and backend settings per
environment at runtime. Record which artifact version and full commit SHA is active.
docs/release/RELEASE_README.md-42-
docs/release/RELEASE_README.md-43-To roll back application code, stop the affected
processes and point the deployment supervisor to the previous known-good versioned release
directory by its metadata version and commit SHA. Verify its checksum before reuse, start it with
the appropriate environment configuration, then run the deployment's readiness and
authentication smoke checks. Keep the database at its current migrated version unless the
```

migration has a reviewed, tested, explicitly approved down migration. Do not restore an older application against a schema it cannot safely read.
docs/release/RELEASE_README.md-44-
docs/release/RELEASE_README.md-45-Database migration policy for safe application rollback:
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
cation smoke checks. Keep the database at its current migrated version unless the migration has a reviewed, tested, explicitly approved down migration. Do not restore an older application against a schema it cannot safely read.
docs/release/RELEASE_README.md-44-
docs/release/RELEASE_README.md-45-Database migration policy for safe application rollback:
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
cd "$HOME/Documents/NVDIA_Play_Book/FinSight_AI"

```
.venv/bin/python - <<'PY'
from pathlib import Path

requirements = Path("apps/api/requirements.txt")
lock = Path("apps/api/requirements.lock")

req_text = requirements.read_text()
lock_text = lock.read_text()

if "cryptography==" in req_text or "cryptography==" in lock_text:
    raise SystemExit(
        "STOP: cryptography already appears in a dependency file. "
        "Inspect both files before making changes."
    )

if "email-validator==2.3.0\n" not in req_text:
    raise SystemExit("STOP: runtime requirements anchor not found; unchanged.")

if "cffi==2.1.1\n" not in lock_text or "click==8.5.0\n" not in lock_text:
    raise SystemExit("STOP: lock-file anchors not found; unchanged.")

req_text = req_text.replace(
    "email-validator==2.3.0\n",
    "email-validator==2.3.0\ncryptography==",
    1,
)
# Leave the edit incomplete rather than invent a version; restore and exit safely.
req_text = req_text.replace("cryptography==", "", 1)
```

```
print("Dependency files verified; no files changed.")
print("Next, resolve the installed-compatible cryptography version before pinning.")
PY

printf '\n===== PYTHON / PIP VERSION =====\n'
.venv/bin/python --version
.venv/bin/python -m pip --version

Dependency files verified; no files changed.
Next, resolve the installed-compatible cryptography version before pinning.

===== PYTHON / PIP VERSION =====
Python 3.14.6
pip 26.1.2 from
/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/pip (python 3.14)
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %(.venv) (base)
sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....

if "cryptography==" in r or "cryptography==" in l:
    raise SystemExit("STOP: cryptography already present; inspect files first.")

if not r.endswith("email-validator==2.3.0\n"):
    raise SystemExit("STOP: unexpected runtime requirements ending; unchanged.")

if "cffi==2.1.1\n" not in l or "email-validator==2.3.0\n" not in l:
    raise SystemExit("STOP: lock dependency anchors missing; unchanged.")

req.write_text(r + "cryptography==50.0.2\n")
l += "cryptography==50.0.2\n"
l = "".join(sorted(l.splitlines(keepends=True), key=str.lower))
lock.write_text(l)

print("Pinned cryptography==50.0.2 in runtime requirements and release lock.")
PY

.venv/bin/python -m pip install -r apps/api/requirements.txt

printf '\n===== CRYPTOGRAPHY VERSION =====\n'
.venv/bin/python - <<'PY'
from importlib.metadata import version
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
```

```
print("cryptography:", version("cryptography"))
print("AESGCM import: OK")
PY

printf '\n===== DEPENDENCY DIFF =====\n'
git diff -- apps/api/requirements.txt apps/api/requirements.lock
git diff --check

Pinned cryptography==50.0.2 in runtime requirements and release lock.
Requirement already satisfied: fastapi==0.133.0 in ./.venv/lib/python3.14/site-packages (from -r
apps/api/requirements.txt (line 1)) (0.133.0)
Requirement already satisfied: starlette==1.3.1 in ./.venv/lib/python3.14/site-packages (from -r
apps/api/requirements.txt (line 2)) (1.3.1)
Requirement already satisfied: uvicorn==0.40.0 in ./.venv/lib/python3.14/site-packages (from -r
apps/api/requirements.txt (line 3)) (0.40.0)
Requirement already satisfied: argon2-cffi==25.1.0 in ./.venv/lib/python3.14/site-packages (from
-r apps/api/requirements.txt (line 4)) (25.1.0)
Requirement already satisfied: psycopg==3.2.13 in ./.venv/lib/python3.14/site-packages (from
psycopg[binary]==3.2.13->-r apps/api/requirements.txt (line 5)) (3.2.13)
Requirement already satisfied: python-dotenv==1.2.2 in ./.venv/lib/python3.14/site-packages
(from -r apps/api/requirements.txt (line 6)) (1.2.2)
Requirement already satisfied: email-validator==2.3.0 in ./.venv/lib/python3.14/site-packages
(from -r apps/api/requirements.txt (line 7)) (2.3.0)
Collecting cryptography==50.0.2 (from -r apps/api/requirements.txt (line 8))
  Downloading cryptography-50.0.2-cp311-abi3-macosx_11_0_arm64.whl.metadata (4.4 kB)
Requirement already satisfied: pydantic>=2.7.0 in ./.venv/lib/python3.14/site-packages (from
fastapi==0.133.0->-r apps/api/requirements.txt (line 1)) (2.14.1)
Requirement already satisfied: typing-extensions>=4.8.0 in ./.venv/lib/python3.14/site-packages
(from fastapi==0.133.0->-r apps/api/requirements.txt (line 1)) (4.16.0)
Requirement already satisfied: typing-inspection>=0.4.2 in ./.venv/lib/python3.14/site-packages
(from fastapi==0.133.0->-r apps/api/requirements.txt (line 1)) (0.4.4)
Requirement already satisfied: annotated-doc>=0.0.2 in ./.venv/lib/python3.14/site-packages
(from fastapi==0.133.0->-r apps/api/requirements.txt (line 1)) (0.0.5)
Requirement already satisfied: anyio<5,>=3.6.2 in ./.venv/lib/python3.14/site-packages (from
starlette==1.3.1->-r apps/api/requirements.txt (line 2)) (4.15.1)
Requirement already satisfied: click>=7.0 in ./.venv/lib/python3.14/site-packages (from
uvicorn==0.40.0->-r apps/api/requirements.txt (line 3)) (8.5.0)
Requirement already satisfied: h11>=0.8 in ./.venv/lib/python3.14/site-packages (from
uvicorn==0.40.0->-r apps/api/requirements.txt (line 3)) (0.16.0)
Requirement already satisfied: argon2-cffi-bindings in ./.venv/lib/python3.14/site-packages (from
argon2-cffi==25.1.0->-r apps/api/requirements.txt (line 4)) (26.1.0)
Requirement already satisfied: dnspython>=2.0.0 in ./.venv/lib/python3.14/site-packages (from
email-validator==2.3.0->-r apps/api/requirements.txt (line 7)) (2.9.0)
```

Requirement already satisfied: idna>=2.0.0 in ./.venv/lib/python3.14/site-packages (from email-validator==2.3.0->-r apps/api/requirements.txt (line 7)) (3.20)
Requirement already satisfied: cffi>=2.0.0 in ./.venv/lib/python3.14/site-packages (from cryptography==50.0.2->-r apps/api/requirements.txt (line 8)) (2.1.1)
Requirement already satisfied: psycopg-binary==3.2.13 in ./.venv/lib/python3.14/site-packages (from psycopg[binary]==3.2.13->-r apps/api/requirements.txt (line 5)) (3.2.13)
Requirement already satisfied: pycparser in ./.venv/lib/python3.14/site-packages (from cffi>=2.0.0->cryptography==50.0.2->-r apps/api/requirements.txt (line 8)) (3.11)
Requirement already satisfied: annotated-types>=0.6.0 in ./.venv/lib/python3.14/site-packages (from pydantic>=2.7.0->fastapi==0.133.0->-r apps/api/requirements.txt (line 1)) (0.8.0)
Requirement already satisfied: pydantic-core==2.50.1 in ./.venv/lib/python3.14/site-packages (from pydantic>=2.7.0->fastapi==0.133.0->-r apps/api/requirements.txt (line 1)) (2.50.1)
Downloading cryptography-50.0.2-cp311-abi3-macosx_11_0_arm64.whl (3.9 MB)
━━
3.9/3.9 MB 3.1 MB/s 0:00:01
Installing collected packages: cryptography
Successfully installed cryptography-50.0.2

[notice] A new release of pip is available: 26.1.2 -> 26.2.1
[notice] To update, run:
/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/bin/python -m pip install --upgrade pip

===== CRYPTOGRAPHY VERSION =====
cryptography: 50.0.2
AESGCM import: OK

===== DEPENDENCY DIFF =====
diff --git a/apps/api/requirements.lock b/apps/api/requirements.lock
index d2d9bfa..e20d8ca 100644
--- a/apps/api/requirements.lock
+++ b/apps/api/requirements.lock
@@ -1,17 +1,18 @@
 annotated-doc==0.0.5
 annotated-types==0.8.0
 anyio==4.15.1
-argon2-cffi==25.1.0
 argon2-cffi-bindings==26.1.0
+argon2-cffi==25.1.0
 cffi==2.1.1
 click==8.5.0
+cryptography==50.0.2
 dnspython==2.9.0
 email-validator==2.3.0

```
 fastapi==0.133.0
 h11==0.16.0
 idna==3.20
-psycopg==3.2.13
 psycopg-binary==3.2.13
+psycopg==3.2.13
 pycparser==3.11
 pydantic==2.14.1
 pydantic_core==2.50.1
diff --git a/apps/api/requirements.txt b/apps/api/requirements.txt
index 83b2eed..ca04cc3 100644
--- a/apps/api/requirements.txt
+++ b/apps/api/requirements.txt
@@ -5,3 +5,4 @@ argon2-cffi==25.1.0
 psycopg[binary]==3.2.13
 python-dotenv==1.2.2
 email-validator==2.3.0
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
        secret_store_from_environment()


def test_test_environment_requires_explicit_fake_backend(monkeypatch):
    monkeypatch.setenv("APP_ENV", "test")
    monkeypatch.delenv("MFA_SECRET_STORE_BACKEND", raising=False)

    with pytest.raises(SecretStoreConfigurationError):
        secret_store_from_environment()

    monkeypatch.setenv("MFA_SECRET_STORE_BACKEND", "fake")
    assert isinstance(secret_store_from_environment(), FakeSecretStore)


def test_development_requires_explicit_backend_and_key(monkeypatch, tmp_path):
    monkeypatch.setenv("APP_ENV", "development")
    monkeypatch.setenv("MFA_SECRET_STORE_BACKEND", "dev-encrypted")
    monkeypatch.delenv("MFA_DEV_ENCRYPTION_KEY", raising=False)
    monkeypatch.setenv("MFA_SECRET_STORE_PATH", str(tmp_path / "private-mfa"))

    with pytest.raises(SecretStoreConfigurationError, match="requires an encryption key"):
        secret_store_from_environment()
PY

printf '\n===== COMPILE =====\n'
```

```
PYTHONPATH=apps/api .venv/bin/python -m py_compile apps/api/app/secret_store.py
apps/api/tests/test_secret_store.py

printf '\n===== FOCUSED TESTS =====\n'
PYTHONPATH=apps/api .venv/bin/python -m pytest apps/api/tests/test_secret_store.py -q

printf '\n===== DIFF CHECK =====\n'
git diff --check


===== COMPILE =====

===== FOCUSED TESTS =====
........
[100%]
8 passed in 0.11s

===== DIFF CHECK =====
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
printf '\n===== MIGRATION 0001: IDENTITY SCHEMA =====\n'
sed -n '1,240p' apps/api/migrations/0001_auth_identity.sql

printf '\n===== MIGRATION 0004: AUDIT CONVENTIONS =====\n'
sed -n '1,180p' apps/api/migrations/0004_immutable_auth_audit.sql

printf '\n===== RELEASE MIGRATION REGISTRATION =====\n'
grep -n -C 4 '000[1-9]_.*sql\|0004\|requirements.lock' scripts/package_release.py

printf '\n===== RELEASE README MIGRATIONS =====\n'
grep -n -C 3 'migration\|0004\|requirements.lock' docs/release/RELEASE_README.md

printf '\n===== ENVIRONMENT TEMPLATE =====\n'
cat .env.example

printf '\n===== DATABASE MIGRATION TESTS =====\n'
ls apps/api/tests


===== MIGRATION 0001: IDENTITY SCHEMA =====
-- FIN-13 identity/session subset. Derived from Database/schema (1).sql.
-- Apply only to a new development/test database; this is not the full target schema.
```

```
BEGIN;

CREATE TABLE roles (
   role_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
   role_key VARCHAR(50) NOT NULL UNIQUE,
   display_name VARCHAR(100) NOT NULL,
   scope VARCHAR(20) NOT NULL CHECK (scope IN ('platform','workspace')),
   description TEXT,
   created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE users (
   user_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
   email VARCHAR(320) NOT NULL,
   password_hash TEXT,
   display_name VARCHAR(160) NOT NULL,
   account_status VARCHAR(20) NOT NULL DEFAULT 'active'
      CHECK (account_status IN ('invited','active','locked','disabled','deleted')),
   platform_role_id UUID REFERENCES roles(role_id) ON DELETE SET NULL,
   email_verified_at TIMESTAMPTZ,
   mfa_required BOOLEAN NOT NULL DEFAULT false,
   last_login_at TIMESTAMPTZ,
   created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
   updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
   deleted_at TIMESTAMPTZ
);
CREATE UNIQUE INDEX uq_users_email_lower ON users (lower(email));

CREATE TABLE auth_sessions (
   session_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
   user_id UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
   refresh_token_hash TEXT UNIQUE,
   ip_address INET,
   user_agent TEXT,
   created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
   last_seen_at TIMESTAMPTZ NOT NULL DEFAULT now(),
   expires_at TIMESTAMPTZ NOT NULL,
   revoked_at TIMESTAMPTZ,
   revoke_reason VARCHAR(80)
);
CREATE INDEX ix_auth_sessions_active ON auth_sessions(user_id, expires_at) WHERE
revoked_at IS NULL;

COMMIT;
```

```
===== MIGRATION 0004: AUDIT CONVENTIONS =====
BEGIN;

-- Preserve the actor UUID and prevent hard deletion from mutating history.
ALTER TABLE audit_events
   DROP CONSTRAINT audit_events_actor_user_id_fkey;

ALTER TABLE audit_events
   ADD CONSTRAINT audit_events_actor_user_id_fkey
   FOREIGN KEY (actor_user_id)
   REFERENCES users(user_id)
   ON DELETE RESTRICT;

-- Audit events are append-only. Corrections must be represented by new events.
CREATE FUNCTION reject_audit_events_mutation()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
   RAISE EXCEPTION 'audit_events is append-only; % is not permitted', TG_OP
      USING ERRCODE = '55000';
END;
$$;

CREATE TRIGGER audit_events_append_only
BEFORE UPDATE OR DELETE ON audit_events
FOR EACH ROW
EXECUTE FUNCTION reject_audit_events_mutation();

COMMIT;

===== RELEASE MIGRATION REGISTRATION =====
42-
43-       backend = payload / "backend"
44-       shutil.copytree(ROOT / "apps/api/app", backend / "app",
ignore=shutil.ignore_patterns("__pycache__", "*.pyc"))
45-       shutil.copytree(ROOT / "apps/api/migrations", backend / "migrations")
46:       shutil.copy2(ROOT / "apps/api/requirements.lock", backend / "requirements.lock")
47-
48-       metadata = {
49-           "artifact": artifact_base,
50-           "version": version,
51-           "commit_sha": commit_sha,
```

```
52-            "frontend": {"runtime": "Node.js 22", "entrypoint": "node frontend/server.js"},
53-            "backend": {"runtime": "Python 3.12", "entrypoint": "python -m uvicorn
app.main:app"},
54:            "database_migrations": ["0001_auth_identity.sql", "0002_platform_authorization.sql",
"0003_auth_audit_and_login_throttling.sql", "0004_immutable_auth_audit.sql"],
55-            "configuration": "Runtime values are supplied through environment variables; see
RELEASE_README.md.",
56-        }
57-        (payload / "RELEASE-METADATA.json").write_text(json.dumps(metadata, indent=2) +
"\n", encoding="utf-8")
58-        release_readme = ROOT / "docs/release/RELEASE_README.md"

===== RELEASE README MIGRATIONS =====
1-# FinSight AI release bundle
2-
3:This bundle is built by `.github/workflows/ci.yml` after the frontend build, backend
dependency/migration/tests, and repository hygiene jobs pass. It is an application package, not
a deployment. The repository has no approved hosting target or production environment
configured.
4-
5-## Identify and verify a release
6-
7:CI archives are named `finsight-<version>-<12-character-commit-prefix>.tar.gz`. A local
bundle without a source commit is named by version and records a null commit SHA; do not
promote such a worktree package as a committed release. `RELEASE-METADATA.json`
records the version, full Git commit SHA when available, application entry points, and included
migration names. A sibling `.build-info.json` records the UTC packaging timestamp and archive
checksum; the `.sha256` file contains the checksum independently. Verify before extracting:
8-
9-```sh
10-sha256sum --check finsight-<version>-<12-character-commit-prefix>.tar.gz.sha256
--
30-cd backend
31-python3.12 -m venv .venv
32-. .venv/bin/activate
33:python -m pip install -r requirements.lock
34-python -m uvicorn app.main:app --host 0.0.0.0 --port 8000
35-```
36-
37:Run database migrations as a separate, explicitly approved deployment step with a
migration-owner role. The bundle includes the current versioned migration under
`backend/migrations`; it does not include or apply the supplied target schema wholesale. Use a
separate least-privileged application database role at runtime.
38-
```

Promotion and rollback

Retain each verified archive, checksum, metadata, migration result, smoke-test report, and deployment record. Promote the same versioned bundle through environments; configure `API_BASE_URL` and backend settings per environment at runtime. Record which artifact version and full commit SHA is active.

To roll back application code, stop the affected processes and point the deployment supervisor to the previous known-good versioned release directory by its metadata version andcommit SHA. Verify its checksum before reuse, start it with the appropriate environment configuration, then run the deployment's readiness and authentication smoke checks. Keep thedatabase at its current migrated version unless the migration has a reviewed, tested, explicitly approved down migration. Do not restore an older application against a schema it cannot safely read.

Database migration policy for safe application rollback:

1. Prefer additive expand migrations that preserve existing columns/records and remain compatible with both old and new application versions.
2. Deploy code that can operate with both schema shapes before any contract/removal migration.
3. Delay destructive or incompatible schema changes to a separately reviewed maintenance window after backup and restore validation; document a forward-fix or recovery strategy and owner.
4. Never run the optional trading schema in Release 1. Never automatically reverse a migration as part of application rollback.

Deployment prerequisite

Before deployment is automated, the deployment owner must choose the hosting target and staging/production topology, configure protected environments and secret references, establish TLS and database network policy, define migration ownership, backups/RTO/RPO, monitoring/readiness, and approval gates. No deployment target or credentials are configured by this repository task. No deployment has been performed.

===== ENVIRONMENT TEMPLATE =====

```
# Local development placeholders only. Replace DATABASE_URL locally; never commit it.
DATABASE_URL=postgresql://USER:PASSWORD@127.0.0.1:5432/finsight
WEB_ORIGIN=http://localhost:3000
SESSION_TTL_MINUTES=60
COOKIE_SECURE=false

# Generate a unique secret locally; do not commit the actual secret.
# Example: python -c 'import secrets; print(secrets.token_urlsafe(48))'
```

```
AUTH_THROTTLE_SECRET=replace-with-a-random-secret-of-at-least-32-bytes
AUTH_LOGIN_MAX_FAILURES=5
AUTH_LOGIN_WINDOW_SECONDS=900
AUTH_LOGIN_BLOCK_SECONDS=900

===== DATABASE MIGRATION TESTS =====
__pycache__                 test_audit_service.py        test_auth_unit.py
test_identity_constraints.py  test_secret_store.py
test_audit_immutability.py    test_auth.py                test_fin14_auth_security.py
test_provision_user.py        test_throttle.py
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....

def test_totp_rejects_expired_code_outside_allowed_window():
    secret = b"S" * 20
    old_code = totp_code(secret, at_time=30)
    assert not verify_totp(secret, old_code, at_time=120)


@pytest.mark.parametrize("bad_code", ["", "123", "1234567", "12a456", "123456", " 12345"])
def test_totp_rejects_malformed_codes(bad_code):
    assert not verify_totp(b"S" * 20, bad_code, at_time=1_700_000_000)


def test_secret_generation_has_expected_size_and_randomness():
    first = generate_totp_secret()
    second = generate_totp_secret()
    assert len(first) == 20
    assert first != second


def test_secret_generation_rejects_unsafe_lengths():
    with pytest.raises(ValueError):
        generate_totp_secret(10)
PY

printf '\n===== COMPILE =====\n'
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
git status --short

Created apps/api/migrations/0005_mfa_factors.sql
Registered migration 0005 in scripts/package_release.py
```

```
===== MIGRATION REVIEW =====
-- FIN-14 MFA factor metadata; secret material remains in the configured secret store.
BEGIN;

CREATE TABLE mfa_factors (
    mfa_factor_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(user_id) ON DELETE RESTRICT,
    factor_type VARCHAR(30) NOT NULL CHECK (factor_type IN ('totp')),
    label VARCHAR(120),
    secret_ref TEXT,
    public_key_data JSONB,
    factor_status VARCHAR(20) NOT NULL DEFAULT 'pending'
        CHECK (factor_status IN ('pending', 'active', 'revoked')),
    enrolled_at TIMESTAMPTZ,
    last_used_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT ck_mfa_factor_secret_reference
        CHECK (factor_status = 'revoked' OR secret_ref IS NOT NULL)
);

CREATE INDEX ix_mfa_factors_user_status
    ON mfa_factors(user_id, factor_status);

COMMIT;

===== RELEASE MANIFEST REVIEW =====
54:             "database_migrations": ["0001_auth_identity.sql", "0002_platform_authorization.sql",
"0003_auth_audit_and_login_throttling.sql", "0004_immutable_auth_audit.sql",
"0005_mfa_factors.sql"],

===== DIFF VALIDATION =====
 M .env.example
 M FUNCTION_TEST_TRACEABILITY.txt
 M IMPLEMENTATION_STATUS.txt
 M SPRINT_1_TEST_EXECUTION_STATUS.txt
 M apps/api/app/main.py
 M apps/api/requirements.lock
 M apps/api/requirements.txt
 M apps/api/tests/test_auth.py
 M conversation.txt
 M docs/release/RELEASE_README.md
 M scripts/package_release.py
?? apps/api/app/audit.py
?? apps/api/app/main.py.backup-20261022-080642
```

```
?? apps/api/app/secret_store.py
?? apps/api/app/throttle.py
?? apps/api/app/totp.py
?? apps/api/migrations/0003_auth_audit_and_login_throttling.sql
?? apps/api/migrations/0004_immutable_auth_audit.sql
?? apps/api/migrations/0005_mfa_factors.sql
?? apps/api/tests/test_audit_immutability.py
?? apps/api/tests/test_audit_service.py
?? apps/api/tests/test_fin14_auth_security.py
?? apps/api/tests/test_secret_store.py
?? apps/api/tests/test_throttle.py
?? apps/api/tests/test_totp.py

PYTHONPATH=apps/api .venv/bin/python -m py_compile apps/api/app/totp.py
apps/api/tests/test_totp.py

printf '\n===== TOTP TESTS =====\n'
PYTHONPATH=apps/api .venv/bin/python -m pytest apps/api/tests/test_totp.py
apps/api/tests/test_secret_store.py -q

printf '\n===== DIFF CHECK =====\n'
git diff --check


===== COMPILE =====

===== TOTP TESTS =====
.....................
[100%]
19 passed in 0.04s

===== DIFF CHECK =====
===== PATCH HYGIENE =====
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
printf '\n===== CANONICAL MFA TABLE DEFINITION =====\n'
grep -n -i -A 30 -B 5 'CREATE TABLE.*mfa_factors\|mfa_factors' \
  "Database/schema (1).sql"

printf '\n===== MFA ENUMS AND CHECK CONSTRAINTS =====\n'
grep -n -i -E 'factor_type|factor_status|secret_ref|public_key_data|enrolled_at|last_used_at' \
  "Database/schema (1).sql"

printf '\n===== EXISTING MIGRATION STYLE =====\n'
```

```
sed -n '1,220p' apps/api/migrations/0002_platform_authorization.sql
sed -n '1,220p' apps/api/migrations/0003_auth_audit_and_login_throttling.sql

printf '\n===== TEST DATABASE CONNECTION =====\n'
PYTHONPATH=apps/api
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test' .venv/bin/python
- <<'PY'
from app.database import connect

with connect() as conn, conn.cursor() as cur:
    cur.execute("SELECT current_database(), current_user")
    print("Database and user:", cur.fetchone())
PY


===== CANONICAL MFA TABLE DEFINITION =====
106-    expires_at TIMESTAMPTZ NOT NULL,
107-    consumed_at TIMESTAMPTZ,
108-    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
109-);
110-
111:CREATE TABLE mfa_factors (
112-    mfa_factor_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
113-    user_id UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
114-    factor_type VARCHAR(30) NOT NULL CHECK (factor_type IN
('totp','webauthn','recovery_code')),
115-    label VARCHAR(120),
116-    secret_ref TEXT,
117-    public_key_data JSONB,
118-    factor_status VARCHAR(20) NOT NULL DEFAULT 'pending'
119-        CHECK (factor_status IN ('pending','active','revoked')),
120-    enrolled_at TIMESTAMPTZ,
121-    last_used_at TIMESTAMPTZ,
122-    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
123-);
124-
125-CREATE TABLE auth_events (
126-    auth_event_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
127-    user_id UUID REFERENCES users(user_id) ON DELETE SET NULL,
128-    email_attempted VARCHAR(320),
129-    event_type VARCHAR(50) NOT NULL,
130-    outcome VARCHAR(20) NOT NULL CHECK (outcome IN
('success','failure','blocked','challenged')),
131-    ip_address INET,
```

```
132-    user_agent TEXT,
133-    detail JSONB NOT NULL DEFAULT '{}'::jsonb,
134-    occurred_at TIMESTAMPTZ NOT NULL DEFAULT now()
135-);
136-
137-CREATE TABLE data_export_requests (
138-    data_export_request_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
139-    user_id UUID NOT NULL REFERENCES users(user_id) ON DELETE RESTRICT,
140-    workspace_id UUID REFERENCES workspaces(workspace_id) ON DELETE SET
NULL,
141-    requested_by_user_id UUID NOT NULL REFERENCES users(user_id) ON DELETE
RESTRICT,

===== MFA ENUMS AND CHECK CONSTRAINTS =====
114:    factor_type VARCHAR(30) NOT NULL CHECK (factor_type IN
('totp','webauthn','recovery_code')),
116:    secret_ref TEXT,
117:    public_key_data JSONB,
118:    factor_status VARCHAR(20) NOT NULL DEFAULT 'pending'
119:        CHECK (factor_status IN ('pending','active','revoked')),
120:    enrolled_at TIMESTAMPTZ,
121:    last_used_at TIMESTAMPTZ,
174:    last_used_at TIMESTAMPTZ,
274:    secret_ref TEXT,
595:    secret_ref TEXT,
858:    connection_secret_ref TEXT,

===== EXISTING MIGRATION STYLE =====
-- FIN-15: only the platform-scoped administrator permission is implemented.
-- Workspace roles and workspace permissions remain unavailable until workspace
membership exists.
BEGIN;

CREATE TABLE permissions (
    permission_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    permission_key VARCHAR(120) NOT NULL UNIQUE,
    description TEXT NOT NULL
);

CREATE TABLE role_permissions (
    role_id UUID NOT NULL REFERENCES roles(role_id) ON DELETE CASCADE,
    permission_id UUID NOT NULL REFERENCES permissions(permission_id) ON DELETE
CASCADE,
    PRIMARY KEY (role_id, permission_id)
```

```
);

INSERT INTO roles (role_key, display_name, scope, description)
VALUES ('platform_admin', 'Platform Administrator', 'platform',
        'Administrator privileges for currently implemented platform operations only.')
ON CONFLICT (role_key) DO NOTHING;

DO $$
BEGIN
   IF NOT EXISTS (SELECT 1 FROM roles WHERE role_key = 'platform_admin' AND scope =
'platform') THEN
      RAISE EXCEPTION 'Existing platform_admin role has non-platform scope';
   END IF;
END
$$;

INSERT INTO permissions (permission_key, description)
VALUES ('platform.admin', 'Access operations explicitly assigned platform administrator
permission.')
ON CONFLICT (permission_key) DO NOTHING;

INSERT INTO role_permissions (role_id, permission_id)
SELECT r.role_id, p.permission_id
FROM roles r CROSS JOIN permissions p
WHERE r.role_key = 'platform_admin' AND r.scope = 'platform'
  AND p.permission_key = 'platform.admin'
ON CONFLICT DO NOTHING;

CREATE FUNCTION enforce_platform_role_scope() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
   IF NEW.platform_role_id IS NOT NULL AND NOT EXISTS (
      SELECT 1 FROM roles WHERE role_id = NEW.platform_role_id AND scope = 'platform'
   ) THEN
      RAISE EXCEPTION 'platform_role_id must reference a platform-scoped role'
         USING ERRCODE = '23514';
   END IF;
   RETURN NEW;
END
$$;

CREATE TRIGGER users_platform_role_scope_check
BEFORE INSERT OR UPDATE OF platform_role_id ON users
FOR EACH ROW EXECUTE FUNCTION enforce_platform_role_scope();
```

```
CREATE FUNCTION prevent_platform_role_scope_drift() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
   IF OLD.scope = 'platform' AND NEW.scope <> 'platform' AND EXISTS (
      SELECT 1 FROM users WHERE platform_role_id = OLD.role_id
   ) THEN
      RAISE EXCEPTION 'Cannot change scope of an assigned platform role';
   END IF;
   RETURN NEW;
END
$$;

CREATE TRIGGER roles_prevent_scope_drift
BEFORE UPDATE OF scope ON roles
FOR EACH ROW EXECUTE FUNCTION prevent_platform_role_scope_drift();

COMMIT;
BEGIN;

-- Authentication/security audit trail.
-- workspace_id remains nullable because the current application schema
-- does not yet contain the workspaces table.
CREATE TABLE audit_events (
   audit_event_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
   workspace_id UUID NULL,
   actor_user_id UUID NULL REFERENCES users(user_id) ON DELETE SET NULL,
   action_key VARCHAR(120) NOT NULL,
   resource_type VARCHAR(80) NOT NULL,
   resource_id UUID NULL,
   outcome VARCHAR(20) NOT NULL
      CHECK (outcome IN ('success', 'failure', 'denied')),
   correlation_id UUID NULL,
   details_redacted JSONB NOT NULL DEFAULT '{}'::jsonb,
   occurred_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_audit_events_occurred_at
   ON audit_events (occurred_at DESC);

CREATE INDEX idx_audit_events_actor_occurred
   ON audit_events (actor_user_id, occurred_at DESC);

CREATE INDEX idx_audit_events_action_occurred
```

```
    ON audit_events (action_key, occurred_at DESC);

-- Persistent, shared login-failure counters.
-- key_hash must be a keyed digest generated by the application;
-- raw email addresses, IP addresses, passwords and tokens do not belong here.
CREATE TABLE auth_login_throttle (
    scope_type VARCHAR(16) NOT NULL
        CHECK (scope_type IN ('account', 'ip')),
    key_hash CHAR(64) NOT NULL,
    failed_attempts INTEGER NOT NULL DEFAULT 0
        CHECK (failed_attempts >= 0),
    window_started_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    blocked_until TIMESTAMPTZ NULL,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (scope_type, key_hash)
);

CREATE INDEX idx_auth_login_throttle_blocked_until
    ON auth_login_throttle (blocked_until)
    WHERE blocked_until IS NOT NULL;

COMMIT;

===== TEST DATABASE CONNECTION =====
Database and user: ('finsight_fin15_test', 'sandeeph')
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
=== TEST DATABASE CONNECTION =====
Database and user: ('finsight_fin15_test', 'sandeeph')
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
from pathlib import Path

path = Path("apps/api/migrations/0005_mfa_factors.sql")
sql = path.read_text()

if "factor_type IN ('totp')" not in sql:
    raise SystemExit("STOP: expected original factor_type constraint not found.")
if "ON DELETE RESTRICT" not in sql:
    raise SystemExit("STOP: expected original user foreign key not found.")
if "ck_mfa_factor_secret_reference" not in sql:
    raise SystemExit("STOP: expected custom secret-reference constraint not found.")

sql = sql.replace(
    "user_id UUID NOT NULL REFERENCES users(user_id) ON DELETE RESTRICT,",
```

```
    "user_id UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,"
)
sql = sql.replace(
    "factor_type VARCHAR(30) NOT NULL CHECK (factor_type IN ('totp')),",
    "factor_type VARCHAR(30) NOT NULL CHECK "
    "(factor_type IN ('totp','webauthn','recovery_code')),"
)
sql = sql.replace(
    "    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),\n"
    "    CONSTRAINT ck_mfa_factor_secret_reference\n"
    "        CHECK (factor_status = 'revoked' OR secret_ref IS NOT NULL)\n",
    "    created_at TIMESTAMPTZ NOT NULL DEFAULT now()\n"
)

path.write_text(sql)
print("Updated migration 0005 to match canonical MFA columns and constraints.")
PY

printf '\n===== CORRECTED MIGRATION =====\n'
cat apps/api/migrations/0005_mfa_factors.sql

printf '\n===== CONFIRM CANONICAL CONSTRAINTS =====\n'
grep -n -E 'user_id
UUID|factor_type|secret_ref|public_key_data|factor_status|enrolled_at|last_used_at|created_at'
\
  apps/api/migrations/0005_mfa_factors.sql

printf '\n===== PATCH HYGIENE =====\n'
git diff --check

Updated migration 0005 to match canonical MFA columns and constraints.

===== CORRECTED MIGRATION =====
-- FIN-14 MFA factor metadata; secret material remains in the configured secret store.
BEGIN;

CREATE TABLE mfa_factors (
    mfa_factor_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    factor_type VARCHAR(30) NOT NULL CHECK (factor_type IN
('totp','webauthn','recovery_code')),
    label VARCHAR(120),
    secret_ref TEXT,
    public_key_data JSONB,
```

```
    factor_status VARCHAR(20) NOT NULL DEFAULT 'pending'
        CHECK (factor_status IN ('pending', 'active', 'revoked')),
    enrolled_at TIMESTAMPTZ,
    last_used_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX ix_mfa_factors_user_status
    ON mfa_factors(user_id, factor_status);

COMMIT;

===== CONFIRM CANONICAL CONSTRAINTS =====
6:    user_id UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
7:    factor_type VARCHAR(30) NOT NULL CHECK (factor_type IN
('totp','webauthn','recovery_code')),
9:    secret_ref TEXT,
10:    public_key_data JSONB,
11:    factor_status VARCHAR(20) NOT NULL DEFAULT 'pending'
12:        CHECK (factor_status IN ('pending', 'active', 'revoked')),
13:    enrolled_at TIMESTAMPTZ,
14:    last_used_at TIMESTAMPTZ,
15:    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
19:    ON mfa_factors(user_id, factor_status);

===== PATCH HYGIENE =====
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
except Exception:
    raise

with connect() as conn, conn.cursor() as cur:
    cur.execute("""
        SELECT column_name, data_type, is_nullable, column_default
        FROM information_schema.columns
        WHERE table_schema = 'public' AND table_name = 'mfa_factors'
        ORDER BY ordinal_position
    """)
    print("Created mfa_factors columns:")
    for row in cur.fetchall():
        print(row)

    cur.execute("""
```

```
        SELECT indexname
        FROM pg_indexes
        WHERE schemaname = 'public' AND tablename = 'mfa_factors'
        ORDER BY indexname
    """)
    print("Indexes:", [row[0] for row in cur.fetchall()])
PY

Created mfa_factors columns:
('mfa_factor_id', 'uuid', 'NO', 'gen_random_uuid()')
('user_id', 'uuid', 'NO', None)
('factor_type', 'character varying', 'NO', None)
('label', 'character varying', 'YES', None)
('secret_ref', 'text', 'YES', None)
('public_key_data', 'jsonb', 'YES', None)
('factor_status', 'character varying', 'NO', "'pending'::character varying")
('enrolled_at', 'timestamp with time zone', 'YES', None)
('last_used_at', 'timestamp with time zone', 'YES', None)
('created_at', 'timestamp with time zone', 'NO', 'now()')
Indexes: ['ix_mfa_factors_user_status', 'mfa_factors_pkey']
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
ndexes: ['ix_mfa_factors_user_status', 'mfa_factors_pkey']
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
printf '\n===== IDENTITY CONSTRAINT TEST CONVENTIONS =====\n'
sed -n '1,240p' apps/api/tests/test_identity_constraints.py

printf '\n===== AUDIT IMMUTABILITY TEST CONVENTIONS =====\n'
sed -n '1,220p' apps/api/tests/test_audit_immutability.py

printf '\n===== TEST FIXTURES AND DATABASE HELPERS =====\n'
grep -R -n -E 'def setup|def teardown|@pytest.fixture|def
connect|DATABASE_URL|soft.delete|account_status' \
  apps/api/tests/conftest.py apps/api/tests/test_identity_constraints.py \
  apps/api/tests/test_audit_immutability.py apps/api/tests/test_auth.py 2>/dev/null | head -120

printf '\n===== MFA DATABASE CONSTRAINTS =====\n'
PYTHONPATH=apps/api
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test' .venv/bin/python
- <<'PY'
from app.database import connect

with connect() as conn, conn.cursor() as cur:
    cur.execute("""
```

```
        SELECT conname, pg_get_constraintdef(oid)
        FROM pg_constraint
        WHERE conrelid = 'public.mfa_factors'::regclass
        ORDER BY conname
    """)
    for row in cur.fetchall():
        print(row)
PY


===== IDENTITY CONSTRAINT TEST CONVENTIONS =====
from __future__ import annotations

from uuid import uuid4

import psycopg
import pytest
from fastapi.testclient import TestClient

from app.auth import new_session_token, session_expiry
from app.database import connect
from app.main import SESSION_COOKIE, app, require_permission


def test_identity_schema_rejects_duplicate_email_and_cascades_sessions():
    email = f"constraint-{uuid4().hex}@example.com"
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "INSERT INTO users (email, password_hash, display_name) VALUES (%s, %s, %s)
RETURNING user_id",
            (email, "synthetic-hash", "Synthetic Constraint Test"),
        )
        user_id = cur.fetchone()[0]

    try:
        with pytest.raises(psycopg.errors.UniqueViolation):
            with connect() as conn, conn.cursor() as cur:
                cur.execute(
                    "INSERT INTO users (email, password_hash, display_name) VALUES (%s, %s,
%s)",
                    (email.upper(), "synthetic-hash", "Duplicate Synthetic Test"),
                )

        _, digest = new_session_token()
```

```
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                "INSERT INTO auth_sessions (user_id, refresh_token_hash, expires_at) VALUES
(%s, %s, %s)",
                (user_id, digest, session_expiry(5)),
            )
            cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))
            cur.execute("SELECT count(*) FROM auth_sessions WHERE user_id = %s", (user_id,))
            assert cur.fetchone()[0] == 0
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))


def test_platform_permission_is_resolved_from_database_role_and_session():
    admin_email = f"platform-admin-{uuid4().hex}@example.test"
    user_email = f"unprivileged-{uuid4().hex}@example.test"
    tokens = []
    with connect() as conn, conn.cursor() as cur:
        cur.execute("SELECT role_id FROM roles WHERE role_key = 'platform_admin' AND
scope = 'platform'")
        admin_role_id = cur.fetchone()[0]
        cur.execute(
            "INSERT INTO users (email, display_name, platform_role_id) VALUES (%s, %s, %s)
RETURNING user_id",
            (admin_email, "Synthetic Platform Admin", admin_role_id),
        )
        admin_id = cur.fetchone()[0]
        cur.execute(
            "INSERT INTO users (email, display_name) VALUES (%s, %s) RETURNING user_id",
            (user_email, "Synthetic Standard User"),
        )
        user_id = cur.fetchone()[0]
        for owner_id in (admin_id, user_id):
            token, digest = new_session_token()
            tokens.append(token)
            cur.execute(
                "INSERT INTO auth_sessions (user_id, refresh_token_hash, expires_at) VALUES
(%s, %s, %s)",
                (owner_id, digest, session_expiry(5)),
            )

    try:
        with TestClient(app) as client:
```

```
        anonymous = client.get("/api/v1/admin/authorization/capabilities")
        assert anonymous.status_code == 401
        assert anonymous.json() == {"detail": "Authentication required"}

        invalid_session = client.get(
            "/api/v1/admin/authorization/capabilities",
            cookies={SESSION_COOKIE: "invalid-session-token"},
        )
        assert invalid_session.status_code == 401

        allowed = client.get(
            "/api/v1/admin/authorization/capabilities",
            cookies={SESSION_COOKIE: tokens[0]},
        )
        #₹ The request_id header is optional, but if supplied it must be validated and preserved
as the correlation_id.
        correlated = client.get(
            "/api/v1/admin/authorization/capabilities",
            cookies={SESSION_COOKIE: tokens[0]},
            headers={"request_id": "client-correlation-123"},
        )
        assert correlated.status_code == 200
        correlated_body = correlated.json()
        assert correlated_body["request_id"].startswith("req_")
        assert correlated_body["request_id"] != "client-correlation-123"
        assert correlated_body["correlation_id"] == "client-correlation-123"

        malformed = client.get(
            "/api/v1/admin/authorization/capabilities",
            cookies={SESSION_COOKIE: tokens[0]},
            headers={"request_id": "x" * 129},
        )
        assert malformed.status_code == 400

        # The platform-admin user should be allowed to inspect the authorization policy.
        assert allowed.status_code == 200
        body = allowed.json()
        assert body["data"]["subject"]["user_id"] == str(admin_id)
        assert body["data"]["subject"]["platform_role_code"] == "platform_admin"
        assert body["data"]["effective_permissions"] == ["platform.admin"]
        assert body["data"]["capabilities"] == ["inspect_authorization_policy"]
        assert body["data"]["policy_version"] == "sprint-1-platform-rbac-v1"
        assert body["request_id"].startswith("req_")
        assert body["correlation_id"] == body["request_id"]
```

```
            denied = client.get(
                "/api/v1/admin/authorization/capabilities",
                cookies={SESSION_COOKIE: tokens[1]},
                headers={"X-Role": "platform_admin"},
            )
            assert denied.status_code == 403
            assert denied.json() == {"detail": "Permission denied"}
        finally:
            with connect() as conn, conn.cursor() as cur:
                cur.execute("DELETE FROM users WHERE user_id IN (%s, %s)", (admin_id, user_id))



def test_permission_grant_revocation_takes_effect_on_next_request():
    suffix = uuid4().hex
    role_key = f"test-admin-{suffix}"
    email = f"permission-revocation-{suffix}@example.test"
    user_id = None
    role_id = None
    permission_id = None

    try:
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                """
                INSERT INTO roles (role_key, display_name, scope, description)
                VALUES (%s, %s, 'platform', %s)
                RETURNING role_id
                """,
                (role_key, "Synthetic Permission Revocation Role", "Test-only role"),
            )
            role_id = cur.fetchone()[0]

            cur.execute(
                "SELECT permission_id FROM permissions WHERE permission_key = %s",
                ("platform.admin",),
            )
            row = cur.fetchone()
            assert row is not None, "platform.admin permission must exist"
            permission_id = row[0]

            cur.execute(
                "INSERT INTO role_permissions (role_id, permission_id) VALUES (%s, %s)",
```

```
            (role_id, permission_id),
        )
        cur.execute(
            """
            INSERT INTO users (email, display_name, platform_role_id)
            VALUES (%s, %s, %s)
            RETURNING user_id
            """,
            (email, "Synthetic Permission Revocation User", role_id),
        )
        user_id = cur.fetchone()[0]

        token, digest = new_session_token()
        cur.execute(
            """
            INSERT INTO auth_sessions (user_id, refresh_token_hash, expires_at)
            VALUES (%s, %s, %s)
            """,
            (user_id, digest, session_expiry(5)),
        )

    with TestClient(app) as client:
        first_response = client.get(
            "/api/v1/admin/authorization/capabilities",
            cookies={SESSION_COOKIE: token},
        )
        assert first_response.status_code == 200
        assert first_response.json()["data"]["effective_permissions"] == [
            "platform.admin"
        ]

        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                "DELETE FROM role_permissions WHERE role_id = %s AND permission_id =
%s",
                (role_id, permission_id),
            )

        next_response = client.get(
            "/api/v1/admin/authorization/capabilities",
            cookies={SESSION_COOKIE: token},
        )
        assert next_response.status_code == 403
        assert next_response.json() == {"detail": "Permission denied"}
```

```
    finally:
        with connect() as conn, conn.cursor() as cur:
            if user_id is not None:
                cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))
            if role_id is not None:
                cur.execute("DELETE FROM roles WHERE role_id = %s", (role_id,))

def test_platform_role_assignment_rejects_workspace_scoped_roles():
    email = f"wrong-scope-{uuid4().hex}@example.test"
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "INSERT INTO roles (role_key, display_name, scope) VALUES (%s, %s, 'workspace')
RETURNING role_id",
            (f"ws-{uuid4().hex[:12]}", "Synthetic Workspace Role"),
        )
        workspace_role_id = cur.fetchone()[0]
        cur.execute(
            "INSERT INTO users (email, display_name) VALUES (%s, 'Synthetic Scope User')
RETURNING user_id",
            (email,),
        )
        user_id = cur.fetchone()[0]
    try:
        with pytest.raises(psycopg.errors.CheckViolation):
            with connect() as conn, conn.cursor() as cur:
                cur.execute("UPDATE users SET platform_role_id = %s WHERE user_id = %s",
(workspace_role_id, user_id))
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))
            cur.execute("DELETE FROM roles WHERE role_id = %s", (workspace_role_id,))




def test_platform_role_revocation_takes_effect_on_next_request():
    email = f"revoked-platform-admin-{uuid4().hex}@example.test"
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "SELECT role_id FROM roles WHERE role_key = 'platform_admin' AND scope =
'platform'"
        )
```

```
===== AUDIT IMMUTABILITY TEST CONVENTIONS =====
from __future__ import annotations

from uuid import uuid4

import psycopg
import pytest

from app.auth import hash_password
from app.database import connect


def test_audit_events_are_immutable_and_user_history_is_preserved():
    email = f"audit-immutable-{uuid4().hex}@example.test"
    password = "Synthetic-Audit-Immutability-Test-Password"
    action_key = "test.audit_immutability"
    resource_id = str(uuid4())

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """
            INSERT INTO users (email, password_hash, display_name, account_status)
            VALUES (%s, %s, %s, 'active')
            RETURNING user_id
            """,
            (email, hash_password(password), "Audit Immutability Test"),
        )
        user_id = cur.fetchone()[0]

        cur.execute(
            """
            INSERT INTO audit_events
                (actor_user_id, action_key, resource_type, resource_id,
                 outcome, details_redacted)
            VALUES (%s, %s, %s, %s, %s, %s::jsonb)
            RETURNING audit_event_id
            """,
            (
                user_id,
                action_key,
                "test",
                resource_id,
                "success",
                '{"test_record": true}',
```

```
        ),
    )
    audit_event_id = cur.fetchone()[0]

with pytest.raises(psycopg.errors.ForeignKeyViolation):
    with connect() as conn, conn.cursor() as cur:
        cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))

with pytest.raises(psycopg.Error) as update_error:
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "UPDATE audit_events SET outcome = 'denied' WHERE audit_event_id = %s",
            (audit_event_id,),
        )
assert update_error.value.sqlstate == "55000"

with pytest.raises(psycopg.Error) as delete_error:
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "DELETE FROM audit_events WHERE audit_event_id = %s",
            (audit_event_id,),
        )
assert delete_error.value.sqlstate == "55000"

with connect() as conn, conn.cursor() as cur:
    cur.execute(
        """
        UPDATE users
        SET account_status = 'deleted',
            deleted_at = COALESCE(deleted_at, now())
        WHERE user_id = %s
        """,
        (user_id,),
    )
    cur.execute(
        """
        SELECT actor_user_id, action_key, outcome
        FROM audit_events
        WHERE audit_event_id = %s
        """,
        (audit_event_id,),
    )
    event = cur.fetchone()
```

```
    assert event == (user_id, action_key, "success")

===== TEST FIXTURES AND DATABASE HELPERS =====
apps/api/tests/test_audit_immutability.py:21:            INSERT INTO users (email,
password_hash, display_name, account_status)
apps/api/tests/test_audit_immutability.py:72:            SET account_status = 'deleted',
apps/api/tests/test_auth.py:18:@pytest.fixture
apps/api/tests/test_auth.py:24:            "INSERT INTO users (email, password_hash,
display_name, account_status) VALUES (%s, %s, %s, 'active') RETURNING user_id",
apps/api/tests/test_auth.py:39:                "UPDATE users SET account_status = 'deleted',
deleted_at = COALESCE(deleted_at, now()) "
apps/api/tests/test_auth.py:45:@pytest.fixture
apps/api/tests/test_auth.py:237:                    (email, password_hash, display_name,
account_status)
apps/api/tests/test_auth.py:293:                    SET account_status = 'deleted',

===== MFA DATABASE CONSTRAINTS =====
('mfa_factors_factor_status_check', "CHECK (((factor_status)::text = ANY
((ARRAY['pending'::character varying, 'active'::character varying, 'revoked'::character
varying])::text[])))")
('mfa_factors_factor_type_check', "CHECK (((factor_type)::text = ANY ((ARRAY['totp'::character
varying, 'webauthn'::character varying, 'recovery_code'::character varying])::text[])))")
('mfa_factors_pkey', 'PRIMARY KEY (mfa_factor_id)')
('mfa_factors_user_id_fkey', 'FOREIGN KEY (user_id) REFERENCES users(user_id) ON
DELETE CASCADE')
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
        with pytest.raises(psycopg.errors.CheckViolation):
            with connect() as conn, conn.cursor() as cur:
                cur.execute(
                    f"""
                    INSERT INTO mfa_factors (user_id, factor_type, factor_status)
                    VALUES (%s, %s, 'pending')
                    """.replace(
                        "factor_type, factor_status",
                        f"{column}, factor_status",
                    ),
                    (user_id, value),
                )
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                """
```

```
            UPDATE users
            SET account_status = 'deleted',
                deleted_at = COALESCE(deleted_at, now())
            WHERE user_id = %s
            """,
            (user_id,),
        )


def test_mfa_factor_requires_existing_user():
    with pytest.raises(psycopg.errors.ForeignKeyViolation):
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                """
                INSERT INTO mfa_factors (user_id, factor_type)
                VALUES (%s, 'totp')
                """,
                (str(uuid4()),),
            )


def test_mfa_factor_cascades_when_user_is_hard_deleted_without_audit_history():
    user_id = create_test_user()
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """
            INSERT INTO mfa_factors (user_id, factor_type, secret_ref)
            VALUES (%s, 'totp', 'fake://schema-test/opaque-reference')
            """,
            (user_id,),
        )
        cur.execute("DELETE FROM users WHERE user_id = %s", (user_id,))
        cur.execute(
            "SELECT count(*) FROM mfa_factors WHERE user_id = %s",
            (user_id,),
        )
        assert cur.fetchone()[0] == 0
PY
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % PYTHONPATH=apps/api \
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test' \
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' \
.venv/bin/python -m pytest \
  apps/api/tests/test_mfa_schema.py \
  apps/api/tests/test_totp.py \
```

```
  apps/api/tests/test_secret_store.py -q
.........F.....................
[100%]
================================================================================
============ FAILURES
================================================================================
============
_____________________________________________________________
test_mfa_factor_rejects_noncanonical_values[factor_status-enabled]
_____________________________________________________________

column = 'factor_status', value = 'enabled'

    @pytest.mark.parametrize(
        ("column", "value"),
        [
            ("factor_type", "sms"),
            ("factor_status", "enabled"),
        ],
    )
    def test_mfa_factor_rejects_noncanonical_values(column, value):
        user_id = create_test_user()
        try:
            with pytest.raises(psycopg.errors.CheckViolation):
                with connect() as conn, conn.cursor() as cur:
>                   cur.execute(
                        f"""
                        INSERT INTO mfa_factors (user_id, factor_type, factor_status)
                        VALUES (%s, %s, 'pending')
                        """.replace(
                            "factor_type, factor_status",
                            f"{column}, factor_status",
                        ),
                        (user_id, value),
                    )

apps/api/tests/test_mfa_schema.py:117:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

self = <psycopg.Cursor [closed] [BAD] at 0x1091be450>
query = "\n                        INSERT INTO mfa_factors (user_id, factor_status, factor_status)\n
VALUES (%s, %s, 'pending')\n                        "
params = ('1ae4daa0-14eb-4d1b-b49f-909bd871b233', 'enabled')
```

```
    def execute(
        self,
        query: Query,
        params: Params | None = None,
        *,
        prepare: bool | None = None,
        binary: bool | None = None,
    ) -> Self:
        """
        Execute a query or command to the database.
        """
        try:
            with self._conn.lock:
                self._conn.wait(
                    self._execute_gen(query, params, prepare=prepare, binary=binary)
                )
        except e._NO_TRACEBACK as ex:
>           raise ex.with_traceback(None)
E           psycopg.errors.DuplicateColumn: column "factor_status" specified more than once
E           LINE 2: ... INSERT INTO mfa_factors (user_id, factor_status, factor_sta...
E                                                ^

.venv/lib/python3.14/site-packages/psycopg/cursor.py:97: DuplicateColumn
=========================================================================
==== short test summary info
=========================================================================
=====
FAILED
apps/api/tests/test_mfa_schema.py::test_mfa_factor_rejects_noncanonical_values[factor_status
-enabled] - psycopg.errors.DuplicateColumn: column "factor_status" specified more than once
1 failed, 29 passed in 2.05s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % python - <<'PY'
from pathlib import Path

path = Path("apps/api/tests/test_mfa_schema.py")
source = path.read_text()

start = source.index(
    '@pytest.mark.parametrize(\n'
    '    ("column", "value"),'
)
```

```
end = source.index(
    '\n\ndef test_mfa_factor_requires_existing_user():',
    start,
)

replacement = '''@pytest.mark.parametrize(
    ("factor_type", "factor_status"),
    [
        ("sms", "pending"),
        ("totp", "enabled"),
    ],
)
def test_mfa_factor_rejects_noncanonical_values(factor_type, factor_status):
    user_id = create_test_user()
    try:
        with pytest.raises(psycopg.errors.CheckViolation):
            with connect() as conn, conn.cursor() as cur:
                cur.execute(
                    """
                    INSERT INTO mfa_factors (user_id, factor_type, factor_status)
                    VALUES (%s, %s, %s)
                    """,
                    (user_id, factor_type, factor_status),
                )
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                """
                UPDATE users
                SET account_status = 'deleted',
                    deleted_at = COALESCE(deleted_at, now())
                WHERE user_id = %s
                """,
                (user_id,),
            )
'''

path.write_text(source[:start] + replacement + source[end:])
print(f"Updated {path}")
PY
Updated apps/api/tests/test_mfa_schema.py
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % PYTHONPATH=apps/api \
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test' \
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' \
```

```
.venv/bin/python -m pytest \
  apps/api/tests/test_mfa_schema.py \
  apps/api/tests/test_totp.py \
  apps/api/tests/test_secret_store.py -q
.............................
[100%]
30 passed in 0.32s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % PYTHONPATH=apps/api \
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test' \
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' \
.venv/bin/python -m pytest \
  apps/api/tests/test_mfa_schema.py \
  apps/api/tests/test_totp.py \
  apps/api/tests/test_secret_store.py -q
.............................
[100%]
30 passed in 0.32s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % PYTHONPATH=apps/api \
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test' \
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' \
.venv/bin/python -m pytest apps/api/tests -q
.....................................................................................................
[100%]
=============================================================================
======== warnings summary
=============================================================================
========
.venv/lib/python3.14/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is
deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

.venv/lib/python3.14/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is
deprecated, use anyio.from_thread.BlockingPortal instead.
    _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

apps/api/tests/test_auth.py: 8 warnings
apps/api/tests/test_auth_unit.py: 3 warnings
```

```
apps/api/tests/test_fin14_auth_security.py: 1 warning
apps/api/tests/test_identity_constraints.py: 9 warnings

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being
deprecated, because the expected behaviour on cookie persistence is ambiguous. Set cookies
directly on the client instance instead.
    return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
123 passed, 23 warnings in 4.00s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
123 passed, 23 warnings in 4.00s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
printf '\n===== AUTHENTICATION ROUTES AND MODELS =====\n'
grep -n -E 'class LoginRequest|class .*Request|def
login|@app\.(get|post|delete).*auth|SESSION_COOKIE|mfa_required|def require_permission'
apps/api/app/main.py

printf '\n===== LOGIN AND SESSION IMPLEMENTATION =====\n'
sed -n '1,260p' apps/api/app/main.py

printf '\n===== REMAINING AUTHENTICATION ROUTES =====\n'
sed -n '260,560p' apps/api/app/main.py

printf '\n===== AUDIT SERVICE INTERFACE =====\n'
sed -n '1,240p' apps/api/app/audit.py

printf '\n===== SECRET STORE FACTORY AND ADAPTERS =====\n'
sed -n '1,280p' apps/api/app/secret_store.py

printf '\n===== TOTP IMPLEMENTATION =====\n'
sed -n '1,240p' apps/api/app/totp.py


===== AUTHENTICATION ROUTES AND MODELS =====
71:SESSION_COOKIE = "finsight_session"
76:class LoginRequest(BaseModel):
132:    session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None,
168:def require_permission(permission_key: str):
178:@app.post("/auth/login", response_model=UserView)
179:def login(payload: LoginRequest, request: Request, response: Response) -> UserView:
306:        SESSION_COOKIE,
```

```
321:@app.post("/auth/logout", status_code=204)
325:    session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None,
364:        SESSION_COOKIE,
384:@app.get("/auth/sessions", response_model=list[SessionView])
388:    session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None,
418:@app.delete("/auth/sessions/{session_id}", status_code=204)
470:@app.get("/auth/me", response_model=UserView)
475:@app.get("/api/v1/admin/authorization/capabilities")

===== LOGIN AND SESSION IMPLEMENTATION =====
"""FinSight AI API with PostgreSQL-backed browser sessions."""

from __future__ import annotations

import os
import secrets
from datetime import UTC, datetime
from pathlib import Path
from typing import Annotated, Any
from uuid import UUID, uuid4

from app.audit import record_auth_event
from app.throttle import (
    is_blocked,
    load_throttle_config,
    make_throttle_key,
    record_failure,
    reset_bucket,
)

import psycopg
from dotenv import load_dotenv
from fastapi import Cookie, Depends, FastAPI, HTTPException, Request, Response, status
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel, EmailStr, Field
from psycopg.rows import dict_row

from app.auth import hash_password, new_session_token, session_expiry, token_digest,
verify_password
from app.database import connect

load_dotenv(Path(__file__).resolve().parents[3] / ".env")

app = FastAPI(title="FinSight AI API", version="0.2.0")
```

```
@app.middleware("http")
async def auth_correlation_middleware(request: Request, call_next):
    """Return a correlation ID on authentication responses, including errors."""
    if (
        request.url.path not in {"/auth/login", "/auth/logout"}
        and not request.url.path.startswith("/auth/sessions")
    ):
        return await call_next(request)

    supplied = request.headers.get("X-Correlation-ID", "").strip()
    try:
        correlation_id = UUID(supplied) if supplied else uuid4()
    except ValueError:
        from starlette.responses import JSONResponse

        return JSONResponse(
            status_code=400,
            content={"detail": "Invalid X-Correlation-ID"},
        )

    request.state.auth_correlation_id = correlation_id
    response = await call_next(request)
    response.headers["X-Correlation-ID"] = str(correlation_id)
    return response


web_origin = os.environ.get("WEB_ORIGIN", "http://localhost:3000")
app.add_middleware(
    CORSMiddleware,
    allow_origins=[web_origin],
    allow_credentials=True,
    allow_methods=["GET", "POST", "DELETE"],
    allow_headers=["Content-Type", "X-Correlation-ID"],
)

SESSION_COOKIE = "finsight_session"
GENERIC_LOGIN_ERROR = "Email or password is incorrect."
DUMMY_PASSWORD_HASH = hash_password(secrets.token_urlsafe(32))


class LoginRequest(BaseModel):
    email: EmailStr
```

```
    password: str = Field(min_length=1, max_length=1024)


class UserView(BaseModel):
    user_id: UUID
    email: str
    display_name: str


def _ttl_minutes() -> int:
    try:
        value = int(os.environ.get("SESSION_TTL_MINUTES", "60"))
        session_expiry(value)
        return value
    except ValueError as exc:
        raise HTTPException(status_code=500, detail="Session configuration is invalid") from exc


def _secure_cookie() -> bool:
    configured = os.environ.get("COOKIE_SECURE")
    if configured is not None:
        return configured.lower() in {"1", "true", "yes"}
    return os.environ.get("APP_ENV", "development").lower() == "production"


def _check_origin(request: Request) -> None:
    origin = request.headers.get("origin")
    if origin and origin != web_origin:
        raise HTTPException(status_code=403, detail="Request origin is not allowed")


def _auth_correlation_id(request: Request) -> UUID:
    """Reuse the middleware's correlation ID, or create one if needed."""
    existing = getattr(request.state, "auth_correlation_id", None)
    if existing is not None:
        return existing

    supplied = request.headers.get("X-Correlation-ID", "").strip()
    if supplied:
        try:
            correlation_id = UUID(supplied)
        except ValueError as exc:
            raise HTTPException(
                status_code=400,
```

```
            detail="Invalid X-Correlation-ID",
        ) from exc
    else:
        correlation_id = uuid4()

    request.state.auth_correlation_id = correlation_id
    return correlation_id


def get_current_user(
    session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None,
) -> dict[str, Any]:
    if not session_token:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
detail="Authentication required")
    try:
        with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
            cur.execute(
                """SELECT u.user_id, u.email, u.display_name, r.role_key,
                        COALESCE(array_agg(DISTINCT p.permission_key)
                            FILTER (WHERE p.permission_key IS NOT NULL), ARRAY[]::varchar[])
AS permissions
                    FROM auth_sessions s JOIN users u ON u.user_id = s.user_id
                    LEFT JOIN roles r ON r.role_id = u.platform_role_id AND r.scope = 'platform'
                    LEFT JOIN role_permissions rp ON rp.role_id = r.role_id
                    LEFT JOIN permissions p ON p.permission_id = rp.permission_id
                    WHERE s.refresh_token_hash = %s AND s.revoked_at IS NULL
                      AND s.expires_at > %s AND u.account_status = 'active'
                      AND s.last_seen_at > %s - make_interval(mins => %s)
                      AND u.deleted_at IS NULL
                    GROUP BY u.user_id, u.email, u.display_name, r.role_key""",
                (token_digest(session_token), datetime.now(UTC), datetime.now(UTC),
_ttl_minutes()),
            )
            user = cur.fetchone()
            if not user:
                raise HTTPException(status_code=401, detail="Authentication required")
            cur.execute(
                "UPDATE auth_sessions SET last_seen_at = %s WHERE refresh_token_hash = %s",
                (datetime.now(UTC), token_digest(session_token)),
            )
            return user
    except psycopg.Error as exc:
```

```
        raise HTTPException(status_code=503, detail="Authentication service unavailable") from
exc


CurrentUser = Annotated[dict[str, Any], Depends(get_current_user)]


def require_permission(permission_key: str):
    """Build a deny-by-default backend permission dependency."""
    def dependency(user: CurrentUser) -> dict[str, Any]:
        if permission_key not in user.get("permissions", []):
            raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Permission
denied")
        return user

    return dependency


@app.post("/auth/login", response_model=UserView)
def login(payload: LoginRequest, request: Request, response: Response) -> UserView:
    _check_origin(request)
    correlation_id = _auth_correlation_id(request)
    response.headers['X-Correlation-ID'] = str(correlation_id)
    email = payload.email.strip().lower()
    try:
        config = load_throttle_config()
    except ValueError as exc:
        raise HTTPException(
            status_code=503,
            detail="Authentication service unavailable",
        ) from exc

    client_ip = request.client.host if request.client else "unknown"
    account_key = make_throttle_key(config.secret, "account", email)
    ip_key = make_throttle_key(config.secret, "ip", client_ip)
    login_succeeded = False
    user = None

    try:
        with connect() as conn:
            account_blocked = is_blocked(
                conn, scope_type="account", key_hash=account_key
            )
            ip_blocked = is_blocked(
```

```
    conn, scope_type="ip", key_hash=ip_key
)

if account_blocked or ip_blocked:
    record_auth_event(
        conn,
        action_key="auth.login",
        outcome="denied",
        actor_user_id=None,
        details={"reason": "throttled"},
        correlation_id=correlation_id,
    )
else:
    with conn.cursor(row_factory=dict_row) as cur:
        cur.execute(
            """SELECT user_id, email, display_name, password_hash
               FROM users
               WHERE lower(email) = %s
                 AND account_status = 'active'
                 AND deleted_at IS NULL""",
            (email,),
        )
        user = cur.fetchone()

    password_hash = (
        user["password_hash"]
        if user and user["password_hash"]
        else DUMMY_PASSWORD_HASH
    )
    password_valid = verify_password(
        password_hash, payload.password
    )
    login_succeeded = bool(
        user and user["password_hash"] and password_valid
    )

    if not login_succeeded:
        record_failure(
            conn,
            scope_type="account",
            key_hash=account_key,
            config=config,
        )
        record_failure(
```

```
                    conn,
                    scope_type="ip",
                    key_hash=ip_key,
                    config=config,
                )
                record_auth_event(
                    conn,
                    action_key="auth.login",
                    outcome="failure",
                    actor_user_id=user["user_id"] if user else None,
                    details={"reason": "invalid_credentials"},
                    correlation_id=correlation_id,
                )

===== REMAINING AUTHENTICATION ROUTES =====
                )
            else:
                # Clear account failures only. Preserve IP-wide history.
                reset_bucket(
                    conn,
                    scope_type="account",
                    key_hash=account_key,
                )

                token, digest = new_session_token()
                expires_at = session_expiry(_ttl_minutes())

                with conn.cursor() as cur:
                    cur.execute(
                        """INSERT INTO auth_sessions
                               (user_id, refresh_token_hash, expires_at)
                           VALUES (%s, %s, %s)""",
                        (user["user_id"], digest, expires_at),
                    )
                    now = datetime.now(UTC)
                    cur.execute(
                        """UPDATE users
                           SET last_login_at = %s, updated_at = %s
                           WHERE user_id = %s""",
                        (now, now, user["user_id"]),
                    )

                record_auth_event(
                    conn,
```

```
                    action_key="auth.login",
                    outcome="success",
                    actor_user_id=user["user_id"],
                    details={"method": "password"},
                    correlation_id=correlation_id,
                )

    except psycopg.Error as exc:
        raise HTTPException(
            status_code=503,
            detail="Authentication service unavailable",
        ) from exc

    if not login_succeeded:
        raise HTTPException(status_code=401, detail=GENERIC_LOGIN_ERROR)

    response.set_cookie(
        SESSION_COOKIE,
        token,
        httponly=True,
        secure=_secure_cookie(),
        samesite="lax",
        max_age=_ttl_minutes() * 60,
        path="/",
    )
    return UserView(
        user_id=user["user_id"],
        email=user["email"],
        display_name=user["display_name"],
    )


@app.post("/auth/logout", status_code=204)
def logout(
    request: Request,
    response: Response,
    session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None,
) -> Response:
    _check_origin(request)
    correlation_id = _auth_correlation_id(request)
    response.headers['X-Correlation-ID'] = str(correlation_id)

    try:
        with connect() as conn:
```

```
            actor_user_id = None

            if session_token:
                with conn.cursor() as cur:
                    cur.execute(
                        """UPDATE auth_sessions
                           SET revoked_at = %s, revoke_reason = 'logout'
                           WHERE refresh_token_hash = %s
                             AND revoked_at IS NULL
                           RETURNING user_id""",
                        (datetime.now(UTC), token_digest(session_token)),
                    )
                    row = cur.fetchone()
                    if row:
                        actor_user_id = row[0]

            record_auth_event(
                conn,
                action_key="auth.logout",
                outcome="success" if actor_user_id else "denied",
                actor_user_id=actor_user_id,
                details={"reason": "logout"},
                correlation_id=correlation_id,
            )
        except psycopg.Error as exc:
            raise HTTPException(
                status_code=503,
                detail="Authentication service unavailable",
            ) from exc

        response.delete_cookie(
            SESSION_COOKIE,
            path="/",
            httponly=True,
            secure=_secure_cookie(),
            samesite="lax",
        )
        response.status_code = status.HTTP_204_NO_CONTENT
        return response


    class SessionView(BaseModel):
        session_id: UUID
        created_at: datetime
```

```
    last_seen_at: datetime
    expires_at: datetime
    ip_address: str | None
    user_agent: str | None
    is_current: bool


@app.get("/auth/sessions", response_model=list[SessionView])
def list_active_sessions(
    request: Request,
    user: CurrentUser,
    session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None,
) -> list[SessionView]:
    correlation_id = _auth_correlation_id(request)
    token_hash = token_digest(session_token) if session_token else None
    now = datetime.now(UTC)

    try:
        with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
            cur.execute(
                """SELECT session_id, created_at, last_seen_at, expires_at,
                          host(ip_address) AS ip_address, user_agent,
                          (refresh_token_hash = %s) AS is_current
                   FROM auth_sessions
                   WHERE user_id = %s
                     AND revoked_at IS NULL
                     AND expires_at > %s
                     AND last_seen_at > %s - make_interval(mins => %s)
                   ORDER BY created_at DESC""",
                (token_hash, user["user_id"], now, now, _ttl_minutes()),
            )
            rows = cur.fetchall()
    except psycopg.Error as exc:
        raise HTTPException(
            status_code=503,
            detail="Authentication service unavailable",
        ) from exc

    return [SessionView(**row) for row in rows]


@app.delete("/auth/sessions/{session_id}", status_code=204)
def terminate_session(
    session_id: UUID,
```

```
    request: Request,
    response: Response,
    user: CurrentUser,
) -> Response:
    _check_origin(request)
    correlation_id = _auth_correlation_id(request)
    response.headers["X-Correlation-ID"] = str(correlation_id)
    now = datetime.now(UTC)
    terminated = False

    try:
        with connect() as conn:
            with conn.cursor() as cur:
                cur.execute(
                    """UPDATE auth_sessions
                       SET revoked_at = %s, revoke_reason = 'user_terminated'
                       WHERE session_id = %s
                         AND user_id = %s
                         AND revoked_at IS NULL
                         AND expires_at > %s
                       RETURNING session_id""",
                    (now, session_id, user["user_id"], now),
                )
                terminated = cur.fetchone() is not None

            record_auth_event(
                conn,
                action_key="auth.session.terminate",
                outcome="success" if terminated else "denied",
                actor_user_id=user["user_id"],
                details={
                    "session_id": str(session_id),
                    "reason": "user_terminated" if terminated else "not_found_or_inactive",
                },
                correlation_id=correlation_id,
            )
    except psycopg.Error as exc:
        raise HTTPException(
            status_code=503,
            detail="Authentication service unavailable",
        ) from exc

    if not terminated:
        raise HTTPException(status_code=404, detail="Active session not found")
```

```
    response.status_code = status.HTTP_204_NO_CONTENT
    return response


@app.get("/auth/me", response_model=UserView)
def who_am_i(user: CurrentUser) -> UserView:
    return UserView(**user)


@app.get("/api/v1/admin/authorization/capabilities")
def platform_admin_capabilities(
    request: Request,
    _authorized: Annotated[dict[str, Any], Depends(require_permission("platform.admin"))],
) -> dict[str, Any]:
    """Return the authenticated caller's database-backed platform capabilities."""

    supplied_request_id = request.headers.get("request_id", "").strip()
    if supplied_request_id and (
        len(supplied_request_id) > 128
        or any(ord(char) < 32 or ord(char) == 127 for char in supplied_request_id)
    ):
        raise HTTPException(status_code=400, detail="Invalid request_id")

    # Always issue a server-generated canonical request ID.
    request_id = f"req_{secrets.token_urlsafe(12)}"
    correlation_id = supplied_request_id or request_id
    permissions = sorted(set(_authorized.get("permissions", [])))
    capabilities = (
        ["inspect_authorization_policy"]
        if "platform.admin" in permissions
        else []
    )

    return {
        "data": {
            "subject": {
                "user_id": str(_authorized["user_id"]),
                "platform_role_code": _authorized.get("role_key"),
            },
            "effective_permissions": permissions,
            "capabilities": capabilities,
            "policy_version": "sprint-1-platform-rbac-v1",
        },
```

```
        "request_id": request_id,
        "correlation_id": correlation_id,
    }

===== AUDIT SERVICE INTERFACE =====
"""Reusable, redacted audit persistence for authentication security events."""

from __future__ import annotations

import json
import re
from datetime import UTC, datetime
from typing import Any
from uuid import UUID

_ALLOWED_OUTCOMES = frozenset({"success", "failure", "denied"})
_SECRET_KEY = re.compile(
    r"(password|passwd|token|cookie|authorization|api[_-]?key|secret|credential)",
    re.IGNORECASE,
)


def _redact(value: Any) -> Any:
    """Return JSON-safe audit details without values under secret-like keys."""
    if isinstance(value, dict):
        return {
            str(key): (
                "[REDACTED]" if _SECRET_KEY.search(str(key)) else _redact(item)
            )
            for key, item in value.items()
        }
    if isinstance(value, (list, tuple)):
        return [_redact(item) for item in value]
    if isinstance(value, UUID):
        return str(value)
    if isinstance(value, datetime):
        return value.astimezone(UTC).isoformat()
    if value is None or isinstance(value, (str, int, float, bool)):
        return value
    return str(value)


def record_auth_event(
    conn: Any,
```

```
    *,
    action_key: str,
    outcome: str,
    actor_user_id: UUID | None,
    details: dict[str, Any] | None = None,
    correlation_id: UUID | None = None,
    occurred_at: datetime | None = None,
) -> None:
    """Persist one authentication audit event on the caller's transaction.

    Persistence errors intentionally propagate to the caller. The caller must
    roll back and apply the documented fail-closed policy; events are never
    reported as recorded when this insert fails.
    """
    if outcome not in _ALLOWED_OUTCOMES:
        raise ValueError("Unsupported audit outcome")
    if not action_key or len(action_key) > 120:
        raise ValueError("action_key must contain 1 to 120 characters")

    redacted_details = _redact(details or {})
    payload = json.dumps(redacted_details, separators=(",", ":"), sort_keys=True)

    with conn.cursor() as cur:
        cur.execute(
            """
            INSERT INTO audit_events (
                workspace_id,
                actor_user_id,
                action_key,
                resource_type,
                resource_id,
                outcome,
                correlation_id,
                details_redacted,
                occurred_at
            )
            VALUES (NULL, %s, %s, 'authentication', NULL, %s, %s, %s::jsonb, %s)
            """,
            (
                actor_user_id,
                action_key,
                outcome,
                correlation_id,
                payload,
```

```
                occurred_at or datetime.now(UTC),
            ),
        )

===== SECRET STORE FACTORY AND ADAPTERS =====
"""Provider-neutral storage for MFA secrets.

The development adapter is for local development only. Production provider
integration must be selected and implemented separately.
"""

from __future__ import annotations

import base64
import os
import secrets
import stat
from pathlib import Path
from typing import Protocol
from uuid import uuid4

from cryptography.exceptions import InvalidTag
from cryptography.hazmat.primitives.ciphers.aead import AESGCM


class SecretStoreError(RuntimeError):
    """Base error for secret-store operations."""


class SecretNotFoundError(SecretStoreError):
    """The requested secret reference does not exist."""


class SecretStoreConfigurationError(SecretStoreError):
    """The configured secret store is missing or unsafe."""


class SecretStore(Protocol):
    """Interface consumed by MFA application services."""

    def put(self, secret: bytes) -> str: ...
    def get(self, ref: str) -> bytes: ...
    def delete(self, ref: str) -> None: ...
```

```
class FakeSecretStore:
    """In-memory adapter with predictable references for automated tests."""

    def __init__(self) -> None:
        self._values: dict[str, bytes] = {}
        self._next_id = 1

    def put(self, secret: bytes) -> str:
        if not isinstance(secret, bytes) or not secret:
            raise ValueError("Secret must be non-empty bytes")
        ref = f"fake://mfa/{self._next_id:06d}"
        self._next_id += 1
        self._values[ref] = bytes(secret)
        return ref

    def get(self, ref: str) -> bytes:
        try:
            return self._values[ref]
        except KeyError as exc:
            raise SecretNotFoundError("Secret reference not found") from exc

    def delete(self, ref: str) -> None:
        if ref not in self._values:
            raise SecretNotFoundError("Secret reference not found")
        del self._values[ref]


class EncryptedFileSecretStore:
    """Local-only AES-GCM store. The encryption key is never stored here."""

    _PREFIX = "dev-encrypted://"

    def __init__(self, root: Path, key: bytes) -> None:
        if len(key) != 32:
            raise SecretStoreConfigurationError(
                "Development encryption key must be exactly 32 bytes"
            )

        self._root = root.expanduser().resolve()
        self._cipher = AESGCM(key)
        self._root.mkdir(mode=0o700, parents=True, exist_ok=True)

        mode = stat.S_IMODE(self._root.stat().st_mode)
```

```
    if mode & 0o077:
        raise SecretStoreConfigurationError(
            "Secret-store directory permissions must exclude group and other access"
        )

def _path_for(self, ref: str) -> Path:
    if not ref.startswith(self._PREFIX):
        raise SecretNotFoundError("Secret reference not found")
    identifier = ref.removeprefix(self._PREFIX)
    try:
        if str(uuid4()) == identifier:
            pass
        from uuid import UUID
        UUID(identifier)
    except (ValueError, AttributeError) as exc:
        raise SecretNotFoundError("Secret reference not found") from exc
    return self._root / f"{identifier}.enc"

def put(self, secret: bytes) -> str:
    if not isinstance(secret, bytes) or not secret:
        raise ValueError("Secret must be non-empty bytes")

    identifier = str(uuid4())
    ref = f"{self._PREFIX}{identifier}"
    nonce = secrets.token_bytes(12)
    ciphertext = nonce + self._cipher.encrypt(nonce, secret, ref.encode("utf-8"))
    path = self._root / f"{identifier}.enc"

    flags = os.O_WRONLY | os.O_CREAT | os.O_EXCL
    fd = os.open(path, flags, 0o600)
    try:
        with os.fdopen(fd, "wb") as handle:
            handle.write(ciphertext)
            handle.flush()
            os.fsync(handle.fileno())
    except Exception:
        path.unlink(missing_ok=True)
        raise
    return ref

def get(self, ref: str) -> bytes:
    path = self._path_for(ref)
    try:
        data = path.read_bytes()
```

```
        except FileNotFoundError as exc:
            raise SecretNotFoundError("Secret reference not found") from exc

        if len(data) < 13:
            raise SecretStoreError("Stored secret record is invalid")

        try:
            return self._cipher.decrypt(data[:12], data[12:], ref.encode("utf-8"))
        except InvalidTag as exc:
            raise SecretStoreError("Stored secret could not be authenticated") from exc

    def delete(self, ref: str) -> None:
        path = self._path_for(ref)
        try:
            path.unlink()
        except FileNotFoundError as exc:
            raise SecretNotFoundError("Secret reference not found") from exc


def secret_store_from_environment() -> SecretStore:
    """Select an explicitly permitted adapter; never fall back silently."""

    app_env = os.environ.get("APP_ENV", "development").strip().lower()
    backend = os.environ.get("MFA_SECRET_STORE_BACKEND", "").strip().lower()

    if app_env == "production":
        raise SecretStoreConfigurationError(
            "No production MFA secret-store provider is configured"
        )

    if app_env == "test":
        if backend != "fake":
            raise SecretStoreConfigurationError(
                "Tests must explicitly select MFA_SECRET_STORE_BACKEND=fake"
            )
        return FakeSecretStore()

    if app_env != "development":
        raise SecretStoreConfigurationError("Unsupported APP_ENV for MFA secret storage")

    if backend != "dev-encrypted":
        raise SecretStoreConfigurationError(
            "Development requires MFA_SECRET_STORE_BACKEND=dev-encrypted"
        )
```

```
    encoded_key = os.environ.get("MFA_DEV_ENCRYPTION_KEY", "")
    store_path = os.environ.get("MFA_SECRET_STORE_PATH", "").strip()
    if not encoded_key or not store_path:
        raise SecretStoreConfigurationError(
            "Development secret storage requires an encryption key and storage path"
        )

    try:
        key = base64.b64decode(encoded_key, validate=True)
    except (ValueError, base64.binascii.Error) as exc:
        raise SecretStoreConfigurationError(
            "Development encryption key must be valid Base64"
        ) from exc

    if len(key) != 32:
        raise SecretStoreConfigurationError(
            "Development encryption key must decode to exactly 32 bytes"
        )

    return EncryptedFileSecretStore(Path(store_path), key)

===== TOTP IMPLEMENTATION =====
"""TOTP primitives for MFA. No logging or secret provisioning is performed here."""

from __future__ import annotations

import hashlib
import hmac
import secrets
import struct
import time


def generate_totp_secret(length: int = 20) -> bytes:
    """Generate a cryptographically random TOTP secret."""
    if length < 16 or length > 64:
        raise ValueError("TOTP secret length must be between 16 and 64 bytes")
    return secrets.token_bytes(length)


def totp_code(
    secret: bytes,
    at_time: float | None = None,
```

```
    *,
    period: int = 30,
    digits: int = 6,
) -> str:
    """Generate a TOTP code for a specific time (injectable for tests)."""
    if not isinstance(secret, bytes) or len(secret) < 16:
        raise ValueError("TOTP secret must contain at least 16 bytes")
    if period <= 0:
        raise ValueError("TOTP period must be positive")
    if digits < 6 or digits > 8:
        raise ValueError("TOTP digits must be between 6 and 8")

    timestamp = time.time() if at_time is None else at_time
    if timestamp < 0:
        raise ValueError("TOTP timestamp cannot be negative")

    counter = int(timestamp) // period
    digest = hmac.new(secret, struct.pack(">Q", counter), hashlib.sha1).digest()
    offset = digest[-1] & 0x0F
    binary_code = struct.unpack(">I", digest[offset:offset + 4])[0] & 0x7FFFFFFF
    return str(binary_code % (10 ** digits)).zfill(digits)


def verify_totp(
    secret: bytes,
    supplied_code: str,
    at_time: float | None = None,
    *,
    period: int = 30,
    digits: int = 6,
    window: int = 1,
) -> bool:
    """Verify a numeric TOTP code, allowing a bounded clock-skew window."""
    if not isinstance(supplied_code, str):
        return False
    if len(supplied_code) != digits or not supplied_code.isascii() or not supplied_code.isdigit():
        return False
    if window < 0 or window > 2:
        raise ValueError("TOTP window must be between 0 and 2")

    timestamp = time.time() if at_time is None else at_time
    if timestamp < 0 or period <= 0:
        return False
```

```
    current_counter = int(timestamp) // period
    for offset in range(-window, window + 1):
        counter = current_counter + offset
        if counter < 0:
            continue
        candidate = totp_code(secret, counter * period, period=period, digits=digits)
        if hmac.compare_digest(candidate, supplied_code):
            return True
    return False
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %

    return False
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
cat > apps/api/migrations/0006_mfa_login_challenges.sql <<'SQL'
BEGIN;

CREATE TABLE mfa_login_challenges (
    challenge_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    challenge_token_hash TEXT NOT NULL UNIQUE,
    expires_at TIMESTAMPTZ NOT NULL,
    attempt_count INTEGER NOT NULL DEFAULT 0
        CHECK (attempt_count >= 0 AND attempt_count <= 5),
    consumed_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX ix_mfa_login_challenges_user_expiry
    ON mfa_login_challenges(user_id, expires_at);

CREATE TABLE mfa_totp_replay_guard (
    mfa_factor_id UUID PRIMARY KEY
        REFERENCES mfa_factors(mfa_factor_id) ON DELETE CASCADE,
    last_accepted_step BIGINT NOT NULL
        CHECK (last_accepted_step >= 0),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

COMMIT;
SQL

printf '\n===== RELEASE MIGRATION MANIFEST =====\n'
grep -n -A18 -B4 '0005_mfa_factors.sql' scripts/package_release.py
```

```
printf '\n===== CURRENT MFA MIGRATION FILES =====\n'
ls apps/api/migrations/000*.sql


===== RELEASE MIGRATION MANIFEST =====
50-            "version": version,
51-            "commit_sha": commit_sha,
52-            "frontend": {"runtime": "Node.js 22", "entrypoint": "node frontend/server.js"},
53-            "backend": {"runtime": "Python 3.12", "entrypoint": "python -m uvicorn
app.main:app"},
54:            "database_migrations": ["0001_auth_identity.sql", "0002_platform_authorization.sql",
"0003_auth_audit_and_login_throttling.sql", "0004_immutable_auth_audit.sql",
"0005_mfa_factors.sql"],
55-            "configuration": "Runtime values are supplied through environment variables; see
RELEASE_README.md.",
56-        }
57-        (payload / "RELEASE-METADATA.json").write_text(json.dumps(metadata, indent=2) +
"\n", encoding="utf-8")
58-        release_readme = ROOT / "docs/release/RELEASE_README.md"
59-        shutil.copy2(release_readme, payload / "RELEASE_README.md")
60-
61-        artifact_path = output_dir / f"{artifact_base}.tar.gz"
62-        def normalized_member(member: tarfile.TarInfo) -> tarfile.TarInfo:
63-            member.mtime = 0
64-            member.uid = 0
65-            member.gid = 0
66-            member.uname = ""
67-            member.gname = ""
68-            return member
69-
70-        with artifact_path.open("wb") as raw:
71-            import gzip
72-

===== CURRENT MFA MIGRATION FILES =====
apps/api/migrations/0001_auth_identity.sql
apps/api/migrations/0004_immutable_auth_audit.sql
apps/api/migrations/0002_platform_authorization.sql
apps/api/migrations/0005_mfa_factors.sql
apps/api/migrations/0003_auth_audit_and_login_throttling.sql
apps/api/migrations/0006_mfa_login_challenges.sql
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
```

```
python - <<'PY'
from pathlib import Path

path = Path("scripts/package_release.py")
source = path.read_text()

old = '"0005_mfa_factors.sql"]'
new = '"0005_mfa_factors.sql", "0006_mfa_login_challenges.sql"]'

if old not in source:
    raise SystemExit("Expected migration manifest entry not found; no changes made.")

if new in source:
    print("Migration 0006 is already listed.")
else:
    path.write_text(source.replace(old, new, 1))
    print("Added 0006_mfa_login_challenges.sql to release metadata.")
PY

printf '\n===== VERIFY MIGRATION MANIFEST =====\n'
grep -n 'database_migrations' scripts/package_release.py

printf '\n===== CHECK PATCH FORMATTING =====\n'
git diff --check

Added 0006_mfa_login_challenges.sql to release metadata.

===== VERIFY MIGRATION MANIFEST =====
54:            "database_migrations": ["0001_auth_identity.sql", "0002_platform_authorization.sql",
"0003_auth_audit_and_login_throttling.sql", "0004_immutable_auth_audit.sql",
"0005_mfa_factors.sql", "0006_mfa_login_challenges.sql"],

===== CHECK PATCH FORMATTING =====
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %

grep -R -n -E 'migrations/|migration.*apply|schema_migrations|Migration' \
  scripts apps/api --include='*.py' --include='*.sql' --include='*.md' \
  | head -100

printf '\n===== MFA FACTOR MIGRATION =====\n'
sed -n '1,180p' apps/api/migrations/0005_mfa_factors.sql

printf '\n===== NEW MFA CHALLENGE MIGRATION =====\n'
sed -n '1,180p' apps/api/migrations/0006_mfa_login_challenges.sql
```

```
===== MIGRATION RUNNER REFERENCES =====

===== MFA FACTOR MIGRATION =====
-- FIN-14 MFA factor metadata; secret material remains in the configured secret store.
BEGIN;

CREATE TABLE mfa_factors (
    mfa_factor_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    factor_type VARCHAR(30) NOT NULL CHECK (factor_type IN
('totp','webauthn','recovery_code')),
    label VARCHAR(120),
    secret_ref TEXT,
    public_key_data JSONB,
    factor_status VARCHAR(20) NOT NULL DEFAULT 'pending'
        CHECK (factor_status IN ('pending', 'active', 'revoked')),
    enrolled_at TIMESTAMPTZ,
    last_used_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX ix_mfa_factors_user_status
    ON mfa_factors(user_id, factor_status);

COMMIT;

===== NEW MFA CHALLENGE MIGRATION =====
BEGIN;

CREATE TABLE mfa_login_challenges (
    challenge_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    challenge_token_hash TEXT NOT NULL UNIQUE,
    expires_at TIMESTAMPTZ NOT NULL,
    attempt_count INTEGER NOT NULL DEFAULT 0
        CHECK (attempt_count >= 0 AND attempt_count <= 5),
    consumed_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX ix_mfa_login_challenges_user_expiry
    ON mfa_login_challenges(user_id, expires_at);
```

```
CREATE TABLE mfa_totp_replay_guard (
    mfa_factor_id UUID PRIMARY KEY
        REFERENCES mfa_factors(mfa_factor_id) ON DELETE CASCADE,
    last_accepted_step BIGINT NOT NULL
        CHECK (last_accepted_step >= 0),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

COMMIT;
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
PYTHONPATH=apps/api \
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test' \
.venv/bin/python - <<'PY'
from app.database import connect

with connect() as conn, conn.cursor() as cur:
    cur.execute("SELECT current_database(), current_user")
    print("Database:", cur.fetchone())

    cur.execute("""
        SELECT to_regclass('public.mfa_factors'),
               to_regclass('public.mfa_login_challenges'),
               to_regclass('public.mfa_totp_replay_guard')
    """)
    print("MFA tables:", cur.fetchone())

    cur.execute("""
        SELECT table_name
        FROM information_schema.tables
        WHERE table_schema = 'public'
          AND table_name ILIKE '%migration%'
        ORDER BY table_name
    """)
    print("Migration tracking tables:", cur.fetchall())

    cur.execute("""
        SELECT column_name, data_type, is_nullable
        FROM information_schema.columns
        WHERE table_schema = 'public'
          AND table_name = 'users'
          AND column_name = 'mfa_required'
    """)
```

```
    print("MFA policy column:", cur.fetchall())
PY

Database: ('finsight_fin15_test', 'sandeeph')
MFA tables: ('mfa_factors', None, None)
Migration tracking tables: []
MFA policy column: [('mfa_required', 'boolean', 'NO')]
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % psql
'postgresql://sandeeph@localhost:55439/finsight_fin15_test' \
  -v ON_ERROR_STOP=1 \
  -f apps/api/migrations/0006_mfa_login_challenges.sql
BEGIN
CREATE TABLE
CREATE INDEX
CREATE TABLE
COMMIT
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % PYTHONPATH=apps/api \
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test' \
.venv/bin/python - <<'PY'
from app.database import connect

with connect() as conn, conn.cursor() as cur:
    for table in ("mfa_login_challenges", "mfa_totp_replay_guard"):
        cur.execute(
            """
            SELECT column_name, data_type, is_nullable
            FROM information_schema.columns
            WHERE table_schema = 'public' AND table_name = %s
            ORDER BY ordinal_position
            """,
            (table,),
        )
        columns = cur.fetchall()
        print(f"\n{table}:")
        for column in columns:
            print("  ", column)
        assert columns, f"Missing table: {table}"

    cur.execute("""
        SELECT conrelid::regclass::text, conname,
               pg_get_constraintdef(oid)
        FROM pg_constraint
```

```
        WHERE conrelid IN (
            'public.mfa_login_challenges'::regclass,
            'public.mfa_totp_replay_guard'::regclass
        )
        ORDER BY conrelid::regclass::text, conname
    """)
    print("\nConstraints:")
    for constraint in cur.fetchall():
        print(" ", constraint)

print("\nMigration 0006 schema verification passed.")
PY

mfa_login_challenges:
  ('challenge_id', 'uuid', 'NO')
  ('user_id', 'uuid', 'NO')
  ('challenge_token_hash', 'text', 'NO')
  ('expires_at', 'timestamp with time zone', 'NO')
  ('attempt_count', 'integer', 'NO')
  ('consumed_at', 'timestamp with time zone', 'YES')
  ('created_at', 'timestamp with time zone', 'NO')

mfa_totp_replay_guard:
  ('mfa_factor_id', 'uuid', 'NO')
  ('last_accepted_step', 'bigint', 'NO')
  ('updated_at', 'timestamp with time zone', 'NO')

Constraints:
  ('mfa_login_challenges', 'mfa_login_challenges_attempt_count_check', 'CHECK
(((attempt_count >= 0) AND (attempt_count <= 5)))')
  ('mfa_login_challenges', 'mfa_login_challenges_challenge_token_hash_key', 'UNIQUE
(challenge_token_hash)')
  ('mfa_login_challenges', 'mfa_login_challenges_pkey', 'PRIMARY KEY (challenge_id)')
  ('mfa_login_challenges', 'mfa_login_challenges_user_id_fkey', 'FOREIGN KEY (user_id)
REFERENCES users(user_id) ON DELETE CASCADE')
  ('mfa_totp_replay_guard', 'mfa_totp_replay_guard_last_accepted_step_check', 'CHECK
((last_accepted_step >= 0))')
  ('mfa_totp_replay_guard', 'mfa_totp_replay_guard_mfa_factor_id_fkey', 'FOREIGN KEY
(mfa_factor_id) REFERENCES mfa_factors(mfa_factor_id) ON DELETE CASCADE')
  ('mfa_totp_replay_guard', 'mfa_totp_replay_guard_pkey', 'PRIMARY KEY (mfa_factor_id)')

Migration 0006 schema verification passed.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
```

```
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % printf '\n===== AUTH
HELPER FUNCTIONS =====\n'
sed -n '1,260p' apps/api/app/auth.py

printf '\n===== MFA AND AUTH TEST FIXTURES =====\n'
sed -n '1,100p' apps/api/tests/test_auth.py

printf '\n===== SECRET STORE TESTS =====\n'
sed -n '1,260p' apps/api/tests/test_secret_store.py

printf '\n===== TOTP TESTS =====\n'
sed -n '1,260p' apps/api/tests/test_totp.py

printf '\n===== CURRENT MFA ENVIRONMENT SETTINGS =====\n'
grep -n -E 'MFA_|APP_ENV|SESSION_TTL|AUTH_LOGIN' .env.example

===== AUTH HELPER FUNCTIONS =====
"""Password and opaque session helpers for the authentication API."""

from __future__ import annotations

import hashlib
import secrets
from datetime import UTC, datetime, timedelta

from argon2 import PasswordHasher
from argon2.exceptions import InvalidHashError, VerificationError

PASSWORD_HASHER = PasswordHasher()


def hash_password(password: str) -> str:
    return PASSWORD_HASHER.hash(password)


def verify_password(encoded_hash: str | None, password: str) -> bool:
    if not encoded_hash:
        return False
    try:
        return PASSWORD_HASHER.verify(encoded_hash, password)
    except (InvalidHashError, VerificationError):
        return False
```

```
def new_session_token() -> tuple[str, str]:
    token = secrets.token_urlsafe(32)
    return token, hashlib.sha256(token.encode("utf-8")).hexdigest()


def session_expiry(ttl_minutes: int, now: datetime | None = None) -> datetime:
    if ttl_minutes < 1 or ttl_minutes > 10080:
        raise ValueError("SESSION_TTL_MINUTES must be between 1 and 10080")
    return (now or datetime.now(UTC)) + timedelta(minutes=ttl_minutes)


def token_digest(token: str) -> str:
    return hashlib.sha256(token.encode("utf-8")).hexdigest()

===== MFA AND AUTH TEST FIXTURES =====
from __future__ import annotations

import secrets
from uuid import uuid4

import pytest
from fastapi.testclient import TestClient
from psycopg.rows import dict_row

from app.auth import hash_password, token_digest, verify_password
from app.throttle import make_throttle_key
from app.database import connect
from app.main import SESSION_COOKIE, app

TEST_PASSWORD = secrets.token_urlsafe(32)


@pytest.fixture
def identity():
    email = f"{uuid4().hex}@example.com"
    encoded = hash_password(TEST_PASSWORD)
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "INSERT INTO users (email, password_hash, display_name, account_status) VALUES
(%s, %s, %s, 'active') RETURNING user_id",
            (email, encoded, "Synthetic Analyst"),
        )
        user_id = cur.fetchone()[0]
    try:
```

```
        yield user_id, email, encoded
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                "UPDATE auth_sessions SET revoked_at = COALESCE(revoked_at, now()), "
                "revoke_reason = COALESCE(revoke_reason, 'test_cleanup') "
                "WHERE user_id = %s",
                (user_id,),
            )
            cur.execute(
                "UPDATE users SET account_status = 'deleted', deleted_at =
COALESCE(deleted_at, now()) "
                "WHERE user_id = %s",
                (user_id,),
            )


@pytest.fixture
def client(monkeypatch):
    monkeypatch.setenv("SESSION_TTL_MINUTES", "60")
    monkeypatch.setenv("COOKIE_SECURE", "false")
    monkeypatch.setenv("WEB_ORIGIN", "http://localhost:3000")
    throttle_secret = "test-only-secret-material-at-least-32-bytes"
    monkeypatch.setenv("AUTH_THROTTLE_SECRET", throttle_secret)
    ip_key = make_throttle_key(
        throttle_secret.encode("utf-8"), "ip", "testclient"
    )

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "DELETE FROM auth_login_throttle WHERE scope_type = 'ip' AND key_hash = %s",
            (ip_key,),
        )

    try:
        with TestClient(app) as test_client:
            yield test_client
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                "DELETE FROM auth_login_throttle WHERE scope_type = 'ip' AND key_hash = %s",
                (ip_key,),
            )
```

```
def test_password_hash_is_argon2_and_never_plaintext():
    encoded = hash_password(TEST_PASSWORD)
    assert encoded.startswith("argon2id")
    assert encoded != TEST_PASSWORD
    assert verify_password(encoded, TEST_PASSWORD)
    assert not verify_password(encoded, "wrong")


def test_login_creates_protected_hashed_expiring_session(client, identity, caplog, capsys):
    user_id, email, encoded = identity
    response = client.post("/auth/login", json={"email": email, "password": TEST_PASSWORD})
    assert response.status_code == 200
    assert response.json() == {"user_id": str(user_id), "email": email, "display_name": "Synthetic
Analyst"}
    assert TEST_PASSWORD not in response.text and encoded not in response.text
    cookie = response.cookies.get(SESSION_COOKIE)
    assert cookie
    set_cookie = response.headers["set-cookie"].lower()
    assert "httponly" in set_cookie and "samesite=lax" in set_cookie and "path=/" in set_cookie
    with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
        cur.execute("SELECT refresh_token_hash, expires_at, revoked_at FROM auth_sessions
WHERE user_id = %s", (user_id,))
        stored = cur.fetchone()
    assert stored["refresh_token_hash"] == token_digest(cookie)
    assert stored["refresh_token_hash"] != cookie
    assert stored["expires_at"] is not None and stored["revoked_at"] is None
    protected = client.get("/auth/me", cookies={SESSION_COOKIE: cookie})
    assert protected.status_code == 200 and protected.json()["user_id"] == str(user_id)
    captured = capsys.readouterr()
    assert TEST_PASSWORD not in caplog.text + captured.out + captured.err

===== SECRET STORE TESTS =====
import base64
import os
import stat
from pathlib import Path

import pytest

from app.secret_store import (
    EncryptedFileSecretStore,
    FakeSecretStore,
    SecretNotFoundError,
```

```
    SecretStoreConfigurationError,
    SecretStoreError,
    secret_store_from_environment,
)


def test_fake_store_round_trip_and_delete():
    store = FakeSecretStore()
    first = store.put(b"test-secret-one")
    second = store.put(b"test-secret-two")

    assert first == "fake://mfa/000001"
    assert second == "fake://mfa/000002"
    assert store.get(first) == b"test-secret-one"
    assert store.get(second) == b"test-secret-two"

    store.delete(first)
    with pytest.raises(SecretNotFoundError):
        store.get(first)


def test_fake_store_rejects_empty_secret():
    with pytest.raises(ValueError):
        FakeSecretStore().put(b"")


def test_encrypted_file_store_round_trip_and_permissions(tmp_path):
    root = tmp_path / "private-mfa"
    store = EncryptedFileSecretStore(root, b"K" * 32)
    ref = store.put(b"totp-seed-for-test")

    assert store.get(ref) == b"totp-seed-for-test"
    record = root / (ref.rsplit("/", 1)[1] + ".enc")
    assert stat.S_IMODE(record.stat().st_mode) == 0o600
    assert b"totp-seed-for-test" not in record.read_bytes()

    store.delete(ref)
    with pytest.raises(SecretNotFoundError):
        store.get(ref)


def test_encrypted_store_rejects_wrong_key(tmp_path):
    root = tmp_path / "private-mfa"
    writer = EncryptedFileSecretStore(root, b"A" * 32)
```

```
    ref = writer.put(b"secret")
    reader = EncryptedFileSecretStore(root, b"B" * 32)

    with pytest.raises(SecretStoreError):
        reader.get(ref)


def test_encrypted_store_rejects_missing_reference(tmp_path):
    store = EncryptedFileSecretStore(tmp_path / "private-mfa", b"K" * 32)
    with pytest.raises(SecretNotFoundError):
        store.get("dev-encrypted://00000000-0000-0000-0000-000000000000")


def test_production_fails_closed(monkeypatch):
    monkeypatch.setenv("APP_ENV", "production")
    monkeypatch.setenv("MFA_SECRET_STORE_BACKEND", "dev-encrypted")
    monkeypatch.setenv("MFA_DEV_ENCRYPTION_KEY", base64.b64encode(b"K" *
32).decode())
    monkeypatch.setenv("MFA_SECRET_STORE_PATH", "/tmp/should-not-be-used")

    with pytest.raises(SecretStoreConfigurationError, match="No production"):
        secret_store_from_environment()


def test_test_environment_requires_explicit_fake_backend(monkeypatch):
    monkeypatch.setenv("APP_ENV", "test")
    monkeypatch.delenv("MFA_SECRET_STORE_BACKEND", raising=False)

    with pytest.raises(SecretStoreConfigurationError):
        secret_store_from_environment()

    monkeypatch.setenv("MFA_SECRET_STORE_BACKEND", "fake")
    assert isinstance(secret_store_from_environment(), FakeSecretStore)


def test_development_requires_explicit_backend_and_key(monkeypatch, tmp_path):
    monkeypatch.setenv("APP_ENV", "development")
    monkeypatch.setenv("MFA_SECRET_STORE_BACKEND", "dev-encrypted")
    monkeypatch.delenv("MFA_DEV_ENCRYPTION_KEY", raising=False)
    monkeypatch.setenv("MFA_SECRET_STORE_PATH", str(tmp_path / "private-mfa"))

    with pytest.raises(SecretStoreConfigurationError, match="requires an encryption key"):
        secret_store_from_environment()
```

```
===== TOTP TESTS =====
import pytest

from app.totp import generate_totp_secret, totp_code, verify_totp


def test_totp_matches_rfc_6238_sha1_six_digit_vector():
    # RFC 6238 Appendix B uses ASCII "12345678901234567890" for SHA-1.
    # The application requires 16+ byte secrets; this 20-byte vector is valid.
    secret = b"12345678901234567890"
    assert totp_code(secret, at_time=59) == "287082"


def test_totp_accepts_current_code_and_small_clock_skew():
    secret = b"S" * 20
    now = 1_700_000_010
    code = totp_code(secret, at_time=now)
    assert verify_totp(secret, code, at_time=now)
    assert verify_totp(secret, code, at_time=now + 30)


def test_totp_rejects_expired_code_outside_allowed_window():
    secret = b"S" * 20
    old_code = totp_code(secret, at_time=30)
    assert not verify_totp(secret, old_code, at_time=120)


@pytest.mark.parametrize("bad_code", ["", "123", "1234567", "12a456", "123456", " 12345"])
def test_totp_rejects_malformed_codes(bad_code):
    assert not verify_totp(b"S" * 20, bad_code, at_time=1_700_000_000)


def test_secret_generation_has_expected_size_and_randomness():
    first = generate_totp_secret()
    second = generate_totp_secret()
    assert len(first) == 20
    assert first != second


def test_secret_generation_rejects_unsafe_lengths():
    with pytest.raises(ValueError):
        generate_totp_secret(10)

===== CURRENT MFA ENVIRONMENT SETTINGS =====
```

```
4:SESSION_TTL_MINUTES=60
10:AUTH_LOGIN_MAX_FAILURES=5
11:AUTH_LOGIN_WINDOW_SECONDS=900
12:AUTH_LOGIN_BLOCK_SECONDS=900
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
    """Return the current TOTP time-step index."""
    if period <= 0:
        raise ValueError("TOTP period must be positive")

    import time

    timestamp = time.time() if at_time is None else at_time
    if timestamp < 0:
        raise ValueError("TOTP timestamp cannot be negative")

    return int(timestamp) // period


def matching_totp_step(
    *,
    secret: bytes,
    supplied_code: str,
    at_time: float | None = None,
    period: int = 30,
    digits: int = 6,
    window: int = 1,
) -> int | None:
    """Return the matching counter for replay protection, or None."""
    if not isinstance(supplied_code, str):
        return None
    if len(supplied_code) != digits or not supplied_code.isascii() or not supplied_code.isdigit():
        return None
    if period <= 0 or window < 0 or window > 2:
        raise ValueError("Invalid TOTP verification configuration")

    import time

    timestamp = time.time() if at_time is None else at_time
    if timestamp < 0:
        return None

    counter = int(timestamp) // period
    for offset in range(-window, window + 1):
```

```
        step = counter + offset
        if step < 0:
            continue
        candidate = totp_code(secret, step * period, period=period, digits=digits)
        if hmac.compare_digest(candidate, supplied_code):
            return step

    return None
PY

printf '\n===== COMPILE MFA SERVICE =====\n'
PYTHONPATH=apps/api .venv/bin/python -m py_compile apps/api/app/mfa_service.py

printf '\n===== CHECK DIFF FORMATTING =====\n'
git diff --check


===== COMPILE MFA SERVICE =====

===== CHECK DIFF FORMATTING =====
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
    store = FakeSecretStore()
    reference = store.put(b"S" * 20)
    now = 1_700_000_010
    code = totp_code(b"S" * 20, at_time=now)

    assert verify_totp_code(
        store=store,
        secret_ref=reference,
        supplied_code=code,
        at_time=now,
    )
    assert not verify_totp_code(
        store=store,
        secret_ref=reference,
        supplied_code="000000",
        at_time=now,
    )


def test_matching_totp_step_returns_counter_for_valid_code():
    secret = b"S" * 20
    now = 1_700_000_010
    code = totp_code(secret, at_time=now)
```

```
    assert matching_totp_step(
        secret=secret,
        supplied_code=code,
        at_time=now,
    ) == now // 30


def test_matching_totp_step_rejects_invalid_or_expired_code():
    secret = b"S" * 20

    assert matching_totp_step(
        secret=secret,
        supplied_code="bad-code",
        at_time=1_700_000_010,
    ) is None

    old_code = totp_code(secret, at_time=30)
    assert matching_totp_step(
        secret=secret,
        supplied_code=old_code,
        at_time=120,
    ) is None
PY

PYTHONPATH=apps/api \
.venv/bin/python -m pytest \
  apps/api/tests/test_mfa_service.py \
  apps/api/tests/test_totp.py \
  apps/api/tests/test_secret_store.py -q

........................
[100%]
24 passed in 0.05s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
printf '\n===== DATABASE CONNECTION MANAGEMENT =====\n'
sed -n '1,240p' apps/api/app/database.py

printf '\n===== LOGIN TESTS AND ERROR EXPECTATIONS =====\n'
grep -n -E 'login|mfa|401|429|503|audit_events|last_login_at' apps/api/tests/test_auth.py
apps/api/tests/test_fin14_auth_security.py
```

```
printf '\n===== MFA FACTOR AND CHALLENGE SCHEMA =====\n'
psql 'postgresql://sandeeph@localhost:55439/finsight_fin15_test' -c '\d+ mfa_factors' -c '\d+
mfa_login_challenges' -c '\d+ mfa_totp_replay_guard'

printf '\n===== CURRENT CHANGE SUMMARY =====\n'
git status --short
git diff --check


===== DATABASE CONNECTION MANAGEMENT =====
"""Small synchronous PostgreSQL boundary for the auth endpoints."""

from __future__ import annotations

import os
from pathlib import Path

import psycopg
from dotenv import load_dotenv

load_dotenv(Path(__file__).resolve().parents[3] / ".env")


def connect() -> psycopg.Connection:
    database_url = os.environ.get("DATABASE_URL")
    if not database_url:
        raise RuntimeError("DATABASE_URL is required")
    return psycopg.connect(database_url, connect_timeout=3)

===== LOGIN TESTS AND ERROR EXPECTATIONS =====
apps/api/tests/test_auth.py:58:            "DELETE FROM auth_login_throttle WHERE scope_type
= 'ip' AND key_hash = %s",
apps/api/tests/test_auth.py:68:                "DELETE FROM auth_login_throttle WHERE
scope_type = 'ip' AND key_hash = %s",
apps/api/tests/test_auth.py:81:def
test_login_creates_protected_hashed_expiring_session(client, identity, caplog, capsys):
apps/api/tests/test_auth.py:83:    response = client.post("/auth/login", json={"email": email,
"password": TEST_PASSWORD})
apps/api/tests/test_auth.py:106:    known = client.post("/auth/login", json={"email": email,
"password": "wrong-password"})
apps/api/tests/test_auth.py:107:    absent = client.post("/auth/login", json={"email":
"absent@example.com", "password": "wrong-password"})
apps/api/tests/test_auth.py:109:    assert known.status_code == 401 and known.json() ==
{"detail": "Email or password is incorrect."}
```

```
apps/api/tests/test_auth.py:115:    login = client.post("/auth/login", json={"email": email,
"password": TEST_PASSWORD})
apps/api/tests/test_auth.py:116:    cookie = login.cookies.get(SESSION_COOKIE)
apps/api/tests/test_auth.py:121:    assert replay.status_code == 401
apps/api/tests/test_auth.py:136:    login = client.post("/auth/login", json={"email": email,
"password": TEST_PASSWORD})
apps/api/tests/test_auth.py:137:    cookie = login.cookies.get(SESSION_COOKIE)
apps/api/tests/test_auth.py:141:    assert client.get("/auth/me", cookies={SESSION_COOKIE:
cookie}).status_code == 401
apps/api/tests/test_auth.py:159:    response = client.post("/auth/login", json={"email": email,
"password": TEST_PASSWORD})
apps/api/tests/test_auth.py:167:    login = client.post(
apps/api/tests/test_auth.py:168:        "/auth/login", json={"email": email, "password":
TEST_PASSWORD}
apps/api/tests/test_auth.py:170:    assert login.status_code == 200
apps/api/tests/test_auth.py:171:    cookie = login.cookies.get(SESSION_COOKIE)
apps/api/tests/test_auth.py:187:    assert response.status_code == 401
apps/api/tests/test_auth.py:192:    login = client.post(
apps/api/tests/test_auth.py:193:        "/auth/login", json={"email": email, "password":
TEST_PASSWORD}
apps/api/tests/test_auth.py:195:    assert login.status_code == 200
apps/api/tests/test_auth.py:196:    cookie = login.cookies.get(SESSION_COOKIE)
apps/api/tests/test_auth.py:225:    assert replay.status_code == 401
apps/api/tests/test_auth.py:245:        owner_login = client.post(
apps/api/tests/test_auth.py:246:            "/auth/login",
apps/api/tests/test_auth.py:249:        other_login = client.post(
apps/api/tests/test_auth.py:250:            "/auth/login",
apps/api/tests/test_auth.py:253:        assert owner_login.status_code == 200
apps/api/tests/test_auth.py:254:        assert other_login.status_code == 200
apps/api/tests/test_auth.py:256:        owner_cookie =
owner_login.cookies.get(SESSION_COOKIE)
apps/api/tests/test_auth.py:257:        other_cookie =
other_login.cookies.get(SESSION_COOKIE)
apps/api/tests/test_fin14_auth_security.py:15:def
test_successful_login_is_audited_without_secrets(monkeypatch):
apps/api/tests/test_fin14_auth_security.py:32:              "/auth/login", json={"email": email,
"password": password}
apps/api/tests/test_fin14_auth_security.py:39:                FROM audit_events
apps/api/tests/test_fin14_auth_security.py:40:                WHERE action_key = 'auth.login' AND
actor_user_id = %s
apps/api/tests/test_fin14_auth_security.py:47:        assert event[0] == "auth.login"
apps/api/tests/test_fin14_auth_security.py:67:def
test_failed_login_is_audited_and_repeated_failures_are_blocked(monkeypatch):
```

```
apps/api/tests/test_fin14_auth_security.py:84:          "DELETE FROM auth_login_throttle
WHERE scope_type = 'ip' AND key_hash = %s",
apps/api/tests/test_fin14_auth_security.py:97:              "/auth/login", json={"email": email,
"password": "wrong-password"}
apps/api/tests/test_fin14_auth_security.py:100:             "/auth/login", json={"email": email,
"password": "wrong-password"}
apps/api/tests/test_fin14_auth_security.py:103:             "/auth/login", json={"email": email,
"password": password}
apps/api/tests/test_fin14_auth_security.py:106:       assert first.status_code == 401
apps/api/tests/test_fin14_auth_security.py:107:       assert second.status_code == 401
apps/api/tests/test_fin14_auth_security.py:108:       assert blocked.status_code == 401
apps/api/tests/test_fin14_auth_security.py:114:             FROM audit_events
apps/api/tests/test_fin14_auth_security.py:115:             WHERE action_key = 'auth.login'
apps/api/tests/test_fin14_auth_security.py:134:           "DELETE FROM auth_login_throttle
WHERE scope_type = 'ip' AND key_hash = %s",
apps/api/tests/test_fin14_auth_security.py:168:         login = client.post(
apps/api/tests/test_fin14_auth_security.py:169:             "/auth/login",
apps/api/tests/test_fin14_auth_security.py:172:         assert login.status_code == 200
apps/api/tests/test_fin14_auth_security.py:173:         session_token =
login.cookies.get("finsight_session")
apps/api/tests/test_fin14_auth_security.py:185:             FROM audit_events
apps/api/tests/test_fin14_auth_security.py:233:           "/auth/login",
apps/api/tests/test_fin14_auth_security.py:244:             FROM audit_events
apps/api/tests/test_fin14_auth_security.py:245:             WHERE action_key = 'auth.login'
apps/api/tests/test_fin14_auth_security.py:278:         "/auth/login",
apps/api/tests/test_fin14_auth_security.py:287:def
test_failed_login_returns_the_audited_correlation_id(monkeypatch):
apps/api/tests/test_fin14_auth_security.py:296:         "/auth/login",
apps/api/tests/test_fin14_auth_security.py:304:   assert response.status_code == 401
apps/api/tests/test_fin14_auth_security.py:310:           FROM audit_events
apps/api/tests/test_fin14_auth_security.py:311:           WHERE action_key = 'auth.login'
apps/api/tests/test_fin14_auth_security.py:332:         "/auth/login",
apps/api/tests/test_fin14_auth_security.py:336:   assert response.status_code == 401
apps/api/tests/test_fin14_auth_security.py:344:           FROM audit_events
apps/api/tests/test_fin14_auth_security.py:345:           WHERE action_key = 'auth.login'
apps/api/tests/test_fin14_auth_security.py:356:def
test_throttle_expires_and_correct_login_succeeds_after_retry(monkeypatch):
apps/api/tests/test_fin14_auth_security.py:373:         """DELETE FROM auth_login_throttle
apps/api/tests/test_fin14_auth_security.py:387:             "/auth/login",
apps/api/tests/test_fin14_auth_security.py:391:         assert failed.status_code == 401
apps/api/tests/test_fin14_auth_security.py:394:             "/auth/login",
apps/api/tests/test_fin14_auth_security.py:398:         assert blocked.status_code == 401
apps/api/tests/test_fin14_auth_security.py:404:             "/auth/login",
apps/api/tests/test_fin14_auth_security.py:415:             FROM audit_events
```

```
apps/api/tests/test_fin14_auth_security.py:428:                "DELETE FROM auth_login_throttle
WHERE key_hash IN (%s, %s)",

===== MFA FACTOR AND CHALLENGE SCHEMA =====
                                          Table "public.mfa_factors"
    Column      |           Type           | Collation | Nullable |          Default           | Storage  |
Compression | Stats target | Description
----------------+--------------------------+-----------+----------+----------------------------+----------+-------------
+--------------+-------------
 mfa_factor_id  | uuid                     |           | not null | gen_random_uuid()          | plain    |
|              |
 user_id        | uuid                     |           | not null |                            | plain    |              |
 factor_type    | character varying(30)    |           | not null |                            | extended |
|              |
 label          | character varying(120)   |           |          |                            | extended |              |
|
 secret_ref     | text                     |           |          |                            | extended |              |
 public_key_data | jsonb                   |           |          |                            | extended |              |
|
 factor_status  | character varying(20)    |           | not null | 'pending'::character varying |
extended |              |              |
 enrolled_at    | timestamp with time zone |           |          |                            | plain    |              |
|
 last_used_at   | timestamp with time zone |           |          |                            | plain    |              |
|
 created_at     | timestamp with time zone |           | not null | now()                      | plain    |
|              |
Indexes:
    "mfa_factors_pkey" PRIMARY KEY, btree (mfa_factor_id)
    "ix_mfa_factors_user_status" btree (user_id, factor_status)
Check constraints:
    "mfa_factors_factor_status_check" CHECK (factor_status::text = ANY
(ARRAY['pending'::character varying, 'active'::character varying, 'revoked'::character
varying]::text[]))
    "mfa_factors_factor_type_check" CHECK (factor_type::text = ANY (ARRAY['totp'::character
varying, 'webauthn'::character varying, 'recovery_code'::character varying]::text[]))
Foreign-key constraints:
    "mfa_factors_user_id_fkey" FOREIGN KEY (user_id) REFERENCES users(user_id) ON
DELETE CASCADE
Referenced by:
    TABLE "mfa_totp_replay_guard" CONSTRAINT "mfa_totp_replay_guard_mfa_factor_id_fkey"
FOREIGN KEY (mfa_factor_id) REFERENCES mfa_factors(mfa_factor_id) ON DELETE
CASCADE
Access method: heap
```

```
                                        Table "public.mfa_login_challenges"
       Column        |           Type           | Collation | Nullable |      Default      | Storage  |
Compression | Stats target | Description
---------------------+--------------------------+-----------+----------+-------------------+----------+-------------+----
----------+-------------
 challenge_id        | uuid                     |           | not null | gen_random_uuid() | plain    |             |
|
 user_id             | uuid                     |           | not null |                   | plain    |             |
 challenge_token_hash | text                    |           | not null |                   | extended |             |
|
 expires_at          | timestamp with time zone |           | not null |                   | plain    |             |
|
 attempt_count       | integer                  |           | not null | 0                 | plain    |             |
|
 consumed_at         | timestamp with time zone |           |          |                   | plain    |             |
|
 created_at          | timestamp with time zone |           | not null | now()             | plain    |             |
|
Indexes:
    "mfa_login_challenges_pkey" PRIMARY KEY, btree (challenge_id)
    "ix_mfa_login_challenges_user_expiry" btree (user_id, expires_at)
    "mfa_login_challenges_challenge_token_hash_key" UNIQUE CONSTRAINT, btree
(challenge_token_hash)
Check constraints:
    "mfa_login_challenges_attempt_count_check" CHECK (attempt_count >= 0 AND
attempt_count <= 5)
Foreign-key constraints:
    "mfa_login_challenges_user_id_fkey" FOREIGN KEY (user_id) REFERENCES
users(user_id) ON DELETE CASCADE
Access method: heap

                                        Table "public.mfa_totp_replay_guard"
      Column       |           Type           | Collation | Nullable | Default | Storage | Compression |
Stats target | Description
-------------------+--------------------------+-----------+----------+---------+---------+-------------+-------------+-
------------
 mfa_factor_id     | uuid                     |           | not null |         | plain   |             |             |
 last_accepted_step | bigint                  |           | not null |         | plain   |             |             |
 updated_at        | timestamp with time zone |           | not null | now()   | plain   |             |
|
Indexes:
    "mfa_totp_replay_guard_pkey" PRIMARY KEY, btree (mfa_factor_id)
Check constraints:
```

```
    "mfa_totp_replay_guard_last_accepted_step_check" CHECK (last_accepted_step >= 0)
Foreign-key constraints:
    "mfa_totp_replay_guard_mfa_factor_id_fkey" FOREIGN KEY (mfa_factor_id) REFERENCES
mfa_factors(mfa_factor_id) ON DELETE CASCADE
Access method: heap


===== CURRENT CHANGE SUMMARY =====
 M .env.example
 M FUNCTION_TEST_TRACEABILITY.txt
 M IMPLEMENTATION_STATUS.txt
 M SPRINT_1_TEST_EXECUTION_STATUS.txt
 M apps/api/app/main.py
 M apps/api/requirements.lock
 M apps/api/requirements.txt
 M apps/api/tests/test_auth.py
 M conversation.txt
 M docs/release/RELEASE_README.md
 M scripts/package_release.py
?? apps/api/app/audit.py
?? apps/api/app/main.py.backup-20261022-080642
?? apps/api/app/mfa_service.py
?? apps/api/app/secret_store.py
?? apps/api/app/throttle.py
?? apps/api/app/totp.py
?? apps/api/migrations/0003_auth_audit_and_login_throttling.sql
?? apps/api/migrations/0004_immutable_auth_audit.sql
?? apps/api/migrations/0005_mfa_factors.sql
?? apps/api/migrations/0006_mfa_login_challenges.sql
?? apps/api/tests/test_audit_immutability.py
?? apps/api/tests/test_audit_service.py
?? apps/api/tests/test_fin14_auth_security.py
?? apps/api/tests/test_mfa_schema.py
?? apps/api/tests/test_mfa_service.py
?? apps/api/tests/test_secret_store.py
?? apps/api/tests/test_throttle.py
?? apps/api/tests/test_totp.py
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % sed -n '1,320p'
apps/api/app/mfa_repository.py
"""Database-backed TOTP factor operations.

The database stores only opaque secret references. Callers must not log
provisioning URIs, TOTP seeds, challenge tokens, or submitted codes.
```

```
"""

from __future__ import annotations

from datetime import UTC, datetime
from typing import Any
from uuid import UUID

import psycopg
from psycopg.rows import dict_row

from app.database import connect
from app.mfa_service import matching_totp_step
from app.secret_store import SecretStore


class MFAFactorError(RuntimeError):
    """Base class for safe MFA factor operation failures."""


class MFAFactorNotFoundError(MFAFactorError):
    """The requested factor is unavailable."""


class MFAInvalidCodeError(MFAFactorError):
    """The submitted TOTP code is invalid or already used."""


def create_pending_totp_factor(
    *,
    user_id: UUID,
    account_name: str,
    store: SecretStore,
    label: str | None = None,
) -> tuple[UUID, str]:
    """Create a pending TOTP factor; return its ID and one-time URI."""
    from app.mfa_service import create_totp_provisioning

    secret_ref, provisioning_uri = create_totp_provisioning(
        account_name=account_name,
        store=store,
    )

    try:
```

```
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                """
                INSERT INTO mfa_factors
                    (user_id, factor_type, label, secret_ref, factor_status)
                VALUES (%s, 'totp', %s, %s, 'pending')
                RETURNING mfa_factor_id
                """,
                (user_id, label, secret_ref),
            )
            factor_id = cur.fetchone()[0]
    except Exception:
        try:
            store.delete(secret_ref)
        except Exception:
            pass
        raise

    return factor_id, provisioning_uri


def activate_totp_factor(
    *,
    user_id: UUID,
    factor_id: UUID,
    supplied_code: str,
    store: SecretStore,
    at_time: float | None = None,
) -> None:
    """Activate a pending factor only when its TOTP code is valid."""
    with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
        cur.execute(
            """
            SELECT mfa_factor_id, secret_ref
            FROM mfa_factors
            WHERE mfa_factor_id = %s
              AND user_id = %s
              AND factor_type = 'totp'
              AND factor_status = 'pending'
            FOR UPDATE
            """,
            (factor_id, user_id),
        )
        factor = cur.fetchone()
```

```
if not factor or not factor["secret_ref"]:
    raise MFAFactorNotFoundError("Pending MFA factor not found")

secret = store.get(factor["secret_ref"])
step = matching_totp_step(
    secret=secret,
    supplied_code=supplied_code,
    at_time=at_time,
)
if step is None:
    raise MFAInvalidCodeError("Invalid verification code")

now = datetime.now(UTC)
cur.execute(
    """
    INSERT INTO mfa_totp_replay_guard
        (mfa_factor_id, last_accepted_step, updated_at)
    VALUES (%s, %s, %s)
    ON CONFLICT (mfa_factor_id) DO UPDATE
    SET last_accepted_step = EXCLUDED.last_accepted_step,
        updated_at = EXCLUDED.updated_at
    WHERE mfa_totp_replay_guard.last_accepted_step
          < EXCLUDED.last_accepted_step
    RETURNING mfa_factor_id
    """,
    (factor_id, step, now),
)
if cur.fetchone() is None:
    raise MFAInvalidCodeError("Invalid verification code")

cur.execute(
    """
    UPDATE mfa_factors
    SET factor_status = 'active',
        enrolled_at = %s,
        last_used_at = %s
    WHERE mfa_factor_id = %s
      AND user_id = %s
      AND factor_status = 'pending'
    RETURNING mfa_factor_id
    """,
    (now, now, factor_id, user_id),
)
```

```
        if cur.fetchone() is None:
            raise MFAFactorNotFoundError("Pending MFA factor not found")


def verify_active_totp_factor(
    *,
    user_id: UUID,
    factor_id: UUID,
    supplied_code: str,
    store: SecretStore,
    at_time: float | None = None,
) -> bool:
    """Verify an active factor and atomically prevent time-step replay."""
    with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
        cur.execute(
            """
            SELECT mfa_factor_id, secret_ref
            FROM mfa_factors
            WHERE mfa_factor_id = %s
              AND user_id = %s
              AND factor_type = 'totp'
              AND factor_status = 'active'
            """,
            (factor_id, user_id),
        )
        factor = cur.fetchone()

        if not factor or not factor["secret_ref"]:
            raise MFAFactorNotFoundError("Active MFA factor not found")

        secret = store.get(factor["secret_ref"])
        step = matching_totp_step(
            secret=secret,
            supplied_code=supplied_code,
            at_time=at_time,
        )
        if step is None:
            return False

        now = datetime.now(UTC)
        cur.execute(
            """
            INSERT INTO mfa_totp_replay_guard
                (mfa_factor_id, last_accepted_step, updated_at)
```

```
            VALUES (%s, %s, %s)
            ON CONFLICT (mfa_factor_id) DO UPDATE
            SET last_accepted_step = EXCLUDED.last_accepted_step,
                updated_at = EXCLUDED.updated_at
            WHERE mfa_totp_replay_guard.last_accepted_step
                < EXCLUDED.last_accepted_step
            RETURNING mfa_factor_id
            """,
            (factor_id, step, now),
        )
        if cur.fetchone() is None:
            return False

        cur.execute(
            """
            UPDATE mfa_factors
            SET last_used_at = %s
            WHERE mfa_factor_id = %s
              AND user_id = %s
              AND factor_status = 'active'
            """,
            (now, factor_id, user_id),
        )
        return cur.rowcount == 1


def revoke_totp_factor(*, user_id: UUID, factor_id: UUID) -> bool:
    """Revoke an owned TOTP factor and remove its replay-guard state."""
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """
            UPDATE mfa_factors
            SET factor_status = 'revoked'
            WHERE mfa_factor_id = %s
              AND user_id = %s
              AND factor_type = 'totp'
              AND factor_status IN ('pending', 'active')
            RETURNING mfa_factor_id
            """,
            (factor_id, user_id),
        )
        row = cur.fetchone()
        if row is None:
            return False
```

```
        cur.execute(
            "DELETE FROM mfa_totp_replay_guard WHERE mfa_factor_id = %s",
            (factor_id,),
        )
        return True


def list_totp_factors(*, user_id: UUID) -> list[dict[str, Any]]:
    """List factor metadata only; never return secret references."""
    with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
        cur.execute(
            """
            SELECT mfa_factor_id, label, factor_status, enrolled_at,
                   last_used_at, created_at
            FROM mfa_factors
            WHERE user_id = %s AND factor_type = 'totp'
            ORDER BY created_at DESC
            """,
            (user_id,),
        )
        return list(cur.fetchall())
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
        store=store,
    )
    secret_ref = next(iter(store._secrets))
    code = totp_code(store.get(secret_ref), at_time=now)

    activate_totp_factor(
        user_id=mfa_identity,
        factor_id=factor_id,
        supplied_code=code,
        store=store,
        at_time=now,
    )

    assert revoke_totp_factor(user_id=mfa_identity, factor_id=factor_id) is True

    with pytest.raises(MFAFactorNotFoundError):
        verify_active_totp_factor(
            user_id=mfa_identity,
            factor_id=factor_id,
```

```
            supplied_code=code,
            store=store,
            at_time=now + 30,
        )
PY

PYTHONPATH=apps/api
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test'
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' .venv/bin/python -m
pytest apps/api/tests/test_mfa_repository.py -q

EEEEE
[100%]
==============================================================================
============= ERRORS
==============================================================================
=============
_____________________________________________________________ ERROR at setup of
test_enroll_activate_list_verify_and_revoke
_____________________________________________________________

    @pytest.fixture
    def mfa_identity():
        email = "mfa-repository-test@example.com"
        with connect() as conn, conn.cursor() as cur:
            cur.execute("DELETE FROM users WHERE email = %s", (email,))
>           cur.execute(
                """
                INSERT INTO users (email, display_name, password_hash, status)
                VALUES (%s, %s, %s, 'active')
                RETURNING user_id
                """,
                (email, "MFA Repository Test", hash_password("Mfa-Test-Password-2026!")),
            )

apps/api/tests/test_mfa_repository.py:26:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

self = <psycopg.Cursor [closed] [BAD] at 0x119191cd0>
query = "\n            INSERT INTO users (email, display_name, password_hash, status)\n
VALUES (%s, %s, %s, 'active')\n            RETURNING user_id\n            "
```

params = ('mfa-repository-test@example.com', 'MFA Repository Test', '$argon2id$v=19$m=65536,t=3,p=4$RloQjO+YEYDdzIWgiaPdSw$zDkzx+qMhbHX4SLzZowoBu0e+0gLviyGRA3VC4zmbdQ')

```
    def execute(
        self,
        query: Query,
        params: Params | None = None,
        *,
        prepare: bool | None = None,
        binary: bool | None = None,
    ) -> Self:
        """
        Execute a query or command to the database.
        """
        try:
            with self._conn.lock:
                self._conn.wait(
                    self._execute_gen(query, params, prepare=prepare, binary=binary)
                )
        except e._NO_TRACEBACK as ex:
>           raise ex.with_traceback(None)
E           psycopg.errors.UndefinedColumn: column "status" of relation "users" does not exist
E           LINE 2: ...SERT INTO users (email, display_name, password_hash, status)
E                                                            ^

.venv/lib/python3.14/site-packages/psycopg/cursor.py:97: UndefinedColumn
_____ ERROR at setup of test_active_factor_rejects_reused_totp_step _____

    @pytest.fixture
    def mfa_identity():
        email = "mfa-repository-test@example.com"
        with connect() as conn, conn.cursor() as cur:
            cur.execute("DELETE FROM users WHERE email = %s", (email,))
>           cur.execute(
                """
                INSERT INTO users (email, display_name, password_hash, status)
                VALUES (%s, %s, %s, 'active')
                RETURNING user_id
                """,
                (email, "MFA Repository Test", hash_password("Mfa-Test-Password-2026!")),
            )
```

apps/api/tests/test_mfa_repository.py:26:
_ _ _ _

```
>           cur.execute(
              """
              INSERT INTO users (email, display_name, password_hash, status)
              VALUES (%s, %s, %s, 'active')
              RETURNING user_id
              """,
              (email, "MFA Repository Test", hash_password("Mfa-Test-Password-2026!")),
            )

apps/api/tests/test_mfa_repository.py:26:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

self = <psycopg.Cursor [closed] [BAD] at 0x119192810>
query = "\n            INSERT INTO users (email, display_name, password_hash, status)\n
VALUES (%s, %s, %s, 'active')\n            RETURNING user_id\n            "
params = ('mfa-repository-test@example.com', 'MFA Repository Test', '$argon2id$v=19
$m = 65536, t = 3, p = 4$
nA/BrOeDYQT9DgBZl7cFVQ$m1GJMITfBl75T3IL5a/A6fCfjYUewcoCCWNMfPHPG50')

    def execute(
        self,
        query: Query,
        params: Params | None = None,
        *,
        prepare: bool | None = None,
        binary: bool | None = None,
    ) -> Self:
        """
        Execute a query or command to the database.
        """
        try:
            with self._conn.lock:
                self._conn.wait(
                    self._execute_gen(query, params, prepare=prepare, binary=binary)
                )
        except e._NO_TRACEBACK as ex:
>           raise ex.with_traceback(None)
E           psycopg.errors.UndefinedColumn: column "status" of relation "users" does not exist
E           LINE 2: ...SERT INTO users (email, display_name, password_hash, status)
E                                                              ^

.venv/lib/python3.14/site-packages/psycopg/cursor.py:97: UndefinedColumn
```

```
_____ ERROR at setup of test_factor_cannot_be_accessed_by_another_user _____

    @pytest.fixture
    def mfa_identity():
        email = "mfa-repository-test@example.com"
        with connect() as conn, conn.cursor() as cur:
            cur.execute("DELETE FROM users WHERE email = %s", (email,))
>           cur.execute(
                """
                INSERT INTO users (email, display_name, password_hash, status)
                VALUES (%s, %s, %s, 'active')
                RETURNING user_id
                """,
                (email, "MFA Repository Test", hash_password("Mfa-Test-Password-2026!")),
            )

apps/api/tests/test_mfa_repository.py:26:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

self = <psycopg.Cursor [closed] [BAD] at 0x119192690>
query = "\n            INSERT INTO users (email, display_name, password_hash, status)\n
VALUES (%s, %s, %s, 'active')\n            RETURNING user_id\n            "
params = ('mfa-repository-test@example.com', 'MFA Repository Test', '$argon2id$v=19
$m = 65536, t = 3, p = 4
NnxaFL2ViVy0D21hAV5yHA$eVy7hl3lHjwkyqvBiLA2g6mW+/Rmt+4hfAaw8IUnIyg')

    def execute(
        self,
        query: Query,
        params: Params | None = None,
        *,
        prepare: bool | None = None,
        binary: bool | None = None,
    ) -> Self:
        """
        Execute a query or command to the database.
        """
        try:
            with self._conn.lock:
                self._conn.wait(
                    self._execute_gen(query, params, prepare=prepare, binary=binary)
```

```
            )
        except e._NO_TRACEBACK as ex:
>           raise ex.with_traceback(None)
E           psycopg.errors.UndefinedColumn: column "status" of relation "users" does not exist
E           LINE 2: ...SERT INTO users (email, display_name, password_hash, status)
E                                                           ^

.venv/lib/python3.14/site-packages/psycopg/cursor.py:97: UndefinedColumn
_____ ERROR at setup of test_revoked_factor_cannot_be_verified _____

    @pytest.fixture
    def mfa_identity():
        email = "mfa-repository-test@example.com"
        with connect() as conn, conn.cursor() as cur:
            cur.execute("DELETE FROM users WHERE email = %s", (email,))
>           cur.execute(
                """
                INSERT INTO users (email, display_name, password_hash, status)
                VALUES (%s, %s, %s, 'active')
                RETURNING user_id
                """,
                (email, "MFA Repository Test", hash_password("Mfa-Test-Password-2026!")),
            )

apps/api/tests/test_mfa_repository.py:26:
_ _ _ _
```

```
    ) -> Self:
        """
        Execute a query or command to the database.
        """
        try:
            with self._conn.lock:
                self._conn.wait(
                    self._execute_gen(query, params, prepare=prepare, binary=binary)
                )
        except e._NO_TRACEBACK as ex:
>           raise ex.with_traceback(None)
E           psycopg.errors.UndefinedColumn: column "status" of relation "users" does not exist
E           LINE 2: ...SERT INTO users (email, display_name, password_hash, status)
E                                                            ^

.venv/lib/python3.14/site-packages/psycopg/cursor.py:97: UndefinedColumn
=========================================================================
==== short test summary info
=========================================================================
=====
ERROR apps/api/tests/test_mfa_repository.py::test_enroll_activate_list_verify_and_revoke -
psycopg.errors.UndefinedColumn: column "status" of relation "users" does not exist
ERROR apps/api/tests/test_mfa_repository.py::test_active_factor_rejects_reused_totp_step -
psycopg.errors.UndefinedColumn: column "status" of relation "users" does not exist
ERROR
apps/api/tests/test_mfa_repository.py::test_invalid_activation_code_does_not_activate_factor -
psycopg.errors.UndefinedColumn: column "status" of relation "users" does notexist
ERROR
apps/api/tests/test_mfa_repository.py::test_factor_cannot_be_accessed_by_another_user -
psycopg.errors.UndefinedColumn: column "status" of relation "users" does not exist
ERROR apps/api/tests/test_mfa_repository.py::test_revoked_factor_cannot_be_verified -
psycopg.errors.UndefinedColumn: column "status" of relation "users" does not exist
5 errors in 0.35s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....

path = Path("apps/api/tests/test_mfa_repository.py")
text = path.read_text()

text = text.replace(
    "INSERT INTO users (email, display_name, password_hash, status)\n"
    "            VALUES (%s, %s, %s, 'active')",
    "INSERT INTO users (email, display_name, password_hash, account_status)\n"
```

```
    "            VALUES (%s, %s, %s, 'active')",
)

text = text.replace(
    '    with connect() as conn, conn.cursor() as cur:\n'
    '        cur.execute("DELETE FROM users WHERE email = %s", (email,))\n',
    '    with connect() as conn, conn.cursor() as cur:\n',
)

text = text.replace(
    '    with connect() as conn, conn.cursor() as cur:\n'
    '        cur.execute(\n'
    '            "DELETE FROM users WHERE user_id = %s",\n'
    '            (user_id,),\n'
    '        )',
    '    with connect() as conn, conn.cursor() as cur:\n'
    '        cur.execute(\n'
    '            "UPDATE auth_sessions SET revoked_at = COALESCE(revoked_at, now()), "\n'
    '            "revoke_reason = COALESCE(revoke_reason, \'test_cleanup\') "\n'
    '            "WHERE user_id = %s",\n'
    '            (user_id,),\n'
    '        )\n'
    '        cur.execute(\n'
    '            "UPDATE users SET account_status = \'deleted\', "\n'
    '            "deleted_at = COALESCE(deleted_at, now()) WHERE user_id = %s",\n'
    '            (user_id,),\n'
    '        )',
)

path.write_text(text)
print("Updated test fixture to use account_status and safe user cleanup.")
PY

grep -n -A24 'def mfa_identity' apps/api/tests/test_mfa_repository.py
git diff --check

Updated test fixture to use account_status and safe user cleanup.
22:def mfa_identity():
23-    email = "mfa-repository-test@example.com"
24-    with connect() as conn, conn.cursor() as cur:
25-        cur.execute(
26-            """
27-            INSERT INTO users (email, display_name, password_hash, account_status)
28-            VALUES (%s, %s, %s, 'active')
```

```
29-            RETURNING user_id
30-            """,
31-            (email, "MFA Repository Test", hash_password("Mfa-Test-Password-2026!")),
32-        )
33-        user_id = cur.fetchone()[0]
34-
35-    yield user_id
36-
37-    with connect() as conn, conn.cursor() as cur:
38-        cur.execute(
39-            "UPDATE auth_sessions SET revoked_at = COALESCE(revoked_at, now()), "
40-            "revoke_reason = COALESCE(revoke_reason, 'test_cleanup') "
41-            "WHERE user_id = %s",
42-            (user_id,),
43-        )
44-        cur.execute(
45-            "UPDATE users SET account_status = 'deleted', "
46-            "deleted_at = COALESCE(deleted_at, now()) WHERE user_id = %s",
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
PYTHONPATH=apps/api
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test'
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' .venv/bin/python -m
pytest apps/api/tests/test_mfa_repository.py -q

FEEEE
[100%]
==========================================================================
============ ERRORS
==========================================================================
=============
_____ ERROR at setup of
test_active_factor_rejects_reused_totp_step
_____

    @pytest.fixture
    def mfa_identity():
        email = "mfa-repository-test@example.com"
        with connect() as conn, conn.cursor() as cur:
>           cur.execute(
                """
                INSERT INTO users (email, display_name, password_hash, account_status)
                VALUES (%s, %s, %s, 'active')
                RETURNING user_id
                """,
```

```
            (email, "MFA Repository Test", hash_password("Mfa-Test-Password-2026!")),
        )

apps/api/tests/test_mfa_repository.py:25:
_ _ _ _
```

```
        with connect() as conn, conn.cursor() as cur:
>           cur.execute(
              """
              INSERT INTO users (email, display_name, password_hash, account_status)
              VALUES (%s, %s, %s, 'active')
              RETURNING user_id
              """,
              (email, "MFA Repository Test", hash_password("Mfa-Test-Password-2026!")),
            )

apps/api/tests/test_mfa_repository.py:25:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

self = <psycopg.Cursor [closed] [BAD] at 0x1136925d0>
query = "\n            INSERT INTO users (email, display_name, password_hash,
account_status)\n            VALUES (%s, %s, %s, 'active')\n            RETURNING user_id\n    "
params = ('mfa-repository-test@example.com', 'MFA Repository Test', '$argon2id$v=19
$m = 65536, t = 3, p = 4
teJ3DFBj3UcQjTGHjIX2mg$xVk6Bs9UlnJXiTkZiSNDirXXEiJJ0nblInOd2pVW0Es')

    def execute(
        self,
        query: Query,
        params: Params | None = None,
        *,
        prepare: bool | None = None,
        binary: bool | None = None,
    ) -> Self:
        """
        Execute a query or command to the database.
        """
        try:
            with self._conn.lock:
                self._conn.wait(
                    self._execute_gen(query, params, prepare=prepare, binary=binary)
                )
        except e._NO_TRACEBACK as ex:
>           raise ex.with_traceback(None)
E           psycopg.errors.UniqueViolation: duplicate key value violates unique constraint
"uq_users_email_lower"
E           DETAIL:  Key (lower(email::text))=(mfa-repository-test@example.com) already exists.

.venv/lib/python3.14/site-packages/psycopg/cursor.py:97: UniqueViolation
```

_____ ERROR at setup of test_factor_cannot_be_accessed_by_another_user _____

```
    @pytest.fixture
    def mfa_identity():
        email = "mfa-repository-test@example.com"
        with connect() as conn, conn.cursor() as cur:
>           cur.execute(
                """
                INSERT INTO users (email, display_name, password_hash, account_status)
                VALUES (%s, %s, %s, 'active')
                RETURNING user_id
                """,
                (email, "MFA Repository Test", hash_password("Mfa-Test-Password-2026!")),
            )

apps/api/tests/test_mfa_repository.py:25:
_ _ _ _
```

```
        except e._NO_TRACEBACK as ex:
>           raise ex.with_traceback(None)
E           psycopg.errors.UniqueViolation: duplicate key value violates unique constraint
"uq_users_email_lower"
E           DETAIL:  Key (lower(email::text))=(mfa-repository-test@example.com) already exists.

.venv/lib/python3.14/site-packages/psycopg/cursor.py:97: UniqueViolation
_____ ERROR at setup of
test_revoked_factor_cannot_be_verified
_____

    @pytest.fixture
    def mfa_identity():
        email = "mfa-repository-test@example.com"
        with connect() as conn, conn.cursor() as cur:
>           cur.execute(
                """
                INSERT INTO users (email, display_name, password_hash, account_status)
                VALUES (%s, %s, %s, 'active')
                RETURNING user_id
                """,
                (email, "MFA Repository Test", hash_password("Mfa-Test-Password-2026!")),
            )

apps/api/tests/test_mfa_repository.py:25:
_ _ _ _
```

```
        Execute a query or command to the database.
        """
        try:
            with self._conn.lock:
                self._conn.wait(
                    self._execute_gen(query, params, prepare=prepare, binary=binary)
                )
        except e._NO_TRACEBACK as ex:
>           raise ex.with_traceback(None)
E           psycopg.errors.UniqueViolation: duplicate key value violates unique constraint
"uq_users_email_lower"
E           DETAIL:  Key (lower(email::text))=(mfa-repository-test@example.com) already exists.

.venv/lib/python3.14/site-packages/psycopg/cursor.py:97: UniqueViolation
=============================================================================
============ FAILURES
=============================================================================
============
_____________________________________________________________________________
test_enroll_activate_list_verify_and_revoke
_____________________________________________________________________________

mfa_identity = UUID('8dcdf8bd-87e0-410f-8fa9-c9f981b965a7')

    def test_enroll_activate_list_verify_and_revoke(mfa_identity):
        store = FakeSecretStore()
        now = 1_800_000_000
        factor_id, uri = create_pending_totp_factor(
            user_id=mfa_identity,
            account_name="mfa-repository-test@example.com",
            label="Test authenticator",
            store=store,
        )

        assert isinstance(factor_id, UUID)
        assert uri.startswith("otpauth://totp/")
        assert "secret=" in uri

        listed = list_totp_factors(user_id=mfa_identity)
        assert len(listed) == 1
        assert listed[0]["factor_status"] == "pending"
        assert "secret_ref" not in listed[0]

>       secret_ref = store._secrets.popitem()[0]
```

```
                ^^^^^^^^^^^^^
E       AttributeError: 'FakeSecretStore' object has no attribute '_secrets'

apps/api/tests/test_mfa_repository.py:70: AttributeError
=========================================================================
==== short test summary info
=========================================================================
=====
FAILED apps/api/tests/test_mfa_repository.py::test_enroll_activate_list_verify_and_revoke -
AttributeError: 'FakeSecretStore' object has no attribute '_secrets'
ERROR apps/api/tests/test_mfa_repository.py::test_active_factor_rejects_reused_totp_step -
psycopg.errors.UniqueViolation: duplicate key value violates unique constraint
"uq_users_email_lower"
ERROR
apps/api/tests/test_mfa_repository.py::test_invalid_activation_code_does_not_activate_factor -
psycopg.errors.UniqueViolation: duplicate key value violates unique constraint
"uq_users_email_lower"
ERROR
apps/api/tests/test_mfa_repository.py::test_factor_cannot_be_accessed_by_another_user -
psycopg.errors.UniqueViolation: duplicate key value violates unique constraint
"uq_users_email_lower"
ERROR apps/api/tests/test_mfa_repository.py::test_revoked_factor_cannot_be_verified -
psycopg.errors.UniqueViolation: duplicate key value violates unique constraint
"uq_users_email_lower"
1 failed, 4 errors in 0.40s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
printf '\n===== FAKE SECRET STORE IMPLEMENTATION =====\n'
sed -n '1,240p' apps/api/app/secret_store.py

printf '\n===== TOTP SERVICE IMPLEMENTATION =====\n'
sed -n '1,240p' apps/api/app/totp.py

printf '\n===== CURRENT REPOSITORY TESTS =====\n'
sed -n '1,240p' apps/api/tests/test_mfa_repository.py


===== FAKE SECRET STORE IMPLEMENTATION =====
"""Provider-neutral storage for MFA secrets.

The development adapter is for local development only. Production provider
integration must be selected and implemented separately.
"""
```

```
from __future__ import annotations

import base64
import os
import secrets
import stat
from pathlib import Path
from typing import Protocol
from uuid import uuid4

from cryptography.exceptions import InvalidTag
from cryptography.hazmat.primitives.ciphers.aead import AESGCM


class SecretStoreError(RuntimeError):
    """Base error for secret-store operations."""


class SecretNotFoundError(SecretStoreError):
    """The requested secret reference does not exist."""


class SecretStoreConfigurationError(SecretStoreError):
    """The configured secret store is missing or unsafe."""


class SecretStore(Protocol):
    """Interface consumed by MFA application services."""

    def put(self, secret: bytes) -> str: ...
    def get(self, ref: str) -> bytes: ...
    def delete(self, ref: str) -> None: ...


class FakeSecretStore:
    """In-memory adapter with predictable references for automated tests."""

    def __init__(self) -> None:
        self._values: dict[str, bytes] = {}
        self._next_id = 1

    def put(self, secret: bytes) -> str:
        if not isinstance(secret, bytes) or not secret:
```

```
            raise ValueError("Secret must be non-empty bytes")
        ref = f"fake://mfa/{self._next_id:06d}"
        self._next_id += 1
        self._values[ref] = bytes(secret)
        return ref

    def get(self, ref: str) -> bytes:
        try:
            return self._values[ref]
        except KeyError as exc:
            raise SecretNotFoundError("Secret reference not found") from exc

    def delete(self, ref: str) -> None:
        if ref not in self._values:
            raise SecretNotFoundError("Secret reference not found")
        del self._values[ref]


class EncryptedFileSecretStore:
    """Local-only AES-GCM store. The encryption key is never stored here."""

    _PREFIX = "dev-encrypted://"

    def __init__(self, root: Path, key: bytes) -> None:
        if len(key) != 32:
            raise SecretStoreConfigurationError(
                "Development encryption key must be exactly 32 bytes"
            )

        self._root = root.expanduser().resolve()
        self._cipher = AESGCM(key)
        self._root.mkdir(mode=0o700, parents=True, exist_ok=True)

        mode = stat.S_IMODE(self._root.stat().st_mode)
        if mode & 0o077:
            raise SecretStoreConfigurationError(
                "Secret-store directory permissions must exclude group and other access"
            )

    def _path_for(self, ref: str) -> Path:
        if not ref.startswith(self._PREFIX):
            raise SecretNotFoundError("Secret reference not found")
        identifier = ref.removeprefix(self._PREFIX)
        try:
```

```
        if str(uuid4()) == identifier:
            pass
        from uuid import UUID
        UUID(identifier)
    except (ValueError, AttributeError) as exc:
        raise SecretNotFoundError("Secret reference not found") from exc
    return self._root / f"{identifier}.enc"

def put(self, secret: bytes) -> str:
    if not isinstance(secret, bytes) or not secret:
        raise ValueError("Secret must be non-empty bytes")

    identifier = str(uuid4())
    ref = f"{self._PREFIX}{identifier}"
    nonce = secrets.token_bytes(12)
    ciphertext = nonce + self._cipher.encrypt(nonce, secret, ref.encode("utf-8"))
    path = self._root / f"{identifier}.enc"

    flags = os.O_WRONLY | os.O_CREAT | os.O_EXCL
    fd = os.open(path, flags, 0o600)
    try:
        with os.fdopen(fd, "wb") as handle:
            handle.write(ciphertext)
            handle.flush()
            os.fsync(handle.fileno())
    except Exception:
        path.unlink(missing_ok=True)
        raise
    return ref

def get(self, ref: str) -> bytes:
    path = self._path_for(ref)
    try:
        data = path.read_bytes()
    except FileNotFoundError as exc:
        raise SecretNotFoundError("Secret reference not found") from exc

    if len(data) < 13:
        raise SecretStoreError("Stored secret record is invalid")

    try:
        return self._cipher.decrypt(data[:12], data[12:], ref.encode("utf-8"))
    except InvalidTag as exc:
        raise SecretStoreError("Stored secret could not be authenticated") from exc
```

```
    def delete(self, ref: str) -> None:
        path = self._path_for(ref)
        try:
            path.unlink()
        except FileNotFoundError as exc:
            raise SecretNotFoundError("Secret reference not found") from exc


def secret_store_from_environment() -> SecretStore:
    """Select an explicitly permitted adapter; never fall back silently."""

    app_env = os.environ.get("APP_ENV", "development").strip().lower()
    backend = os.environ.get("MFA_SECRET_STORE_BACKEND", "").strip().lower()

    if app_env == "production":
        raise SecretStoreConfigurationError(
            "No production MFA secret-store provider is configured"
        )

    if app_env == "test":
        if backend != "fake":
            raise SecretStoreConfigurationError(
                "Tests must explicitly select MFA_SECRET_STORE_BACKEND=fake"
            )
        return FakeSecretStore()

    if app_env != "development":
        raise SecretStoreConfigurationError("Unsupported APP_ENV for MFA secret storage")

    if backend != "dev-encrypted":
        raise SecretStoreConfigurationError(
            "Development requires MFA_SECRET_STORE_BACKEND=dev-encrypted"
        )

    encoded_key = os.environ.get("MFA_DEV_ENCRYPTION_KEY", "")
    store_path = os.environ.get("MFA_SECRET_STORE_PATH", "").strip()
    if not encoded_key or not store_path:
        raise SecretStoreConfigurationError(
            "Development secret storage requires an encryption key and storage path"
        )

    try:
        key = base64.b64decode(encoded_key, validate=True)
```

```
    except (ValueError, base64.binascii.Error) as exc:
        raise SecretStoreConfigurationError(
            "Development encryption key must be valid Base64"
        ) from exc

    if len(key) != 32:
        raise SecretStoreConfigurationError(
            "Development encryption key must decode to exactly 32 bytes"
        )

    return EncryptedFileSecretStore(Path(store_path), key)

===== TOTP SERVICE IMPLEMENTATION =====
"""TOTP primitives for MFA. No logging or secret provisioning is performed here."""

from __future__ import annotations

import hashlib
import hmac
import secrets
import struct
import time


def generate_totp_secret(length: int = 20) -> bytes:
    """Generate a cryptographically random TOTP secret."""
    if length < 16 or length > 64:
        raise ValueError("TOTP secret length must be between 16 and 64 bytes")
    return secrets.token_bytes(length)


def totp_code(
    secret: bytes,
    at_time: float | None = None,
    *,
    period: int = 30,
    digits: int = 6,
) -> str:
    """Generate a TOTP code for a specific time (injectable for tests)."""
    if not isinstance(secret, bytes) or len(secret) < 16:
        raise ValueError("TOTP secret must contain at least 16 bytes")
    if period <= 0:
        raise ValueError("TOTP period must be positive")
    if digits < 6 or digits > 8:
```

```
        raise ValueError("TOTP digits must be between 6 and 8")

    timestamp = time.time() if at_time is None else at_time
    if timestamp < 0:
        raise ValueError("TOTP timestamp cannot be negative")

    counter = int(timestamp) // period
    digest = hmac.new(secret, struct.pack(">Q", counter), hashlib.sha1).digest()
    offset = digest[-1] & 0x0F
    binary_code = struct.unpack(">I", digest[offset:offset + 4])[0] & 0x7FFFFFFF
    return str(binary_code % (10 ** digits)).zfill(digits)


def verify_totp(
    secret: bytes,
    supplied_code: str,
    at_time: float | None = None,
    *,
    period: int = 30,
    digits: int = 6,
    window: int = 1,
) -> bool:
    """Verify a numeric TOTP code, allowing a bounded clock-skew window."""
    if not isinstance(supplied_code, str):
        return False
    if len(supplied_code) != digits or not supplied_code.isascii() or not supplied_code.isdigit():
        return False
    if window < 0 or window > 2:
        raise ValueError("TOTP window must be between 0 and 2")

    timestamp = time.time() if at_time is None else at_time
    if timestamp < 0 or period <= 0:
        return False

    current_counter = int(timestamp) // period
    for offset in range(-window, window + 1):
        counter = current_counter + offset
        if counter < 0:
            continue
        candidate = totp_code(secret, counter * period, period=period, digits=digits)
        if hmac.compare_digest(candidate, supplied_code):
            return True
    return False
```

```
===== CURRENT REPOSITORY TESTS =====
from datetime import UTC, datetime
from uuid import UUID

import pytest

from app.auth import hash_password
from app.database import connect
from app.mfa_repository import (
    MFAFactorNotFoundError,
    MFAInvalidCodeError,
    activate_totp_factor,
    create_pending_totp_factor,
    list_totp_factors,
    revoke_totp_factor,
    verify_active_totp_factor,
)
from app.secret_store import FakeSecretStore
from app.totp import totp_code


@pytest.fixture
def mfa_identity():
    email = "mfa-repository-test@example.com"
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """
            INSERT INTO users (email, display_name, password_hash, account_status)
            VALUES (%s, %s, %s, 'active')
            RETURNING user_id
            """,
            (email, "MFA Repository Test", hash_password("Mfa-Test-Password-2026!")),
        )
        user_id = cur.fetchone()[0]

    yield user_id

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "UPDATE auth_sessions SET revoked_at = COALESCE(revoked_at, now()), "
            "revoke_reason = COALESCE(revoke_reason, 'test_cleanup') "
            "WHERE user_id = %s",
            (user_id,),
        )
```

```
        cur.execute(
            "UPDATE users SET account_status = 'deleted', "
            "deleted_at = COALESCE(deleted_at, now()) WHERE user_id = %s",
            (user_id,),
        )


def test_enroll_activate_list_verify_and_revoke(mfa_identity):
    store = FakeSecretStore()
    now = 1_800_000_000
    factor_id, uri = create_pending_totp_factor(
        user_id=mfa_identity,
        account_name="mfa-repository-test@example.com",
        label="Test authenticator",
        store=store,
    )

    assert isinstance(factor_id, UUID)
    assert uri.startswith("otpauth://totp/")
    assert "secret=" in uri

    listed = list_totp_factors(user_id=mfa_identity)
    assert len(listed) == 1
    assert listed[0]["factor_status"] == "pending"
    assert "secret_ref" not in listed[0]

    secret_ref = store._secrets.popitem()[0]
    secret = store.get(secret_ref)
    code = totp_code(secret, at_time=now)

    activate_totp_factor(
        user_id=mfa_identity,
        factor_id=factor_id,
        supplied_code=code,
        store=store,
        at_time=now,
    )

    assert list_totp_factors(user_id=mfa_identity)[0]["factor_status"] == "active"


def test_active_factor_rejects_reused_totp_step(mfa_identity):
    store = FakeSecretStore()
    now = 1_800_000_000
```

```
    factor_id, _ = create_pending_totp_factor(
        user_id=mfa_identity,
        account_name="mfa-repository-test@example.com",
        store=store,
    )
    secret_ref = next(iter(store._secrets))
    code = totp_code(store.get(secret_ref), at_time=now)

    activate_totp_factor(
        user_id=mfa_identity,
        factor_id=factor_id,
        supplied_code=code,
        store=store,
        at_time=now,
    )

    assert verify_active_totp_factor(
        user_id=mfa_identity,
        factor_id=factor_id,
        supplied_code=code,
        store=store,
        at_time=now + 30,
    ) is False


def test_invalid_activation_code_does_not_activate_factor(mfa_identity):
    store = FakeSecretStore()
    factor_id, _ = create_pending_totp_factor(
        user_id=mfa_identity,
        account_name="mfa-repository-test@example.com",
        store=store,
    )

    with pytest.raises(MFAInvalidCodeError):
        activate_totp_factor(
            user_id=mfa_identity,
            factor_id=factor_id,
            supplied_code="000000",
            store=store,
            at_time=1_800_000_000,
        )

    assert list_totp_factors(user_id=mfa_identity)[0]["factor_status"] == "pending"
```

```
def test_factor_cannot_be_accessed_by_another_user(mfa_identity):
    store = FakeSecretStore()
    factor_id, _ = create_pending_totp_factor(
        user_id=mfa_identity,
        account_name="mfa-repository-test@example.com",
        store=store,
    )

    other_user_id = UUID("00000000-0000-4000-8000-000000000001")

    with pytest.raises(MFAFactorNotFoundError):
        activate_totp_factor(
            user_id=other_user_id,
            factor_id=factor_id,
            supplied_code="123456",
            store=store,
            at_time=1_800_000_000,
        )

    assert revoke_totp_factor(user_id=other_user_id, factor_id=factor_id) is False


def test_revoked_factor_cannot_be_verified(mfa_identity):
    store = FakeSecretStore()
    now = 1_800_000_000
    factor_id, _ = create_pending_totp_factor(
        user_id=mfa_identity,
        account_name="mfa-repository-test@example.com",
        store=store,
    )
    secret_ref = next(iter(store._secrets))
    code = totp_code(store.get(secret_ref), at_time=now)

    activate_totp_factor(
        user_id=mfa_identity,
        factor_id=factor_id,
        supplied_code=code,
        store=store,
        at_time=now,
    )

    assert revoke_totp_factor(user_id=mfa_identity, factor_id=factor_id) is True
```

```
    with pytest.raises(MFAFactorNotFoundError):
        verify_active_totp_factor(
            user_id=mfa_identity,
            factor_id=factor_id,
            supplied_code=code,
            store=store,
            at_time=now + 30,
        )
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
            supplied_code="123456",
            store=store,
            at_time=1_800_000_000,
        )

    assert revoke_totp_factor(
        user_id=other_user_id,
        factor_id=factor_id,
    ) is False


def test_revoked_factor_cannot_be_verified(mfa_identity):
    user_id, email = mfa_identity
    store = FakeSecretStore()
    now = 1_800_000_000

    factor_id, _ = enroll(store, user_id, email)
    secret = store.get("fake://mfa/000001")
    code = totp_code(secret, at_time=now)

    activate_totp_factor(
        user_id=user_id,
        factor_id=factor_id,
        supplied_code=code,
        store=store,
        at_time=now,
    )

    assert revoke_totp_factor(user_id=user_id, factor_id=factor_id) is True

    with pytest.raises(MFAFactorNotFoundError):
        verify_active_totp_factor(
            user_id=user_id,
            factor_id=factor_id,
```

```
        supplied_code=code,
        store=store,
        at_time=now + 30,
    )
PY

printf '\n===== RUN MFA REPOSITORY TESTS =====\n'
PYTHONPATH=apps/api
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test'
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' .venv/bin/python -m
pytest apps/api/tests/test_mfa_repository.py -q


===== RUN MFA REPOSITORY TESTS =====
.....
[100%]
5 passed in 0.42s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
printf '\n===== AUTHENTICATION AND MFA REGRESSION TESTS =====\n'
PYTHONPATH=apps/api
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test'
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' .venv/bin/python -m
pytest apps/api/tests/test_auth.py apps/api/tests/test_fin14_auth_security.py
apps/api/tests/test_mfa_repository.py apps/api/tests/test_mfa_service.py
apps/api/tests/test_totp.py apps/api/tests/test_secret_store.py -q

printf '\n===== LOGIN ROUTE AND USER MFA POLICY =====\n'
grep -n -A145 -B20 'def login' apps/api/app/main.py

printf '\n===== MFA-RELATED CONFIGURATION =====\n'
grep -n -E 'APP_ENV|MFA_|SESSION_TTL|AUTH_LOGIN' .env.example apps/api/app/main.py


===== AUTHENTICATION AND MFA REGRESSION TESTS =====
.....................................................
[100%]
=============================================================================
======== warnings summary
=============================================================================
========
.venv/lib/python3.14/site-packages/fastapi/testclient.py:1
```

```
/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is
deprecated; install `httpx2` instead.
  from starlette.testclient import TestClient as TestClient  # noqa

.venv/lib/python3.14/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is
deprecated, use anyio.from_thread.BlockingPortal instead.
  _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

apps/api/tests/test_auth.py::test_login_creates_protected_hashed_expiring_session
apps/api/tests/test_auth.py::test_logout_revokes_session_and_replay_fails
apps/api/tests/test_auth.py::test_logout_revokes_session_and_replay_fails
apps/api/tests/test_auth.py::test_expired_session_is_rejected
apps/api/tests/test_auth.py::test_list_active_sessions_shows_current_session
apps/api/tests/test_auth.py::test_terminate_own_session_revokes_it
apps/api/tests/test_auth.py::test_terminate_own_session_revokes_it
apps/api/tests/test_auth.py::test_user_cannot_terminate_another_users_session
apps/api/tests/test_fin14_auth_security.py::test_logout_is_audited_without_recording_session_t
oken

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being
deprecated, because the expected behaviour on cookie persistence is ambiguous. Set cookies
directly on the client instance instead.
  return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
50 passed, 11 warnings in 3.66s

===== LOGIN ROUTE AND USER MFA POLICY =====
159-            )
160-            return user
161-    except psycopg.Error as exc:
162-        raise HTTPException(status_code=503, detail="Authentication service unavailable")
from exc
163-
164-
165-CurrentUser = Annotated[dict[str, Any], Depends(get_current_user)]
166-
167-
```

```
168-def require_permission(permission_key: str):
169-    """Build a deny-by-default backend permission dependency."""
170-    def dependency(user: CurrentUser) -> dict[str, Any]:
171-        if permission_key not in user.get("permissions", []):
172-            raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,
detail="Permission denied")
173-        return user
174-
175-    return dependency
176-
177-
178-@app.post("/auth/login", response_model=UserView)
179:def login(payload: LoginRequest, request: Request, response: Response) -> UserView:
180-    _check_origin(request)
181-    correlation_id = _auth_correlation_id(request)
182-    response.headers['X-Correlation-ID'] = str(correlation_id)
183-    email = payload.email.strip().lower()
184-    try:
185-        config = load_throttle_config()
186-    except ValueError as exc:
187-        raise HTTPException(
188-            status_code=503,
189-            detail="Authentication service unavailable",
190-        ) from exc
191-
192-    client_ip = request.client.host if request.client else "unknown"
193-    account_key = make_throttle_key(config.secret, "account", email)
194-    ip_key = make_throttle_key(config.secret, "ip", client_ip)
195-    login_succeeded = False
196-    user = None
197-
198-    try:
199-        with connect() as conn:
200-            account_blocked = is_blocked(
201-                conn, scope_type="account", key_hash=account_key
202-            )
203-            ip_blocked = is_blocked(
204-                conn, scope_type="ip", key_hash=ip_key
205-            )
206-
207-            if account_blocked or ip_blocked:
208-                record_auth_event(
209-                    conn,
210-                    action_key="auth.login",
```

```
                outcome="denied",
                actor_user_id=None,
                details={"reason": "throttled"},
                correlation_id=correlation_id,
            )
        else:
            with conn.cursor(row_factory=dict_row) as cur:
                cur.execute(
                    """SELECT user_id, email, display_name, password_hash
                       FROM users
                       WHERE lower(email) = %s
                         AND account_status = 'active'
                         AND deleted_at IS NULL""",
                    (email,),
                )
                user = cur.fetchone()

            password_hash = (
                user["password_hash"]
                if user and user["password_hash"]
                else DUMMY_PASSWORD_HASH
            )
            password_valid = verify_password(
                password_hash, payload.password
            )
            login_succeeded = bool(
                user and user["password_hash"] and password_valid
            )

            if not login_succeeded:
                record_failure(
                    conn,
                    scope_type="account",
                    key_hash=account_key,
                    config=config,
                )
                record_failure(
                    conn,
                    scope_type="ip",
                    key_hash=ip_key,
                    config=config,
                )
                record_auth_event(
                    conn,
```

```
                    action_key="auth.login",
                    outcome="failure",
                    actor_user_id=user["user_id"] if user else None,
                    details={"reason": "invalid_credentials"},
                    correlation_id=correlation_id,
                )
            else:
                # Clear account failures only. Preserve IP-wide history.
                reset_bucket(
                    conn,
                    scope_type="account",
                    key_hash=account_key,
                )

                token, digest = new_session_token()
                expires_at = session_expiry(_ttl_minutes())

                with conn.cursor() as cur:
                    cur.execute(
                        """INSERT INTO auth_sessions
                               (user_id, refresh_token_hash, expires_at)
                           VALUES (%s, %s, %s)""",
                        (user["user_id"], digest, expires_at),
                    )
                    now = datetime.now(UTC)
                    cur.execute(
                        """UPDATE users
                           SET last_login_at = %s, updated_at = %s
                           WHERE user_id = %s""",
                        (now, now, user["user_id"]),
                    )

                record_auth_event(
                    conn,
                    action_key="auth.login",
                    outcome="success",
                    actor_user_id=user["user_id"],
                    details={"method": "password"},
                    correlation_id=correlation_id,
                )

    except psycopg.Error as exc:
        raise HTTPException(
            status_code=503,
```

```
299-            detail="Authentication service unavailable",
300-        ) from exc
301-
302-    if not login_succeeded:
303-        raise HTTPException(status_code=401, detail=GENERIC_LOGIN_ERROR)
304-
305-    response.set_cookie(
306-        SESSION_COOKIE,
307-        token,
308-        httponly=True,
309-        secure=_secure_cookie(),
310-        samesite="lax",
311-        max_age=_ttl_minutes() * 60,
312-        path="/",
313-    )
314-    return UserView(
315-        user_id=user["user_id"],
316-        email=user["email"],
317-        display_name=user["display_name"],
318-    )
319-
320-
321-@app.post("/auth/logout", status_code=204)
322-def logout(
323-    request: Request,
324-    response: Response,

===== MFA-RELATED CONFIGURATION =====
.env.example:4:SESSION_TTL_MINUTES=60
.env.example:10:AUTH_LOGIN_MAX_FAILURES=5
.env.example:11:AUTH_LOGIN_WINDOW_SECONDS=900
.env.example:12:AUTH_LOGIN_BLOCK_SECONDS=900
apps/api/app/main.py:89:        value = int(os.environ.get("SESSION_TTL_MINUTES", "60"))
apps/api/app/main.py:100:    return os.environ.get("APP_ENV", "development").lower() ==
"production"
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
printf '\n===== MAIN.PY IMPORTS AND CONFIGURATION =====\n'
sed -n '1,165p' apps/api/app/main.py

printf '\n===== MFA REPOSITORY =====\n'
sed -n '1,280p' apps/api/app/mfa_repository.py
```

```
printf '\n===== MFA SERVICE AND TOKEN HELPERS =====\n'
sed -n '1,220p' apps/api/app/mfa_service.py
sed -n '1,140p' apps/api/app/auth.py

printf '\n===== LOGIN AND MFA TEST LOCATIONS =====\n'
grep -n -E 'def
test_.*(login|mfa|session)|/auth/login|mfa_required|create_pending|activate_totp|verify_active'
apps/api/tests/test_auth.py apps/api/tests/test_mfa_repository.py apps/api/app/main.py


===== MAIN.PY IMPORTS AND CONFIGURATION =====
"""FinSight AI API with PostgreSQL-backed browser sessions."""

from __future__ import annotations

import os
import secrets
from datetime import UTC, datetime
from pathlib import Path
from typing import Annotated, Any
from uuid import UUID, uuid4

from app.audit import record_auth_event
from app.throttle import (
    is_blocked,
    load_throttle_config,
    make_throttle_key,
    record_failure,
    reset_bucket,
)

import psycopg
from dotenv import load_dotenv
from fastapi import Cookie, Depends, FastAPI, HTTPException, Request, Response, status
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel, EmailStr, Field
from psycopg.rows import dict_row

from app.auth import hash_password, new_session_token, session_expiry, token_digest,
verify_password
from app.database import connect

load_dotenv(Path(__file__).resolve().parents[3] / ".env")
```

```
app = FastAPI(title="FinSight AI API", version="0.2.0")


@app.middleware("http")
async def auth_correlation_middleware(request: Request, call_next):
    """Return a correlation ID on authentication responses, including errors."""
    if (
        request.url.path not in {"/auth/login", "/auth/logout"}
        and not request.url.path.startswith("/auth/sessions")
    ):
        return await call_next(request)

    supplied = request.headers.get("X-Correlation-ID", "").strip()
    try:
        correlation_id = UUID(supplied) if supplied else uuid4()
    except ValueError:
        from starlette.responses import JSONResponse

        return JSONResponse(
            status_code=400,
            content={"detail": "Invalid X-Correlation-ID"},
        )

    request.state.auth_correlation_id = correlation_id
    response = await call_next(request)
    response.headers["X-Correlation-ID"] = str(correlation_id)
    return response


web_origin = os.environ.get("WEB_ORIGIN", "http://localhost:3000")
app.add_middleware(
    CORSMiddleware,
    allow_origins=[web_origin],
    allow_credentials=True,
    allow_methods=["GET", "POST", "DELETE"],
    allow_headers=["Content-Type", "X-Correlation-ID"],
)

SESSION_COOKIE = "finsight_session"
GENERIC_LOGIN_ERROR = "Email or password is incorrect."
DUMMY_PASSWORD_HASH = hash_password(secrets.token_urlsafe(32))


class LoginRequest(BaseModel):
```

```
    email: EmailStr
    password: str = Field(min_length=1, max_length=1024)


class UserView(BaseModel):
    user_id: UUID
    email: str
    display_name: str


def _ttl_minutes() -> int:
    try:
        value = int(os.environ.get("SESSION_TTL_MINUTES", "60"))
        session_expiry(value)
        return value
    except ValueError as exc:
        raise HTTPException(status_code=500, detail="Session configuration is invalid") from exc


def _secure_cookie() -> bool:
    configured = os.environ.get("COOKIE_SECURE")
    if configured is not None:
        return configured.lower() in {"1", "true", "yes"}
    return os.environ.get("APP_ENV", "development").lower() == "production"


def _check_origin(request: Request) -> None:
    origin = request.headers.get("origin")
    if origin and origin != web_origin:
        raise HTTPException(status_code=403, detail="Request origin is not allowed")


def _auth_correlation_id(request: Request) -> UUID:
    """Reuse the middleware's correlation ID, or create one if needed."""
    existing = getattr(request.state, "auth_correlation_id", None)
    if existing is not None:
        return existing

    supplied = request.headers.get("X-Correlation-ID", "").strip()
    if supplied:
        try:
            correlation_id = UUID(supplied)
        except ValueError as exc:
            raise HTTPException(
```

```
            status_code=400,
            detail="Invalid X-Correlation-ID",
        ) from exc
    else:
        correlation_id = uuid4()

    request.state.auth_correlation_id = correlation_id
    return correlation_id


def get_current_user(
    session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None,
) -> dict[str, Any]:
    if not session_token:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
detail="Authentication required")
    try:
        with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
            cur.execute(
                """SELECT u.user_id, u.email, u.display_name, r.role_key,
                       COALESCE(array_agg(DISTINCT p.permission_key)
                           FILTER (WHERE p.permission_key IS NOT NULL), ARRAY[]::varchar[])
AS permissions
                  FROM auth_sessions s JOIN users u ON u.user_id = s.user_id
                  LEFT JOIN roles r ON r.role_id = u.platform_role_id AND r.scope = 'platform'
                  LEFT JOIN role_permissions rp ON rp.role_id = r.role_id
                  LEFT JOIN permissions p ON p.permission_id = rp.permission_id
                  WHERE s.refresh_token_hash = %s AND s.revoked_at IS NULL
                    AND s.expires_at > %s AND u.account_status = 'active'
                    AND s.last_seen_at > %s - make_interval(mins => %s)
                    AND u.deleted_at IS NULL
                  GROUP BY u.user_id, u.email, u.display_name, r.role_key""",
                (token_digest(session_token), datetime.now(UTC), datetime.now(UTC),
_ttl_minutes()),
            )
            user = cur.fetchone()
            if not user:
                raise HTTPException(status_code=401, detail="Authentication required")
            cur.execute(
                "UPDATE auth_sessions SET last_seen_at = %s WHERE refresh_token_hash = %s",
                (datetime.now(UTC), token_digest(session_token)),
            )
            return user
    except psycopg.Error as exc:
```

```
        raise HTTPException(status_code=503, detail="Authentication service unavailable") from
exc


CurrentUser = Annotated[dict[str, Any], Depends(get_current_user)]

===== MFA REPOSITORY =====
"""Database-backed TOTP factor operations.

The database stores only opaque secret references. Callers must not log
provisioning URIs, TOTP seeds, challenge tokens, or submitted codes.
"""

from __future__ import annotations

from datetime import UTC, datetime
from typing import Any
from uuid import UUID

import psycopg
from psycopg.rows import dict_row

from app.database import connect
from app.mfa_service import matching_totp_step
from app.secret_store import SecretStore


class MFAFactorError(RuntimeError):
    """Base class for safe MFA factor operation failures."""


class MFAFactorNotFoundError(MFAFactorError):
    """The requested factor is unavailable."""


class MFAInvalidCodeError(MFAFactorError):
    """The submitted TOTP code is invalid or already used."""


def create_pending_totp_factor(
    *,
    user_id: UUID,
    account_name: str,
    store: SecretStore,
```

```
    label: str | None = None,
) -> tuple[UUID, str]:
    """Create a pending TOTP factor; return its ID and one-time URI."""
    from app.mfa_service import create_totp_provisioning

    secret_ref, provisioning_uri = create_totp_provisioning(
        account_name=account_name,
        store=store,
    )

    try:
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                """
                INSERT INTO mfa_factors
                    (user_id, factor_type, label, secret_ref, factor_status)
                VALUES (%s, 'totp', %s, %s, 'pending')
                RETURNING mfa_factor_id
                """,
                (user_id, label, secret_ref),
            )
            factor_id = cur.fetchone()[0]
    except Exception:
        try:
            store.delete(secret_ref)
        except Exception:
            pass
        raise

    return factor_id, provisioning_uri


def activate_totp_factor(
    *,
    user_id: UUID,
    factor_id: UUID,
    supplied_code: str,
    store: SecretStore,
    at_time: float | None = None,
) -> None:
    """Activate a pending factor only when its TOTP code is valid."""
    with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
        cur.execute(
            """
```

```
    SELECT mfa_factor_id, secret_ref
    FROM mfa_factors
    WHERE mfa_factor_id = %s
      AND user_id = %s
      AND factor_type = 'totp'
      AND factor_status = 'pending'
    FOR UPDATE
    """,
    (factor_id, user_id),
)
factor = cur.fetchone()

if not factor or not factor["secret_ref"]:
    raise MFAFactorNotFoundError("Pending MFA factor not found")

secret = store.get(factor["secret_ref"])
step = matching_totp_step(
    secret=secret,
    supplied_code=supplied_code,
    at_time=at_time,
)
if step is None:
    raise MFAInvalidCodeError("Invalid verification code")

now = datetime.now(UTC)
cur.execute(
    """
    INSERT INTO mfa_totp_replay_guard
        (mfa_factor_id, last_accepted_step, updated_at)
    VALUES (%s, %s, %s)
    ON CONFLICT (mfa_factor_id) DO UPDATE
    SET last_accepted_step = EXCLUDED.last_accepted_step,
        updated_at = EXCLUDED.updated_at
    WHERE mfa_totp_replay_guard.last_accepted_step
         < EXCLUDED.last_accepted_step
    RETURNING mfa_factor_id
    """,
    (factor_id, step, now),
)
if cur.fetchone() is None:
    raise MFAInvalidCodeError("Invalid verification code")

cur.execute(
    """
```

```
            UPDATE mfa_factors
            SET factor_status = 'active',
                enrolled_at = %s,
                last_used_at = %s
            WHERE mfa_factor_id = %s
              AND user_id = %s
              AND factor_status = 'pending'
            RETURNING mfa_factor_id
            """,
            (now, now, factor_id, user_id),
        )
        if cur.fetchone() is None:
            raise MFAFactorNotFoundError("Pending MFA factor not found")


def verify_active_totp_factor(
    *,
    user_id: UUID,
    factor_id: UUID,
    supplied_code: str,
    store: SecretStore,
    at_time: float | None = None,
) -> bool:
    """Verify an active factor and atomically prevent time-step replay."""
    with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
        cur.execute(
            """
            SELECT mfa_factor_id, secret_ref
            FROM mfa_factors
            WHERE mfa_factor_id = %s
              AND user_id = %s
              AND factor_type = 'totp'
              AND factor_status = 'active'
            """,
            (factor_id, user_id),
        )
        factor = cur.fetchone()

        if not factor or not factor["secret_ref"]:
            raise MFAFactorNotFoundError("Active MFA factor not found")

        secret = store.get(factor["secret_ref"])
        step = matching_totp_step(
            secret=secret,
```

```
            supplied_code=supplied_code,
            at_time=at_time,
        )
        if step is None:
            return False

        now = datetime.now(UTC)
        cur.execute(
            """
            INSERT INTO mfa_totp_replay_guard
                (mfa_factor_id, last_accepted_step, updated_at)
            VALUES (%s, %s, %s)
            ON CONFLICT (mfa_factor_id) DO UPDATE
            SET last_accepted_step = EXCLUDED.last_accepted_step,
                updated_at = EXCLUDED.updated_at
            WHERE mfa_totp_replay_guard.last_accepted_step
                 < EXCLUDED.last_accepted_step
            RETURNING mfa_factor_id
            """,
            (factor_id, step, now),
        )
        if cur.fetchone() is None:
            return False

        cur.execute(
            """
            UPDATE mfa_factors
            SET last_used_at = %s
            WHERE mfa_factor_id = %s
              AND user_id = %s
              AND factor_status = 'active'
            """,
            (now, factor_id, user_id),
        )
        return cur.rowcount == 1


def revoke_totp_factor(*, user_id: UUID, factor_id: UUID) -> bool:
    """Revoke an owned TOTP factor and remove its replay-guard state."""
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """
            UPDATE mfa_factors
            SET factor_status = 'revoked'
```

```
            WHERE mfa_factor_id = %s
              AND user_id = %s
              AND factor_type = 'totp'
              AND factor_status IN ('pending', 'active')
            RETURNING mfa_factor_id
            """,
            (factor_id, user_id),
        )
        row = cur.fetchone()
        if row is None:
            return False

        cur.execute(
            "DELETE FROM mfa_totp_replay_guard WHERE mfa_factor_id = %s",
            (factor_id,),
        )
        return True


def list_totp_factors(*, user_id: UUID) -> list[dict[str, Any]]:
    """List factor metadata only; never return secret references."""
    with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
        cur.execute(
            """
            SELECT mfa_factor_id, label, factor_status, enrolled_at,
                   last_used_at, created_at
            FROM mfa_factors
            WHERE user_id = %s AND factor_type = 'totp'
            ORDER BY created_at DESC
            """,
            (user_id,),
        )
        return list(cur.fetchall())

===== MFA SERVICE AND TOKEN HELPERS =====
"""Provider-neutral TOTP enrollment and verification services.

Provisioning material is returned only to the caller performing enrollment.
Callers must never log the returned URI or the underlying TOTP secret.
"""

from __future__ import annotations

import base64
```

```
import hmac
from urllib.parse import quote, urlencode

from app.secret_store import SecretStore
from app.totp import generate_totp_secret, totp_code, verify_totp


DEFAULT_ISSUER = "FinSight AI"


def create_totp_provisioning(
    *,
    account_name: str,
    issuer: str = DEFAULT_ISSUER,
    store: SecretStore,
) -> tuple[str, str]:
    """Store a new TOTP seed and return (opaque reference, provisioning URI).

    The caller must persist only the reference. The URI contains the seed and
    must be shown only in the authenticated, one-time enrollment response.
    """
    account_name = account_name.strip()
    issuer = issuer.strip()

    if not account_name or not issuer:
        raise ValueError("Account name and issuer are required")

    secret = generate_totp_secret()
    reference = store.put(secret)

    try:
        encoded_secret = base64.b32encode(secret).decode("ascii").rstrip("=")
        label = f"{issuer}:{account_name}"
        query = urlencode(
            {
                "secret": encoded_secret,
                "issuer": issuer,
                "algorithm": "SHA1",
                "digits": 6,
                "period": 30,
            }
        )
        uri = f"otpauth://totp/{quote(label, safe='')}?{query}"
        return reference, uri
```

```
    except Exception:
        store.delete(reference)
        raise


def verify_totp_code(
    *,
    store: SecretStore,
    secret_ref: str,
    supplied_code: str,
    at_time: float | None = None,
) -> bool:
    """Verify a TOTP code without returning or exposing the underlying seed."""
    secret = store.get(secret_ref)
    return verify_totp(secret, supplied_code, at_time=at_time)


def current_totp_step(at_time: float | None = None, *, period: int = 30) -> int:
    """Return the current TOTP time-step index."""
    if period <= 0:
        raise ValueError("TOTP period must be positive")

    import time

    timestamp = time.time() if at_time is None else at_time
    if timestamp < 0:
        raise ValueError("TOTP timestamp cannot be negative")

    return int(timestamp) // period


def matching_totp_step(
    *,
    secret: bytes,
    supplied_code: str,
    at_time: float | None = None,
    period: int = 30,
    digits: int = 6,
    window: int = 1,
) -> int | None:
    """Return the matching counter for replay protection, or None."""
    if not isinstance(supplied_code, str):
        return None
    if len(supplied_code) != digits or not supplied_code.isascii() or not supplied_code.isdigit():
```

```
        return None
    if period <= 0 or window < 0 or window > 2:
        raise ValueError("Invalid TOTP verification configuration")

    import time

    timestamp = time.time() if at_time is None else at_time
    if timestamp < 0:
        return None

    counter = int(timestamp) // period
    for offset in range(-window, window + 1):
        step = counter + offset
        if step < 0:
            continue
        candidate = totp_code(secret, step * period, period=period, digits=digits)
        if hmac.compare_digest(candidate, supplied_code):
            return step

    return None
"""Password and opaque session helpers for the authentication API."""

from __future__ import annotations

import hashlib
import secrets
from datetime import UTC, datetime, timedelta

from argon2 import PasswordHasher
from argon2.exceptions import InvalidHashError, VerificationError

PASSWORD_HASHER = PasswordHasher()


def hash_password(password: str) -> str:
    return PASSWORD_HASHER.hash(password)


def verify_password(encoded_hash: str | None, password: str) -> bool:
    if not encoded_hash:
        return False
    try:
        return PASSWORD_HASHER.verify(encoded_hash, password)
    except (InvalidHashError, VerificationError):
```

```
        return False


def new_session_token() -> tuple[str, str]:
    token = secrets.token_urlsafe(32)
    return token, hashlib.sha256(token.encode("utf-8")).hexdigest()


def session_expiry(ttl_minutes: int, now: datetime | None = None) -> datetime:
    if ttl_minutes < 1 or ttl_minutes > 10080:
        raise ValueError("SESSION_TTL_MINUTES must be between 1 and 10080")
    return (now or datetime.now(UTC)) + timedelta(minutes=ttl_minutes)


def token_digest(token: str) -> str:
    return hashlib.sha256(token.encode("utf-8")).hexdigest()

===== LOGIN AND MFA TEST LOCATIONS =====
apps/api/tests/test_auth.py:81:def
test_login_creates_protected_hashed_expiring_session(client, identity, caplog, capsys):
apps/api/tests/test_auth.py:83:    response = client.post("/auth/login", json={"email": email,
"password": TEST_PASSWORD})
apps/api/tests/test_auth.py:106:    known = client.post("/auth/login", json={"email": email,
"password": "wrong-password"})
apps/api/tests/test_auth.py:107:    absent = client.post("/auth/login", json={"email":
"absent@example.com", "password": "wrong-password"})
apps/api/tests/test_auth.py:113:def test_logout_revokes_session_and_replay_fails(client,
identity):
apps/api/tests/test_auth.py:115:    login = client.post("/auth/login", json={"email": email,
"password": TEST_PASSWORD})
apps/api/tests/test_auth.py:127:def test_logout_without_session_is_idempotent(client):
apps/api/tests/test_auth.py:134:def test_expired_session_is_rejected(client, identity):
apps/api/tests/test_auth.py:136:    login = client.post("/auth/login", json={"email": email,
"password": TEST_PASSWORD})
apps/api/tests/test_auth.py:159:    response = client.post("/auth/login", json={"email": email,
"password": TEST_PASSWORD})
apps/api/tests/test_auth.py:165:def test_list_active_sessions_shows_current_session(client,
identity):
apps/api/tests/test_auth.py:168:        "/auth/login", json={"email": email, "password":
TEST_PASSWORD}
apps/api/tests/test_auth.py:185:def test_list_active_sessions_requires_authentication(client):
apps/api/tests/test_auth.py:190:def test_terminate_own_session_revokes_it(client, identity):
apps/api/tests/test_auth.py:193:        "/auth/login", json={"email": email, "password":
TEST_PASSWORD}
```

```
apps/api/tests/test_auth.py:228:def test_user_cannot_terminate_another_users_session(client,
identity):
apps/api/tests/test_auth.py:246:            "/auth/login",
apps/api/tests/test_auth.py:250:            "/auth/login",
apps/api/tests/test_mfa_repository.py:10:    activate_totp_factor,
apps/api/tests/test_mfa_repository.py:11:    create_pending_totp_factor,
apps/api/tests/test_mfa_repository.py:14:    verify_active_totp_factor,
apps/api/tests/test_mfa_repository.py:60:    factor_id, uri = create_pending_totp_factor(
apps/api/tests/test_mfa_repository.py:71:def test_enroll_activate_and_list_factor(mfa_identity):
apps/api/tests/test_mfa_repository.py:88:    activate_totp_factor(
apps/api/tests/test_mfa_repository.py:101:def
test_active_factor_rejects_reused_totp_step(mfa_identity):
apps/api/tests/test_mfa_repository.py:110:    activate_totp_factor(
apps/api/tests/test_mfa_repository.py:120:    assert verify_active_totp_factor(
apps/api/tests/test_mfa_repository.py:129:def
test_invalid_activation_code_leaves_factor_pending(mfa_identity):
apps/api/tests/test_mfa_repository.py:139:        activate_totp_factor(
apps/api/tests/test_mfa_repository.py:150:def
test_factor_cannot_be_accessed_by_another_user(mfa_identity):
apps/api/tests/test_mfa_repository.py:158:        activate_totp_factor(
apps/api/tests/test_mfa_repository.py:172:def
test_revoked_factor_cannot_be_verified(mfa_identity):
apps/api/tests/test_mfa_repository.py:181:    activate_totp_factor(
apps/api/tests/test_mfa_repository.py:192:        verify_active_totp_factor(
apps/api/app/main.py:40:        request.url.path not in {"/auth/login", "/auth/logout"}
apps/api/app/main.py:178:@app.post("/auth/login", response_model=UserView)
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
ps/api/app/main.py:178:@app.post("/auth/login", response_model=UserView)
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
printf '\n===== MFA TEST FIXTURES =====\n'
sed -n '1,220p' apps/api/tests/test_mfa_repository.py

printf '\n===== AUTH TEST FIXTURES =====\n'
sed -n '1,115p' apps/api/tests/test_auth.py

printf '\n===== SECRET STORE INTERFACE =====\n'
sed -n '1,260p' apps/api/app/secret_store.py

printf '\n===== MFA MIGRATION =====\n'
cat apps/api/migrations/0006_mfa_login_challenges.sql

printf '\n===== AUDIT EVENT SIGNATURE =====\n'
grep -n -A55 'def record_auth_event' apps/api/app/audit.py
```

```
===== MFA TEST FIXTURES =====
from uuid import uuid4

import pytest

from app.auth import hash_password
from app.database import connect
from app.mfa_repository import (
    MFAFactorNotFoundError,
    MFAInvalidCodeError,
    activate_totp_factor,
    create_pending_totp_factor,
    list_totp_factors,
    revoke_totp_factor,
    verify_active_totp_factor,
)
from app.secret_store import FakeSecretStore
from app.totp import totp_code


@pytest.fixture
def mfa_identity():
    email = f"mfa-repository-{uuid4().hex}@example.com"
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """
            INSERT INTO users
                (email, display_name, password_hash, account_status)
            VALUES (%s, %s, %s, 'active')
            RETURNING user_id
            """,
            (email, "MFA Repository Test", hash_password("Mfa-Test-Password-2026!")),
        )
        user_id = cur.fetchone()[0]

    try:
        yield user_id, email
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                """
                UPDATE auth_sessions
```

```
            SET revoked_at = COALESCE(revoked_at, now()),
                revoke_reason = COALESCE(revoke_reason, 'test_cleanup')
            WHERE user_id = %s
            """,
            (user_id,),
        )
        cur.execute(
            """
            UPDATE users
            SET account_status = 'deleted',
                deleted_at = COALESCE(deleted_at, now())
            WHERE user_id = %s
            """,
            (user_id,),
        )


def enroll(store, user_id, email):
    factor_id, uri = create_pending_totp_factor(
        user_id=user_id,
        account_name=email,
        label="Test authenticator",
        store=store,
    )
    assert uri.startswith("otpauth://totp/")
    assert "secret=" in uri
    return factor_id, uri


def test_enroll_activate_and_list_factor(mfa_identity):
    user_id, email = mfa_identity
    store = FakeSecretStore()
    now = 1_800_000_000

    factor_id, _ = enroll(store, user_id, email)
    listed = list_totp_factors(user_id=user_id)

    assert len(listed) == 1
    assert listed[0]["factor_status"] == "pending"
    assert "secret_ref" not in listed[0]

    # The provisioning URI contains the seed, so retrieve it through the
    # fake store's public interface using the deterministic first reference.
    secret = store.get("fake://mfa/000001")
```

```
    code = totp_code(secret, at_time=now)

    activate_totp_factor(
        user_id=user_id,
        factor_id=factor_id,
        supplied_code=code,
        store=store,
        at_time=now,
    )

    listed = list_totp_factors(user_id=user_id)
    assert listed[0]["factor_status"] == "active"
    assert "secret_ref" not in listed[0]


def test_active_factor_rejects_reused_totp_step(mfa_identity):
    user_id, email = mfa_identity
    store = FakeSecretStore()
    now = 1_800_000_000

    factor_id, _ = enroll(store, user_id, email)
    secret = store.get("fake://mfa/000001")
    code = totp_code(secret, at_time=now)

    activate_totp_factor(
        user_id=user_id,
        factor_id=factor_id,
        supplied_code=code,
        store=store,
        at_time=now,
    )

    # Activation consumes this time step. The same step must not be accepted
    # again, even though the code is still inside the TOTP clock window.
    assert verify_active_totp_factor(
        user_id=user_id,
        factor_id=factor_id,
        supplied_code=code,
        store=store,
        at_time=now,
    ) is False


def test_invalid_activation_code_leaves_factor_pending(mfa_identity):
```

```
    user_id, email = mfa_identity
    store = FakeSecretStore()
    factor_id, _ = enroll(store, user_id, email)

    secret = store.get("fake://mfa/000001")
    correct_code = totp_code(secret, at_time=1_800_000_000)
    wrong_code = "000000" if correct_code != "000000" else "000001"

    with pytest.raises(MFAInvalidCodeError):
        activate_totp_factor(
            user_id=user_id,
            factor_id=factor_id,
            supplied_code=wrong_code,
            store=store,
            at_time=1_800_000_000,
        )

    assert list_totp_factors(user_id=user_id)[0]["factor_status"] == "pending"


def test_factor_cannot_be_accessed_by_another_user(mfa_identity):
    user_id, email = mfa_identity
    store = FakeSecretStore()
    factor_id, _ = enroll(store, user_id, email)

    other_user_id = uuid4()

    with pytest.raises(MFAFactorNotFoundError):
        activate_totp_factor(
            user_id=other_user_id,
            factor_id=factor_id,
            supplied_code="123456",
            store=store,
            at_time=1_800_000_000,
        )

    assert revoke_totp_factor(
        user_id=other_user_id,
        factor_id=factor_id,
    ) is False


def test_revoked_factor_cannot_be_verified(mfa_identity):
    user_id, email = mfa_identity
```

```
    store = FakeSecretStore()
    now = 1_800_000_000

    factor_id, _ = enroll(store, user_id, email)
    secret = store.get("fake://mfa/000001")
    code = totp_code(secret, at_time=now)

    activate_totp_factor(
        user_id=user_id,
        factor_id=factor_id,
        supplied_code=code,
        store=store,
        at_time=now,
    )

    assert revoke_totp_factor(user_id=user_id, factor_id=factor_id) is True

    with pytest.raises(MFAFactorNotFoundError):
        verify_active_totp_factor(
            user_id=user_id,
            factor_id=factor_id,
            supplied_code=code,
            store=store,
            at_time=now + 30,
        )

===== AUTH TEST FIXTURES =====
from __future__ import annotations

import secrets
from uuid import uuid4

import pytest
from fastapi.testclient import TestClient
from psycopg.rows import dict_row

from app.auth import hash_password, token_digest, verify_password
from app.throttle import make_throttle_key
from app.database import connect
from app.main import SESSION_COOKIE, app

TEST_PASSWORD = secrets.token_urlsafe(32)
```

```
@pytest.fixture
def identity():
    email = f"{uuid4().hex}@example.com"
    encoded = hash_password(TEST_PASSWORD)
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "INSERT INTO users (email, password_hash, display_name, account_status) VALUES
(%s, %s, %s, 'active') RETURNING user_id",
            (email, encoded, "Synthetic Analyst"),
        )
        user_id = cur.fetchone()[0]
    try:
        yield user_id, email, encoded
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                "UPDATE auth_sessions SET revoked_at = COALESCE(revoked_at, now()), "
                "revoke_reason = COALESCE(revoke_reason, 'test_cleanup') "
                "WHERE user_id = %s",
                (user_id,),
            )
            cur.execute(
                "UPDATE users SET account_status = 'deleted', deleted_at =
COALESCE(deleted_at, now()) "
                "WHERE user_id = %s",
                (user_id,),
            )


@pytest.fixture
def client(monkeypatch):
    monkeypatch.setenv("SESSION_TTL_MINUTES", "60")
    monkeypatch.setenv("COOKIE_SECURE", "false")
    monkeypatch.setenv("WEB_ORIGIN", "http://localhost:3000")
    throttle_secret = "test-only-secret-material-at-least-32-bytes"
    monkeypatch.setenv("AUTH_THROTTLE_SECRET", throttle_secret)
    ip_key = make_throttle_key(
        throttle_secret.encode("utf-8"), "ip", "testclient"
    )

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "DELETE FROM auth_login_throttle WHERE scope_type = 'ip' AND key_hash = %s",
            (ip_key,),
```

```
        )

    try:
        with TestClient(app) as test_client:
            yield test_client
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                "DELETE FROM auth_login_throttle WHERE scope_type = 'ip' AND key_hash = %s",
                (ip_key,),
            )


def test_password_hash_is_argon2_and_never_plaintext():
    encoded = hash_password(TEST_PASSWORD)
    assert encoded.startswith("argon2id")
    assert encoded != TEST_PASSWORD
    assert verify_password(encoded, TEST_PASSWORD)
    assert not verify_password(encoded, "wrong")


def test_login_creates_protected_hashed_expiring_session(client, identity, caplog, capsys):
    user_id, email, encoded = identity
    response = client.post("/auth/login", json={"email": email, "password": TEST_PASSWORD})
    assert response.status_code == 200
    assert response.json() == {"user_id": str(user_id), "email": email, "display_name": "Synthetic
Analyst"}
    assert TEST_PASSWORD not in response.text and encoded not in response.text
    cookie = response.cookies.get(SESSION_COOKIE)
    assert cookie
    set_cookie = response.headers["set-cookie"].lower()
    assert "httponly" in set_cookie and "samesite=lax" in set_cookie and "path=/" in set_cookie
    with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
        cur.execute("SELECT refresh_token_hash, expires_at, revoked_at FROM auth_sessions
WHERE user_id = %s", (user_id,))
        stored = cur.fetchone()
    assert stored["refresh_token_hash"] == token_digest(cookie)
    assert stored["refresh_token_hash"] != cookie
    assert stored["expires_at"] is not None and stored["revoked_at"] is None
    protected = client.get("/auth/me", cookies={SESSION_COOKIE: cookie})
    assert protected.status_code == 200 and protected.json()["user_id"] == str(user_id)
    captured = capsys.readouterr()
    assert TEST_PASSWORD not in caplog.text + captured.out + captured.err
    assert cookie not in caplog.text + captured.out + captured.err
```

```
def test_invalid_credentials_have_same_generic_response(client, identity):
    _, email, _ = identity
    known = client.post("/auth/login", json={"email": email, "password": "wrong-password"})
    absent = client.post("/auth/login", json={"email": "absent@example.com", "password":
"wrong-password"})
    assert (known.status_code, known.json()) == (absent.status_code, absent.json())
    assert known.status_code == 401 and known.json() == {"detail": "Email or password is
incorrect."}
    assert SESSION_COOKIE not in known.cookies and SESSION_COOKIE not in
absent.cookies


def test_logout_revokes_session_and_replay_fails(client, identity):
    _, email, _ = identity
    login = client.post("/auth/login", json={"email": email, "password": TEST_PASSWORD})

===== SECRET STORE INTERFACE =====
"""Provider-neutral storage for MFA secrets.

The development adapter is for local development only. Production provider
integration must be selected and implemented separately.
"""

from __future__ import annotations

import base64
import os
import secrets
import stat
from pathlib import Path
from typing import Protocol
from uuid import uuid4

from cryptography.exceptions import InvalidTag
from cryptography.hazmat.primitives.ciphers.aead import AESGCM


class SecretStoreError(RuntimeError):
    """Base error for secret-store operations."""


class SecretNotFoundError(SecretStoreError):
```

```
    """The requested secret reference does not exist."""


class SecretStoreConfigurationError(SecretStoreError):
    """The configured secret store is missing or unsafe."""


class SecretStore(Protocol):
    """Interface consumed by MFA application services."""

    def put(self, secret: bytes) -> str: ...
    def get(self, ref: str) -> bytes: ...
    def delete(self, ref: str) -> None: ...


class FakeSecretStore:
    """In-memory adapter with predictable references for automated tests."""

    def __init__(self) -> None:
        self._values: dict[str, bytes] = {}
        self._next_id = 1

    def put(self, secret: bytes) -> str:
        if not isinstance(secret, bytes) or not secret:
            raise ValueError("Secret must be non-empty bytes")
        ref = f"fake://mfa/{self._next_id:06d}"
        self._next_id += 1
        self._values[ref] = bytes(secret)
        return ref

    def get(self, ref: str) -> bytes:
        try:
            return self._values[ref]
        except KeyError as exc:
            raise SecretNotFoundError("Secret reference not found") from exc

    def delete(self, ref: str) -> None:
        if ref not in self._values:
            raise SecretNotFoundError("Secret reference not found")
        del self._values[ref]


class EncryptedFileSecretStore:
    """Local-only AES-GCM store. The encryption key is never stored here."""
```

```
_PREFIX = "dev-encrypted://"

def __init__(self, root: Path, key: bytes) -> None:
    if len(key) != 32:
        raise SecretStoreConfigurationError(
            "Development encryption key must be exactly 32 bytes"
        )

    self._root = root.expanduser().resolve()
    self._cipher = AESGCM(key)
    self._root.mkdir(mode=0o700, parents=True, exist_ok=True)

    mode = stat.S_IMODE(self._root.stat().st_mode)
    if mode & 0o077:
        raise SecretStoreConfigurationError(
            "Secret-store directory permissions must exclude group and other access"
        )

def _path_for(self, ref: str) -> Path:
    if not ref.startswith(self._PREFIX):
        raise SecretNotFoundError("Secret reference not found")
    identifier = ref.removeprefix(self._PREFIX)
    try:
        if str(uuid4()) == identifier:
            pass
        from uuid import UUID
        UUID(identifier)
    except (ValueError, AttributeError) as exc:
        raise SecretNotFoundError("Secret reference not found") from exc
    return self._root / f"{identifier}.enc"

def put(self, secret: bytes) -> str:
    if not isinstance(secret, bytes) or not secret:
        raise ValueError("Secret must be non-empty bytes")

    identifier = str(uuid4())
    ref = f"{self._PREFIX}{identifier}"
    nonce = secrets.token_bytes(12)
    ciphertext = nonce + self._cipher.encrypt(nonce, secret, ref.encode("utf-8"))
    path = self._root / f"{identifier}.enc"

    flags = os.O_WRONLY | os.O_CREAT | os.O_EXCL
    fd = os.open(path, flags, 0o600)
```

```
        try:
            with os.fdopen(fd, "wb") as handle:
                handle.write(ciphertext)
                handle.flush()
                os.fsync(handle.fileno())
        except Exception:
            path.unlink(missing_ok=True)
            raise
        return ref

    def get(self, ref: str) -> bytes:
        path = self._path_for(ref)
        try:
            data = path.read_bytes()
        except FileNotFoundError as exc:
            raise SecretNotFoundError("Secret reference not found") from exc

        if len(data) < 13:
            raise SecretStoreError("Stored secret record is invalid")

        try:
            return self._cipher.decrypt(data[:12], data[12:], ref.encode("utf-8"))
        except InvalidTag as exc:
            raise SecretStoreError("Stored secret could not be authenticated") from exc

    def delete(self, ref: str) -> None:
        path = self._path_for(ref)
        try:
            path.unlink()
        except FileNotFoundError as exc:
            raise SecretNotFoundError("Secret reference not found") from exc


def secret_store_from_environment() -> SecretStore:
    """Select an explicitly permitted adapter; never fall back silently."""

    app_env = os.environ.get("APP_ENV", "development").strip().lower()
    backend = os.environ.get("MFA_SECRET_STORE_BACKEND", "").strip().lower()

    if app_env == "production":
        raise SecretStoreConfigurationError(
            "No production MFA secret-store provider is configured"
        )
```

```
    if app_env == "test":
        if backend != "fake":
            raise SecretStoreConfigurationError(
                "Tests must explicitly select MFA_SECRET_STORE_BACKEND=fake"
            )
        return FakeSecretStore()

    if app_env != "development":
        raise SecretStoreConfigurationError("Unsupported APP_ENV for MFA secret storage")

    if backend != "dev-encrypted":
        raise SecretStoreConfigurationError(
            "Development requires MFA_SECRET_STORE_BACKEND=dev-encrypted"
        )

    encoded_key = os.environ.get("MFA_DEV_ENCRYPTION_KEY", "")
    store_path = os.environ.get("MFA_SECRET_STORE_PATH", "").strip()
    if not encoded_key or not store_path:
        raise SecretStoreConfigurationError(
            "Development secret storage requires an encryption key and storage path"
        )

    try:
        key = base64.b64decode(encoded_key, validate=True)
    except (ValueError, base64.binascii.Error) as exc:
        raise SecretStoreConfigurationError(
            "Development encryption key must be valid Base64"
        ) from exc

    if len(key) != 32:
        raise SecretStoreConfigurationError(
            "Development encryption key must decode to exactly 32 bytes"
        )

    return EncryptedFileSecretStore(Path(store_path), key)

===== MFA MIGRATION =====
BEGIN;

CREATE TABLE mfa_login_challenges (
    challenge_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    challenge_token_hash TEXT NOT NULL UNIQUE,
    expires_at TIMESTAMPTZ NOT NULL,
```

```
    attempt_count INTEGER NOT NULL DEFAULT 0
        CHECK (attempt_count >= 0 AND attempt_count <= 5),
    consumed_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX ix_mfa_login_challenges_user_expiry
    ON mfa_login_challenges(user_id, expires_at);

CREATE TABLE mfa_totp_replay_guard (
    mfa_factor_id UUID PRIMARY KEY
        REFERENCES mfa_factors(mfa_factor_id) ON DELETE CASCADE,
    last_accepted_step BIGINT NOT NULL
        CHECK (last_accepted_step >= 0),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

COMMIT;

===== AUDIT EVENT SIGNATURE =====
38:def record_auth_event(
39-    conn: Any,
40-    *,
41-    action_key: str,
42-    outcome: str,
43-    actor_user_id: UUID | None,
44-    details: dict[str, Any] | None = None,
45-    correlation_id: UUID | None = None,
46-    occurred_at: datetime | None = None,
47-) -> None:
48-    """Persist one authentication audit event on the caller's transaction.
49-
50-    Persistence errors intentionally propagate to the caller. The caller must
51-    roll back and apply the documented fail-closed policy; events are never
52-    reported as recorded when this insert fails.
53-    """
54-    if outcome not in _ALLOWED_OUTCOMES:
55-        raise ValueError("Unsupported audit outcome")
56-    if not action_key or len(action_key) > 120:
57-        raise ValueError("action_key must contain 1 to 120 characters")
58-
59-    redacted_details = _redact(details or {})
60-    payload = json.dumps(redacted_details, separators=(",", ":"), sort_keys=True)
61-
```

```
62-    with conn.cursor() as cur:
63-        cur.execute(
64-            """
65-            INSERT INTO audit_events (
66-                workspace_id,
67-                actor_user_id,
68-                action_key,
69-                resource_type,
70-                resource_id,
71-                outcome,
72-                correlation_id,
73-                details_redacted,
74-                occurred_at
75-            )
76-            VALUES (NULL, %s, %s, 'authentication', NULL, %s, %s, %s::jsonb, %s)
77-            """,
78-            (
79-                actor_user_id,
80-                action_key,
81-                outcome,
82-                correlation_id,
83-                payload,
84-                occurred_at or datetime.now(UTC),
85-            ),
86-        )
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
-       )
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
            SET last_used_at = %s
            WHERE mfa_factor_id = %s
              AND user_id = %s
              AND factor_status = 'active'
            RETURNING mfa_factor_id
            """,
            (now, factor["mfa_factor_id"], challenge["user_id"]),
        )
        if cur.fetchone() is None:
            raise MFAChallengeInvalidError("Invalid MFA challenge")

        cur.execute(
            """
            UPDATE mfa_login_challenges
            SET consumed_at = %s
            WHERE challenge_id = %s
```

```
            AND consumed_at IS NULL
          RETURNING user_id
          """,
          (now, challenge["challenge_id"]),
      )
      consumed = cur.fetchone()
      if consumed is None:
          raise MFAChallengeInvalidError("Invalid MFA challenge")

      record_auth_event(
          conn,
          action_key="auth.mfa_challenge",
          outcome="success",
          actor_user_id=challenge["user_id"],
          details={"reason": "challenge_verified"},
          correlation_id=correlation_id,
          occurred_at=now,
      )

      return challenge["user_id"]
PY

printf '\n===== VERIFY NEW FILE SYNTAX =====\n'
PYTHONPATH=apps/api .venv/bin/python -m py_compile
apps/api/app/mfa_challenge_repository.py

printf '\n===== CHECK PATCH WHITESPACE =====\n'
git diff --check


===== VERIFY NEW FILE SYNTAX =====

===== CHECK PATCH WHITESPACE =====
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(base) sandeeph@Sandeeps-MacBook-Pro NVDIA_Play_Book %  source
/Users/sandeeph/Documents/NVDIA_Play_Book/.venv/bin/activate
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro NVDIA_Play_Book % cd FinSight_AI
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
          )

          if cur.fetchone() is None:
              raise MFAChallengeInvalidError("Invalid MFA challenge")

          cur.execute(
```

```
            """
            UPDATE mfa_login_challenges
            SET consumed_at = %s
            WHERE challenge_id = %s
              AND consumed_at IS NULL
            RETURNING user_id
            """,
            (now, challenge["challenge_id"]),
        )
        consumed = cur.fetchone()

        if consumed is None:
            raise MFAChallengeInvalidError("Invalid MFA challenge")

        verified_user_id = challenge["user_id"]
        record_auth_event(
            conn,
            action_key="auth.mfa_challenge",
            outcome="success",
            actor_user_id=verified_user_id,
            details={"reason": "challenge_verified"},
            correlation_id=correlation_id,
            occurred_at=now,
        )

    # Raise only after the invalid attempt count and audit event have committed.
    if invalid or verified_user_id is None:
        raise MFAChallengeInvalidError("Invalid MFA challenge")

    return verified_user_id
PY

printf '\n===== VERIFY REPOSITORY SYNTAX =====\n'
PYTHONPATH=apps/api .venv/bin/python -m py_compile
apps/api/app/mfa_challenge_repository.py

printf '\n===== CHECK WHITESPACE =====\n'
git diff --check


===== VERIFY REPOSITORY SYNTAX =====

===== CHECK WHITESPACE =====
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
```

```
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
            )

            if cur.fetchone() is None:
                raise MFAChallengeInvalidError("Invalid MFA challenge")

            cur.execute(
                """
                UPDATE mfa_login_challenges
                SET consumed_at = %s
                WHERE challenge_id = %s
                  AND consumed_at IS NULL
                RETURNING user_id
                """,
                (now, challenge["challenge_id"]),
            )
            consumed = cur.fetchone()

            if consumed is None:
                raise MFAChallengeInvalidError("Invalid MFA challenge")

            verified_user_id = challenge["user_id"]
            record_auth_event(
                conn,
                action_key="auth.mfa_challenge",
                outcome="success",
                actor_user_id=verified_user_id,
                details={"reason": "challenge_verified"},
                correlation_id=correlation_id,
                occurred_at=now,
            )

        # Raise only after the invalid attempt count and audit event have committed.
        if invalid or verified_user_id is None:
            raise MFAChallengeInvalidError("Invalid MFA challenge")

        return verified_user_id
PY

printf '\n===== VERIFY REPOSITORY SYNTAX =====\n'
PYTHONPATH=apps/api .venv/bin/python -m py_compile
apps/api/app/mfa_challenge_repository.py

printf '\n===== CHECK WHITESPACE =====\n'
```

```
git diff --check


===== VERIFY REPOSITORY SYNTAX =====

===== CHECK WHITESPACE =====
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
            store=store,
            at_time=replay_time,
        )

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """
            SELECT attempt_count
            FROM mfa_login_challenges
            WHERE user_id = %s
            ORDER BY created_at DESC
            LIMIT 1
            """,
            (user_id,),
        )
        assert cur.fetchone()[0] == 1


def test_expired_challenge_is_rejected(challenge_identity):
    user_id, store, secret = challenge_identity
    token, _ = issue_mfa_challenge(user_id=user_id)

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """
            UPDATE mfa_login_challenges
            SET expires_at = %s
            WHERE user_id = %s
            """,
            (datetime.now(UTC) - timedelta(seconds=1), user_id),
        )

    with pytest.raises(MFAChallengeInvalidError):
        verify_mfa_challenge(
            challenge_token=token,
            supplied_code=totp_code(secret),
            store=store,
```

```
    )
PY

printf '\n===== RUN MFA CHALLENGE REPOSITORY TESTS =====\n'
PYTHONPATH=apps/api
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test'
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' .venv/bin/python -m
pytest apps/api/tests/test_mfa_challenge_repository.py -q


===== RUN MFA CHALLENGE REPOSITORY TESTS =====
....
[100%]
4 passed in 0.40s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
cat >> apps/api/tests/test_mfa_challenge_repository.py <<'PY'


def test_challenge_locks_after_five_invalid_attempts(challenge_identity):
    user_id, store, _ = challenge_identity
    token, _ = issue_mfa_challenge(user_id=user_id)

    for _ in range(5):
        with pytest.raises(MFAChallengeInvalidError):
            verify_mfa_challenge(
                challenge_token=token,
                supplied_code="not-a-code",
                store=store,
            )

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """
            SELECT attempt_count
            FROM mfa_login_challenges
            WHERE user_id = %s
            ORDER BY created_at DESC
            LIMIT 1
            """,
            (user_id,),
        )
        assert cur.fetchone()[0] == 5
```

```
    with pytest.raises(MFAChallengeInvalidError):
        verify_mfa_challenge(
            challenge_token=token,
            supplied_code="not-a-code",
            store=store,
        )
PY

printf '\n===== RUN MFA CHALLENGE TESTS =====\n'
PYTHONPATH=apps/api
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test'
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' .venv/bin/python -m
pytest apps/api/tests/test_mfa_challenge_repository.py -q

printf '\n===== CHECK WHITESPACE =====\n'
git diff --check


===== RUN MFA CHALLENGE TESTS =====
.....
[100%]
5 passed in 0.52s

===== CHECK WHITESPACE =====
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
helper_anchor = 'app = FastAPI(title="FinSight AI API", version="0.2.0")\n'

if source.count(import_anchor) != 1:
    raise SystemExit("STOP: expected exactly one database import; no changes made.")
if source.count(helper_anchor) != 1:
    raise SystemExit("STOP: expected exactly one FastAPI initialization; no changes made.")
if "def _get_mfa_secret_store(" in source:
    raise SystemExit("STOP: MFA secret-store helper already exists; no changes made.")

source = source.replace(
    import_anchor,
    import_anchor
    + "from app.secret_store import SecretStore, secret_store_from_environment\n",
)
source = source.replace(
    helper_anchor,
    helper_anchor
```

```
    + '''

def _get_mfa_secret_store(request: Request) -> SecretStore:
    """Return the app-scoped secret store; never silently change backends."""
    store = getattr(request.app.state, "mfa_secret_store", None)
    if store is None:
        store = secret_store_from_environment()
        request.app.state.mfa_secret_store = store
    return store


''',
)

path.write_text(source)
print("Added app-scoped MFA secret-store helper.")
PY

printf '\n===== VERIFY SYNTAX =====\n'
PYTHONPATH=apps/api .venv/bin/python -m py_compile apps/api/app/main.py

printf '\n===== CHECK WHITESPACE =====\n'
git diff --check

printf '\n===== REVIEW CHANGE =====\n'
git diff -- apps/api/app/main.py

Added app-scoped MFA secret-store helper.

===== VERIFY SYNTAX =====

===== CHECK WHITESPACE =====

===== REVIEW CHANGE =====
diff --git a/apps/api/app/main.py b/apps/api/app/main.py
index 5e0e618..92d7074 100644
--- a/apps/api/app/main.py
+++ b/apps/api/app/main.py
@@ -7,7 +7,16 @@ import secrets
 from datetime import UTC, datetime
 from pathlib import Path
 from typing import Annotated, Any
-from uuid import UUID
+from uuid import UUID, uuid4
```

```
+
+from app.audit import record_auth_event
+from app.throttle import (
+    is_blocked,
+    load_throttle_config,
+    make_throttle_key,
+    record_failure,
+    reset_bucket,
+)

 import psycopg
 from dotenv import load_dotenv
@@ -18,17 +27,57 @@ from psycopg.rows import dict_row

 from app.auth import hash_password, new_session_token, session_expiry, token_digest,
verify_password
 from app.database import connect
+from app.secret_store import SecretStore, secret_store_from_environment

 load_dotenv(Path(__file__).resolve().parents[3] / ".env")

 app = FastAPI(title="FinSight AI API", version="0.2.0")
+
+
+def _get_mfa_secret_store(request: Request) -> SecretStore:
+    """Return the app-scoped secret store; never silently change backends."""
+    store = getattr(request.app.state, "mfa_secret_store", None)
+    if store is None:
+        store = secret_store_from_environment()
+        request.app.state.mfa_secret_store = store
+    return store
+
+
+
+
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %

===== MFA CHALLENGE REPOSITORY TESTS =====
.....
[100%]
5 passed in 0.50s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
printf '\n===== LOGIN ROUTE BOUNDARIES =====\n'
```

```
grep -n -E '^@app.post\("/auth/(login|logout)|^def login|^def logout|^class
(LoginRequest|UserView)' apps/api/app/main.py

printf '\n===== SECRET STORE EXCEPTIONS =====\n'
grep -n -E '^class (SecretStoreError|SecretStoreConfigurationError|SecretNotFoundError)'
apps/api/app/secret_store.py

printf '\n===== MFA CHALLENGE PUBLIC API =====\n'
grep -n -E '^class MFAChallenge|^def (issue_mfa_challenge|verify_mfa_challenge)'
apps/api/app/mfa_challenge_repository.py


===== LOGIN ROUTE BOUNDARIES =====
88:class LoginRequest(BaseModel):
93:class UserView(BaseModel):
190:@app.post("/auth/login", response_model=UserView)
191:def login(payload: LoginRequest, request: Request, response: Response) -> UserView:
333:@app.post("/auth/logout", status_code=204)
334:def logout(

===== SECRET STORE EXCEPTIONS =====
21:class SecretStoreError(RuntimeError):
25:class SecretNotFoundError(SecretStoreError):
29:class SecretStoreConfigurationError(SecretStoreError):

===== MFA CHALLENGE PUBLIC API =====
22:class MFAChallengeError(RuntimeError):
26:class MFAChallengeInvalidError(MFAChallengeError):
34:def issue_mfa_challenge(
66:def verify_mfa_challenge(
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
zsh: parse error near `('
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
printf '\n===== MFA VERIFY ENDING =====\n'
nl -ba apps/api/app/main.py | sed -n '450,535p'

printf '\n===== EXACT CORRUPTED FRAGMENTS =====\n'
grep -n -E 'GIN_ERROR|MFAChallengeInvalidError|SecretStoreError|^@app.post|^def logout'
apps/api/app/main.py

printf '\n===== LAST KNOWN GOOD COMMIT VERSION =====\n'
git show HEAD:apps/api/app/main.py | nl -ba | sed -n '175,225p'
```

```
===== MFA VERIFY ENDING =====
            )

        cur.execute(
            """INSERT INTO auth_sessions
                   (user_id, refresh_token_hash, expires_at)
               VALUES (%s, %s, %s)""",
            (user_id, digest, expires_at),
        )
        cur.execute(
            """UPDATE users
               SET last_login_at = %s, updated_at = %s
               WHERE user_id = %s""",
            (now, now, user_id),
        )
        record_auth_event(
            conn,
            action_key="auth.login",
            outcome="success",
            actor_user_id=user_id,
            details={"method": "password_and_totp"},
            correlation_id=correlation_id,
            occurred_at=now,
        )

    except MFAChallengeInvalidError as exc:
        raise HTTPException(
            status_code=401,
            detail="MFA verification failed",
        ) from exc
    except (SecretStoreError, SecretStoreConfigurationError) as exc:
        raise HTTPException(
            status_code=503,
            detail="MFA service unavailable",
        ) from exc
    except psycopg.Error as exc:
        raise HTTPException(
            status_code=503,
            detail="Authentication service unavailable",
        ) from exc

    response.set_cookie(
        SESSION_COOKIE,
```

```
        token,
        httponly=True,
        secure=_secure_cookie(),
        samesite="lax",
        max_age=_ttl_minutes() * 60,
        path="/",
    )
    return UserView(
        user_id=user["user_id"],
        email=user["email"],
        display_name=user["display_name"],
    )
GIN_ERROR)

    response.set_cookie(
        SESSION_COOKIE,
        token,
        httponly=True,
        secure=_secure_cookie(),
        samesite="lax",
        max_age=_ttl_minutes() * 60,
        path="/",
    )
    return UserView(
        user_id=user["user_id"],
        email=user["email"],
        display_name=user["display_name"],
    )


@app.post("/auth/logout", status_code=204)
def logout(
    request: Request,
    response: Response,
    session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None,
) -> Response:
    _check_origin(request)
    correlation_id = _auth_correlation_id(request)
    response.headers['X-Correlation-ID'] = str(correlation_id)

    try:
        with connect() as conn:
            actor_user_id = None

```

```
===== EXACT CORRUPTED FRAGMENTS =====
33:    SecretStoreError,
37:    MFAChallengeInvalidError,
94:GENERIC_LOGIN_ERROR = "Email or password is incorrect."
209:@app.post("/auth/login")
370:        raise HTTPException(status_code=401, detail=GENERIC_LOGIN_ERROR)
379:        except (SecretStoreError, SecretStoreConfigurationError) as exc:
412:@app.post("/auth/mfa/verify", response_model=UserView)
474:    except MFAChallengeInvalidError as exc:
479:    except (SecretStoreError, SecretStoreConfigurationError) as exc:
504:GIN_ERROR)
522:@app.post("/auth/logout", status_code=204)
523:def logout(

===== LAST KNOWN GOOD COMMIT VERSION =====
  175      response.status_code = status.HTTP_204_NO_CONTENT
  176      return response
  177
  178
  179  @app.get("/auth/me", response_model=UserView)
  180  def who_am_i(user: CurrentUser) -> UserView:
  181      return UserView(**user)
  182
  183
  184  @app.get("/api/v1/admin/authorization/capabilities")
  185  def platform_admin_capabilities(
  186      request: Request,
  187      _authorized: Annotated[dict[str, Any], Depends(require_permission("platform.admin"))],
  188  ) -> dict[str, Any]:
  189      """Return the authenticated caller's database-backed platform capabilities."""
  190
  191      supplied_request_id = request.headers.get("request_id", "").strip()
  192      if supplied_request_id and (
  193          len(supplied_request_id) > 128
  194          or any(ord(char) < 32 or ord(char) == 127 for char in supplied_request_id)
  195      ):
  196          raise HTTPException(status_code=400, detail="Invalid request_id")
  197
  198      # Always issue a server-generated canonical request ID.
  199      request_id = f"req_{secrets.token_urlsafe(12)}"
  200      correlation_id = supplied_request_id or request_id
  201      permissions = sorted(set(_authorized.get("permissions", [])))
  202      capabilities = (
```

```
  203         ["inspect_authorization_policy"]
  204         if "platform.admin" in permissions
  205         else []
  206     )
  207
  208     return {
  209         "data": {
  210             "subject": {
  211                 "user_id": str(_authorized["user_id"]),
  212                 "platform_role_code": _authorized.get("role_key"),
  213             },
  214             "effective_permissions": permissions,
  215             "capabilities": capabilities,
  216             "policy_version": "sprint-1-platform-rbac-v1",
  217         },
  218         "request_id": request_id,
  219         "correlation_id": correlation_id,
  220     }
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
.venv/bin/python - <<'PY'
from pathlib import Path

path = Path("apps/api/app/main.py")
source = path.read_text()

duplicate = '''GIN_ERROR)

    response.set_cookie(
        SESSION_COOKIE,
        token,
        httponly=True,
        secure=_secure_cookie(),
        samesite="lax",
        max_age=_ttl_minutes() * 60,
        path="/",
    )
    return UserView(
        user_id=user["user_id"],
        email=user["email"],
        display_name=user["display_name"],
    )
'''
```

```
count = source.count(duplicate)
if count != 1:
    raise SystemExit(
        f"STOP: expected one duplicate block, found {count}; no changes made."
    )

path.write_text(source.replace(duplicate, "", 1))
print("Removed the confirmed stray fragment and duplicate response block.")
PY

printf '\n===== VERIFY SYNTAX =====\n'
PYTHONPATH=apps/api .venv/bin/python -m py_compile apps/api/app/main.py

printf '\n===== CHECK WHITESPACE =====\n'
git diff --check

printf '\n===== VERIFY ROUTES =====\n'
grep -n -E '^def require_permission|^@app.post|^def login|^def verify_mfa_login|^def logout'
apps/api/app/main.py

Removed the confirmed stray fragment and duplicate response block.

===== VERIFY SYNTAX =====

===== CHECK WHITESPACE =====

===== VERIFY ROUTES =====
196:def require_permission(permission_key: str):
209:@app.post("/auth/login")
210:def login(payload: LoginRequest, request: Request, response: Response):
412:@app.post("/auth/mfa/verify", response_model=UserView)
413:def verify_mfa_login(
506:@app.post("/auth/logout", status_code=204)
507:def logout(
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
PYTHONPATH=apps/api
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test'
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' .venv/bin/python -m
pytest apps/api/tests/test_auth.py apps/api/tests/test_fin14_auth_security.py
apps/api/tests/test_mfa_challenge_repository.py apps/api/tests/test_mfa_repository.py -q
```

```
.............................
[100%]
=============================================================================
======== warnings summary
=============================================================================
========
.venv/lib/python3.14/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is
deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

.venv/lib/python3.14/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is
deprecated, use anyio.from_thread.BlockingPortal instead.
    _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

apps/api/tests/test_auth.py::test_login_creates_protected_hashed_expiring_session
apps/api/tests/test_auth.py::test_logout_revokes_session_and_replay_fails
apps/api/tests/test_auth.py::test_logout_revokes_session_and_replay_fails
apps/api/tests/test_auth.py::test_expired_session_is_rejected
apps/api/tests/test_auth.py::test_list_active_sessions_shows_current_session
apps/api/tests/test_auth.py::test_terminate_own_session_revokes_it
apps/api/tests/test_auth.py::test_terminate_own_session_revokes_it
apps/api/tests/test_auth.py::test_user_cannot_terminate_another_users_session
apps/api/tests/test_fin14_auth_security.py::test_logout_is_audited_without_recording_session_t
oken

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being
deprecated, because the expected behaviour on cookie persistence is ambiguous. Set cookies
directly on the client instance instead.
    return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
31 passed, 11 warnings in 4.11s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
printf '\n===== LOGIN MFA BRANCH =====\n'
nl -ba apps/api/app/main.py | sed -n '200,415p'
```

```
printf '\n===== MFA VERIFY ENDPOINT =====\n'
nl -ba apps/api/app/main.py | sed -n '410,510p'

printf '\n===== MFA CHALLENGE REPOSITORY API =====\n'
grep -n -E '^def |^class ' apps/api/app/mfa_challenge_repository.py

printf '\n===== CURRENT CHANGE SUMMARY =====\n'
git diff --stat
git diff --check


===== LOGIN MFA BRANCH =====
  200             raise HTTPException(
  201                 status_code=status.HTTP_403_FORBIDDEN,
  202                 detail="Permission denied",
  203             )
  204         return user
  205
  206     return dependency
  207
  208
  209 @app.post("/auth/login")
  210 def login(payload: LoginRequest, request: Request, response: Response):
  211     _check_origin(request)
  212     correlation_id = _auth_correlation_id(request)
  213     response.headers["X-Correlation-ID"] = str(correlation_id)
  214     email = payload.email.strip().lower()
  215
  216     try:
  217         config = load_throttle_config()
  218     except ValueError as exc:
  219         raise HTTPException(
  220             status_code=503,
  221             detail="Authentication service unavailable",
  222         ) from exc
  223
  224     client_ip = request.client.host if request.client else "unknown"
  225     account_key = make_throttle_key(config.secret, "account", email)
  226     ip_key = make_throttle_key(config.secret, "ip", client_ip)
  227     login_succeeded = False
  228     user = None
  229     session_token = None
  230     mfa_required = False
```

```

    try:
        with connect() as conn:
            account_blocked = is_blocked(
                conn, scope_type="account", key_hash=account_key
            )
            ip_blocked = is_blocked(
                conn, scope_type="ip", key_hash=ip_key
            )

            if account_blocked or ip_blocked:
                record_auth_event(
                    conn,
                    action_key="auth.login",
                    outcome="denied",
                    actor_user_id=None,
                    details={"reason": "throttled"},
                    correlation_id=correlation_id,
                )
            else:
                with conn.cursor(row_factory=dict_row) as cur:
                    cur.execute(
                        """SELECT user_id, email, display_name, password_hash,
                                mfa_required
                           FROM users
                           WHERE lower(email) = %s
                             AND account_status = 'active'
                             AND deleted_at IS NULL""",
                        (email,),
                    )
                    user = cur.fetchone()

                password_hash = (
                    user["password_hash"]
                    if user and user["password_hash"]
                    else DUMMY_PASSWORD_HASH
                )
                password_valid = verify_password(
                    password_hash, payload.password
                )
                login_succeeded = bool(
                    user and user["password_hash"] and password_valid
                )

```

```
            if not login_succeeded:
                record_failure(
                    conn,
                    scope_type="account",
                    key_hash=account_key,
                    config=config,
                )
                record_failure(
                    conn,
                    scope_type="ip",
                    key_hash=ip_key,
                    config=config,
                )
                record_auth_event(
                    conn,
                    action_key="auth.login",
                    outcome="failure",
                    actor_user_id=user["user_id"] if user else None,
                    details={"reason": "invalid_credentials"},
                    correlation_id=correlation_id,
                )
            else:
                reset_bucket(
                    conn,
                    scope_type="account",
                    key_hash=account_key,
                )
                mfa_required = bool(user["mfa_required"])

                if mfa_required:
                    with conn.cursor() as cur:
                        cur.execute(
                            """SELECT 1
                               FROM mfa_factors
                               WHERE user_id = %s
                                 AND factor_type = 'totp'
                                 AND factor_status = 'active'
                               LIMIT 1""",
                            (user["user_id"],),
                        )
                        has_active_factor = cur.fetchone() is not None

                    if not has_active_factor:
                        record_auth_event(
```

```
                    conn,
                    action_key="auth.login",
                    outcome="denied",
                    actor_user_id=user["user_id"],
                    details={"reason": "mfa_factor_unavailable"},
                    correlation_id=correlation_id,
                )
            else:
                record_auth_event(
                    conn,
                    action_key="auth.login",
                    outcome="success",
                    actor_user_id=user["user_id"],
                    details={"method": "password", "next_step": "mfa"},
                    correlation_id=correlation_id,
                )
        else:
            session_token, digest = new_session_token()
            expires_at = session_expiry(_ttl_minutes())

            with conn.cursor() as cur:
                cur.execute(
                    """INSERT INTO auth_sessions
                           (user_id, refresh_token_hash, expires_at)
                       VALUES (%s, %s, %s)""",
                    (user["user_id"], digest, expires_at),
                )
                now = datetime.now(UTC)
                cur.execute(
                    """UPDATE users
                       SET last_login_at = %s, updated_at = %s
                       WHERE user_id = %s""",
                    (now, now, user["user_id"]),
                )

            record_auth_event(
                conn,
                action_key="auth.login",
                outcome="success",
                actor_user_id=user["user_id"],
                details={"method": "password"},
                correlation_id=correlation_id,
            )

```

```
    except psycopg.Error as exc:
        raise HTTPException(
            status_code=503,
            detail="Authentication service unavailable",
        ) from exc

    if not login_succeeded:
        raise HTTPException(status_code=401, detail=GENERIC_LOGIN_ERROR)

    if mfa_required:
        try:
            store = _get_mfa_secret_store(request)
            challenge_token, expires_at = issue_mfa_challenge(
                user_id=user["user_id"],
                correlation_id=correlation_id,
            )
        except (SecretStoreError, SecretStoreConfigurationError) as exc:
            raise HTTPException(
                status_code=503,
                detail="MFA service unavailable",
            ) from exc
        except psycopg.Error as exc:
            raise HTTPException(
                status_code=503,
                detail="Authentication service unavailable",
            ) from exc

        return {
            "mfa_required": True,
            "challenge_token": challenge_token,
            "expires_at": expires_at,
        }

    response.set_cookie(
        SESSION_COOKIE,
        session_token,
        httponly=True,
        secure=_secure_cookie(),
        samesite="lax",
        max_age=_ttl_minutes() * 60,
        path="/",
    )
    return UserView(
        user_id=user["user_id"],
```

```
  407         email=user["email"],
  408         display_name=user["display_name"],
  409     )
  410
  411
  412 @app.post("/auth/mfa/verify", response_model=UserView)
  413 def verify_mfa_login(
  414     payload: MFAVerifyRequest,
  415     request: Request,

===== MFA VERIFY ENDPOINT =====
  410
  411
  412 @app.post("/auth/mfa/verify", response_model=UserView)
  413 def verify_mfa_login(
  414     payload: MFAVerifyRequest,
  415     request: Request,
  416     response: Response,
  417 ) -> UserView:
  418     _check_origin(request)
  419     correlation_id = _auth_correlation_id(request)
  420     response.headers["X-Correlation-ID"] = str(correlation_id)
  421
  422     try:
  423         store = _get_mfa_secret_store(request)
  424         user_id = verify_mfa_challenge(
  425             challenge_token=payload.challenge_token,
  426             supplied_code=payload.code,
  427             store=store,
  428             correlation_id=correlation_id,
  429         )
  430
  431         token, digest = new_session_token()
  432         expires_at = session_expiry(_ttl_minutes())
  433         now = datetime.now(UTC)
  434
  435         with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
  436             cur.execute(
  437                 """SELECT user_id, email, display_name
  438                    FROM users
  439                    WHERE user_id = %s
  440                      AND account_status = 'active'
  441                      AND deleted_at IS NULL
  442                      AND mfa_required = TRUE""",
```

```
            (user_id,),
        )
        user = cur.fetchone()
        if not user:
            raise HTTPException(
                status_code=401,
                detail="MFA verification failed",
            )

        cur.execute(
            """INSERT INTO auth_sessions
                   (user_id, refresh_token_hash, expires_at)
               VALUES (%s, %s, %s)""",
            (user_id, digest, expires_at),
        )
        cur.execute(
            """UPDATE users
               SET last_login_at = %s, updated_at = %s
               WHERE user_id = %s""",
            (now, now, user_id),
        )
        record_auth_event(
            conn,
            action_key="auth.login",
            outcome="success",
            actor_user_id=user_id,
            details={"method": "password_and_totp"},
            correlation_id=correlation_id,
            occurred_at=now,
        )

    except MFAChallengeInvalidError as exc:
        raise HTTPException(
            status_code=401,
            detail="MFA verification failed",
        ) from exc
    except (SecretStoreError, SecretStoreConfigurationError) as exc:
        raise HTTPException(
            status_code=503,
            detail="MFA service unavailable",
        ) from exc
    except psycopg.Error as exc:
        raise HTTPException(
            status_code=503,
```

```
  487             detail="Authentication service unavailable",
  488         ) from exc
  489
  490     response.set_cookie(
  491         SESSION_COOKIE,
  492         token,
  493         httponly=True,
  494         secure=_secure_cookie(),
  495         samesite="lax",
  496         max_age=_ttl_minutes() * 60,
  497         path="/",
  498     )
  499     return UserView(
  500         user_id=user["user_id"],
  501         email=user["email"],
  502         display_name=user["display_name"],
  503     )
  504
  505
  506 @app.post("/auth/logout", status_code=204)
  507 def logout(
  508     request: Request,
  509     response: Response,
  510     session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None,

===== MFA CHALLENGE REPOSITORY API =====
22:class MFAChallengeError(RuntimeError):
26:class MFAChallengeInvalidError(MFAChallengeError):
30:def _challenge_digest(token: str) -> str:
34:def issue_mfa_challenge(
66:def verify_mfa_challenge(

===== CURRENT CHANGE SUMMARY =====
 .env.example                    |   7 ++
 FUNCTION_TEST_TRACEABILITY.txt  |  35 +++++++++
 IMPLEMENTATION_STATUS.txt       |  52 ++++++++++++
 SPRINT_1_TEST_EXECUTION_STATUS.txt |  44 ++++++++++-
 apps/api/app/main.py            | 562
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++----------
 apps/api/requirements.lock      |   5 +-
 apps/api/requirements.txt       |   1 +
 apps/api/tests/test_auth.py     | 172 ++++++++++++++++++++++++++++++++++++++++++-
 conversation.txt                |  30 ++++++++
```

```
 docs/release/RELEASE_README.md     |   2 +-
 scripts/package_release.py         |   2 +-
 11 files changed, 859 insertions(+), 53 deletions(-)
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
printf '\n===== MFA CHALLENGE TRANSACTION =====\n'
nl -ba apps/api/app/mfa_challenge_repository.py | sed -n '1,180p'

printf '\n===== LOGIN TEST FIXTURES AND MFA TESTS =====\n'
grep -n -E '^def test_|^def
setup|^@pytest.fixture|mfa_required|mfa_factors|FakeSecretStore|secret_store'
apps/api/tests/test_auth.py

printf '\n===== LOGIN MODELS AND SECRET STORE HELPER =====\n'
nl -ba apps/api/app/main.py | sed -n '1,145p'

printf '\n===== MFA TESTS ALREADY PRESENT =====\n'
grep -R -n -E 'auth/mfa/verify|mfa_required.*login|challenge_token' apps/api/tests


===== MFA CHALLENGE TRANSACTION =====
     1  """Atomic, single-use MFA login challenges with TOTP replay protection."""
     2
     3  from __future__ import annotations
     4
     5  import hashlib
     6  import secrets
     7  from datetime import UTC, datetime, timedelta
     8  from uuid import UUID
     9
    10  from psycopg.rows import dict_row
    11
    12  from app.audit import record_auth_event
    13  from app.database import connect
    14  from app.mfa_service import matching_totp_step
    15  from app.secret_store import SecretStore
    16
    17
    18  CHALLENGE_TTL_SECONDS = 300
    19  MAX_CHALLENGE_ATTEMPTS = 5
    20
    21
    22  class MFAChallengeError(RuntimeError):
    23      """Base class for safe MFA challenge failures."""
```

```


class MFAChallengeInvalidError(MFAChallengeError):
    """Challenge is invalid, expired, consumed, or exhausted."""


def _challenge_digest(token: str) -> str:
    return hashlib.sha256(token.encode("utf-8")).hexdigest()


def issue_mfa_challenge(
    *,
    user_id: UUID,
    correlation_id: UUID | None = None,
) -> tuple[str, datetime]:
    """Persist only the challenge-token hash and return the token once."""
    token = secrets.token_urlsafe(32)
    now = datetime.now(UTC)
    expires_at = now + timedelta(seconds=CHALLENGE_TTL_SECONDS)

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """
            INSERT INTO mfa_login_challenges
                (user_id, challenge_token_hash, expires_at)
            VALUES (%s, %s, %s)
            """,
            (user_id, _challenge_digest(token), expires_at),
        )
        record_auth_event(
            conn,
            action_key="auth.mfa_challenge",
            outcome="success",
            actor_user_id=user_id,
            details={"reason": "challenge_issued"},
            correlation_id=correlation_id,
            occurred_at=now,
        )

    return token, expires_at


def verify_mfa_challenge(
    *,
```

```
    challenge_token: str,
    supplied_code: str,
    store: SecretStore,
    correlation_id: UUID | None = None,
    at_time: float | None = None,
) -> UUID:
    """Verify a challenge and TOTP step atomically, returning the user ID."""
    now = datetime.now(UTC)
    invalid = False
    verified_user_id: UUID | None = None

    with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
        cur.execute(
            """
            SELECT challenge_id, user_id, expires_at, attempt_count, consumed_at
            FROM mfa_login_challenges
            WHERE challenge_token_hash = %s
            FOR UPDATE
            """,
            (_challenge_digest(challenge_token),),
        )
        challenge = cur.fetchone()

        if (
            not challenge
            or challenge["consumed_at"] is not None
            or challenge["expires_at"] <= now
            or challenge["attempt_count"] >= MAX_CHALLENGE_ATTEMPTS
        ):
            raise MFAChallengeInvalidError("Invalid MFA challenge")

        cur.execute(
            """
            SELECT mfa_factor_id, secret_ref
            FROM mfa_factors
            WHERE user_id = %s
              AND factor_type = 'totp'
              AND factor_status = 'active'
            ORDER BY created_at DESC
            LIMIT 1
            FOR UPDATE
            """,
            (challenge["user_id"],),
        )
```

```
        factor = cur.fetchone()

        if not factor or not factor["secret_ref"]:
            raise MFAChallengeInvalidError("Invalid MFA challenge")

        # Secret-store errors deliberately propagate. No session should be
        # created if the configured store cannot provide the factor secret.
        secret = store.get(factor["secret_ref"])
        step = matching_totp_step(
            secret=secret,
            supplied_code=supplied_code,
            at_time=at_time,
        )

        if step is None:
            invalid = True
            reason = "invalid_code"
        else:
            cur.execute(
                """
                INSERT INTO mfa_totp_replay_guard
                    (mfa_factor_id, last_accepted_step, updated_at)
                VALUES (%s, %s, %s)
                ON CONFLICT (mfa_factor_id) DO UPDATE
                SET last_accepted_step = EXCLUDED.last_accepted_step,
                    updated_at = EXCLUDED.updated_at
                WHERE mfa_totp_replay_guard.last_accepted_step
                      < EXCLUDED.last_accepted_step
                RETURNING mfa_factor_id
                """,
                (factor["mfa_factor_id"], step, now),
            )

            if cur.fetchone() is None:
                invalid = True
                reason = "replayed_code"

        if invalid:
            cur.execute(
                """
                UPDATE mfa_login_challenges
                SET attempt_count = LEAST(attempt_count + 1, %s)
                WHERE challenge_id = %s
                """,
```

```
156                 (MAX_CHALLENGE_ATTEMPTS, challenge["challenge_id"]),
157             )
158             record_auth_event(
159                 conn,
160                 action_key="auth.mfa_challenge",
161                 outcome="failure",
162                 actor_user_id=challenge["user_id"],
163                 details={"reason": reason},
164                 correlation_id=correlation_id,
165                 occurred_at=now,
166             )
167         else:
168             cur.execute(
169                 """
170                 UPDATE mfa_factors
171                 SET last_used_at = %s
172                 WHERE mfa_factor_id = %s
173                   AND user_id = %s
174                   AND factor_status = 'active'
175                 RETURNING mfa_factor_id
176                 """,
177                 (now, factor["mfa_factor_id"], challenge["user_id"]),
178             )
179 
180             if cur.fetchone() is None:
```

```
===== LOGIN TEST FIXTURES AND MFA TESTS =====
18:@pytest.fixture
45:@pytest.fixture
73:def test_password_hash_is_argon2_and_never_plaintext():
81:def test_login_creates_protected_hashed_expiring_session(client, identity, caplog, capsys):
104:def test_invalid_credentials_have_same_generic_response(client, identity):
113:def test_logout_revokes_session_and_replay_fails(client, identity):
127:def test_logout_without_session_is_idempotent(client):
134:def test_expired_session_is_rejected(client, identity):
144:def test_public_registration_is_not_available(client):
149:def test_auth_mutation_rejects_foreign_origin(client):
156:def test_secure_cookie_can_be_required_by_deployment(client, identity, monkeypatch):
165:def test_list_active_sessions_shows_current_session(client, identity):
185:def test_list_active_sessions_requires_authentication(client):
190:def test_terminate_own_session_revokes_it(client, identity):
228:def test_user_cannot_terminate_another_users_session(client, identity):

===== LOGIN MODELS AND SECRET STORE HELPER =====
```

```python
"""FinSight AI API with PostgreSQL-backed browser sessions."""

from __future__ import annotations

import os
import secrets
from datetime import UTC, datetime
from pathlib import Path
from typing import Annotated, Any
from uuid import UUID, uuid4

from app.audit import record_auth_event
from app.throttle import (
    is_blocked,
    load_throttle_config,
    make_throttle_key,
    record_failure,
    reset_bucket,
)

import psycopg
from dotenv import load_dotenv
from fastapi import Cookie, Depends, FastAPI, HTTPException, Request, Response, status
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel, EmailStr, Field
from psycopg.rows import dict_row

from app.auth import hash_password, new_session_token, session_expiry, token_digest, verify_password
from app.database import connect
from app.secret_store import (
    SecretStore,
    SecretStoreConfigurationError,
    SecretStoreError,
    secret_store_from_environment,
)
from app.mfa_challenge_repository import (
    MFAChallengeInvalidError,
    issue_mfa_challenge,
    verify_mfa_challenge,
)

load_dotenv(Path(__file__).resolve().parents[3] / ".env")
```

```
43
44 app = FastAPI(title="FinSight AI API", version="0.2.0")
45
46
47 def _get_mfa_secret_store(request: Request) -> SecretStore:
48     """Return the app-scoped secret store; never silently change backends."""
49     store = getattr(request.app.state, "mfa_secret_store", None)
50     if store is None:
51         store = secret_store_from_environment()
52         request.app.state.mfa_secret_store = store
53     return store
54
55
56
57
58 @app.middleware("http")
59 async def auth_correlation_middleware(request: Request, call_next):
60     """Return a correlation ID on authentication responses, including errors."""
61     if (
62         request.url.path not in {"/auth/login", "/auth/logout"}
63         and not request.url.path.startswith("/auth/sessions")
64     ):
65         return await call_next(request)
66
67     supplied = request.headers.get("X-Correlation-ID", "").strip()
68     try:
69         correlation_id = UUID(supplied) if supplied else uuid4()
70     except ValueError:
71         from starlette.responses import JSONResponse
72
73         return JSONResponse(
74             status_code=400,
75             content={"detail": "Invalid X-Correlation-ID"},
76         )
77
78     request.state.auth_correlation_id = correlation_id
79     response = await call_next(request)
80     response.headers["X-Correlation-ID"] = str(correlation_id)
81     return response
82
83
84 web_origin = os.environ.get("WEB_ORIGIN", "http://localhost:3000")
85 app.add_middleware(
86     CORSMiddleware,
```

```
    allow_origins=[web_origin],
    allow_credentials=True,
    allow_methods=["GET", "POST", "DELETE"],
    allow_headers=["Content-Type", "X-Correlation-ID"],
)

SESSION_COOKIE = "finsight_session"
GENERIC_LOGIN_ERROR = "Email or password is incorrect."
DUMMY_PASSWORD_HASH = hash_password(secrets.token_urlsafe(32))


class LoginRequest(BaseModel):
    email: EmailStr
    password: str = Field(min_length=1, max_length=1024)


class UserView(BaseModel):
    user_id: UUID
    email: str
    display_name: str



class MFAVerifyRequest(BaseModel):
    challenge_token: str = Field(min_length=32, max_length=256)
    code: str = Field(min_length=1, max_length=32)


def _ttl_minutes() -> int:
    try:
        value = int(os.environ.get("SESSION_TTL_MINUTES", "60"))
        session_expiry(value)
        return value
    except ValueError as exc:
        raise HTTPException(status_code=500, detail="Session configuration is invalid")
from exc


def _secure_cookie() -> bool:
    configured = os.environ.get("COOKIE_SECURE")
    if configured is not None:
        return configured.lower() in {"1", "true", "yes"}
    return os.environ.get("APP_ENV", "development").lower() == "production"

```

```
  130
  131  def _check_origin(request: Request) -> None:
  132      origin = request.headers.get("origin")
  133      if origin and origin != web_origin:
  134          raise HTTPException(status_code=403, detail="Request origin is not allowed")
  135
  136
  137  def _auth_correlation_id(request: Request) -> UUID:
  138      """Reuse the middleware's correlation ID, or create one if needed."""
  139      existing = getattr(request.state, "auth_correlation_id", None)
  140      if existing is not None:
  141          return existing
  142
  143      supplied = request.headers.get("X-Correlation-ID", "").strip()
  144      if supplied:
  145          try:

===== MFA TESTS ALREADY PRESENT =====
Binary file
apps/api/tests/__pycache__/test_mfa_challenge_repository.cpython-314-pytest-9.0.3.pyc
matches
apps/api/tests/test_mfa_challenge_repository.py:76:def
test_challenge_token_is_hashed_and_success_consumes_challenge(challenge_identity):
apps/api/tests/test_mfa_challenge_repository.py:85:            SELECT challenge_token_hash,
consumed_at
apps/api/tests/test_mfa_challenge_repository.py:108:        challenge_token=token,
apps/api/tests/test_mfa_challenge_repository.py:116:            challenge_token=token,
apps/api/tests/test_mfa_challenge_repository.py:129:            challenge_token=token,
apps/api/tests/test_mfa_challenge_repository.py:159:            challenge_token=token,
apps/api/tests/test_mfa_challenge_repository.py:195:            challenge_token=token,
apps/api/tests/test_mfa_challenge_repository.py:208:                challenge_token=token,
apps/api/tests/test_mfa_challenge_repository.py:228:            challenge_token=token,
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
old_gate = '''    if mfa_required:
        try:
            store = _get_mfa_secret_store(request)
'''

new_gate = '''    if mfa_required and not mfa_challenge_allowed:
        raise HTTPException(status_code=401, detail=GENERIC_LOGIN_ERROR)

    if mfa_required:
        try:
            _get_mfa_secret_store(request)
```

```
'''

checks = [
    ("initialization", old_init, new_init),
    ("MFA audit branch", old_branch, new_branch),
    ("challenge gate", old_gate, new_gate),
]

for name, old, new in checks:
    count = source.count(old)
    if count != 1:
        raise SystemExit(
            f"STOP: expected exactly one {name} fragment, found {count}; "
            "no changes written."
        )
    source = source.replace(old, new, 1)

path.write_text(source)
print("Updated MFA challenge gating and removed premature login-success audit.")
PY

printf '\n===== VERIFY SYNTAX =====\n'
PYTHONPATH=apps/api .venv/bin/python -m py_compile apps/api/app/main.py

printf '\n===== CHECK WHITESPACE =====\n'
git diff --check

STOP: expected exactly one MFA audit branch fragment, found 0; no changes written.

===== VERIFY SYNTAX =====

===== CHECK WHITESPACE =====
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
PYTHONPATH=apps/api
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test'
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' .venv/bin/python -m
pytest apps/api/tests/test_auth.py apps/api/tests/test_fin14_auth_security.py
apps/api/tests/test_mfa_challenge_repository.py apps/api/tests/test_mfa_repository.py -q

.....................................
[100%]
==============================================================================
======== warnings summary
```

```
================================================================================
.venv/lib/python3.14/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-packages/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

.venv/lib/python3.14/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-packages/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is deprecated, use anyio.from_thread.BlockingPortal instead.
    _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

apps/api/tests/test_auth.py::test_login_creates_protected_hashed_expiring_session
apps/api/tests/test_auth.py::test_logout_revokes_session_and_replay_fails
apps/api/tests/test_auth.py::test_logout_revokes_session_and_replay_fails
apps/api/tests/test_auth.py::test_expired_session_is_rejected
apps/api/tests/test_auth.py::test_list_active_sessions_shows_current_session
apps/api/tests/test_auth.py::test_terminate_own_session_revokes_it
apps/api/tests/test_auth.py::test_terminate_own_session_revokes_it
apps/api/tests/test_auth.py::test_user_cannot_terminate_another_users_session
apps/api/tests/test_fin14_auth_security.py::test_logout_is_audited_without_recording_session_token

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-packages/starlette/testclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being deprecated, because the expected behaviour on cookie persistence is ambiguous. Set cookies directly on the client instance instead.
    return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
31 passed, 11 warnings in 3.94s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
printf '\n===== MFA LOGIN BRANCH =====\n'
nl -ba apps/api/app/main.py | sed -n '290,405p'

printf '\n===== CURRENT MFA FLAGS =====\n'
grep -n -E 'mfa_required|mfa_challenge_allowed|next_step.*mfa|mfa_factor_unavailable' apps/api/app/main.py
```

```
===== MFA LOGIN BRANCH =====
                    action_key="auth.login",
                    outcome="failure",
                    actor_user_id=user["user_id"] if user else None,
                    details={"reason": "invalid_credentials"},
                    correlation_id=correlation_id,
                )
            else:
                reset_bucket(
                    conn,
                    scope_type="account",
                    key_hash=account_key,
                )
                mfa_required = bool(user["mfa_required"])

                if mfa_required:
                    with conn.cursor() as cur:
                        cur.execute(
                            """SELECT 1
                              FROM mfa_factors
                              WHERE user_id = %s
                                AND factor_type = 'totp'
                                AND factor_status = 'active'
                              LIMIT 1""",
                            (user["user_id"],),
                        )
                        has_active_factor = cur.fetchone() is not None

                    if not has_active_factor:
                        record_auth_event(
                            conn,
                            action_key="auth.login",
                            outcome="denied",
                            actor_user_id=user["user_id"],
                            details={"reason": "mfa_factor_unavailable"},
                            correlation_id=correlation_id,
                        )
                    else:
                        record_auth_event(
                            conn,
                            action_key="auth.login",
                            outcome="success",
                            actor_user_id=user["user_id"],
```

```
                        details={"method": "password", "next_step": "mfa"},
                        correlation_id=correlation_id,
                    )
                else:
                    session_token, digest = new_session_token()
                    expires_at = session_expiry(_ttl_minutes())

                    with conn.cursor() as cur:
                        cur.execute(
                            """INSERT INTO auth_sessions
                                   (user_id, refresh_token_hash, expires_at)
                               VALUES (%s, %s, %s)""",
                            (user["user_id"], digest, expires_at),
                        )
                        now = datetime.now(UTC)
                        cur.execute(
                            """UPDATE users
                               SET last_login_at = %s, updated_at = %s
                               WHERE user_id = %s""",
                            (now, now, user["user_id"]),
                        )

                    record_auth_event(
                        conn,
                        action_key="auth.login",
                        outcome="success",
                        actor_user_id=user["user_id"],
                        details={"method": "password"},
                        correlation_id=correlation_id,
                    )

    except psycopg.Error as exc:
        raise HTTPException(
            status_code=503,
            detail="Authentication service unavailable",
        ) from exc

    if not login_succeeded:
        raise HTTPException(status_code=401, detail=GENERIC_LOGIN_ERROR)

    if mfa_required:
        try:
            store = _get_mfa_secret_store(request)
            challenge_token, expires_at = issue_mfa_challenge(
```

```
  376                 user_id=user["user_id"],
  377                 correlation_id=correlation_id,
  378             )
  379         except (SecretStoreError, SecretStoreConfigurationError) as exc:
  380             raise HTTPException(
  381                 status_code=503,
  382                 detail="MFA service unavailable",
  383             ) from exc
  384         except psycopg.Error as exc:
  385             raise HTTPException(
  386                 status_code=503,
  387                 detail="Authentication service unavailable",
  388             ) from exc
  389 
  390         return {
  391             "mfa_required": True,
  392             "challenge_token": challenge_token,
  393             "expires_at": expires_at,
  394         }
  395 
  396     response.set_cookie(
  397         SESSION_COOKIE,
  398         session_token,
  399         httponly=True,
  400         secure=_secure_cookie(),
  401         samesite="lax",
  402         max_age=_ttl_minutes() * 60,
  403         path="/",
  404     )
  405     return UserView(

===== CURRENT MFA FLAGS =====
230:     mfa_required = False
254:                                mfa_required
302:                 mfa_required = bool(user["mfa_required"])
304:                 if mfa_required:
323:                             details={"reason": "mfa_factor_unavailable"},
332:                             details={"method": "password", "next_step": "mfa"},
372:     if mfa_required:
391:             "mfa_required": True,
442:                     AND mfa_required = TRUE""",
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
```

```
new_audit = '''                        else:
                        mfa_challenge_allowed = True
'''

old_gate = '''    if mfa_required:
        try:
            store = _get_mfa_secret_store(request)
'''

new_gate = '''    if mfa_required and not mfa_challenge_allowed:
        raise HTTPException(status_code=401, detail=GENERIC_LOGIN_ERROR)

    if mfa_required:
        try:
            _get_mfa_secret_store(request)
'''

for label, old, new in (
    ("MFA flag initialization", old_init, new_init),
    ("premature MFA login audit", old_audit, new_audit),
    ("MFA challenge gate", old_gate, new_gate),
):
    count = source.count(old)
    if count != 1:
        raise SystemExit(
            f"STOP: {label}: expected one match, found {count}. "
            "No file changes written."
        )
    source = source.replace(old, new, 1)

path.write_text(source)
print("Updated MFA fail-closed gate and audit behavior.")
PY

printf '\n===== VERIFY SYNTAX =====\n'
PYTHONPATH=apps/api .venv/bin/python -m py_compile apps/api/app/main.py

printf '\n===== CHECK WHITESPACE =====\n'
git diff --check

printf '\n===== VERIFY MFA GATE =====\n'
grep -n -E 'mfa_required|mfa_challenge_allowed|next_step.*mfa|mfa_factor_unavailable'
apps/api/app/main.py
```

STOP: premature MFA login audit: expected one match, found 0. No file changes written.

===== VERIFY SYNTAX =====

===== CHECK WHITESPACE =====

===== VERIFY MFA GATE =====
230:    mfa_required = False
254:                                mfa_required
302:                    mfa_required = bool(user["mfa_required"])
304:                    if mfa_required:
323:                                details={"reason": "mfa_factor_unavailable"},
332:                                details={"method": "password", "next_step": "mfa"},
372:    if mfa_required:
391:            "mfa_required": True,
442:                    AND mfa_required = TRUE""",
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
PYTHONPATH=apps/api
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test'
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' .venv/bin/python -m
pytest apps/api/tests/test_auth.py apps/api/tests/test_fin14_auth_security.py
apps/api/tests/test_mfa_challenge_repository.py apps/api/tests/test_mfa_repository.py -q

.....
[100%]
==========
======== warnings summary
==========
========
.venv/lib/python3.14/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is
deprecated; install `httpx2` instead.
  from starlette.testclient import TestClient as TestClient # noqa

.venv/lib/python3.14/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is
deprecated, use anyio.from_thread.BlockingPortal instead.
  _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

apps/api/tests/test_auth.py::test_login_creates_protected_hashed_expiring_session

```
apps/api/tests/test_auth.py::test_logout_revokes_session_and_replay_fails
apps/api/tests/test_auth.py::test_logout_revokes_session_and_replay_fails
apps/api/tests/test_auth.py::test_expired_session_is_rejected
apps/api/tests/test_auth.py::test_list_active_sessions_shows_current_session
apps/api/tests/test_auth.py::test_terminate_own_session_revokes_it
apps/api/tests/test_auth.py::test_terminate_own_session_revokes_it
apps/api/tests/test_auth.py::test_user_cannot_terminate_another_users_session
apps/api/tests/test_fin14_auth_security.py::test_logout_is_audited_without_recording_session_t
oken

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being
deprecated, because the expected behaviour on cookie persistence is ambiguous. Set cookies
directly on the client instance instead.
    return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
31 passed, 11 warnings in 4.03s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
.venv/bin/python - <<'PY'
from pathlib import Path

source = Path("apps/api/app/main.py").read_text().splitlines()

for start, end in ((225, 238), (312, 340), (365, 380)):
    print(f"\n===== main.py lines {start}-{end} =====")
    for index in range(start - 1, min(end, len(source))):
        print(f"{index + 1:4}: {source[index]!r}")
PY


===== main.py lines 225-238 =====
 225: '    account_key = make_throttle_key(config.secret, "account", email)'
 226: '    ip_key = make_throttle_key(config.secret, "ip", client_ip)'
 227: '    login_succeeded = False'
 228: '    user = None'
 229: '    session_token = None'
 230: '    mfa_required = False'
 231: ''
 232: '    try:'
 233: '        with connect() as conn:'
 234: '            account_blocked = is_blocked('
 235: '                conn, scope_type="account", key_hash=account_key'
```

```
 236: '            )'
 237: '            ip_blocked = is_blocked('
 238: '                conn, scope_type="ip", key_hash=ip_key'

===== main.py lines 312-340 =====
 312: '                                LIMIT 1""",'
 313: '                            (user["user_id"],),'
 314: '                        )'
 315: '                        has_active_factor = cur.fetchone() is not None'
 316: "
 317: '                    if not has_active_factor:'
 318: '                        record_auth_event('
 319: '                            conn,'
 320: '                            action_key="auth.login",'
 321: '                            outcome="denied",'
 322: '                            actor_user_id=user["user_id"],'
 323: '                            details={"reason": "mfa_factor_unavailable"},'
 324: '                            correlation_id=correlation_id,'
 325: '                        )'
 326: '                    else:'
 327: '                        record_auth_event('
 328: '                            conn,'
 329: '                            action_key="auth.login",'
 330: '                            outcome="success",'
 331: '                            actor_user_id=user["user_id"],'
 332: '                            details={"method": "password", "next_step": "mfa"},'
 333: '                            correlation_id=correlation_id,'
 334: '                        )'
 335: '                else:'
 336: '                    session_token, digest = new_session_token()'
 337: '                    expires_at = session_expiry(_ttl_minutes())'
 338: "
 339: '                    with conn.cursor() as cur:'
 340: '                        cur.execute('

===== main.py lines 365-380 =====
 365: '            status_code=503,'
 366: '            detail="Authentication service unavailable",'
 367: '        ) from exc'
 368: "
 369: '    if not login_succeeded:'
 370: '        raise HTTPException(status_code=401, detail=GENERIC_LOGIN_ERROR)'
 371: "
 372: '    if mfa_required:'
```

```
 373: '        try:'
 374: '            store = _get_mfa_secret_store(request)'
 375: '            challenge_token, expires_at = issue_mfa_challenge('
 376: '                user_id=user["user_id"],'
 377: '                correlation_id=correlation_id,'
 378: '            )'
 379: '        except (SecretStoreError, SecretStoreConfigurationError) as exc:'
 380: '            raise HTTPException('
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
if len(start_indices) != 1:
    raise SystemExit(
        "STOP: expected one password-only MFA audit block; no changes written."
    )

start = start_indices[0]
end = None
for i in range(start, min(start + 12, len(lines))):
    if lines[i] == '                        )\n':
        end = i + 1
        break
if end is None:
    raise SystemExit("STOP: could not locate audit block end; no changes written.")

lines[start:end] = ['                        mfa_challenge_allowed = True\n']

# 3. Gate challenge issuance when there is no active factor.
source = ''.join(lines)
old_gate = '    if mfa_required:\n        try:\n            store = _get_mfa_secret_store(request)\n'
new_gate = (
    '    if mfa_required and not mfa_challenge_allowed:\n'
    '        raise HTTPException(status_code=401, detail=GENERIC_LOGIN_ERROR)\n'
    '\n'
    '    if mfa_required:\n'
    '        try:\n'
    '            _get_mfa_secret_store(request)\n'
)
if source.count(old_gate) != 1:
    raise SystemExit("STOP: expected one challenge gate; no changes written.")

source = source.replace(old_gate, new_gate, 1)
path.write_text(source)
print("Applied MFA challenge gate and audit correction.")
PY
```

```
printf '\n===== VERIFY SYNTAX =====\n'
PYTHONPATH=apps/api .venv/bin/python -m py_compile apps/api/app/main.py

printf '\n===== CHECK WHITESPACE =====\n'
git diff --check

printf '\n===== REVIEW MODIFIED SECTION =====\n'
nl -ba apps/api/app/main.py | sed -n '225,245p;315,390p'

STOP: expected one password-only MFA audit block; no changes written.

===== VERIFY SYNTAX =====

===== CHECK WHITESPACE =====

===== REVIEW MODIFIED SECTION =====
   225      account_key = make_throttle_key(config.secret, "account", email)
   226      ip_key = make_throttle_key(config.secret, "ip", client_ip)
   227      login_succeeded = False
   228      user = None
   229      session_token = None
   230      mfa_required = False
   231
   232      try:
   233          with connect() as conn:
   234              account_blocked = is_blocked(
   235                  conn, scope_type="account", key_hash=account_key
   236              )
   237              ip_blocked = is_blocked(
   238                  conn, scope_type="ip", key_hash=ip_key
   239              )
   240
   241              if account_blocked or ip_blocked:
   242                  record_auth_event(
   243                      conn,
   244                      action_key="auth.login",
   245                      outcome="denied",
   315                          has_active_factor = cur.fetchone() is not None
   316
   317                      if not has_active_factor:
   318                          record_auth_event(
   319                              conn,
   320                              action_key="auth.login",
```

```
                        outcome="denied",
                        actor_user_id=user["user_id"],
                        details={"reason": "mfa_factor_unavailable"},
                        correlation_id=correlation_id,
                    )
                else:
                    record_auth_event(
                        conn,
                        action_key="auth.login",
                        outcome="success",
                        actor_user_id=user["user_id"],
                        details={"method": "password", "next_step": "mfa"},
                        correlation_id=correlation_id,
                    )
            else:
                session_token, digest = new_session_token()
                expires_at = session_expiry(_ttl_minutes())

                with conn.cursor() as cur:
                    cur.execute(
                        """INSERT INTO auth_sessions
                               (user_id, refresh_token_hash, expires_at)
                           VALUES (%s, %s, %s)""",
                        (user["user_id"], digest, expires_at),
                    )
                    now = datetime.now(UTC)
                    cur.execute(
                        """UPDATE users
                           SET last_login_at = %s, updated_at = %s
                           WHERE user_id = %s""",
                        (now, now, user["user_id"]),
                    )

                record_auth_event(
                    conn,
                    action_key="auth.login",
                    outcome="success",
                    actor_user_id=user["user_id"],
                    details={"method": "password"},
                    correlation_id=correlation_id,
                )

    except psycopg.Error as exc:
        raise HTTPException(
```

```
  365             status_code=503,
  366             detail="Authentication service unavailable",
  367         ) from exc
  368
  369     if not login_succeeded:
  370         raise HTTPException(status_code=401, detail=GENERIC_LOGIN_ERROR)
  371
  372     if mfa_required:
  373         try:
  374             store = _get_mfa_secret_store(request)
  375             challenge_token, expires_at = issue_mfa_challenge(
  376                 user_id=user["user_id"],
  377                 correlation_id=correlation_id,
  378             )
  379         except (SecretStoreError, SecretStoreConfigurationError) as exc:
  380             raise HTTPException(
  381                 status_code=503,
  382                 detail="MFA service unavailable",
  383             ) from exc
  384         except psycopg.Error as exc:
  385             raise HTTPException(
  386                 status_code=503,
  387                 detail="Authentication service unavailable",
  388             ) from exc
  389
  390         return {
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
    )

if source.count("    mfa_challenge_allowed = False") != 0:
    raise SystemExit(
        "STOP: MFA flag may already exist; inspect before editing. No changes written."
    )

source = source.replace(
    "    mfa_required = False\n",
    "    mfa_required = False\n    mfa_challenge_allowed = False\n",
    1,
)
source = source.replace(old, new, 1)

old_gate = '''    if mfa_required:
        try:
```

```
            store = _get_mfa_secret_store(request)"'

new_gate = '''    if mfa_required and not mfa_challenge_allowed:
        raise HTTPException(status_code=401, detail=GENERIC_LOGIN_ERROR)

    if mfa_required:
        try:
            _get_mfa_secret_store(request)"'

if source.count(old_gate) != 1:
    raise SystemExit(
        "STOP: challenge gate did not match; review the diff before proceeding."
    )

source = source.replace(old_gate, new_gate, 1)
path.write_text(source)
print("Applied MFA challenge gating and audit correction.")
PY

printf '\n===== VERIFY SYNTAX =====\n'
PYTHONPATH=apps/api .venv/bin/python -m py_compile apps/api/app/main.py

printf '\n===== CHECK WHITESPACE =====\n'
git diff --check

printf '\n===== VERIFY CORRECTION =====\n'
grep -n -E 'mfa_required|mfa_challenge_allowed|next_step.*mfa|mfa_factor_unavailable'
apps/api/app/main.py

Applied MFA challenge gating and audit correction.

===== VERIFY SYNTAX =====

===== CHECK WHITESPACE =====

===== VERIFY CORRECTION =====
230:    mfa_required = False
231:    mfa_challenge_allowed = False
255:                                mfa_required
303:                    mfa_required = bool(user["mfa_required"])
305:                    if mfa_required:
324:                                details={"reason": "mfa_factor_unavailable"},
328:                            mfa_challenge_allowed = True
366:    if mfa_required and not mfa_challenge_allowed:
```

```
369:    if mfa_required:
388:            "mfa_required": True,
439:                    AND mfa_required = TRUE""",
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
PYTHONPATH=apps/api
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test'
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' .venv/bin/python -m
pytest apps/api/tests/test_auth.py apps/api/tests/test_fin14_auth_security.py
apps/api/tests/test_mfa_challenge_repository.py apps/api/tests/test_mfa_repository.py -q

.....................
[100%]
==========================================================================
======== warnings summary
==========================================================================
========
.venv/lib/python3.14/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is
deprecated; install `httpx2` instead.
  from starlette.testclient import TestClient as TestClient  # noqa

.venv/lib/python3.14/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is
deprecated, use anyio.from_thread.BlockingPortal instead.
  _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

apps/api/tests/test_auth.py::test_login_creates_protected_hashed_expiring_session
apps/api/tests/test_auth.py::test_logout_revokes_session_and_replay_fails
apps/api/tests/test_auth.py::test_logout_revokes_session_and_replay_fails
apps/api/tests/test_auth.py::test_expired_session_is_rejected
apps/api/tests/test_auth.py::test_list_active_sessions_shows_current_session
apps/api/tests/test_auth.py::test_terminate_own_session_revokes_it
apps/api/tests/test_auth.py::test_terminate_own_session_revokes_it
apps/api/tests/test_auth.py::test_user_cannot_terminate_another_users_session
apps/api/tests/test_fin14_auth_security.py::test_logout_is_audited_without_recording_session_t
oken

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being
```

deprecated, because the expected behaviour on cookie persistence is ambiguous. Set cookies directly on the client instance instead.
    return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
31 passed, 11 warnings in 3.93s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
printf '\n===== MFA FACTOR CREATION AND ACTIVATION =====\n'
nl -ba apps/api/app/mfa_repository.py | sed -n '1,175p'

printf '\n===== TOTP CODE GENERATION =====\n'
nl -ba apps/api/app/totp.py | sed -n '1,75p'

printf '\n===== MFA FLAG DATABASE USAGE =====\n'
grep -R -n -E 'mfa_required|CREATE TABLE.*mfa_factors|secret_ref' apps/api/migrations

===== MFA FACTOR CREATION AND ACTIVATION =====
```
     1  """Database-backed TOTP factor operations.
     2
     3  The database stores only opaque secret references. Callers must not log
     4  provisioning URIs, TOTP seeds, challenge tokens, or submitted codes.
     5  """
     6
     7  from __future__ import annotations
     8
     9  from datetime import UTC, datetime
    10  from typing import Any
    11  from uuid import UUID
    12
    13  import psycopg
    14  from psycopg.rows import dict_row
    15
    16  from app.database import connect
    17  from app.mfa_service import matching_totp_step
    18  from app.secret_store import SecretStore
    19
    20
    21  class MFAFactorError(RuntimeError):
    22      """Base class for safe MFA factor operation failures."""
    23
    24
    25  class MFAFactorNotFoundError(MFAFactorError):
```

```
    """The requested factor is unavailable."""


class MFAInvalidCodeError(MFAFactorError):
    """The submitted TOTP code is invalid or already used."""


def create_pending_totp_factor(
    *,
    user_id: UUID,
    account_name: str,
    store: SecretStore,
    label: str | None = None,
) -> tuple[UUID, str]:
    """Create a pending TOTP factor; return its ID and one-time URI."""
    from app.mfa_service import create_totp_provisioning

    secret_ref, provisioning_uri = create_totp_provisioning(
        account_name=account_name,
        store=store,
    )

    try:
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                """
                INSERT INTO mfa_factors
                    (user_id, factor_type, label, secret_ref, factor_status)
                VALUES (%s, 'totp', %s, %s, 'pending')
                RETURNING mfa_factor_id
                """,
                (user_id, label, secret_ref),
            )
            factor_id = cur.fetchone()[0]
    except Exception:
        try:
            store.delete(secret_ref)
        except Exception:
            pass
        raise

    return factor_id, provisioning_uri


```

```
def activate_totp_factor(
    *,
    user_id: UUID,
    factor_id: UUID,
    supplied_code: str,
    store: SecretStore,
    at_time: float | None = None,
) -> None:
    """Activate a pending factor only when its TOTP code is valid."""
    with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
        cur.execute(
            """
            SELECT mfa_factor_id, secret_ref
            FROM mfa_factors
            WHERE mfa_factor_id = %s
              AND user_id = %s
              AND factor_type = 'totp'
              AND factor_status = 'pending'
            FOR UPDATE
            """,
            (factor_id, user_id),
        )
        factor = cur.fetchone()

        if not factor or not factor["secret_ref"]:
            raise MFAFactorNotFoundError("Pending MFA factor not found")

        secret = store.get(factor["secret_ref"])
        step = matching_totp_step(
            secret=secret,
             supplied_code=supplied_code,
             at_time=at_time,
         )
         if step is None:
             raise MFAInvalidCodeError("Invalid verification code")

         now = datetime.now(UTC)
         cur.execute(
             """
             INSERT INTO mfa_totp_replay_guard
                 (mfa_factor_id, last_accepted_step, updated_at)
             VALUES (%s, %s, %s)
             ON CONFLICT (mfa_factor_id) DO UPDATE
             SET last_accepted_step = EXCLUDED.last_accepted_step,
```

```
            updated_at = EXCLUDED.updated_at
        WHERE mfa_totp_replay_guard.last_accepted_step
            < EXCLUDED.last_accepted_step
        RETURNING mfa_factor_id
        """,
        (factor_id, step, now),
    )
    if cur.fetchone() is None:
        raise MFAInvalidCodeError("Invalid verification code")

    cur.execute(
        """
        UPDATE mfa_factors
        SET factor_status = 'active',
            enrolled_at = %s,
            last_used_at = %s
        WHERE mfa_factor_id = %s
          AND user_id = %s
          AND factor_status = 'pending'
        RETURNING mfa_factor_id
        """,
        (now, now, factor_id, user_id),
    )
    if cur.fetchone() is None:
        raise MFAFactorNotFoundError("Pending MFA factor not found")


def verify_active_totp_factor(
    *,
    user_id: UUID,
    factor_id: UUID,
    supplied_code: str,
    store: SecretStore,
    at_time: float | None = None,
) -> bool:
    """Verify an active factor and atomically prevent time-step replay."""
    with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
        cur.execute(
            """
            SELECT mfa_factor_id, secret_ref
            FROM mfa_factors
            WHERE mfa_factor_id = %s
              AND user_id = %s
              AND factor_type = 'totp'
```

```
  158               AND factor_status = 'active'
  159             """,
  160             (factor_id, user_id),
  161         )
  162         factor = cur.fetchone()
  163
  164         if not factor or not factor["secret_ref"]:
  165             raise MFAFactorNotFoundError("Active MFA factor not found")
  166
  167         secret = store.get(factor["secret_ref"])
  168         step = matching_totp_step(
  169             secret=secret,
  170             supplied_code=supplied_code,
  171             at_time=at_time,
  172         )
  173         if step is None:
  174             return False
  175
```

===== TOTP CODE GENERATION =====

```
    1  """TOTP primitives for MFA. No logging or secret provisioning is performed here."""
    2
    3  from __future__ import annotations
    4
    5  import hashlib
    6  import hmac
    7  import secrets
    8  import struct
    9  import time
   10
   11
   12  def generate_totp_secret(length: int = 20) -> bytes:
   13      """Generate a cryptographically random TOTP secret."""
   14      if length < 16 or length > 64:
   15          raise ValueError("TOTP secret length must be between 16 and 64 bytes")
   16      return secrets.token_bytes(length)
   17
   18
   19  def totp_code(
   20      secret: bytes,
   21      at_time: float | None = None,
   22      *,
   23      period: int = 30,
   24      digits: int = 6,
```

```
) -> str:
    """Generate a TOTP code for a specific time (injectable for tests)."""
    if not isinstance(secret, bytes) or len(secret) < 16:
        raise ValueError("TOTP secret must contain at least 16 bytes")
    if period <= 0:
        raise ValueError("TOTP period must be positive")
    if digits < 6 or digits > 8:
        raise ValueError("TOTP digits must be between 6 and 8")

    timestamp = time.time() if at_time is None else at_time
    if timestamp < 0:
        raise ValueError("TOTP timestamp cannot be negative")

    counter = int(timestamp) // period
    digest = hmac.new(secret, struct.pack(">Q", counter), hashlib.sha1).digest()
    offset = digest[-1] & 0x0F
    binary_code = struct.unpack(">I", digest[offset:offset + 4])[0] & 0x7FFFFFFF
    return str(binary_code % (10 ** digits)).zfill(digits)


def verify_totp(
    secret: bytes,
    supplied_code: str,
    at_time: float | None = None,
    *,
    period: int = 30,
    digits: int = 6,
    window: int = 1,
) -> bool:
    """Verify a numeric TOTP code, allowing a bounded clock-skew window."""
    if not isinstance(supplied_code, str):
        return False
    if len(supplied_code) != digits or not supplied_code.isascii() or not
supplied_code.isdigit():
        return False
    if window < 0 or window > 2:
        raise ValueError("TOTP window must be between 0 and 2")

    timestamp = time.time() if at_time is None else at_time
    if timestamp < 0 or period <= 0:
        return False

    current_counter = int(timestamp) // period
    for offset in range(-window, window + 1):
```

```
    counter = current_counter + offset
    if counter < 0:
        continue
    candidate = totp_code(secret, counter * period, period=period, digits=digits)
    if hmac.compare_digest(candidate, supplied_code):
        return True
return False

===== MFA FLAG DATABASE USAGE =====
apps/api/migrations/0001_auth_identity.sql:23:    mfa_required BOOLEAN NOT NULL
DEFAULT false,
apps/api/migrations/0005_mfa_factors.sql:4:CREATE TABLE mfa_factors (
apps/api/migrations/0005_mfa_factors.sql:9:    secret_ref TEXT,
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
)

assert verify_response.status_code == 401
assert SESSION_COOKIE not in verify_response.cookies


def test_mfa_challenge_cannot_be_reused(client, mfa_identity):
    import time

    from app.totp import totp_code

    _, email, secret = mfa_identity
    code = totp_code(secret, at_time=int(time.time()))

    first_login = client.post(
        "/auth/login",
        json={"email": email, "password": TEST_PASSWORD},
    )
    first_challenge = first_login.json()["challenge_token"]

    first_verify = client.post(
        "/auth/mfa/verify",
        json={"challenge_token": first_challenge, "code": code},
    )
    assert first_verify.status_code == 200

    replay = client.post(
        "/auth/mfa/verify",
        json={"challenge_token": first_challenge, "code": code},
```

```
    )
    assert replay.status_code == 401
    assert SESSION_COOKIE not in replay.cookies
'''

path.write_text(source.rstrip() + "\n\n" + block.lstrip())
print("Added five MFA endpoint integration tests.")
PY

printf '\n===== VERIFY SYNTAX =====\n'
PYTHONPATH=apps/api .venv/bin/python -m py_compile apps/api/tests/test_auth.py

printf '\n===== CHECK WHITESPACE =====\n'
git diff --check

Added five MFA endpoint integration tests.

===== VERIFY SYNTAX =====

===== CHECK WHITESPACE =====
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
PYTHONPATH=apps/api
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test'
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' .venv/bin/python -m
pytest apps/api/tests/test_auth.py -k 'mfa_login_returns_challenge or
mfa_required_login_without_active_factor or valid_totp_verification or invalid_totp or
mfa_challenge_cannot_be_reused' -q

.....
[100%]
=============================================================================
======== warnings summary
=============================================================================
========
.venv/lib/python3.14/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is
deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

.venv/lib/python3.14/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
```

s/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is deprecated, use anyio.from_thread.BlockingPortal instead.
  _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
5 passed, 13 deselected, 2 warnings in 1.03s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
PYTHONPATH=apps/api
DATABASE_URL='postgresql://sandeeph@localhost:55439/finsight_fin15_test'
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' .venv/bin/python -m
pytest apps/api/tests/test_auth.py apps/api/tests/test_fin14_auth_security.py
apps/api/tests/test_mfa_challenge_repository.py apps/api/tests/test_mfa_repository.py
apps/api/tests/test_mfa_service.py apps/api/tests/test_totp.py
apps/api/tests/test_secret_store.py -q

....
[100%]
==========
======== warnings summary
==========
========
.venv/lib/python3.14/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
  from starlette.testclient import TestClient as TestClient # noqa

.venv/lib/python3.14/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is deprecated, use anyio.from_thread.BlockingPortal instead.
  _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

apps/api/tests/test_auth.py::test_login_creates_protected_hashed_expiring_session
apps/api/tests/test_auth.py::test_logout_revokes_session_and_replay_fails
apps/api/tests/test_auth.py::test_logout_revokes_session_and_replay_fails
apps/api/tests/test_auth.py::test_expired_session_is_rejected
apps/api/tests/test_auth.py::test_list_active_sessions_shows_current_session
apps/api/tests/test_auth.py::test_terminate_own_session_revokes_it
apps/api/tests/test_auth.py::test_terminate_own_session_revokes_it
apps/api/tests/test_auth.py::test_user_cannot_terminate_another_users_session

```
apps/api/tests/test_fin14_auth_security.py::test_logout_is_audited_without_recording_session_t
oken

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being
deprecated, because the expected behaviour on cookie persistence is ambiguous. Set cookies
directly on the client instance instead.
    return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
60 passed, 11 warnings in 4.61s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
printf '\n===== MFA ROUTES AND MODELS =====\n'
grep -n -E 'mfa|MFA|class .*Request|class .*View' apps/api/app/main.py

printf '\n===== MFA REPOSITORY FUNCTIONS =====\n'
grep -n -E '^def |^class ' apps/api/app/mfa_repository.py

printf '\n===== MFA DATABASE SCHEMA =====\n'
sed -n '1,240p' apps/api/migrations/0005_mfa_factors.sql

printf '\n===== CURRENT CHANGE SUMMARY =====\n'
git status --short
git diff --stat


===== MFA ROUTES AND MODELS =====
36:from app.mfa_challenge_repository import (
37:    MFAChallengeInvalidError,
38:    issue_mfa_challenge,
39:    verify_mfa_challenge,
47:def _get_mfa_secret_store(request: Request) -> SecretStore:
49:    store = getattr(request.app.state, "mfa_secret_store", None)
52:        request.app.state.mfa_secret_store = store
98:class LoginRequest(BaseModel):
103:class UserView(BaseModel):
110:class MFAVerifyRequest(BaseModel):
230:    mfa_required = False
231:    mfa_challenge_allowed = False
255:                                mfa_required
303:                mfa_required = bool(user["mfa_required"])
305:                if mfa_required:
309:                                FROM mfa_factors
```

```
324:                             details={"reason": "mfa_factor_unavailable"},
328:                         mfa_challenge_allowed = True
366:    if mfa_required and not mfa_challenge_allowed:
369:    if mfa_required:
371:            _get_mfa_secret_store(request)
372:            challenge_token, expires_at = issue_mfa_challenge(
379:                detail="MFA service unavailable",
388:            "mfa_required": True,
409:@app.post("/auth/mfa/verify", response_model=UserView)
410:def verify_mfa_login(
411:    payload: MFAVerifyRequest,
420:        store = _get_mfa_secret_store(request)
421:        user_id = verify_mfa_challenge(
439:                    AND mfa_required = TRUE""",
446:                detail="MFA verification failed",
471:    except MFAChallengeInvalidError as exc:
474:            detail="MFA verification failed",
479:            detail="MFA service unavailable",
556:class SessionView(BaseModel):

===== MFA REPOSITORY FUNCTIONS =====
21:class MFAFactorError(RuntimeError):
25:class MFAFactorNotFoundError(MFAFactorError):
29:class MFAInvalidCodeError(MFAFactorError):
33:def create_pending_totp_factor(
70:def activate_totp_factor(
141:def verify_active_totp_factor(
207:def revoke_totp_factor(*, user_id: UUID, factor_id: UUID) -> bool:
233:def list_totp_factors(*, user_id: UUID) -> list[dict[str, Any]]:

===== MFA DATABASE SCHEMA =====
-- FIN-14 MFA factor metadata; secret material remains in the configured secret store.
BEGIN;

CREATE TABLE mfa_factors (
    mfa_factor_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    factor_type VARCHAR(30) NOT NULL CHECK (factor_type IN
('totp','webauthn','recovery_code')),
    label VARCHAR(120),
    secret_ref TEXT,
    public_key_data JSONB,
    factor_status VARCHAR(20) NOT NULL DEFAULT 'pending'
        CHECK (factor_status IN ('pending', 'active', 'revoked')),
```

```
    enrolled_at TIMESTAMPTZ,
    last_used_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX ix_mfa_factors_user_status
    ON mfa_factors(user_id, factor_status);

COMMIT;

===== CURRENT CHANGE SUMMARY =====
 M .env.example
 M FUNCTION_TEST_TRACEABILITY.txt
 M IMPLEMENTATION_STATUS.txt
 M SPRINT_1_TEST_EXECUTION_STATUS.txt
 M apps/api/app/main.py
 M apps/api/requirements.lock
 M apps/api/requirements.txt
 M apps/api/tests/test_auth.py
 M conversation.txt
 M docs/release/RELEASE_README.md
 M scripts/package_release.py
?? apps/api/app/audit.py
?? apps/api/app/main.py.backup-20261022-080642
?? apps/api/app/mfa_challenge_repository.py
?? apps/api/app/mfa_repository.py
?? apps/api/app/mfa_service.py
?? apps/api/app/secret_store.py
?? apps/api/app/throttle.py
?? apps/api/app/totp.py
?? apps/api/migrations/0003_auth_audit_and_login_throttling.sql
?? apps/api/migrations/0004_immutable_auth_audit.sql
?? apps/api/migrations/0005_mfa_factors.sql
?? apps/api/migrations/0006_mfa_login_challenges.sql
?? apps/api/tests/test_audit_immutability.py
?? apps/api/tests/test_audit_service.py
?? apps/api/tests/test_fin14_auth_security.py
?? apps/api/tests/test_mfa_challenge_repository.py
?? apps/api/tests/test_mfa_repository.py
?? apps/api/tests/test_mfa_schema.py
?? apps/api/tests/test_mfa_service.py
?? apps/api/tests/test_secret_store.py
?? apps/api/tests/test_throttle.py
?? apps/api/tests/test_totp.py
```

```
 .env.example                     |   7 ++
 FUNCTION_TEST_TRACEABILITY.txt   |  35 +++++++++
 IMPLEMENTATION_STATUS.txt        |  52 ++++++++++++++
 SPRINT_1_TEST_EXECUTION_STATUS.txt |  44 ++++++++++-
 apps/api/app/main.py             | 559
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++----------
 apps/api/requirements.lock       |   5 +-
 apps/api/requirements.txt        |   1 +
 apps/api/tests/test_auth.py      | 325
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
+++++-
 conversation.txt                 |  30 ++++++++
 docs/release/RELEASE_README.md   |   2 +-
 scripts/package_release.py       |   2 +-
 11 files changed, 1009 insertions(+), 53 deletions(-)
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(base) sandeeph@Sandeeps-MacBook-Pro NVDIA_Play_Book %  source
/Users/sandeeph/Documents/NVDIA_Play_Book/.venv/bin/activate
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro NVDIA_Play_Book % cd FinSight_AI
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
printf '\n===== MAIN.PY IMPORTS AND AUTH DEPENDENCIES =====\n'
sed -n '1,115p' apps/api/app/main.py

printf '\n===== AUTHENTICATION DEPENDENCY DEFINITIONS =====\n'
grep -R -n -E 'CurrentUser|def get_current_user|def require_permission|Depends\('
apps/api/app

printf '\n===== AUDIT API =====\n'
sed -n '1,240p' apps/api/app/audit.py

printf '\n===== MFA REPOSITORY REVOCATION AND LISTING =====\n'
sed -n '190,260p' apps/api/app/mfa_repository.py

printf '\n===== EXISTING API DOCUMENTATION CONVENTIONS =====\n'
sed -n '1,180p' docs/architecture/README.md

printf '\n===== CURRENT TRACKING FILE ENDINGS =====\n'
tail -65 IMPLEMENTATION_STATUS.txt
tail -45 SPRINT_1_TEST_EXECUTION_STATUS.txt
tail -35 FUNCTION_TEST_TRACEABILITY.txt


===== MAIN.PY IMPORTS AND AUTH DEPENDENCIES =====
```

```
"""FinSight AI API with PostgreSQL-backed browser sessions."""

from __future__ import annotations

import os
import secrets
from datetime import UTC, datetime
from pathlib import Path
from typing import Annotated, Any
from uuid import UUID, uuid4

from app.audit import record_auth_event
from app.throttle import (
    is_blocked,
    load_throttle_config,
    make_throttle_key,
    record_failure,
    reset_bucket,
)

import psycopg
from dotenv import load_dotenv
from fastapi import Cookie, Depends, FastAPI, HTTPException, Request, Response, status
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel, EmailStr, Field
from psycopg.rows import dict_row

from app.auth import hash_password, new_session_token, session_expiry, token_digest,
verify_password
from app.database import connect
from app.secret_store import (
    SecretStore,
    SecretStoreConfigurationError,
    SecretStoreError,
    secret_store_from_environment,
)
from app.mfa_challenge_repository import (
    MFAChallengeInvalidError,
    issue_mfa_challenge,
    verify_mfa_challenge,
)

load_dotenv(Path(__file__).resolve().parents[3] / ".env")
```

```
app = FastAPI(title="FinSight AI API", version="0.2.0")


def _get_mfa_secret_store(request: Request) -> SecretStore:
    """Return the app-scoped secret store; never silently change backends."""
    store = getattr(request.app.state, "mfa_secret_store", None)
    if store is None:
        store = secret_store_from_environment()
        request.app.state.mfa_secret_store = store
    return store




@app.middleware("http")
async def auth_correlation_middleware(request: Request, call_next):
    """Return a correlation ID on authentication responses, including errors."""
    if (
        request.url.path not in {"/auth/login", "/auth/logout"}
        and not request.url.path.startswith("/auth/sessions")
    ):
        return await call_next(request)

    supplied = request.headers.get("X-Correlation-ID", "").strip()
    try:
        correlation_id = UUID(supplied) if supplied else uuid4()
    except ValueError:
        from starlette.responses import JSONResponse

        return JSONResponse(
            status_code=400,
            content={"detail": "Invalid X-Correlation-ID"},
        )

    request.state.auth_correlation_id = correlation_id
    response = await call_next(request)
    response.headers["X-Correlation-ID"] = str(correlation_id)
    return response


web_origin = os.environ.get("WEB_ORIGIN", "http://localhost:3000")
app.add_middleware(
    CORSMiddleware,
    allow_origins=[web_origin],
```

```
    allow_credentials=True,
    allow_methods=["GET", "POST", "DELETE"],
    allow_headers=["Content-Type", "X-Correlation-ID"],
)

SESSION_COOKIE = "finsight_session"
GENERIC_LOGIN_ERROR = "Email or password is incorrect."
DUMMY_PASSWORD_HASH = hash_password(secrets.token_urlsafe(32))


class LoginRequest(BaseModel):
    email: EmailStr
    password: str = Field(min_length=1, max_length=1024)


class UserView(BaseModel):
    user_id: UUID
    email: str
    display_name: str



class MFAVerifyRequest(BaseModel):
    challenge_token: str = Field(min_length=32, max_length=256)
    code: str = Field(min_length=1, max_length=32)


def _ttl_minutes() -> int:

===== AUTHENTICATION DEPENDENCY DEFINITIONS =====
Binary file apps/api/app/__pycache__/main.cpython-312.pyc matches
Binary file apps/api/app/__pycache__/main.cpython-314.pyc matches
apps/api/app/main.py.backup-20261022-080642:128:def get_current_user(
apps/api/app/main.py.backup-20261022-080642:162:CurrentUser = Annotated[dict[str, Any],
Depends(get_current_user)]
apps/api/app/main.py.backup-20261022-080642:165:def require_permission(permission_key:
str):
apps/api/app/main.py.backup-20261022-080642:167:    def dependency(user: CurrentUser) ->
dict[str, Any]:
apps/api/app/main.py.backup-20261022-080642:372:def who_am_i(user: CurrentUser) ->
UserView:
apps/api/app/main.py.backup-20261022-080642:379:    _authorized: Annotated[dict[str, Any],
Depends(require_permission("platform.admin"))],
apps/api/app/main.py:159:def get_current_user(
```

```
apps/api/app/main.py:193:CurrentUser = Annotated[dict[str, Any], Depends(get_current_user)]
apps/api/app/main.py:196:def require_permission(permission_key: str):
apps/api/app/main.py:198:    def dependency(user: CurrentUser) -> dict[str, Any]:
apps/api/app/main.py:569:    user: CurrentUser,
apps/api/app/main.py:605:    user: CurrentUser,
apps/api/app/main.py:653:def who_am_i(user: CurrentUser) -> UserView:
apps/api/app/main.py:660:    _authorized: Annotated[dict[str, Any],
Depends(require_permission("platform.admin"))],

===== AUDIT API =====
"""Reusable, redacted audit persistence for authentication security events."""

from __future__ import annotations

import json
import re
from datetime import UTC, datetime
from typing import Any
from uuid import UUID

_ALLOWED_OUTCOMES = frozenset({"success", "failure", "denied"})
_SECRET_KEY = re.compile(
    r"(password|passwd|token|cookie|authorization|api[_-]?key|secret|credential)",
    re.IGNORECASE,
)


def _redact(value: Any) -> Any:
    """Return JSON-safe audit details without values under secret-like keys."""
    if isinstance(value, dict):
        return {
            str(key): (
                "[REDACTED]" if _SECRET_KEY.search(str(key)) else _redact(item)
            )
            for key, item in value.items()
        }
    if isinstance(value, (list, tuple)):
        return [_redact(item) for item in value]
    if isinstance(value, UUID):
        return str(value)
    if isinstance(value, datetime):
        return value.astimezone(UTC).isoformat()
    if value is None or isinstance(value, (str, int, float, bool)):
        return value
```

```
    return str(value)


def record_auth_event(
    conn: Any,
    *,
    action_key: str,
    outcome: str,
    actor_user_id: UUID | None,
    details: dict[str, Any] | None = None,
    correlation_id: UUID | None = None,
    occurred_at: datetime | None = None,
) -> None:
    """Persist one authentication audit event on the caller's transaction.

    Persistence errors intentionally propagate to the caller. The caller must
    roll back and apply the documented fail-closed policy; events are never
    reported as recorded when this insert fails.
    """
    if outcome not in _ALLOWED_OUTCOMES:
        raise ValueError("Unsupported audit outcome")
    if not action_key or len(action_key) > 120:
        raise ValueError("action_key must contain 1 to 120 characters")

    redacted_details = _redact(details or {})
    payload = json.dumps(redacted_details, separators=(",", ":"), sort_keys=True)

    with conn.cursor() as cur:
        cur.execute(
            """
            INSERT INTO audit_events (
                workspace_id,
                actor_user_id,
                action_key,
                resource_type,
                resource_id,
                outcome,
                correlation_id,
                details_redacted,
                occurred_at
            )
            VALUES (NULL, %s, %s, 'authentication', NULL, %s, %s, %s::jsonb, %s)
            """,
            (
```

```
            actor_user_id,
            action_key,
            outcome,
            correlation_id,
            payload,
            occurred_at or datetime.now(UTC),
        ),
    )

===== MFA REPOSITORY REVOCATION AND LISTING =====
    )
    if cur.fetchone() is None:
        return False

    cur.execute(
        """
        UPDATE mfa_factors
        SET last_used_at = %s
        WHERE mfa_factor_id = %s
          AND user_id = %s
          AND factor_status = 'active'
        """,
        (now, factor_id, user_id),
    )
    return cur.rowcount == 1


def revoke_totp_factor(*, user_id: UUID, factor_id: UUID) -> bool:
    """Revoke an owned TOTP factor and remove its replay-guard state."""
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """
            UPDATE mfa_factors
            SET factor_status = 'revoked'
            WHERE mfa_factor_id = %s
              AND user_id = %s
              AND factor_type = 'totp'
              AND factor_status IN ('pending', 'active')
            RETURNING mfa_factor_id
            """,
            (factor_id, user_id),
        )
        row = cur.fetchone()
        if row is None:
```

```
            return False

        cur.execute(
            "DELETE FROM mfa_totp_replay_guard WHERE mfa_factor_id = %s",
            (factor_id,),
        )
        return True


def list_totp_factors(*, user_id: UUID) -> list[dict[str, Any]]:
    """List factor metadata only; never return secret references."""
    with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
        cur.execute(
            """
            SELECT mfa_factor_id, label, factor_status, enrolled_at,
                   last_used_at, created_at
            FROM mfa_factors
            WHERE user_id = %s AND factor_type = 'totp'
            ORDER BY created_at DESC
            """,
            (user_id,),
        )
        return list(cur.fetchall())
```

===== EXISTING API DOCUMENTATION CONVENTIONS =====
FinSight AI architecture and compatibility

Scope: this page describes the implementation currently in the repository, not the full target system in the SRS or technical design. Source references are read-only: `Dcos/FinSight_Software Requirements Specification (SRS).pdf`, `Dcos/FinSight_AI_Technical_Design_and_Implementation_Plan.pdf`, `Dcos/FinSight_AI_Database_Schema_and_ERD_Reference.pdf`, and the FIN-78 entry in the detailed backlog.

Current status

Area	Implemented now	Planned or not implemented
Web	Next.js App Router sign-in page; runtime API-origin endpoint; cookie-based browser requests	Dashboard, research, analytics, settings, and other product areas
API	One modular FastAPI application, currently owning login, current-user lookup, and logout	Domain-service modules, public route version prefix, authorization policies beyond the current-user dependency, rate limiting

Data	PostgreSQL is authoritative for the implemented identity/session records	The wider target schema, pgvector, market and analysis data, job records
Operations	One versioned SQL identity migration; CI quality, test, dependency-security, hygiene, and release-package jobs	Migration runner, configured deployment target, production environment and automated deployment
Integrations	None in the current runtime slice	Ollama, embedding/market-data providers, object storage, email/notifications, workers

Release 1 remains read-only financial research: there are no broker order routes or enabled trading flows. Password recovery, MFA, administrator invitation UI/email, workspace membership, audit events, and a health endpoint are not implemented. User provisioning is currently a trusted local CLI operation, not a public registration or application invitation flow. Platform RBAC currently defines only `platform_admin` with the narrowly scoped `platform.admin` permission; no current product operation consumes that permission. The reusable `require_permission` dependency denies missing permissions. Operator and Standard User remain workspace-scoped and unassigned until workspace membership exists.

Runtime and repository boundaries

- `apps/web/`: Next.js/React presentation. `app/page.js` loads runtime configuration, calls the API, and renders sign-in/session state. The browser is not an authorization boundary.
- `apps/api/app/`: FastAPI application and small auth/database modules. `main.py` validates requests, checks request origin, sets/clears cookies, and uses the identity service logic; `auth.py` hashes passwords and creates/digests opaque tokens; `database.py` connects to PostgreSQL.
- `apps/api/migrations/`: versioned PostgreSQL DDL. `0001_auth_identity.sql` creates the current `roles`, `users`, and `auth_sessions` subset. It is intended for a new development/test database and is not a general migration runner or the supplied full target schema.
- `apps/api/tests/` and `apps/web/tests/`: backend unit/integration, Playwright interaction, and live browser smoke tests. Integration and live smoke require disposable PostgreSQL with the identity migration applied.
- `.github/workflows/ci.yml`: frontend lock/install/build and UI tests; backend locked install, migration, compile, unit/integration tests and coverage; browser smoke; hygiene and dependency security; release packaging after prerequisites pass.
- `scripts/`: release packaging and live browser smoke support.
`docs/release/RELEASE_README.md` describes the artifact and deployment prerequisite/rollback procedure.

The technical design's domain modules, adapters, workers, and additional deployables are target boundaries, not currently present applications or services. The current backend remains one FastAPI process.

Request and authentication flow

1. The browser requests `GET /api/runtime-config` from Next.js. The server reads `API_BASE_URL`, validates that it is an HTTP(S) origin without credentials/path/query/fragment, and returns the origin.
2. The browser calls `GET /auth/me` with credentials to restore a session. FastAPI hashes the cookie token and queries PostgreSQL for a live session and active user.
3. Sign-in sends email/password to `POST /auth/login`. The server verifies the Argon2 password hash (or a dummy hash for an unknown/unusable account), inserts only a SHA-256 digest of a newly generated opaque token in `auth_sessions`, and sets the browser cookie.
4. Sign-out calls `POST /auth/logout`; the backend revokes a matching active session and clears the cookie. Authorization for future protected endpoints must be enforced by the backend, not inferred from web UI state.

Cookie name is `finsight_session`; attributes are `HttpOnly`, `SameSite=Lax`, `Path=/`, a max age from `SESSION_TTL_MINUTES`, and `Secure` according to `COOKIE_SECURE` (defaultsto true when `APP_ENV=production`, otherwise false). Configure `WEB_ORIGIN` to the exact trusted frontend origin. CORS currently allows that origin, credentials, GET/POST, and `Content-Type`. Requests carrying a different `Origin` receive 403.

Implemented HTTP contracts

JSON error bodies from FastAPI use `{"detail": ...}`. Pydantic validation errors use FastAPI's structured `detail` array. Do not expose raw exceptions or database details.

Method and path	Auth	Request	Success	Relevant errors
`GET /api/runtime-config` (Next.js)	No	None	`200 {"apiBaseUrl":"https://api.example.test"}` (shape example)	`503 {"detail":"API_BASE_URL is not configured."}`; `500 {"detail":"API_BASE_URL is invalid."}`
`POST /auth/login`	No; valid allowed `Origin` if supplied	`{ "email": "analyst@example.test", "password": "…" }`; email validated, password length 1–1024	`200 {"user_id":"<uuid>","email":"analyst@example.test","display_name":"Analyst"}` plus session cookie	`401 {"detail":"Email or password is incorrect."}` for unknown/incorrect credentials; `403` disallowed origin; `422` invalid body; `503 {"detail":"Authentication service unavailable"}` for PostgreSQL errors
`GET /auth/me`	Yes, valid `finsight_session` cookie	None	`200` with the same `UserView` fields	`401 {"detail":"Authentication required"}` for absent/invalid/expired/revoked session; `503` database failure
`POST /auth/logout`	Cookie optional; valid allowed `Origin` if supplied	None	`204`, cookie deleted; matching session revoked when present	`403` disallowed origin; `503` if revocation hits a database error

The API also exposes FastAPI's generated OpenAPI document and interactive `/docs` in the current application. These are development/API inspection endpoints, not additional product capabilities. There is no `/auth/register`, password-reset, invitation, or health route.

Persistence and configuration ownership

PostgreSQL is the source of truth for the existing identity data. Migrations create `roles` (unique role key and constrained scope), `users` (case-insensitive unique email, status, nullable password hash and platform-role reference), `auth_sessions` (user FK with cascade, unique token digest, expiry/revocation timestamps), `permissions`, and `role_permissions`. The migration enforces that `users.platform_role_id` references a platform-scoped role. Only `platform_admin` and `platform.admin` are seeded. This is only the implemented subset; consult the schema/ERD reference for canonical broader target entities. No supplied full schema, workspace schema, or optional trading schema is applied by this app.

The current migration is a one-time SQL script with transaction boundaries, not a migration framework. Apply it only to a confirmed new/disposable development or test database, once. Production migration ownership and a general migration runner are unresolved deployment work. Keep application rollback separate from schema rollback; do not reverse a migration automatically. Prefer additive expand/contract migrations and preserve a schema usable by both old and new application versions. Review and test any destructive/contract migration with backup/restore and an explicit recovery owner.

Variable	Consumer	Meaning and safe baseline
`DATABASE_URL`	FastAPI	PostgreSQL connection string; required. Keep credentials in ignored local `.env` or an approved secret manager.
`WEB_ORIGIN`	FastAPI	Exact allowed browser origin; local example `http://localhost:3000`.
`SESSION_TTL_MINUTES`	FastAPI	Idle/session policy input, 1–10080 minutes; default 60.
`COOKIE_SECURE`	FastAPI	Explicit cookie Secure switch; production should be true.
`APP_ENV`	FastAPI	Environment marker; absent defaults to development for cookie behavior. Set production explicitly in deployment.
`API_BASE_URL`	Next.js server	API origin read at runtime; local example `http://localhost:8000`. No credentials or path.

`.env.example` and `apps/web/.env.example` contain placeholders/examples only. Add new configuration through an explicit variable name, purpose, validation, safe local example, deployment guidance, and tests; never add actual credentials or unsafe production defaults to tracked files.

Local development and test entry points

Follow the concrete setup in the root `README.md`. In summary, apply the identity migration to a new local database; install backend dependencies from `apps/api/requirements.lock` and run `python -m uvicorn app.main:app --app-dir apps/api --host 127.0.0.1 --port 8000`; install

frontend dependencies with `npm ci` under `apps/web` and run `npm run dev`. Atrusted operator can provision an account with `python -m app.provision_user EMAIL DISPLAY_NAME` from `apps/api`.

Test entry points documented in the README are backend pytest with coverage, `npm run test:ui`, and `python scripts/run_live_e2e.py`. The latter and PostgreSQL integration testsrequire a disposable migrated database; browser tests require Playwright Chromium. CI uses locked dependencies and test-only PostgreSQL values.

Compatibility and change rules

1. **HTTP contracts:** current API paths are unprefixed and have no formal `/v1` policy. Do not claim route versioning already exists. Preserve existing field meaning, status codes, cookie behavior, and generic auth errors. Additive optional response fields are generally safer than renaming/removing fields or changing error/status behavior. For an incompatible contract, first propose/version the public API strategy, document a deprecation window and consumer migration, update OpenAPI/examples and tests, and obtain review beforerelease.
2. **Errors and privacy:** retain stable status classes and JSON `detail` envelope; do not return stack traces, SQL/provider detail, passwords, raw session tokens, or account-existence distinctions. Validation remains 422, authentication 401, origin rejection 403, and unavailable persistence 503 for the current auth slice.
3. **Database:** schema changes must be new versioned migrations; do not edit the reference SQL/PDF or rewrite applied migration history. Validate forward migration on disposable PostgreSQL. Prefer backward-compatible expand/contract; an app rollback does not imply a database downgrade is safe.
4. **Environment:** runtime configuration is separated from code. Validate new settings, document required/optional status and safe defaults, keep real values out of templates/logs, and add ignored local values only to local env files/secret managers.
5. **Dependencies:** exact direct pins and lockfiles are authoritative for installs (`npm ci`, pip lock files). Update manifests and locks together, run consistency/security checks and relevant builds/tests, and review install scripts rather than broadly permitting them.
6. **Artifacts:** release packages carry a version and commit SHA when built from a commit, timestamp/checksum metadata, and a separate checksum. A local artifact with no commitSHA is not a committed release. Promote the same verified artifact; retain previous known-good artifacts for rollback. Deployment is not configured in this repository.
7. **Acceptance:** relevant unit, PostgreSQL integration, frontend interaction and live smoke checks must pass as applicable. CI also checks build, dependency/security and repository hygiene. Do not call GitHub checks passed based on local runs; record unexecuted or infrastructure-blocked checks honestly.

Architecture Decision Records

- [ADR-0001: Modular Next.js and FastAPI application](adr/0001-modular-nextjs-fastapi.md)

- [ADR-0002: PostgreSQL as structured source of
truth](adr/0002-postgresql-source-of-truth.md)
- [ADR-0003: Opaque database-validated
sessions](adr/0003-opaque-server-validated-sessions.md)
- [ADR-0004: Versioned migrations and environment
configuration](adr/0004-migrations-and-environment-configuration.md)
- [ADR-0005: CI quality and versioned release
artifacts](adr/0005-ci-quality-and-release-artifacts.md)

===== CURRENT TRACKING FILE ENDINGS =====
the full originally specified protected-resource acceptance case is
not established.
- TC_AUTHZ_003: retain the previously recorded historical PASS; the
latest suite is not by itself a new idle-timeout-specific execution.
- TC_PRIV_002: DEFERRED / NOT RUN under the approved platform-only scope.
- TC_SYS_001: NOT RUN for the full planned system acceptance workflow.

Overall FIN-15 status: PARTIAL. Do not mark accepted or VERIFIED COMPLETE
until the remaining applicable acceptance criteria and evidence are
reconciled. Workspace roles, membership, and resource-level authorization
remain deferred. FIN-14 is outside this verification scope.

No commit or push was performed as part of this verification.

FIN-14 / FS-01-03 — AUTHENTICATION SECURITY IMPLEMENTATION (2026-10-21)
Status: PARTIAL — acceptance gaps remain.

Implemented:
- Added migration 0003_auth_audit_and_login_throttling.sql for persistent
authentication audit events and PostgreSQL-backed login throttle state.
- Added apps/api/app/audit.py for authentication audit insertion and
redaction of secret-like fields.
- Added apps/api/app/throttle.py for validated configuration, HMAC-derived
account/IP keys, persistent failure counting, and temporary blocking.
- Integrated login success/failure/denied audit events and throttle checks
into the login flow; logout revokes the presented active session and
records an audit event without storing the session token.
- Updated .env.example, release configuration metadata, and release README
for the throttle settings and migration.
- Added tests in test_audit_service.py, test_throttle.py, and
test_fin14_auth_security.py; extended authentication integration coverage.

Verification evidence:
- Focused auth/security suite: 66 passed, 0 failed, 10 warnings.
- Full API suite: 82 passed, 0 failed, 19 warnings.
- git diff --check passed in the recorded verification.
- Local release package included migration 0003 and the new audit/throttle modules; archive checksum verification passed.
- Test database: finsight_fin15_test. Synthetic test records were cleaned up by the integration tests; do not treat this as production deployment evidence.

Remaining gaps:
- TC_AUTHSEC_001 remains PARTIAL: end-to-end correlation propagation and append-only enforcement are not established.
- TC_AUTHSEC_002 remains PARTIAL: the complete login-level lockout-expiry and successful-retry flow needs explicit integration coverage.
- TC_AUTHSEC_003 remains BLOCKED: MFA policy/provider and challenge flow are not implemented; inspection and termination of other active sessions are also absent. Current-session logout does not satisfy cross-session controls.
- FIN-14 must not be marked accepted or VERIFIED COMPLETE until the relevant acceptance criteria are implemented, tested, and reconciled.

No source requirements, reference SQL/schema, diagrams, or original test plan were modified. No commit or push was performed.

FIN-14 — LATEST CORRELATION AND API VERIFICATION (2026-10-21)

- Focused FIN-14 security suite: 7 passed, 0 failed, 3 warnings.
- Full API suite: 86 passed, 0 failed, 19 warnings.
- Integration tests verify supplied and generated correlation IDs match audit records.
- CORS permits the X-Correlation-ID request header.
- git diff --check passed.
- Open: database-level append-only audit enforcement; complete login-level throttle expiry and successful retry; MFA policy/provider and challenge enforcement; cross-session inspection and termination.
- Overall FIN-14 status remains PARTIAL. No commit or push performed.
the individual acceptance criteria. TC_AUTHZ_001 and TC_AUTHZ_002 remain PARTIAL; TC_AUTHZ_003 is historical PASS only; TC_PRIV_002 is DEFERRED / NOT RUN under the approved platform-only scope; TC_SYS_001 remains NOT RUN for the full planned system acceptance workflow.

Overall FIN-15 / FS-01-02 status: PARTIAL, not accepted or VERIFIED COMPLETE. Workspace/resource authorization remains deferred. FIN-14 is unchanged.

FIN-14 / FS-01-03 — AUTHENTICATION SECURITY EXECUTION ADDENDUM

Execution date: 2026-10-21
Tester/owner: Codex
Environment: dedicated local PostgreSQL test database finsight_fin15_test.

Observed verification:
- Focused authentication/security suites: 66 passed, 0 failed, 10 warnings.
- Full API suite: 82 passed, 0 failed, 19 warnings.
- Local release-package validation included migration 0003, audit/throttle modules, metadata, and checksum verification.
- Warnings were reported as test-client deprecations; no test failures occurred.

Acceptance-case reconciliation:
- TC_AUTHSEC_001: PARTIAL. Login success/failure and logout audit events, plus secret-redaction behavior, have tests. Complete correlation propagation and database-level append-only enforcement have not been established.
- TC_AUTHSEC_002: PARTIAL. Configurable PostgreSQL-backed throttling and repeated-failure blocking are implemented. Throttle behavior/expiry has been tested, but the complete login-level expiry-and-successful-retry flow is not established by the recorded integration tests.
- TC_AUTHSEC_003: BLOCKED. MFA enforcement and listing/terminating other active sessions are not implemented. Existing logout only revokes the session presented to that request.

Overall FIN-14 / FS-01-03 status: PARTIAL, not accepted or VERIFIED COMPLETE.
No source requirements, schema reference documents, diagrams, or original test plan were changed. No commit or push was performed.

FIN-14 — LATEST VERIFICATION ADDENDUM (2026-10-21)

- Focused FIN-14 security suite: 7 passed, 0 failed, 3 warnings.
- Full API suite: 86 passed, 0 failed, 19 warnings.
- Correlation ID propagation is verified for supplied and generated IDs.
- TC_AUTHSEC_001 remains PARTIAL: database-level append-only enforcement is unverified.
- TC_AUTHSEC_002 remains PARTIAL: complete login-level expiry and successful retry are unestablished.
- TC_AUTHSEC_003 remains BLOCKED: MFA and cross-session inspection/termination are not implemented.

FIN-14 — AUTHENTICATION SECURITY TRACEABILITY ADDENDUM (2026-10-21)

Story: FIN-14 / FS-01-03 — Protect Accounts and Audit Authentication.

- TC_AUTHSEC_001 — PARTIAL.

Evidence: apps/api/tests/test_fin14_auth_security.py,
apps/api/tests/test_audit_service.py, apps/api/app/audit.py.
Tested authentication audit events and secret redaction. End-to-end
correlation propagation and append-only enforcement remain unverified.

- TC_AUTHSEC_002 — PARTIAL.
Evidence: apps/api/tests/test_throttle.py,
apps/api/tests/test_fin14_auth_security.py, apps/api/app/throttle.py.
Configuration validation and repeated-failure blocking are tested;
complete login-level expiry-and-successful-retry acceptance remains unverified.

- TC_AUTHSEC_003 — BLOCKED.
Evidence: apps/api/app/main.py and apps/api/tests/test_auth.py.
Current-session logout/revocation is implemented; MFA and inspection/
termination of other active sessions are not implemented.

Latest recorded execution: focused auth/security suite 66 passed;
full API suite 82 passed. Passing test counts do not establish complete
FIN-14 acceptance. Overall story status remains PARTIAL.

FIN-14 — LATEST VERIFICATION ADDENDUM (2026-10-21)

- Focused FIN-14 security suite: 7 passed, 0 failed, 3 warnings.
- Full API suite: 86 passed, 0 failed, 19 warnings.
- Supplied and generated correlation ID propagation is covered by integration tests.
- TC_AUTHSEC_001 remains PARTIAL: append-only enforcement is unverified.
- TC_AUTHSEC_002 remains PARTIAL: complete login-level expiry and successful retry are unestablished.
- TC_AUTHSEC_003 remains BLOCKED: MFA and cross-session controls are not implemented.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % printf '\n===== MFA REPOSITORY PUBLIC FUNCTIONS =====\n'
grep -n -E '^def (create_pending_totp_factor|activate_totp_factor|verify_active_totp_factor|revoke_totp_factor|list_totp_factors)' apps/api/app/mfa_repository.py

printf '\n===== MFA REPOSITORY IMPLEMENTATION =====\n'
sed -n '1,340p' apps/api/app/mfa_repository.py

printf '\n===== MFA SERVICE =====\n'
sed -n '1,240p' apps/api/app/mfa_service.py

printf '\n===== AUTH ROUTES AND EXISTING MFA TESTS =====\n'

```
grep -n -E '^@app\\.(get|post|delete|put|patch)|^def (login|verify_mfa_login|logout|.*session)'
apps/api/app/main.py
grep -n -E 'mfa_|MFA|/auth/mfa' apps/api/tests/test_auth.py

===== MFA REPOSITORY PUBLIC FUNCTIONS =====
33:def create_pending_totp_factor(
70:def activate_totp_factor(
141:def verify_active_totp_factor(
207:def revoke_totp_factor(*, user_id: UUID, factor_id: UUID) -> bool:
233:def list_totp_factors(*, user_id: UUID) -> list[dict[str, Any]]:

===== MFA REPOSITORY IMPLEMENTATION =====
"""Database-backed TOTP factor operations.

The database stores only opaque secret references. Callers must not log
provisioning URIs, TOTP seeds, challenge tokens, or submitted codes.
"""

from __future__ import annotations

from datetime import UTC, datetime
from typing import Any
from uuid import UUID

import psycopg
from psycopg.rows import dict_row

from app.database import connect
from app.mfa_service import matching_totp_step
from app.secret_store import SecretStore


class MFAFactorError(RuntimeError):
    """Base class for safe MFA factor operation failures."""


class MFAFactorNotFoundError(MFAFactorError):
    """The requested factor is unavailable."""


class MFAInvalidCodeError(MFAFactorError):
    """The submitted TOTP code is invalid or already used."""
```

```
def create_pending_totp_factor(
    *,
    user_id: UUID,
    account_name: str,
    store: SecretStore,
    label: str | None = None,
) -> tuple[UUID, str]:
    """Create a pending TOTP factor; return its ID and one-time URI."""
    from app.mfa_service import create_totp_provisioning

    secret_ref, provisioning_uri = create_totp_provisioning(
        account_name=account_name,
        store=store,
    )

    try:
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                """
                INSERT INTO mfa_factors
                    (user_id, factor_type, label, secret_ref, factor_status)
                VALUES (%s, 'totp', %s, %s, 'pending')
                RETURNING mfa_factor_id
                """,
                (user_id, label, secret_ref),
            )
            factor_id = cur.fetchone()[0]
    except Exception:
        try:
            store.delete(secret_ref)
        except Exception:
            pass
        raise

    return factor_id, provisioning_uri


def activate_totp_factor(
    *,
    user_id: UUID,
    factor_id: UUID,
    supplied_code: str,
    store: SecretStore,
    at_time: float | None = None,
```

```
) -> None:
    """Activate a pending factor only when its TOTP code is valid."""
    with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
        cur.execute(
            """
            SELECT mfa_factor_id, secret_ref
            FROM mfa_factors
            WHERE mfa_factor_id = %s
              AND user_id = %s
              AND factor_type = 'totp'
              AND factor_status = 'pending'
            FOR UPDATE
            """,
            (factor_id, user_id),
        )
        factor = cur.fetchone()

        if not factor or not factor["secret_ref"]:
            raise MFAFactorNotFoundError("Pending MFA factor not found")

        secret = store.get(factor["secret_ref"])
        step = matching_totp_step(
            secret=secret,
            supplied_code=supplied_code,
            at_time=at_time,
        )
        if step is None:
            raise MFAInvalidCodeError("Invalid verification code")

        now = datetime.now(UTC)
        cur.execute(
            """
            INSERT INTO mfa_totp_replay_guard
                (mfa_factor_id, last_accepted_step, updated_at)
            VALUES (%s, %s, %s)
            ON CONFLICT (mfa_factor_id) DO UPDATE
            SET last_accepted_step = EXCLUDED.last_accepted_step,
                updated_at = EXCLUDED.updated_at
            WHERE mfa_totp_replay_guard.last_accepted_step
                 < EXCLUDED.last_accepted_step
            RETURNING mfa_factor_id
            """,
            (factor_id, step, now),
        )
```

```
        if cur.fetchone() is None:
            raise MFAInvalidCodeError("Invalid verification code")

        cur.execute(
            """
            UPDATE mfa_factors
            SET factor_status = 'active',
                enrolled_at = %s,
                last_used_at = %s
            WHERE mfa_factor_id = %s
              AND user_id = %s
              AND factor_status = 'pending'
            RETURNING mfa_factor_id
            """,
            (now, now, factor_id, user_id),
        )
        if cur.fetchone() is None:
            raise MFAFactorNotFoundError("Pending MFA factor not found")


def verify_active_totp_factor(
    *,
    user_id: UUID,
    factor_id: UUID,
    supplied_code: str,
    store: SecretStore,
    at_time: float | None = None,
) -> bool:
    """Verify an active factor and atomically prevent time-step replay."""
    with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
        cur.execute(
            """
            SELECT mfa_factor_id, secret_ref
            FROM mfa_factors
            WHERE mfa_factor_id = %s
              AND user_id = %s
              AND factor_type = 'totp'
              AND factor_status = 'active'
            """,
            (factor_id, user_id),
        )
        factor = cur.fetchone()

        if not factor or not factor["secret_ref"]:
```

```
        raise MFAFactorNotFoundError("Active MFA factor not found")

    secret = store.get(factor["secret_ref"])
    step = matching_totp_step(
        secret=secret,
        supplied_code=supplied_code,
        at_time=at_time,
    )
    if step is None:
        return False

    now = datetime.now(UTC)
    cur.execute(
        """
        INSERT INTO mfa_totp_replay_guard
            (mfa_factor_id, last_accepted_step, updated_at)
        VALUES (%s, %s, %s)
        ON CONFLICT (mfa_factor_id) DO UPDATE
        SET last_accepted_step = EXCLUDED.last_accepted_step,
            updated_at = EXCLUDED.updated_at
        WHERE mfa_totp_replay_guard.last_accepted_step
            < EXCLUDED.last_accepted_step
        RETURNING mfa_factor_id
        """,
        (factor_id, step, now),
    )
    if cur.fetchone() is None:
        return False

    cur.execute(
        """
        UPDATE mfa_factors
        SET last_used_at = %s
        WHERE mfa_factor_id = %s
          AND user_id = %s
          AND factor_status = 'active'
        """,
        (now, factor_id, user_id),
    )
    return cur.rowcount == 1


def revoke_totp_factor(*, user_id: UUID, factor_id: UUID) -> bool:
    """Revoke an owned TOTP factor and remove its replay-guard state."""
```

```
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """
            UPDATE mfa_factors
            SET factor_status = 'revoked'
            WHERE mfa_factor_id = %s
              AND user_id = %s
              AND factor_type = 'totp'
              AND factor_status IN ('pending', 'active')
            RETURNING mfa_factor_id
            """,
            (factor_id, user_id),
        )
        row = cur.fetchone()
        if row is None:
            return False

        cur.execute(
            "DELETE FROM mfa_totp_replay_guard WHERE mfa_factor_id = %s",
            (factor_id,),
        )
        return True


def list_totp_factors(*, user_id: UUID) -> list[dict[str, Any]]:
    """List factor metadata only; never return secret references."""
    with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
        cur.execute(
            """
            SELECT mfa_factor_id, label, factor_status, enrolled_at,
                   last_used_at, created_at
            FROM mfa_factors
            WHERE user_id = %s AND factor_type = 'totp'
            ORDER BY created_at DESC
            """,
            (user_id,),
        )
        return list(cur.fetchall())

===== MFA SERVICE =====
"""Provider-neutral TOTP enrollment and verification services.

Provisioning material is returned only to the caller performing enrollment.
Callers must never log the returned URI or the underlying TOTP secret.
```

```
"""

from __future__ import annotations

import base64
import hmac
from urllib.parse import quote, urlencode

from app.secret_store import SecretStore
from app.totp import generate_totp_secret, totp_code, verify_totp


DEFAULT_ISSUER = "FinSight AI"


def create_totp_provisioning(
    *,
    account_name: str,
    issuer: str = DEFAULT_ISSUER,
    store: SecretStore,
) -> tuple[str, str]:
    """Store a new TOTP seed and return (opaque reference, provisioning URI).

    The caller must persist only the reference. The URI contains the seed and
    must be shown only in the authenticated, one-time enrollment response.
    """
    account_name = account_name.strip()
    issuer = issuer.strip()

    if not account_name or not issuer:
        raise ValueError("Account name and issuer are required")

    secret = generate_totp_secret()
    reference = store.put(secret)

    try:
        encoded_secret = base64.b32encode(secret).decode("ascii").rstrip("=")
        label = f"{issuer}:{account_name}"
        query = urlencode(
            {
                "secret": encoded_secret,
                "issuer": issuer,
                "algorithm": "SHA1",
                "digits": 6,
```

```
                "period": 30,
            }
        )
        uri = f"otpauth://totp/{quote(label, safe='')}?{query}"
        return reference, uri
    except Exception:
        store.delete(reference)
        raise


def verify_totp_code(
    *,
    store: SecretStore,
    secret_ref: str,
    supplied_code: str,
    at_time: float | None = None,
) -> bool:
    """Verify a TOTP code without returning or exposing the underlying seed."""
    secret = store.get(secret_ref)
    return verify_totp(secret, supplied_code, at_time=at_time)


def current_totp_step(at_time: float | None = None, *, period: int = 30) -> int:
    """Return the current TOTP time-step index."""
    if period <= 0:
        raise ValueError("TOTP period must be positive")

    import time

    timestamp = time.time() if at_time is None else at_time
    if timestamp < 0:
        raise ValueError("TOTP timestamp cannot be negative")

    return int(timestamp) // period


def matching_totp_step(
    *,
    secret: bytes,
    supplied_code: str,
    at_time: float | None = None,
    period: int = 30,
    digits: int = 6,
    window: int = 1,
```

```
) -> int | None:
    """Return the matching counter for replay protection, or None."""
    if not isinstance(supplied_code, str):
        return None
    if len(supplied_code) != digits or not supplied_code.isascii() or not supplied_code.isdigit():
        return None
    if period <= 0 or window < 0 or window > 2:
        raise ValueError("Invalid TOTP verification configuration")

    import time

    timestamp = time.time() if at_time is None else at_time
    if timestamp < 0:
        return None

    counter = int(timestamp) // period
    for offset in range(-window, window + 1):
        step = counter + offset
        if step < 0:
            continue
        candidate = totp_code(secret, step * period, period=period, digits=digits)
        if hmac.compare_digest(candidate, supplied_code):
            return step

    return None

===== AUTH ROUTES AND EXISTING MFA TESTS =====
210:def login(payload: LoginRequest, request: Request, response: Response):
410:def verify_mfa_login(
504:def logout(
567:def list_active_sessions(
601:def terminate_session(
299:# FIN-14 MFA endpoint integration tests
302:def mfa_identity(identity, monkeypatch):
305:    from app.mfa_repository import (
314:    monkeypatch.setattr(app.state, "mfa_secret_store", store, raising=False)
318:            "UPDATE users SET mfa_required = TRUE WHERE user_id = %s",
345:def test_mfa_login_returns_challenge_without_session_cookie(
346:    client, mfa_identity
348:    _, email, _ = mfa_identity
357:    assert body["mfa_required"] is True
363:def test_mfa_required_login_without_active_factor_is_denied(client, identity):
367:            "UPDATE users SET mfa_required = TRUE WHERE user_id = %s",
380:def test_valid_totp_verification_creates_session(client, mfa_identity):
```

```
385:    user_id, email, secret = mfa_identity
394:        "/auth/mfa/verify",
408:def test_invalid_totp_does_not_create_session(client, mfa_identity):
409:    _, email, _ = mfa_identity
417:        "/auth/mfa/verify",
425:def test_mfa_challenge_cannot_be_reused(client, mfa_identity):
430:    _, email, secret = mfa_identity
440:        "/auth/mfa/verify",
446:        "/auth/mfa/verify",
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % printf '\n===== MAIN.PY
IMPORTS AND MODELS =====\n'
sed -n '1,85p' apps/api/app/main.py
sed -n '320,430p' apps/api/app/main.py

printf '\n===== AUTH ROUTE PATTERNS =====\n'
sed -n '480,635p' apps/api/app/main.py

printf '\n===== API TEST FIXTURES =====\n'
sed -n '1,100p' apps/api/tests/test_auth.py

printf '\n===== MFA REPOSITORY AND SECRET-STORE ERRORS =====\n'
grep -n -E 'class MFA|class SecretStore|def secret_store_from_environment'
apps/api/app/mfa_repository.py apps/api/app/secret_store.py

===== MAIN.PY IMPORTS AND MODELS =====
"""FinSight AI API with PostgreSQL-backed browser sessions."""

from __future__ import annotations

import os
import secrets
from datetime import UTC, datetime
from pathlib import Path
from typing import Annotated, Any
from uuid import UUID, uuid4

from app.audit import record_auth_event
from app.throttle import (
    is_blocked,
    load_throttle_config,
    make_throttle_key,
    record_failure,
    reset_bucket,
```

```
)

import psycopg
from dotenv import load_dotenv
from fastapi import Cookie, Depends, FastAPI, HTTPException, Request, Response, status
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel, EmailStr, Field
from psycopg.rows import dict_row

from app.auth import hash_password, new_session_token, session_expiry, token_digest,
verify_password
from app.database import connect
from app.secret_store import (
    SecretStore,
    SecretStoreConfigurationError,
    SecretStoreError,
    secret_store_from_environment,
)
from app.mfa_challenge_repository import (
    MFAChallengeInvalidError,
    issue_mfa_challenge,
    verify_mfa_challenge,
)

load_dotenv(Path(__file__).resolve().parents[3] / ".env")

app = FastAPI(title="FinSight AI API", version="0.2.0")


def _get_mfa_secret_store(request: Request) -> SecretStore:
    """Return the app-scoped secret store; never silently change backends."""
    store = getattr(request.app.state, "mfa_secret_store", None)
    if store is None:
        store = secret_store_from_environment()
        request.app.state.mfa_secret_store = store
    return store




@app.middleware("http")
async def auth_correlation_middleware(request: Request, call_next):
    """Return a correlation ID on authentication responses, including errors."""
    if (
```

```
        request.url.path not in {"/auth/login", "/auth/logout"}
        and not request.url.path.startswith("/auth/sessions")
    ):
        return await call_next(request)

    supplied = request.headers.get("X-Correlation-ID", "").strip()
    try:
        correlation_id = UUID(supplied) if supplied else uuid4()
    except ValueError:
        from starlette.responses import JSONResponse

        return JSONResponse(
            status_code=400,
            content={"detail": "Invalid X-Correlation-ID"},
        )

    request.state.auth_correlation_id = correlation_id
    response = await call_next(request)
    response.headers["X-Correlation-ID"] = str(correlation_id)
    return response


web_origin = os.environ.get("WEB_ORIGIN", "http://localhost:3000")
app.add_middleware(
                            conn,
                            action_key="auth.login",
                            outcome="denied",
                            actor_user_id=user["user_id"],
                            details={"reason": "mfa_factor_unavailable"},
                            correlation_id=correlation_id,
                        )
                    else:
                        mfa_challenge_allowed = True
                else:
                    session_token, digest = new_session_token()
                    expires_at = session_expiry(_ttl_minutes())

                    with conn.cursor() as cur:
                        cur.execute(
                            """INSERT INTO auth_sessions
                                   (user_id, refresh_token_hash, expires_at)
                               VALUES (%s, %s, %s)""",
                            (user["user_id"], digest, expires_at),
                        )
```

```
                    now = datetime.now(UTC)
                    cur.execute(
                        """UPDATE users
                           SET last_login_at = %s, updated_at = %s
                           WHERE user_id = %s""",
                        (now, now, user["user_id"]),
                    )

                record_auth_event(
                    conn,
                    action_key="auth.login",
                    outcome="success",
                    actor_user_id=user["user_id"],
                    details={"method": "password"},
                    correlation_id=correlation_id,
                )

except psycopg.Error as exc:
    raise HTTPException(
        status_code=503,
        detail="Authentication service unavailable",
    ) from exc

if not login_succeeded:
    raise HTTPException(status_code=401, detail=GENERIC_LOGIN_ERROR)

if mfa_required and not mfa_challenge_allowed:
    raise HTTPException(status_code=401, detail=GENERIC_LOGIN_ERROR)

if mfa_required:
    try:
        _get_mfa_secret_store(request)
        challenge_token, expires_at = issue_mfa_challenge(
            user_id=user["user_id"],
            correlation_id=correlation_id,
        )
    except (SecretStoreError, SecretStoreConfigurationError) as exc:
        raise HTTPException(
            status_code=503,
            detail="MFA service unavailable",
        ) from exc
    except psycopg.Error as exc:
        raise HTTPException(
            status_code=503,
```

```
                detail="Authentication service unavailable",
            ) from exc

        return {
            "mfa_required": True,
            "challenge_token": challenge_token,
            "expires_at": expires_at,
        }

    response.set_cookie(
        SESSION_COOKIE,
        session_token,
        httponly=True,
        secure=_secure_cookie(),
        samesite="lax",
        max_age=_ttl_minutes() * 60,
        path="/",
    )
    return UserView(
        user_id=user["user_id"],
        email=user["email"],
        display_name=user["display_name"],
    )


@app.post("/auth/mfa/verify", response_model=UserView)
def verify_mfa_login(
    payload: MFAVerifyRequest,
    request: Request,
    response: Response,
) -> UserView:
    _check_origin(request)
    correlation_id = _auth_correlation_id(request)
    response.headers["X-Correlation-ID"] = str(correlation_id)

    try:
        store = _get_mfa_secret_store(request)
        user_id = verify_mfa_challenge(
            challenge_token=payload.challenge_token,
            supplied_code=payload.code,
            store=store,
            correlation_id=correlation_id,
        )
```

```
        token, digest = new_session_token()
        expires_at = session_expiry(_ttl_minutes())
        now = datetime.now(UTC)

===== AUTH ROUTE PATTERNS =====
        ) from exc
    except psycopg.Error as exc:
        raise HTTPException(
            status_code=503,
            detail="Authentication service unavailable",
        ) from exc

    response.set_cookie(
        SESSION_COOKIE,
        token,
        httponly=True,
        secure=_secure_cookie(),
        samesite="lax",
        max_age=_ttl_minutes() * 60,
        path="/",
    )
    return UserView(
        user_id=user["user_id"],
        email=user["email"],
        display_name=user["display_name"],
    )


@app.post("/auth/logout", status_code=204)
def logout(
    request: Request,
    response: Response,
    session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None,
) -> Response:
    _check_origin(request)
    correlation_id = _auth_correlation_id(request)
    response.headers['X-Correlation-ID'] = str(correlation_id)

    try:
        with connect() as conn:
            actor_user_id = None

            if session_token:
                with conn.cursor() as cur:
```

```
                cur.execute(
                    """UPDATE auth_sessions
                       SET revoked_at = %s, revoke_reason = 'logout'
                       WHERE refresh_token_hash = %s
                         AND revoked_at IS NULL
                       RETURNING user_id""",
                    (datetime.now(UTC), token_digest(session_token)),
                )
                row = cur.fetchone()
                if row:
                    actor_user_id = row[0]

            record_auth_event(
                conn,
                action_key="auth.logout",
                outcome="success" if actor_user_id else "denied",
                actor_user_id=actor_user_id,
                details={"reason": "logout"},
                correlation_id=correlation_id,
            )
    except psycopg.Error as exc:
        raise HTTPException(
            status_code=503,
            detail="Authentication service unavailable",
        ) from exc

    response.delete_cookie(
        SESSION_COOKIE,
        path="/",
        httponly=True,
        secure=_secure_cookie(),
        samesite="lax",
    )
    response.status_code = status.HTTP_204_NO_CONTENT
    return response


class SessionView(BaseModel):
    session_id: UUID
    created_at: datetime
    last_seen_at: datetime
    expires_at: datetime
    ip_address: str | None
    user_agent: str | None
```

```
    is_current: bool


@app.get("/auth/sessions", response_model=list[SessionView])
def list_active_sessions(
    request: Request,
    user: CurrentUser,
    session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None,
) -> list[SessionView]:
    correlation_id = _auth_correlation_id(request)
    token_hash = token_digest(session_token) if session_token else None
    now = datetime.now(UTC)

    try:
        with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
            cur.execute(
                """SELECT session_id, created_at, last_seen_at, expires_at,
                          host(ip_address) AS ip_address, user_agent,
                          (refresh_token_hash = %s) AS is_current
                   FROM auth_sessions
                   WHERE user_id = %s
                     AND revoked_at IS NULL
                     AND expires_at > %s
                     AND last_seen_at > %s - make_interval(mins => %s)
                   ORDER BY created_at DESC""",
                (token_hash, user["user_id"], now, now, _ttl_minutes()),
            )
            rows = cur.fetchall()
    except psycopg.Error as exc:
        raise HTTPException(
            status_code=503,
            detail="Authentication service unavailable",
        ) from exc

    return [SessionView(**row) for row in rows]


@app.delete("/auth/sessions/{session_id}", status_code=204)
def terminate_session(
    session_id: UUID,
    request: Request,
    response: Response,
    user: CurrentUser,
) -> Response:
```

```
    _check_origin(request)
    correlation_id = _auth_correlation_id(request)
    response.headers["X-Correlation-ID"] = str(correlation_id)
    now = datetime.now(UTC)
    terminated = False

    try:
        with connect() as conn:
            with conn.cursor() as cur:
                cur.execute(
                    """UPDATE auth_sessions
                       SET revoked_at = %s, revoke_reason = 'user_terminated'
                       WHERE session_id = %s
                         AND user_id = %s
                         AND revoked_at IS NULL
                         AND expires_at > %s
                       RETURNING session_id""",
                    (now, session_id, user["user_id"], now),
                )
                terminated = cur.fetchone() is not None

            record_auth_event(
                conn,
                action_key="auth.session.terminate",
                outcome="success" if terminated else "denied",
                actor_user_id=user["user_id"],
                details={
                    "session_id": str(session_id),
                    "reason": "user_terminated" if terminated else "not_found_or_inactive",

===== API TEST FIXTURES =====
from __future__ import annotations

import secrets
from uuid import uuid4

import pytest
from fastapi.testclient import TestClient
from psycopg.rows import dict_row

from app.auth import hash_password, token_digest, verify_password
from app.throttle import make_throttle_key
from app.database import connect
from app.main import SESSION_COOKIE, app
```

```
TEST_PASSWORD = secrets.token_urlsafe(32)


@pytest.fixture
def identity():
    email = f"{uuid4().hex}@example.com"
    encoded = hash_password(TEST_PASSWORD)
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "INSERT INTO users (email, password_hash, display_name, account_status) VALUES
(%s, %s, %s, 'active') RETURNING user_id",
            (email, encoded, "Synthetic Analyst"),
        )
        user_id = cur.fetchone()[0]
    try:
        yield user_id, email, encoded
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                "UPDATE auth_sessions SET revoked_at = COALESCE(revoked_at, now()), "
                "revoke_reason = COALESCE(revoke_reason, 'test_cleanup') "
                "WHERE user_id = %s",
                (user_id,),
            )
            cur.execute(
                "UPDATE users SET account_status = 'deleted', deleted_at =
COALESCE(deleted_at, now()) "
                "WHERE user_id = %s",
                (user_id,),
            )


@pytest.fixture
def client(monkeypatch):
    monkeypatch.setenv("SESSION_TTL_MINUTES", "60")
    monkeypatch.setenv("COOKIE_SECURE", "false")
    monkeypatch.setenv("WEB_ORIGIN", "http://localhost:3000")
    throttle_secret = "test-only-secret-material-at-least-32-bytes"
    monkeypatch.setenv("AUTH_THROTTLE_SECRET", throttle_secret)
    ip_key = make_throttle_key(
        throttle_secret.encode("utf-8"), "ip", "testclient"
    )
```

```
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "DELETE FROM auth_login_throttle WHERE scope_type = 'ip' AND key_hash = %s",
            (ip_key,),
        )

    try:
        with TestClient(app) as test_client:
            yield test_client
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                "DELETE FROM auth_login_throttle WHERE scope_type = 'ip' AND key_hash = %s",
                (ip_key,),
            )


def test_password_hash_is_argon2_and_never_plaintext():
    encoded = hash_password(TEST_PASSWORD)
    assert encoded.startswith("argon2id")
    assert encoded != TEST_PASSWORD
    assert verify_password(encoded, TEST_PASSWORD)
    assert not verify_password(encoded, "wrong")


def test_login_creates_protected_hashed_expiring_session(client, identity, caplog, capsys):
    user_id, email, encoded = identity
    response = client.post("/auth/login", json={"email": email, "password": TEST_PASSWORD})
    assert response.status_code == 200
    assert response.json() == {"user_id": str(user_id), "email": email, "display_name": "Synthetic
Analyst"}
    assert TEST_PASSWORD not in response.text and encoded not in response.text
    cookie = response.cookies.get(SESSION_COOKIE)
    assert cookie
    set_cookie = response.headers["set-cookie"].lower()
    assert "httponly" in set_cookie and "samesite=lax" in set_cookie and "path=/" in set_cookie
    with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
        cur.execute("SELECT refresh_token_hash, expires_at, revoked_at FROM auth_sessions
WHERE user_id = %s", (user_id,))
        stored = cur.fetchone()
    assert stored["refresh_token_hash"] == token_digest(cookie)
    assert stored["refresh_token_hash"] != cookie
    assert stored["expires_at"] is not None and stored["revoked_at"] is None
    protected = client.get("/auth/me", cookies={SESSION_COOKIE: cookie})
```

```
    assert protected.status_code == 200 and protected.json()["user_id"] == str(user_id)
    captured = capsys.readouterr()
    assert TEST_PASSWORD not in caplog.text + captured.out + captured.err

===== MFA REPOSITORY AND SECRET-STORE ERRORS =====
apps/api/app/mfa_repository.py:21:class MFAFactorError(RuntimeError):
apps/api/app/mfa_repository.py:25:class MFAFactorNotFoundError(MFAFactorError):
apps/api/app/mfa_repository.py:29:class MFAInvalidCodeError(MFAFactorError):
apps/api/app/secret_store.py:21:class SecretStoreError(RuntimeError):
apps/api/app/secret_store.py:29:class SecretStoreConfigurationError(SecretStoreError):
apps/api/app/secret_store.py:33:class SecretStore(Protocol):
apps/api/app/secret_store.py:147:def secret_store_from_environment() -> SecretStore:
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % printf '\n===== REQUEST
MODELS AND ORIGIN CHECK =====\n'
grep -n -E '^class (MFAVerifyRequest|.*MFA.*Request|.*Factor.*View)|^def _check_origin|^def
_auth_correlation_id' apps/api/app/main.py

printf '\n===== MFA CHALLENGE RESPONSE SHAPE =====\n'
sed -n '330,370p' apps/api/app/main.py

printf '\n===== SECRET STORE FAILURE BEHAVIOR =====\n'
sed -n '1,175p' apps/api/app/secret_store.py

printf '\n===== REPOSITORY STATUS =====\n'
git status --short

===== REQUEST MODELS AND ORIGIN CHECK =====
110:class MFAVerifyRequest(BaseModel):
131:def _check_origin(request: Request) -> None:
137:def _auth_correlation_id(request: Request) -> UUID:

===== MFA CHALLENGE RESPONSE SHAPE =====
                    session_token, digest = new_session_token()
                    expires_at = session_expiry(_ttl_minutes())

                    with conn.cursor() as cur:
                        cur.execute(
                            """INSERT INTO auth_sessions
                                    (user_id, refresh_token_hash, expires_at)
                                VALUES (%s, %s, %s)""",
                            (user["user_id"], digest, expires_at),
                        )
                        now = datetime.now(UTC)
```

```
                    cur.execute(
                        """UPDATE users
                           SET last_login_at = %s, updated_at = %s
                           WHERE user_id = %s""",
                        (now, now, user["user_id"]),
                    )

                record_auth_event(
                    conn,
                    action_key="auth.login",
                    outcome="success",
                    actor_user_id=user["user_id"],
                    details={"method": "password"},
                    correlation_id=correlation_id,
                )

    except psycopg.Error as exc:
        raise HTTPException(
            status_code=503,
            detail="Authentication service unavailable",
        ) from exc

    if not login_succeeded:
        raise HTTPException(status_code=401, detail=GENERIC_LOGIN_ERROR)

    if mfa_required and not mfa_challenge_allowed:
        raise HTTPException(status_code=401, detail=GENERIC_LOGIN_ERROR)

    if mfa_required:
        try:

===== SECRET STORE FAILURE BEHAVIOR =====
"""Provider-neutral storage for MFA secrets.

The development adapter is for local development only. Production provider
integration must be selected and implemented separately.
"""

from __future__ import annotations

import base64
import os
import secrets
import stat
```

```
from pathlib import Path
from typing import Protocol
from uuid import uuid4

from cryptography.exceptions import InvalidTag
from cryptography.hazmat.primitives.ciphers.aead import AESGCM


class SecretStoreError(RuntimeError):
    """Base error for secret-store operations."""


class SecretNotFoundError(SecretStoreError):
    """The requested secret reference does not exist."""


class SecretStoreConfigurationError(SecretStoreError):
    """The configured secret store is missing or unsafe."""


class SecretStore(Protocol):
    """Interface consumed by MFA application services."""

    def put(self, secret: bytes) -> str: ...
    def get(self, ref: str) -> bytes: ...
    def delete(self, ref: str) -> None: ...


class FakeSecretStore:
    """In-memory adapter with predictable references for automated tests."""

    def __init__(self) -> None:
        self._values: dict[str, bytes] = {}
        self._next_id = 1

    def put(self, secret: bytes) -> str:
        if not isinstance(secret, bytes) or not secret:
            raise ValueError("Secret must be non-empty bytes")
        ref = f"fake://mfa/{self._next_id:06d}"
        self._next_id += 1
        self._values[ref] = bytes(secret)
        return ref

    def get(self, ref: str) -> bytes:
```

```
        try:
            return self._values[ref]
        except KeyError as exc:
            raise SecretNotFoundError("Secret reference not found") from exc

    def delete(self, ref: str) -> None:
        if ref not in self._values:
            raise SecretNotFoundError("Secret reference not found")
        del self._values[ref]


class EncryptedFileSecretStore:
    """Local-only AES-GCM store. The encryption key is never stored here."""

    _PREFIX = "dev-encrypted://"

    def __init__(self, root: Path, key: bytes) -> None:
        if len(key) != 32:
            raise SecretStoreConfigurationError(
                "Development encryption key must be exactly 32 bytes"
            )

        self._root = root.expanduser().resolve()
        self._cipher = AESGCM(key)
        self._root.mkdir(mode=0o700, parents=True, exist_ok=True)

        mode = stat.S_IMODE(self._root.stat().st_mode)
        if mode & 0o077:
            raise SecretStoreConfigurationError(
                "Secret-store directory permissions must exclude group and other access"
            )

    def _path_for(self, ref: str) -> Path:
        if not ref.startswith(self._PREFIX):
            raise SecretNotFoundError("Secret reference not found")
        identifier = ref.removeprefix(self._PREFIX)
        try:
            if str(uuid4()) == identifier:
                pass
            from uuid import UUID
            UUID(identifier)
        except (ValueError, AttributeError) as exc:
            raise SecretNotFoundError("Secret reference not found") from exc
        return self._root / f"{identifier}.enc"
```

```
def put(self, secret: bytes) -> str:
    if not isinstance(secret, bytes) or not secret:
        raise ValueError("Secret must be non-empty bytes")

    identifier = str(uuid4())
    ref = f"{self._PREFIX}{identifier}"
    nonce = secrets.token_bytes(12)
    ciphertext = nonce + self._cipher.encrypt(nonce, secret, ref.encode("utf-8"))
    path = self._root / f"{identifier}.enc"

    flags = os.O_WRONLY | os.O_CREAT | os.O_EXCL
    fd = os.open(path, flags, 0o600)
    try:
        with os.fdopen(fd, "wb") as handle:
            handle.write(ciphertext)
            handle.flush()
            os.fsync(handle.fileno())
    except Exception:
        path.unlink(missing_ok=True)
        raise
    return ref

def get(self, ref: str) -> bytes:
    path = self._path_for(ref)
    try:
        data = path.read_bytes()
    except FileNotFoundError as exc:
        raise SecretNotFoundError("Secret reference not found") from exc

    if len(data) < 13:
        raise SecretStoreError("Stored secret record is invalid")

    try:
        return self._cipher.decrypt(data[:12], data[12:], ref.encode("utf-8"))
    except InvalidTag as exc:
        raise SecretStoreError("Stored secret could not be authenticated") from exc

def delete(self, ref: str) -> None:
    path = self._path_for(ref)
    try:
        path.unlink()
    except FileNotFoundError as exc:
        raise SecretNotFoundError("Secret reference not found") from exc
```

```
def secret_store_from_environment() -> SecretStore:
    """Select an explicitly permitted adapter; never fall back silently."""

    app_env = os.environ.get("APP_ENV", "development").strip().lower()
    backend = os.environ.get("MFA_SECRET_STORE_BACKEND", "").strip().lower()

    if app_env == "production":
        raise SecretStoreConfigurationError(
            "No production MFA secret-store provider is configured"
        )

    if app_env == "test":
        if backend != "fake":
            raise SecretStoreConfigurationError(
                "Tests must explicitly select MFA_SECRET_STORE_BACKEND=fake"
            )
        return FakeSecretStore()

    if app_env != "development":
        raise SecretStoreConfigurationError("Unsupported APP_ENV for MFA secret storage")

    if backend != "dev-encrypted":
        raise SecretStoreConfigurationError(
            "Development requires MFA_SECRET_STORE_BACKEND=dev-encrypted"
        )

    encoded_key = os.environ.get("MFA_DEV_ENCRYPTION_KEY", "")
    store_path = os.environ.get("MFA_SECRET_STORE_PATH", "").strip()
    if not encoded_key or not store_path:

===== REPOSITORY STATUS =====
 M .env.example
 M FUNCTION_TEST_TRACEABILITY.txt
 M IMPLEMENTATION_STATUS.txt
 M SPRINT_1_TEST_EXECUTION_STATUS.txt
 M apps/api/app/main.py
 M apps/api/requirements.lock
 M apps/api/requirements.txt
 M apps/api/tests/test_auth.py
 M conversation.txt
 M docs/release/RELEASE_README.md
 M scripts/package_release.py
```

```
?? apps/api/app/audit.py
?? apps/api/app/main.py.backup-20261022-080642
?? apps/api/app/mfa_challenge_repository.py
?? apps/api/app/mfa_repository.py
?? apps/api/app/mfa_service.py
?? apps/api/app/secret_store.py
?? apps/api/app/throttle.py
?? apps/api/app/totp.py
?? apps/api/migrations/0003_auth_audit_and_login_throttling.sql
?? apps/api/migrations/0004_immutable_auth_audit.sql
?? apps/api/migrations/0005_mfa_factors.sql
?? apps/api/migrations/0006_mfa_login_challenges.sql
?? apps/api/tests/test_audit_immutability.py
?? apps/api/tests/test_audit_service.py
?? apps/api/tests/test_fin14_auth_security.py
?? apps/api/tests/test_mfa_challenge_repository.py
?? apps/api/tests/test_mfa_repository.py
?? apps/api/tests/test_mfa_schema.py
?? apps/api/tests/test_mfa_service.py
?? apps/api/tests/test_secret_store.py
?? apps/api/tests/test_throttle.py
?? apps/api/tests/test_totp.py
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
old_import = """from app.mfa_challenge_repository import (
    MFAChallengeInvalidError,
    issue_mfa_challenge,
    verify_mfa_challenge,
)
"""
new_import = old_import + """from app.mfa_repository import (
    MFAFactorNotFoundError,
    MFAInvalidCodeError,
    activate_totp_factor,
    create_pending_totp_factor,
    list_totp_factors,
    revoke_totp_factor,
)
"""

old_model = """class MFAVerifyRequest(BaseModel):
    challenge_token: str = Field(min_length=32, max_length=256)
    code: str = Field(min_length=1, max_length=32)
"""
```

```
new_model = old_model + """

class MFAEnrollRequest(BaseModel):
    label: str | None = Field(default=None, max_length=120)


class MFAActivateRequest(BaseModel):
    factor_id: UUID
    code: str = Field(min_length=1, max_length=32)
"""

if source.count(old_import) != 1:
    raise SystemExit("STOP: expected MFA challenge import block was not unique.")
if source.count(old_model) != 1:
    raise SystemExit("STOP: expected MFA verification model was not unique.")

source = source.replace(old_import, new_import, 1)
source = source.replace(old_model, new_model, 1)
path.write_text(source)
print("Added MFA repository imports and request models.")
PY
Added MFA repository imports and request models.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % .venv/bin/python -m
py_compile apps/api/app/main.py
git diff --check
git diff -- apps/api/app/main.py
diff --git a/apps/api/app/main.py b/apps/api/app/main.py
index 5e0e618..d4c681c 100644
--- a/apps/api/app/main.py
+++ b/apps/api/app/main.py
@@ -7,7 +7,16 @@ import secrets
 from datetime import UTC, datetime
 from pathlib import Path
 from typing import Annotated, Any
-from uuid import UUID
+from uuid import UUID, uuid4
+
+from app.audit import record_auth_event
+from app.throttle import (
+    is_blocked,
+    load_throttle_config,
+    make_throttle_key,
+    record_failure,
```

```
+    reset_bucket,
+)

 import psycopg
 from dotenv import load_dotenv
@@ -18,17 +27,75 @@ from psycopg.rows import dict_row

 from app.auth import hash_password, new_session_token, session_expiry, token_digest,
verify_password
 from app.database import connect
+from app.secret_store import (
+    SecretStore,
+    SecretStoreConfigurationError,
+    SecretStoreError,
+    secret_store_from_environment,
+)
+from app.mfa_challenge_repository import (
+    MFAChallengeInvalidError,
+    issue_mfa_challenge,
+    verify_mfa_challenge,
+)
+from app.mfa_repository import (
+    MFAFactorNotFoundError,
+    MFAInvalidCodeError,
+    activate_totp_factor,
+    create_pending_totp_factor,
+    list_totp_factors,
+    revoke_totp_factor,
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % clear
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % printf '\n===== MFA
IMPORTS =====\n'
sed -n '35,75p' apps/api/app/main.py

printf '\n===== MFA REQUEST MODELS =====\n'
sed -n '105,145p' apps/api/app/main.py

printf '\n===== MFA ROUTE INSERTION POINT =====\n'
grep -n -E '^@app\.post"/auth/mfa/verify"|^@app\.post"/auth/logout"' apps/api/app/main.py

===== MFA IMPORTS =====
)
from app.mfa_challenge_repository import (
    MFAChallengeInvalidError,
```

```
    issue_mfa_challenge,
    verify_mfa_challenge,
)
from app.mfa_repository import (
    MFAFactorNotFoundError,
    MFAInvalidCodeError,
    activate_totp_factor,
    create_pending_totp_factor,
    list_totp_factors,
    revoke_totp_factor,
)

load_dotenv(Path(__file__).resolve().parents[3] / ".env")

app = FastAPI(title="FinSight AI API", version="0.2.0")


def _get_mfa_secret_store(request: Request) -> SecretStore:
    """Return the app-scoped secret store; never silently change backends."""
    store = getattr(request.app.state, "mfa_secret_store", None)
    if store is None:
        store = secret_store_from_environment()
        request.app.state.mfa_secret_store = store
    return store




@app.middleware("http")
async def auth_correlation_middleware(request: Request, call_next):
    """Return a correlation ID on authentication responses, including errors."""
    if (
        request.url.path not in {"/auth/login", "/auth/logout"}
        and not request.url.path.startswith("/auth/sessions")
    ):
        return await call_next(request)

    supplied = request.headers.get("X-Correlation-ID", "").strip()

===== MFA REQUEST MODELS =====

class LoginRequest(BaseModel):
    email: EmailStr
    password: str = Field(min_length=1, max_length=1024)
```

```
class UserView(BaseModel):
    user_id: UUID
    email: str
    display_name: str



class MFAVerifyRequest(BaseModel):
    challenge_token: str = Field(min_length=32, max_length=256)
    code: str = Field(min_length=1, max_length=32)


class MFAEnrollRequest(BaseModel):
    label: str | None = Field(default=None, max_length=120)


class MFAActivateRequest(BaseModel):
    factor_id: UUID
    code: str = Field(min_length=1, max_length=32)


def _ttl_minutes() -> int:
    try:
        value = int(os.environ.get("SESSION_TTL_MINUTES", "60"))
        session_expiry(value)
        return value
    except ValueError as exc:
        raise HTTPException(status_code=500, detail="Session configuration is invalid") from exc


def _secure_cookie() -> bool:
    configured = os.environ.get("COOKIE_SECURE")
    if configured is not None:
        return configured.lower() in {"1", "true", "yes"}
    return os.environ.get("APP_ENV", "development").lower() == "production"

===== MFA ROUTE INSERTION POINT =====
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % printf '\n===== ROUTE
DECORATORS =====\n'
grep -n '@app.post' apps/api/app/main.py
grep -n '@app.get' apps/api/app/main.py
```

```
grep -n '@app.delete' apps/api/app/main.py

printf '\n===== MFA VERIFICATION ROUTE BOUNDARIES =====\n'
grep -n 'auth/mfa/verify' apps/api/app/main.py
grep -n 'auth/logout' apps/api/app/main.py

printf '\n===== MFA ROUTE END =====\n'
sed -n '400,515p' apps/api/app/main.py

===== ROUTE DECORATORS =====
226:@app.post("/auth/login")
426:@app.post("/auth/mfa/verify", response_model=UserView)
520:@app.post("/auth/logout", status_code=204)
583:@app.get("/auth/sessions", response_model=list[SessionView])
669:@app.get("/auth/me", response_model=UserView)
674:@app.get("/api/v1/admin/authorization/capabilities")
617:@app.delete("/auth/sessions/{session_id}", status_code=204)

===== MFA VERIFICATION ROUTE BOUNDARIES =====
426:@app.post("/auth/mfa/verify", response_model=UserView)
70:        request.url.path not in {"/auth/login", "/auth/logout"}
520:@app.post("/auth/logout", status_code=204)

===== MFA ROUTE END =====
                status_code=503,
                detail="Authentication service unavailable",
            ) from exc

        return {
            "mfa_required": True,
            "challenge_token": challenge_token,
            "expires_at": expires_at,
        }

    response.set_cookie(
        SESSION_COOKIE,
        session_token,
        httponly=True,
        secure=_secure_cookie(),
        samesite="lax",
        max_age=_ttl_minutes() * 60,
        path="/",
    )
    return UserView(
```

```
        user_id=user["user_id"],
        email=user["email"],
        display_name=user["display_name"],
    )


@app.post("/auth/mfa/verify", response_model=UserView)
def verify_mfa_login(
    payload: MFAVerifyRequest,
    request: Request,
    response: Response,
) -> UserView:
    _check_origin(request)
    correlation_id = _auth_correlation_id(request)
    response.headers["X-Correlation-ID"] = str(correlation_id)

    try:
        store = _get_mfa_secret_store(request)
        user_id = verify_mfa_challenge(
            challenge_token=payload.challenge_token,
            supplied_code=payload.code,
            store=store,
            correlation_id=correlation_id,
        )

        token, digest = new_session_token()
        expires_at = session_expiry(_ttl_minutes())
        now = datetime.now(UTC)

        with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
            cur.execute(
                """SELECT user_id, email, display_name
                   FROM users
                   WHERE user_id = %s
                     AND account_status = 'active'
                     AND deleted_at IS NULL
                     AND mfa_required = TRUE""",
                (user_id,),
            )
            user = cur.fetchone()
            if not user:
                raise HTTPException(
                    status_code=401,
                    detail="MFA verification failed",
```

```
            )

        cur.execute(
            """INSERT INTO auth_sessions
                   (user_id, refresh_token_hash, expires_at)
               VALUES (%s, %s, %s)""",
            (user_id, digest, expires_at),
        )
        cur.execute(
            """UPDATE users
               SET last_login_at = %s, updated_at = %s
               WHERE user_id = %s""",
            (now, now, user_id),
        )
        record_auth_event(
            conn,
            action_key="auth.login",
            outcome="success",
            actor_user_id=user_id,
            details={"method": "password_and_totp"},
            correlation_id=correlation_id,
            occurred_at=now,
        )

except MFAChallengeInvalidError as exc:
    raise HTTPException(
        status_code=401,
        detail="MFA verification failed",
    ) from exc
except (SecretStoreError, SecretStoreConfigurationError) as exc:
    raise HTTPException(
        status_code=503,
        detail="MFA service unavailable",
    ) from exc
except psycopg.Error as exc:
    raise HTTPException(
        status_code=503,
        detail="Authentication service unavailable",
    ) from exc

response.set_cookie(
    SESSION_COOKIE,
    token,
    httponly=True,
```

```
        secure=_secure_cookie(),
        samesite="lax",
        max_age=_ttl_minutes() * 60,
        path="/",
    )
    return UserView(
        user_id=user["user_id"],
        email=user["email"],
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
    _auth_correlation_id(request)
    try:
        return list_totp_factors(user_id=user["user_id"])
    except psycopg.Error as exc:
        raise HTTPException(status_code=503, detail="MFA service unavailable") from exc


@app.delete("/auth/mfa/factors/{factor_id}", status_code=204)
def delete_mfa_factor(
    factor_id: UUID,
    request: Request,
    user: CurrentUser,
) -> Response:
    _check_origin(request)
    correlation_id = _auth_correlation_id(request)

    try:
        revoked = revoke_totp_factor(
            user_id=user["user_id"],
            factor_id=factor_id,
        )
        if not revoked:
            raise HTTPException(status_code=404, detail="MFA factor not found")
        return Response(status_code=status.HTTP_204_NO_CONTENT)
    except HTTPException:
        raise
    except psycopg.Error as exc:
        raise HTTPException(status_code=503, detail="MFA service unavailable") from exc


'''

if source.count(old_middleware) != 1:
    raise SystemExit("STOP: expected middleware condition was not found exactly once.")
```

```
if source.count(anchor) != 1:
    raise SystemExit("STOP: expected logout route anchor was not found exactly once.")
if '"/auth/mfa/enroll"' in source:
    raise SystemExit("STOP: MFA enrollment route already exists; no changes made.")

source = source.replace(old_middleware, new_middleware, 1)
source = source.replace(anchor, routes + anchor, 1)
path.write_text(source)
print("Added MFA management routes and correlation middleware coverage.")
PY
Added MFA management routes and correlation middleware coverage.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % .venv/bin/python -m
py_compile apps/api/app/main.py
git diff --check

PYTHONPATH=apps/api .venv/bin/python - <<'PY'
from app.main import app

for route in app.routes:
    methods = getattr(route, "methods", set())
    path = getattr(route, "path", "")
    if path.startswith("/auth/mfa"):
        print(sorted(methods), path)
PY
['POST'] /auth/mfa/verify
['POST'] /auth/mfa/enroll
['POST'] /auth/mfa/activate
['GET'] /auth/mfa/factors
['DELETE'] /auth/mfa/factors/{factor_id}
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
        if not rv:
            last_ex = conn_errors[-1][0]
            if len(conn_errors) == 1:
                raise last_ex.with_traceback(None)

            # Create a new exception with the same type as the last one, containing
            # all attempt errors while preserving backward compatibility.
            lines = [str(last_ex)]
            lines.append("Multiple connection attempts failed. All failures were:")
            lines.extend((f"- {descr}: {error}" for error, descr in conn_errors))
>           raise type(last_ex)("\n".join(lines)).with_traceback(None)
E           psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port
55439 failed: FATAL:  could not open file "global/pg_filenode.map": No such file or directory
```

```
E           Multiple connection attempts failed. All failures were:
E           - host: 'localhost', port: '55439', hostaddr: '::1': connection failed: connection to server at
"::1", port 55439 failed: FATAL:  could not open file "global/pg_filenode.map": No such file or
directory
E           - host: 'localhost', port: '55439', hostaddr: '127.0.0.1': connection failed: connection to
server at "127.0.0.1", port 55439 failed: FATAL:  could not open file "global/pg_filenode.map":
No such file or directory

.venv/lib/python3.14/site-packages/psycopg/connection.py:127: OperationalError
_____ ERROR at setup of
test_challenge_locks_after_five_invalid_attempts
_____

    @pytest.fixture
    def challenge_identity():
        email = f"mfa-challenge-{uuid4().hex}@example.com"
>       with connect() as conn, conn.cursor() as cur:
            ^^^^^^^^

apps/api/tests/test_mfa_challenge_repository.py:22:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
apps/api/app/database.py:18: in connect
    return psycopg.connect(database_url, connect_timeout=3)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
cls = <class 'psycopg.Connection'>, conninfo = 'user=sandeepeh dbname=finsight_fin15_test
host=localhost port=55439 connect_timeout=3 hostaddr=127.0.0.1', autocommit = False
prepare_threshold = 5, context = None, row_factory = None, cursor_factory = None, kwargs =
{'connect_timeout': 3}
params = {'connect_timeout': 3, 'dbname': 'finsight_fin15_test', 'host': 'localhost', 'port': '55439',
...}, timeout = 3

    @classmethod
    def connect(
        cls,
        conninfo: str = "",
        *,
        autocommit: bool = False,
        prepare_threshold: int | None = 5,
        context: AdaptContext | None = None,
        row_factory: RowFactory[Row] | None = None,
```

```
        cursor_factory: type[Cursor[Row]] | None = None,
        **kwargs: ConnParam,
    ) -> Self:
        """
        Connect to a database server and return a new `Connection` instance.
        """

        params = cls._get_connection_params(conninfo, **kwargs)
        timeout = timeout_from_conninfo(params)
        rv = None
        attempts = conninfo_attempts(params)
        conn_errors: list[tuple[e.Error, str]] = []
        for attempt in attempts:
            tdescr = (attempt.get("host"), attempt.get("port"), attempt.get("hostaddr"))
            descr = "host: %r, port: %r, hostaddr: %r" % tdescr
            logger.debug("connection attempt: %s", descr)
            try:
                conninfo = make_conninfo("", **attempt)
                gen = cls._connect_gen(conninfo, timeout=timeout)
                rv = waiting.wait_conn(gen, interval=_WAIT_INTERVAL)
            except e.Error as ex:
                logger.debug("connection failed: %s: %s", descr, str(ex))
                conn_errors.append((ex, descr))
            except e._NO_TRACEBACK as ex:
                raise ex.with_traceback(None)
            else:
                logger.debug("connection succeeded: %s", descr)
                break

        if not rv:
            last_ex = conn_errors[-1][0]
            if len(conn_errors) == 1:
                raise last_ex.with_traceback(None)

            # Create a new exception with the same type as the last one, containing
            # all attempt errors while preserving backward compatibility.
            lines = [str(last_ex)]
            lines.append("Multiple connection attempts failed. All failures were:")
            lines.extend((f"- {descr}: {error}" for error, descr in conn_errors))
>           raise type(last_ex)("\n".join(lines)).with_traceback(None)
E           psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port
55439 failed: FATAL:  could not open file "global/pg_filenode.map": No such file or directory
E           Multiple connection attempts failed. All failures were:
```

```
E           - host: 'localhost', port: '55439', hostaddr: '::1': connection failed: connection to server at
"::1", port 55439 failed: FATAL:  could not open file "global/pg_filenode.map": No such file or
directory
E           - host: 'localhost', port: '55439', hostaddr: '127.0.0.1': connection failed: connection to
server at "127.0.0.1", port 55439 failed: FATAL:  could not open file "global/pg_filenode.map":
No such file or directory

.venv/lib/python3.14/site-packages/psycopg/connection.py:127: OperationalError
_____ ERROR at setup
of test_enroll_activate_and_list_factor
_____

    @pytest.fixture
    def mfa_identity():
        email = f"mfa-repository-{uuid4().hex}@example.com"
>       with connect() as conn, conn.cursor() as cur:
            ^^^^^^^^^

apps/api/tests/test_mfa_repository.py:23:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
apps/api/app/database.py:18: in connect
    return psycopg.connect(database_url, connect_timeout=3)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

cls = <class 'psycopg.Connection'>, conninfo = 'user=sandeepeh dbname=finsight_fin15_test
host=localhost port=55439 connect_timeout=3 hostaddr=127.0.0.1', autocommit = False
prepare_threshold = 5, context = None, row_factory = None, cursor_factory = None, kwargs =
{'connect_timeout': 3}
params = {'connect_timeout': 3, 'dbname': 'finsight_fin15_test', 'host': 'localhost', 'port': '55439',
...}, timeout = 3

    @classmethod
    def connect(
        cls,
        conninfo: str = "",
        *,
        autocommit: bool = False,
        prepare_threshold: int | None = 5,
        context: AdaptContext | None = None,
        row_factory: RowFactory[Row] | None = None,
        cursor_factory: type[Cursor[Row]] | None = None,
```

```
        **kwargs: ConnParam,
    ) -> Self:
        """
        Connect to a database server and return a new `Connection` instance.
        """

        params = cls._get_connection_params(conninfo, **kwargs)
        timeout = timeout_from_conninfo(params)
        rv = None
        attempts = conninfo_attempts(params)
        conn_errors: list[tuple[e.Error, str]] = []
        for attempt in attempts:
            tdescr = (attempt.get("host"), attempt.get("port"), attempt.get("hostaddr"))
            descr = "host: %r, port: %r, hostaddr: %r" % tdescr
            logger.debug("connection attempt: %s", descr)
            try:
                conninfo = make_conninfo("", **attempt)
                gen = cls._connect_gen(conninfo, timeout=timeout)
                rv = waiting.wait_conn(gen, interval=_WAIT_INTERVAL)
            except e.Error as ex:
                logger.debug("connection failed: %s: %s", descr, str(ex))
                conn_errors.append((ex, descr))
            except e._NO_TRACEBACK as ex:
                raise ex.with_traceback(None)
            else:
                logger.debug("connection succeeded: %s", descr)
                break

        if not rv:
            last_ex = conn_errors[-1][0]
            if len(conn_errors) == 1:
                raise last_ex.with_traceback(None)

            # Create a new exception with the same type as the last one, containing
            # all attempt errors while preserving backward compatibility.
            lines = [str(last_ex)]
            lines.append("Multiple connection attempts failed. All failures were:")
            lines.extend((f"- {descr}: {error}" for error, descr in conn_errors))
>           raise type(last_ex)("\n".join(lines)).with_traceback(None)
E           psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port
55439 failed: FATAL:  could not open file "global/pg_filenode.map": No such file or directory
E           Multiple connection attempts failed. All failures were:
```

```
E           - host: 'localhost', port: '55439', hostaddr: '::1': connection failed: connection to server at
"::1", port 55439 failed: FATAL:  could not open file "global/pg_filenode.map": No such file or
directory
E           - host: 'localhost', port: '55439', hostaddr: '127.0.0.1': connection failed: connection to
server at "127.0.0.1", port 55439 failed: FATAL:  could not open file "global/pg_filenode.map":
No such file or directory

.venv/lib/python3.14/site-packages/psycopg/connection.py:127: OperationalError
_____ ERROR at setup of
test_active_factor_rejects_reused_totp_step
_____

    @pytest.fixture
    def mfa_identity():
        email = f"mfa-repository-{uuid4().hex}@example.com"
>       with connect() as conn, conn.cursor() as cur:
            ^^^^^^^^^

apps/api/tests/test_mfa_repository.py:23:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
apps/api/app/database.py:18: in connect
    return psycopg.connect(database_url, connect_timeout=3)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

cls = <class 'psycopg.Connection'>, conninfo = 'user=sandeepeh dbname=finsight_fin15_test
host=localhost port=55439 connect_timeout=3 hostaddr=127.0.0.1', autocommit = False
prepare_threshold = 5, context = None, row_factory = None, cursor_factory = None, kwargs =
{'connect_timeout': 3}
params = {'connect_timeout': 3, 'dbname': 'finsight_fin15_test', 'host': 'localhost', 'port': '55439',
...}, timeout = 3

    @classmethod
    def connect(
        cls,
        conninfo: str = "",
        *,
        autocommit: bool = False,
        prepare_threshold: int | None = 5,
        context: AdaptContext | None = None,
        row_factory: RowFactory[Row] | None = None,
        cursor_factory: type[Cursor[Row]] | None = None,
```

```
        **kwargs: ConnParam,
    ) -> Self:
        """
        Connect to a database server and return a new `Connection` instance.
        """

        params = cls._get_connection_params(conninfo, **kwargs)
        timeout = timeout_from_conninfo(params)
        rv = None
        attempts = conninfo_attempts(params)
        conn_errors: list[tuple[e.Error, str]] = []
        for attempt in attempts:
            tdescr = (attempt.get("host"), attempt.get("port"), attempt.get("hostaddr"))
            descr = "host: %r, port: %r, hostaddr: %r" % tdescr
            logger.debug("connection attempt: %s", descr)
            try:
                conninfo = make_conninfo("", **attempt)
                gen = cls._connect_gen(conninfo, timeout=timeout)
                rv = waiting.wait_conn(gen, interval=_WAIT_INTERVAL)
            except e.Error as ex:
                logger.debug("connection failed: %s: %s", descr, str(ex))
                conn_errors.append((ex, descr))
            except e._NO_TRACEBACK as ex:
                raise ex.with_traceback(None)
            else:
                logger.debug("connection succeeded: %s", descr)
                break

        if not rv:
            last_ex = conn_errors[-1][0]
            if len(conn_errors) == 1:
                raise last_ex.with_traceback(None)

            # Create a new exception with the same type as the last one, containing
            # all attempt errors while preserving backward compatibility.
            lines = [str(last_ex)]
            lines.append("Multiple connection attempts failed. All failures were:")
            lines.extend((f"- {descr}: {error}" for error, descr in conn_errors))
>           raise type(last_ex)("\n".join(lines)).with_traceback(None)
E           psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port
55439 failed: FATAL:  could not open file "global/pg_filenode.map": No such file or directory
E           Multiple connection attempts failed. All failures were:
```

```
E           - host: 'localhost', port: '55439', hostaddr: '::1': connection failed: connection to server at
"::1", port 55439 failed: FATAL:  could not open file "global/pg_filenode.map": No such file or
directory
E           - host: 'localhost', port: '55439', hostaddr: '127.0.0.1': connection failed: connection to
server at "127.0.0.1", port 55439 failed: FATAL:  could not open file "global/pg_filenode.map":
No such file or directory

.venv/lib/python3.14/site-packages/psycopg/connection.py:127: OperationalError
_____ ERROR at setup of
test_invalid_activation_code_leaves_factor_pending
_____

    @pytest.fixture
    def mfa_identity():
        email = f"mfa-repository-{uuid4().hex}@example.com"
>       with connect() as conn, conn.cursor() as cur:
            ^^^^^^^^^

apps/api/tests/test_mfa_repository.py:23:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
apps/api/app/database.py:18: in connect
    return psycopg.connect(database_url, connect_timeout=3)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

cls = <class 'psycopg.Connection'>, conninfo = 'user=sandeepeh dbname=finsight_fin15_test
host=localhost port=55439 connect_timeout=3 hostaddr=127.0.0.1', autocommit = False
prepare_threshold = 5, context = None, row_factory = None, cursor_factory = None, kwargs =
{'connect_timeout': 3}
params = {'connect_timeout': 3, 'dbname': 'finsight_fin15_test', 'host': 'localhost', 'port': '55439',
...}, timeout = 3

    @classmethod
    def connect(
        cls,
        conninfo: str = "",
        *,
        autocommit: bool = False,
        prepare_threshold: int | None = 5,
        context: AdaptContext | None = None,
        row_factory: RowFactory[Row] | None = None,
        cursor_factory: type[Cursor[Row]] | None = None,
```

```
        **kwargs: ConnParam,
    ) -> Self:
        """
        Connect to a database server and return a new `Connection` instance.
        """

        params = cls._get_connection_params(conninfo, **kwargs)
        timeout = timeout_from_conninfo(params)
        rv = None
        attempts = conninfo_attempts(params)
        conn_errors: list[tuple[e.Error, str]] = []
        for attempt in attempts:
            tdescr = (attempt.get("host"), attempt.get("port"), attempt.get("hostaddr"))
            descr = "host: %r, port: %r, hostaddr: %r" % tdescr
            logger.debug("connection attempt: %s", descr)
            try:
                conninfo = make_conninfo("", **attempt)
                gen = cls._connect_gen(conninfo, timeout=timeout)
                rv = waiting.wait_conn(gen, interval=_WAIT_INTERVAL)
            except e.Error as ex:
                logger.debug("connection failed: %s: %s", descr, str(ex))
                conn_errors.append((ex, descr))
            except e._NO_TRACEBACK as ex:
                raise ex.with_traceback(None)
            else:
                logger.debug("connection succeeded: %s", descr)
                break

        if not rv:
            last_ex = conn_errors[-1][0]
            if len(conn_errors) == 1:
                raise last_ex.with_traceback(None)

            # Create a new exception with the same type as the last one, containing
            # all attempt errors while preserving backward compatibility.
            lines = [str(last_ex)]
            lines.append("Multiple connection attempts failed. All failures were:")
            lines.extend((f"- {descr}: {error}" for error, descr in conn_errors))
>           raise type(last_ex)("\n".join(lines)).with_traceback(None)
E           psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port
55439 failed: FATAL:  could not open file "global/pg_filenode.map": No such file or directory
E           Multiple connection attempts failed. All failures were:
```

```
E           - host: 'localhost', port: '55439', hostaddr: '::1': connection failed: connection to server at
"::1", port 55439 failed: FATAL:  could not open file "global/pg_filenode.map": No such file or
directory
E           - host: 'localhost', port: '55439', hostaddr: '127.0.0.1': connection failed: connection to
server at "127.0.0.1", port 55439 failed: FATAL:  could not open file "global/pg_filenode.map":
No such file or directory

.venv/lib/python3.14/site-packages/psycopg/connection.py:127: OperationalError
_____ ERROR at setup of
test_factor_cannot_be_accessed_by_another_user
_____

    @pytest.fixture
    def mfa_identity():
        email = f"mfa-repository-{uuid4().hex}@example.com"
>       with connect() as conn, conn.cursor() as cur:
            ^^^^^^^^^

apps/api/tests/test_mfa_repository.py:23:

_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
apps/api/app/database.py:18: in connect
    return psycopg.connect(database_url, connect_timeout=3)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

cls = <class 'psycopg.Connection'>, conninfo = 'user=sandeepeh dbname=finsight_fin15_test
host=localhost port=55439 connect_timeout=3 hostaddr=127.0.0.1', autocommit = False
prepare_threshold = 5, context = None, row_factory = None, cursor_factory = None, kwargs =
{'connect_timeout': 3}
params = {'connect_timeout': 3, 'dbname': 'finsight_fin15_test', 'host': 'localhost', 'port': '55439',
...}, timeout = 3

    @classmethod
    def connect(
        cls,
        conninfo: str = "",
        *,
        autocommit: bool = False,
        prepare_threshold: int | None = 5,
        context: AdaptContext | None = None,
        row_factory: RowFactory[Row] | None = None,
        cursor_factory: type[Cursor[Row]] | None = None,
```

```
        **kwargs: ConnParam,
    ) -> Self:
        """
        Connect to a database server and return a new `Connection` instance.
        """

        params = cls._get_connection_params(conninfo, **kwargs)
        timeout = timeout_from_conninfo(params)
        rv = None
        attempts = conninfo_attempts(params)
        conn_errors: list[tuple[e.Error, str]] = []
        for attempt in attempts:
            tdescr = (attempt.get("host"), attempt.get("port"), attempt.get("hostaddr"))
            descr = "host: %r, port: %r, hostaddr: %r" % tdescr
            logger.debug("connection attempt: %s", descr)
            try:
                conninfo = make_conninfo("", **attempt)
                gen = cls._connect_gen(conninfo, timeout=timeout)
                rv = waiting.wait_conn(gen, interval=_WAIT_INTERVAL)
            except e.Error as ex:
                logger.debug("connection failed: %s: %s", descr, str(ex))
                conn_errors.append((ex, descr))
            except e._NO_TRACEBACK as ex:
                raise ex.with_traceback(None)
            else:
                logger.debug("connection succeeded: %s", descr)
                break

        if not rv:
            last_ex = conn_errors[-1][0]
            if len(conn_errors) == 1:
                raise last_ex.with_traceback(None)

            # Create a new exception with the same type as the last one, containing
            # all attempt errors while preserving backward compatibility.
            lines = [str(last_ex)]
            lines.append("Multiple connection attempts failed. All failures were:")
            lines.extend((f"- {descr}: {error}" for error, descr in conn_errors))
>           raise type(last_ex)("\n".join(lines)).with_traceback(None)
E           psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port
55439 failed: FATAL:  could not open file "global/pg_filenode.map": No such file or directory
E           Multiple connection attempts failed. All failures were:
```

```
E           - host: 'localhost', port: '55439', hostaddr: '::1': connection failed: connection to server at
"::1", port 55439 failed: FATAL:  could not open file "global/pg_filenode.map": No such file or
directory
E           - host: 'localhost', port: '55439', hostaddr: '127.0.0.1': connection failed: connection to
server at "127.0.0.1", port 55439 failed: FATAL:  could not open file "global/pg_filenode.map":
No such file or directory

.venv/lib/python3.14/site-packages/psycopg/connection.py:127: OperationalError
_____ ERROR at setup of
test_revoked_factor_cannot_be_verified
_____

    @pytest.fixture
    def mfa_identity():
        email = f"mfa-repository-{uuid4().hex}@example.com"
>       with connect() as conn, conn.cursor() as cur:
            ^^^^^^^^^

apps/api/tests/test_mfa_repository.py:23:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
apps/api/app/database.py:18: in connect
    return psycopg.connect(database_url, connect_timeout=3)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

cls = <class 'psycopg.Connection'>, conninfo = 'user=sandeepeh dbname=finsight_fin15_test
host=localhost port=55439 connect_timeout=3 hostaddr=127.0.0.1', autocommit = False
prepare_threshold = 5, context = None, row_factory = None, cursor_factory = None, kwargs =
{'connect_timeout': 3}
params = {'connect_timeout': 3, 'dbname': 'finsight_fin15_test', 'host': 'localhost', 'port': '55439',
...}, timeout = 3

    @classmethod
    def connect(
        cls,
        conninfo: str = "",
        *,
        autocommit: bool = False,
        prepare_threshold: int | None = 5,
        context: AdaptContext | None = None,
        row_factory: RowFactory[Row] | None = None,
        cursor_factory: type[Cursor[Row]] | None = None,
```

```
        **kwargs: ConnParam,
    ) -> Self:
        """
        Connect to a database server and return a new `Connection` instance.
        """

        params = cls._get_connection_params(conninfo, **kwargs)
        timeout = timeout_from_conninfo(params)
        rv = None
        attempts = conninfo_attempts(params)
        conn_errors: list[tuple[e.Error, str]] = []
        for attempt in attempts:
            tdescr = (attempt.get("host"), attempt.get("port"), attempt.get("hostaddr"))
            descr = "host: %r, port: %r, hostaddr: %r" % tdescr
            logger.debug("connection attempt: %s", descr)
            try:
                conninfo = make_conninfo("", **attempt)
                gen = cls._connect_gen(conninfo, timeout=timeout)
                rv = waiting.wait_conn(gen, interval=_WAIT_INTERVAL)
            except e.Error as ex:
                logger.debug("connection failed: %s: %s", descr, str(ex))
                conn_errors.append((ex, descr))
            except e._NO_TRACEBACK as ex:
                raise ex.with_traceback(None)
            else:
                logger.debug("connection succeeded: %s", descr)
                break

        if not rv:
            last_ex = conn_errors[-1][0]
            if len(conn_errors) == 1:
                raise last_ex.with_traceback(None)

            # Create a new exception with the same type as the last one, containing
            # all attempt errors while preserving backward compatibility.
            lines = [str(last_ex)]
            lines.append("Multiple connection attempts failed. All failures were:")
            lines.extend((f"- {descr}: {error}" for error, descr in conn_errors))
>           raise type(last_ex)("\n".join(lines)).with_traceback(None)
E           psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port
55439 failed: FATAL:  could not open file "global/pg_filenode.map": No such file or directory
E           Multiple connection attempts failed. All failures were:
```

```
E           - host: 'localhost', port: '55439', hostaddr: '::1': connection failed: connection to server at
"::1", port 55439 failed: FATAL:  could not open file "global/pg_filenode.map": No such file or
directory
E           - host: 'localhost', port: '55439', hostaddr: '127.0.0.1': connection failed: connection to
server at "127.0.0.1", port 55439 failed: FATAL:  could not open file "global/pg_filenode.map":
No such file or directory

.venv/lib/python3.14/site-packages/psycopg/connection.py:127: OperationalError
=============================================================================
============ FAILURES
=============================================================================
============

_____________________________________________________________________
test_successful_login_is_audited_without_secrets
_____________________________________________________________________

monkeypatch = <_pytest.monkeypatch.MonkeyPatch object at 0x10945c370>

    def test_successful_login_is_audited_without_secrets(monkeypatch):
        monkeypatch.setenv("AUTH_THROTTLE_SECRET",
"integration-test-secret-material-32-bytes")
        email = f"{uuid4().hex}@example.com"
        password = "Synthetic-FIN14-Password-Only"
        correlation = None

>       with connect() as conn, conn.cursor() as cur:
            ^^^^^^^^^

apps/api/tests/test_fin14_auth_security.py:21:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
apps/api/app/database.py:18: in connect
    return psycopg.connect(database_url, connect_timeout=3)
           ^^^^^^^^^
```

```
@classmethod
def connect(
    cls,
    conninfo: str = "",
    *,
    autocommit: bool = False,
    prepare_threshold: int | None = 5,
    context: AdaptContext | None = None,
    row_factory: RowFactory[Row] | None = None,
    cursor_factory: type[Cursor[Row]] | None = None,
    **kwargs: ConnParam,
) -> Self:
    """
    Connect to a database server and return a new `Connection` instance.
    """

    params = cls._get_connection_params(conninfo, **kwargs)
    timeout = timeout_from_conninfo(params)
    rv = None
    attempts = conninfo_attempts(params)
    conn_errors: list[tuple[e.Error, str]] = []
    for attempt in attempts:
        tdescr = (attempt.get("host"), attempt.get("port"), attempt.get("hostaddr"))
        descr = "host: %r, port: %r, hostaddr: %r" % tdescr
        logger.debug("connection attempt: %s", descr)
        try:
            conninfo = make_conninfo("", **attempt)
            gen = cls._connect_gen(conninfo, timeout=timeout)
            rv = waiting.wait_conn(gen, interval=_WAIT_INTERVAL)
        except e.Error as ex:
            logger.debug("connection failed: %s: %s", descr, str(ex))
            conn_errors.append((ex, descr))
        except e._NO_TRACEBACK as ex:
            raise ex.with_traceback(None)
        else:
            logger.debug("connection succeeded: %s", descr)
            break

    if not rv:
        last_ex = conn_errors[-1][0]
        if len(conn_errors) == 1:
            raise last_ex.with_traceback(None)

        # Create a new exception with the same type as the last one, containing
```

```
            # all attempt errors while preserving backward compatibility.
            lines = [str(last_ex)]
            lines.append("Multiple connection attempts failed. All failures were:")
            lines.extend((f"- {descr}: {error}" for error, descr in conn_errors))
>           raise type(last_ex)("\n".join(lines)).with_traceback(None)
E           psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port
55439 failed: FATAL:  could not open file "global/pg_filenode.map": No such file or directory
E           Multiple connection attempts failed. All failures were:
E           - host: 'localhost', port: '55439', hostaddr: '::1': connection failed: connection to server at
"::1", port 55439 failed: FATAL:  could not open file "global/pg_filenode.map": No such file or
directory
E           - host: 'localhost', port: '55439', hostaddr: '127.0.0.1': connection failed: connection to
server at "127.0.0.1", port 55439 failed: FATAL:  could not open file "global/pg_filenode.map":
No such file or directory

.venv/lib/python3.14/site-packages/psycopg/connection.py:127: OperationalError
_____________________________________________________________________
test_failed_login_is_audited_and_repeated_failures_are_blocked
_____________________________________________________________________

monkeypatch = <_pytest.monkeypatch.MonkeyPatch object at 0x109519250>

    def test_failed_login_is_audited_and_repeated_failures_are_blocked(monkeypatch):
        monkeypatch.setenv("AUTH_THROTTLE_SECRET",
"integration-test-secret-material-32-bytes")
        monkeypatch.setenv("AUTH_LOGIN_MAX_FAILURES", "2")
        monkeypatch.setenv("AUTH_LOGIN_WINDOW_SECONDS", "600")
        monkeypatch.setenv("AUTH_LOGIN_BLOCK_SECONDS", "60")

        email = f"{uuid4().hex}@example.com"
        password = "Synthetic-FIN14-Password-Only"
        test_started_at = datetime.now(UTC)
        ip_key = make_throttle_key(
            b"integration-test-secret-material-32-bytes",
            "ip",
            "testclient",
        )

>       with connect() as conn, conn.cursor() as cur:
             ^^^^^^^^^

apps/api/tests/test_fin14_auth_security.py:82:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
```

```
apps/api/app/database.py:18: in connect
    return psycopg.connect(database_url, connect_timeout=3)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

cls = <class 'psycopg.Connection'>, conninfo = 'user=sandeepeh dbname=finsight_fin15_test
host=localhost port=55439 connect_timeout=3 hostaddr=127.0.0.1', autocommit = False
prepare_threshold = 5, context = None, row_factory = None, cursor_factory = None, kwargs =
{'connect_timeout': 3}
params = {'connect_timeout': 3, 'dbname': 'finsight_fin15_test', 'host': 'localhost', 'port': '55439',
...}, timeout = 3

    @classmethod
    def connect(
        cls,
        conninfo: str = "",
        *,
        autocommit: bool = False,
        prepare_threshold: int | None = 5,
        context: AdaptContext | None = None,
        row_factory: RowFactory[Row] | None = None,
        cursor_factory: type[Cursor[Row]] | None = None,
        **kwargs: ConnParam,
    ) -> Self:
        """
        Connect to a database server and return a new `Connection` instance.
        """

        params = cls._get_connection_params(conninfo, **kwargs)
        timeout = timeout_from_conninfo(params)
        rv = None
        attempts = conninfo_attempts(params)
        conn_errors: list[tuple[e.Error, str]] = []
        for attempt in attempts:
            tdescr = (attempt.get("host"), attempt.get("port"), attempt.get("hostaddr"))
            descr = "host: %r, port: %r, hostaddr: %r" % tdescr
            logger.debug("connection attempt: %s", descr)
            try:
                conninfo = make_conninfo("", **attempt)
                gen = cls._connect_gen(conninfo, timeout=timeout)
                rv = waiting.wait_conn(gen, interval=_WAIT_INTERVAL)
            except e.Error as ex:
                logger.debug("connection failed: %s: %s", descr, str(ex))
```

```
                conn_errors.append((ex, descr))
            except e._NO_TRACEBACK as ex:
                raise ex.with_traceback(None)
            else:
                logger.debug("connection succeeded: %s", descr)
                break

        if not rv:
            last_ex = conn_errors[-1][0]
            if len(conn_errors) == 1:
                raise last_ex.with_traceback(None)

            # Create a new exception with the same type as the last one, containing
            # all attempt errors while preserving backward compatibility.
            lines = [str(last_ex)]
            lines.append("Multiple connection attempts failed. All failures were:")
            lines.extend((f"- {descr}: {error}" for error, descr in conn_errors))
>           raise type(last_ex)("\n".join(lines)).with_traceback(None)
E           psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port
55439 failed: FATAL:  could not open file "global/pg_filenode.map": No such file or directory
E           Multiple connection attempts failed. All failures were:
E           - host: 'localhost', port: '55439', hostaddr: '::1': connection failed: connection to server at
"::1", port 55439 failed: FATAL:  could not open file "global/pg_filenode.map": No such file or
directory
E           - host: 'localhost', port: '55439', hostaddr: '127.0.0.1': connection failed: connection to
server at "127.0.0.1", port 55439 failed: FATAL:  could not open file "global/pg_filenode.map":
No such file or directory

.venv/lib/python3.14/site-packages/psycopg/connection.py:127: OperationalError
_____________________________________________________________________________
test_logout_is_audited_without_recording_session_token
_____________________________________________________________________________

monkeypatch = <_pytest.monkeypatch.MonkeyPatch object at 0x109366f50>

    def test_logout_is_audited_without_recording_session_token(monkeypatch):
        monkeypatch.setenv(
            "AUTH_THROTTLE_SECRET",
            "integration-test-secret-material-32-bytes",
        )
        email = f"{uuid4().hex}@example.com"
        password = "Synthetic-FIN14-Logout-Password"

>       with connect() as conn, conn.cursor() as cur:
```

```
            ^^^^^^^^^

apps/api/tests/test_fin14_auth_security.py:158:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
apps/api/app/database.py:18: in connect
    return psycopg.connect(database_url, connect_timeout=3)
           ^^^^^^^^^
```

```
                conninfo = make_conninfo("", **attempt)
                gen = cls._connect_gen(conninfo, timeout=timeout)
                rv = waiting.wait_conn(gen, interval=_WAIT_INTERVAL)
            except e.Error as ex:
                logger.debug("connection failed: %s: %s", descr, str(ex))
                conn_errors.append((ex, descr))
            except e._NO_TRACEBACK as ex:
                raise ex.with_traceback(None)
            else:
                logger.debug("connection succeeded: %s", descr)
                break

        if not rv:
            last_ex = conn_errors[-1][0]
            if len(conn_errors) == 1:
                raise last_ex.with_traceback(None)

            # Create a new exception with the same type as the last one, containing
            # all attempt errors while preserving backward compatibility.
            lines = [str(last_ex)]
            lines.append("Multiple connection attempts failed. All failures were:")
            lines.extend((f"- {descr}: {error}" for error, descr in conn_errors))
>           raise type(last_ex)("\n".join(lines)).with_traceback(None)
E           psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port
55439 failed: FATAL:  could not open file "global/pg_filenode.map": No such file or directory
E           Multiple connection attempts failed. All failures were:
E           - host: 'localhost', port: '55439', hostaddr: '::1': connection failed: connection to server at
"::1", port 55439 failed: FATAL:  could not open file "global/pg_filenode.map": No such file or
directory
E           - host: 'localhost', port: '55439', hostaddr: '127.0.0.1': connection failed: connection to
server at "127.0.0.1", port 55439 failed: FATAL:  could not open file "global/pg_filenode.map":
No such file or directory

.venv/lib/python3.14/site-packages/psycopg/connection.py:127: OperationalError
_____ test_auth_correlation_id_is_returned_and_audited _____

monkeypatch = <_pytest.monkeypatch.MonkeyPatch object at 0x1093804d0>

    def test_auth_correlation_id_is_returned_and_audited(monkeypatch):
        monkeypatch.setenv(
            "AUTH_THROTTLE_SECRET",
            "integration-test-secret-material-32-bytes",
```

```
        )
        email = f"{uuid4().hex}@example.com"
        password = "Synthetic-FIN14-Correlation-Password"
        correlation_id = uuid4()

>       with connect() as conn, conn.cursor() as cur:
            ^^^^^^^^^

apps/api/tests/test_fin14_auth_security.py:222:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
apps/api/app/database.py:18: in connect
    return psycopg.connect(database_url, connect_timeout=3)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
cls = <class 'psycopg.Connection'>, conninfo = 'user=sandeepeh dbname=finsight_fin15_test
host=localhost port=55439 connect_timeout=3 hostaddr=127.0.0.1', autocommit = False
prepare_threshold = 5, context = None, row_factory = None, cursor_factory = None, kwargs =
{'connect_timeout': 3}
params = {'connect_timeout': 3, 'dbname': 'finsight_fin15_test', 'host': 'localhost', 'port': '55439',
...}, timeout = 3

    @classmethod
    def connect(
        cls,
        conninfo: str = "",
        *,
        autocommit: bool = False,
        prepare_threshold: int | None = 5,
        context: AdaptContext | None = None,
        row_factory: RowFactory[Row] | None = None,
        cursor_factory: type[Cursor[Row]] | None = None,
        **kwargs: ConnParam,
    ) -> Self:
        """
        Connect to a database server and return a new `Connection` instance.
        """

        params = cls._get_connection_params(conninfo, **kwargs)
        timeout = timeout_from_conninfo(params)
        rv = None
        attempts = conninfo_attempts(params)
```

```
        conn_errors: list[tuple[e.Error, str]] = []
        for attempt in attempts:
            tdescr = (attempt.get("host"), attempt.get("port"), attempt.get("hostaddr"))
            descr = "host: %r, port: %r, hostaddr: %r" % tdescr
            logger.debug("connection attempt: %s", descr)
            try:
                conninfo = make_conninfo("", **attempt)
                gen = cls._connect_gen(conninfo, timeout=timeout)
                rv = waiting.wait_conn(gen, interval=_WAIT_INTERVAL)
            except e.Error as ex:
                logger.debug("connection failed: %s: %s", descr, str(ex))
                conn_errors.append((ex, descr))
            except e._NO_TRACEBACK as ex:
                raise ex.with_traceback(None)
            else:
                logger.debug("connection succeeded: %s", descr)
                break

        if not rv:
            last_ex = conn_errors[-1][0]
            if len(conn_errors) == 1:
                raise last_ex.with_traceback(None)

            # Create a new exception with the same type as the last one, containing
            # all attempt errors while preserving backward compatibility.
            lines = [str(last_ex)]
            lines.append("Multiple connection attempts failed. All failures were:")
            lines.extend((f"- {descr}: {error}" for error, descr in conn_errors))
>           raise type(last_ex)("\n".join(lines)).with_traceback(None)
E           psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port
55439 failed: FATAL:  could not open file "global/pg_filenode.map": No such file or directory
E           Multiple connection attempts failed. All failures were:
E           - host: 'localhost', port: '55439', hostaddr: '::1': connection failed: connection to server at
"::1", port 55439 failed: FATAL:  could not open file "global/pg_filenode.map": No such file or
directory
E           - host: 'localhost', port: '55439', hostaddr: '127.0.0.1': connection failed: connection to
server at "127.0.0.1", port 55439 failed: FATAL:  could not open file "global/pg_filenode.map":
No such file or directory

.venv/lib/python3.14/site-packages/psycopg/connection.py:127: OperationalError
_____________________________________________________________________________
test_failed_login_returns_the_audited_correlation_id
_____________________________________________________________________________
```

```
monkeypatch = <_pytest.monkeypatch.MonkeyPatch object at 0x109457540>

    def test_failed_login_returns_the_audited_correlation_id(monkeypatch):
        monkeypatch.setenv(
            "AUTH_THROTTLE_SECRET",
            "integration-test-secret-material-32-bytes",
        )
        correlation_id = uuid4()

        with TestClient(app) as client:
            response = client.post(
                "/auth/login",
                json={
                    "email": f"{uuid4().hex}@example.com",
                    "password": "Synthetic-Invalid-Password",
                },
                headers={"X-Correlation-ID": str(correlation_id)},
            )

>       assert response.status_code == 401
E       assert 503 == 401
E        +  where 503 = <Response [503 Service Unavailable]>.status_code

apps/api/tests/test_fin14_auth_security.py:304: AssertionError
_____________________________________________________________________
test_generated_auth_correlation_id_matches_audit_record
_____________________________________________________________________

monkeypatch = <_pytest.monkeypatch.MonkeyPatch object at 0x10938f310>

    def test_generated_auth_correlation_id_matches_audit_record(monkeypatch):
        monkeypatch.setenv(
            "AUTH_THROTTLE_SECRET",
            "integration-test-secret-material-32-bytes",
        )
        email = f"{uuid4().hex}@example.com"

        with TestClient(app) as client:
            response = client.post(
                "/auth/login",
                json={"email": email, "password": "Synthetic-Invalid-Password"},
            )

>       assert response.status_code == 401
```

```
E       assert 503 == 401
E        +  where 503 = <Response [503 Service Unavailable]>.status_code

apps/api/tests/test_fin14_auth_security.py:336: AssertionError
_____ test_throttle_expires_and_correct_login_succeeds_after_retry _____

monkeypatch = <_pytest.monkeypatch.MonkeyPatch object at 0x1094a7ee0>

    def test_throttle_expires_and_correct_login_succeeds_after_retry(monkeypatch):
        secret = "integration-test-secret-material-32-bytes"
        monkeypatch.setenv("AUTH_THROTTLE_SECRET", secret)
        monkeypatch.setenv("AUTH_LOGIN_MAX_FAILURES", "1")
        monkeypatch.setenv("AUTH_LOGIN_WINDOW_SECONDS", "60")
        monkeypatch.setenv("AUTH_LOGIN_BLOCK_SECONDS", "1")

        email = f"{uuid4().hex}@example.com"
        password = "Synthetic-FIN14-Expiry-Retry-Password"
        failed_correlation = uuid4()
        blocked_correlation = uuid4()
        retry_correlation = uuid4()
        account_key = make_throttle_key(secret.encode("utf-8"), "account", email.lower())
        ip_key = make_throttle_key(secret.encode("utf-8"), "ip", "testclient")

>       with connect() as conn, conn.cursor() as cur:
            ^^^^^^^^^

apps/api/tests/test_fin14_auth_security.py:371:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
apps/api/app/database.py:18: in connect
    return psycopg.connect(database_url, connect_timeout=3)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

cls = <class 'psycopg.Connection'>, conninfo = 'user=sandeepeh dbname=finsight_fin15_test
host=localhost port=55439 connect_timeout=3 hostaddr=127.0.0.1', autocommit = False
prepare_threshold = 5, context = None, row_factory = None, cursor_factory = None, kwargs =
{'connect_timeout': 3}
params = {'connect_timeout': 3, 'dbname': 'finsight_fin15_test', 'host': 'localhost', 'port': '55439',
...}, timeout = 3
```

```
@classmethod
def connect(
    cls,
    conninfo: str = "",
    *,
    autocommit: bool = False,
    prepare_threshold: int | None = 5,
    context: AdaptContext | None = None,
    row_factory: RowFactory[Row] | None = None,
    cursor_factory: type[Cursor[Row]] | None = None,
    **kwargs: ConnParam,
) -> Self:
    """
    Connect to a database server and return a new `Connection` instance.
    """

    params = cls._get_connection_params(conninfo, **kwargs)
    timeout = timeout_from_conninfo(params)
    rv = None
    attempts = conninfo_attempts(params)
    conn_errors: list[tuple[e.Error, str]] = []
    for attempt in attempts:
        tdescr = (attempt.get("host"), attempt.get("port"), attempt.get("hostaddr"))
        descr = "host: %r, port: %r, hostaddr: %r" % tdescr
        logger.debug("connection attempt: %s", descr)
        try:
            conninfo = make_conninfo("", **attempt)
            gen = cls._connect_gen(conninfo, timeout=timeout)
            rv = waiting.wait_conn(gen, interval=_WAIT_INTERVAL)
        except e.Error as ex:
            logger.debug("connection failed: %s: %s", descr, str(ex))
            conn_errors.append((ex, descr))
        except e._NO_TRACEBACK as ex:
            raise ex.with_traceback(None)
        else:
            logger.debug("connection succeeded: %s", descr)
            break

    if not rv:
        last_ex = conn_errors[-1][0]
        if len(conn_errors) == 1:
            raise last_ex.with_traceback(None)

        # Create a new exception with the same type as the last one, containing
```

```
            # all attempt errors while preserving backward compatibility.
            lines = [str(last_ex)]
            lines.append("Multiple connection attempts failed. All failures were:")
            lines.extend((f"- {descr}: {error}" for error, descr in conn_errors))
>           raise type(last_ex)("\n".join(lines)).with_traceback(None)
E           psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port
55439 failed: FATAL:  could not open file "global/pg_filenode.map": No such file or directory
E           Multiple connection attempts failed. All failures were:
E           - host: 'localhost', port: '55439', hostaddr: '::1': connection failed: connection to server at
"::1", port 55439 failed: FATAL:  could not open file "global/pg_filenode.map": No such file or
directory
E           - host: 'localhost', port: '55439', hostaddr: '127.0.0.1': connection failed: connection to
server at "127.0.0.1", port 55439 failed: FATAL:  could not open file "global/pg_filenode.map":
No such file or directory

.venv/lib/python3.14/site-packages/psycopg/connection.py:127: OperationalError
================================================================================
======== warnings summary
================================================================================
========
.venv/lib/python3.14/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is
deprecated; install `httpx2` instead.
  from starlette.testclient import TestClient as TestClient  # noqa

.venv/lib/python3.14/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is
deprecated, use anyio.from_thread.BlockingPortal instead.
  _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
================================================================================
==== short test summary info
================================================================================
=====
FAILED
apps/api/tests/test_fin14_auth_security.py::test_successful_login_is_audited_without_secrets -
psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port 55439
failed: FATAL:  could not open file "global/pg_filenode.map": No such file or di...
```

FAILED apps/api/tests/test_fin14_auth_security.py::test_failed_login_is_audited_and_repeated_failures_are_blocked - psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port 55439 failed: FATAL: could not open file "global/pg_filenode.map": No such file or di...
FAILED apps/api/tests/test_fin14_auth_security.py::test_logout_is_audited_without_recording_session_token - psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port 55439 failed: FATAL: could not open file "global/pg_filenode.map": No such file or di...
FAILED apps/api/tests/test_fin14_auth_security.py::test_auth_correlation_id_is_returned_and_audited - psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port 55439 failed: FATAL: could not open file "global/pg_filenode.map": No such file or di...
FAILED apps/api/tests/test_fin14_auth_security.py::test_failed_login_returns_the_audited_correlation_id - assert 503 == 401
FAILED apps/api/tests/test_fin14_auth_security.py::test_generated_auth_correlation_id_matches_audit_record - assert 503 == 401
FAILED apps/api/tests/test_fin14_auth_security.py::test_throttle_expires_and_correct_login_succeeds_after_retry - psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port 55439 failed: FATAL: could not open file "global/pg_filenode.map": No such file or di...
ERROR apps/api/tests/test_auth.py::test_login_creates_protected_hashed_expiring_session - psycopg.OperationalError: connection failed: connection to server at "::1", port 55439 failed: FATAL: could not open file "global/pg_filenode.map": No such file or directory
ERROR apps/api/tests/test_auth.py::test_invalid_credentials_have_same_generic_response - psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port 55439 failed: FATAL: could not open file "global/pg_filenode.map": No such file or di...
ERROR apps/api/tests/test_auth.py::test_logout_revokes_session_and_replay_fails - psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port 55439 failed: FATAL: could not open file "global/pg_filenode.map": No such file or di...
ERROR apps/api/tests/test_auth.py::test_logout_without_session_is_idempotent - psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port 55439 failed: FATAL: could not open file "global/pg_filenode.map": No such file or di...
ERROR apps/api/tests/test_auth.py::test_expired_session_is_rejected - psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port 55439 failed: FATAL: could not open file "global/pg_filenode.map": No such file or di...
ERROR apps/api/tests/test_auth.py::test_public_registration_is_not_available - psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port 55439 failed: FATAL: could not open file "global/pg_filenode.map": No such file or di...
ERROR apps/api/tests/test_auth.py::test_auth_mutation_rejects_foreign_origin - psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port 55439 failed: FATAL: could not open file "global/pg_filenode.map": No such file or di...

ERROR apps/api/tests/test_auth.py::test_secure_cookie_can_be_required_by_deployment - psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port 55439 failed: FATAL: could not open file "global/pg_filenode.map": No such file or di...
ERROR apps/api/tests/test_auth.py::test_list_active_sessions_shows_current_session - psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port 55439failed: FATAL: could not open file "global/pg_filenode.map": No such file or di...
ERROR apps/api/tests/test_auth.py::test_list_active_sessions_requires_authentication - psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port 55439 failed: FATAL: could not open file "global/pg_filenode.map": No such file or di...
ERROR apps/api/tests/test_auth.py::test_terminate_own_session_revokes_it - psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port 55439 failed: FATAL: could not open file "global/pg_filenode.map": No such file or di...
ERROR apps/api/tests/test_auth.py::test_user_cannot_terminate_another_users_session - psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port 55439 failed: FATAL: could not open file "global/pg_filenode.map": No such file or di...
ERROR apps/api/tests/test_auth.py::test_mfa_login_returns_challenge_without_session_cookie - psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port 55439 failed: FATAL: could not open file "global/pg_filenode.map": No such file or di...
ERROR apps/api/tests/test_auth.py::test_mfa_required_login_without_active_factor_is_denied - psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port 55439 failed: FATAL: could not open file "global/pg_filenode.map": No such file or di...
ERROR apps/api/tests/test_auth.py::test_valid_totp_verification_creates_session - psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port 55439 failed: FATAL: could not open file "global/pg_filenode.map": No such file or di...
ERROR apps/api/tests/test_auth.py::test_invalid_totp_does_not_create_session - psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port 55439 failed: FATAL: could not open file "global/pg_filenode.map": No such file or di...
ERROR apps/api/tests/test_auth.py::test_mfa_challenge_cannot_be_reused - psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port 55439 failed: FATAL: could not open file "global/pg_filenode.map": No such file or di...
ERROR apps/api/tests/test_mfa_challenge_repository.py::test_challenge_token_is_hashed_and_success_consumes_challenge - psycopg.OperationalError: connection failed: connection toserver at "127.0.0.1", port 55439 failed: FATAL: could not open file "global/pg_filenode.map": No such file or di...
ERROR apps/api/tests/test_mfa_challenge_repository.py::test_invalid_code_attempt_is_committed - psycopg.OperationalError: connection failed: connection to server at "127.0.0.1",port 55439 failed: FATAL: could not open file "global/pg_filenode.map": No such file or di...
ERROR apps/api/tests/test_mfa_challenge_repository.py::test_replayed_totp_code_increments_attempt_count - psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port 55439 failed: FATAL: could not open file "global/pg_filenode.map": No such file or di...

```
ERROR apps/api/tests/test_mfa_challenge_repository.py::test_expired_challenge_is_rejected -
psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port 55439
failed: FATAL:  could not open file "global/pg_filenode.map": No such file or di...
ERROR
apps/api/tests/test_mfa_challenge_repository.py::test_challenge_locks_after_five_invalid_attem
pts - psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port
55439 failed: FATAL:  could not open file "global/pg_filenode.map": No such file or di...
ERROR apps/api/tests/test_mfa_repository.py::test_enroll_activate_and_list_factor -
psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port 55439
failed: FATAL:  could not open file "global/pg_filenode.map": No such file or di...
ERROR apps/api/tests/test_mfa_repository.py::test_active_factor_rejects_reused_totp_step -
psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port55439
failed: FATAL:  could not open file "global/pg_filenode.map": No such file or di...
ERROR
apps/api/tests/test_mfa_repository.py::test_invalid_activation_code_leaves_factor_pending -
psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port 55439
failed: FATAL:  could not open file "global/pg_filenode.map": No such file or di...
ERROR
apps/api/tests/test_mfa_repository.py::test_factor_cannot_be_accessed_by_another_user -
psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port 55439
failed: FATAL:  could not open file "global/pg_filenode.map": No such file or di...
ERROR apps/api/tests/test_mfa_repository.py::test_revoked_factor_cannot_be_verified -
psycopg.OperationalError: connection failed: connection to server at "127.0.0.1", port 55439
failed: FATAL:  could not open file "global/pg_filenode.map": No such file or di...
7 failed, 26 passed, 2 warnings, 27 errors in 4.05s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % printf '\n===== POSTGRES
PROCESS =====\n'
ps aux | grep '[p]ostgres'

printf '\n===== PORT 55439 OWNER =====\n'
lsof -nP -iTCP:55439 -sTCP:LISTEN

printf '\n===== DATABASE CONNECTION CONFIG =====\n'
sed -n '1,100p' apps/api/app/database.py

printf '\n===== POSTGRES CLIENT VERSION =====\n'
pg_isready -h localhost -p 55439
psql --version

===== POSTGRES PROCESS =====
postgres         81537   0.0  0.0 435390976   5152   ??  S    10:26PM   0:00.09 /usr/libexec/trustd
--agent
```

```
postgres         81536   0.0  0.0 435388240   3680   ??  S    10:26PM   0:00.07 /usr/libexec/containermanagerd --runmode=agent --user-container-mode=current --bundle-container-mode=proxy --system-container-mode=none
postgres         81535   0.0  0.0 435355104   1904   ??  S    10:26PM   0:00.15 /usr/sbin/cfprefsd agent
postgres         81534   0.0  0.0 435365776   9856   ??  S    10:26PM   0:00.08 /usr/libexec/pkd
sandeeph         11364   0.0  0.0 435496448   1840   ??  Ss   Tue11AM   0:00.09 postgres: logical replication launcher
sandeeph         11363   0.0  0.0 435496448   2272   ??  Ss   Tue11AM   0:01.63 postgres: autovacuum launcher
sandeeph         11362   0.0  0.0 435495424    640   ??  Ss   Tue11AM   0:00.73 postgres: walwriter
sandeeph         11360   0.0  0.0 435495424    800   ??  Ss   Tue11AM   0:00.69 postgres: background writer
sandeeph         11359   0.0  0.0 435495424    880   ??  Ss   Tue11AM   0:00.23 postgres: checkpointer
sandeeph         11290   0.0  0.0 435499824   6160   ??  S    Tue11AM   0:12.57 /opt/homebrew/opt/postgresql@17/bin/postgres -D /opt/homebrew/var/postgresql@17
postgres         56366   0.0  0.0 435401920   4416   ??  S    Mon05PM   0:01.46 /usr/libexec/lsd
postgres         56354   0.0  0.0 435401744   4272   ??  SN   Mon05PM   0:19.98 /System/Library/Frameworks/CoreServices.framework/Versions/A/Frameworks/Metadata.framework/Versions/A/Support/mdbulkimport -s mdworker-bundle -c MDSImporterBundleFinder -m com.apple.metadata.mdbulkimport
postgres         56353   0.0  0.0 435354144   2080   ??  S    Mon05PM   0:07.65 /usr/sbin/distnoted agent
sandeeph         96435   0.0  0.0 435496336   1840   ??  Ss   Mon11AM   0:00.13 postgres: logical replication launcher
sandeeph         96434   0.0  0.0 435496336   2272   ??  Ss   Mon11AM   0:03.41 postgres: autovacuum launcher
sandeeph         96433   0.0  0.0 435495312    752   ??  Ss   Mon11AM   0:01.09 postgres: walwriter
sandeeph         96431   0.0  0.0 435495312   1024   ??  Ss   Mon11AM   0:00.95 postgres: background writer
sandeeph         96430   0.0  0.0 435495312   1744   ??  Ss   Mon11AM   0:00.75 postgres: checkpointer
sandeeph         96429   0.0  0.0 435499424   6144   ??  Ss   Mon11AM   0:25.25 /opt/homebrew/Cellar/postgresql@17/17.6_1/bin/postgres -D /private/tmp/finsight-fin13-pg -p 55439

===== PORT 55439 OWNER =====
COMMAND    PID     USER   FD   TYPE             DEVICE SIZE/OFF NODE NAME
postgres 96429 sandeeph    7u  IPv4 0xed34230699a28617      0t0  TCP 127.0.0.1:55439 (LISTEN)
postgres 96429 sandeeph    8u  IPv6 0xeed1c910710922ca      0t0  TCP [::1]:55439 (LISTEN)
```

```
===== DATABASE CONNECTION CONFIG =====
"""Small synchronous PostgreSQL boundary for the auth endpoints."""

from __future__ import annotations

import os
from pathlib import Path

import psycopg
from dotenv import load_dotenv

load_dotenv(Path(__file__).resolve().parents[3] / ".env")


def connect() -> psycopg.Connection:
    database_url = os.environ.get("DATABASE_URL")
    if not database_url:
        raise RuntimeError("DATABASE_URL is required")
    return psycopg.connect(database_url, connect_timeout=3)

===== POSTGRES CLIENT VERSION =====
localhost:55439 - accepting connections
psql (PostgreSQL) 17.6 (Homebrew)
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % printf '\n===== DATA
DIRECTORY STATUS =====\n'
ls -ld /private/tmp/finsight-fin13-pg
ls -la /private/tmp/finsight-fin13-pg/global 2>&1
ls -l /private/tmp/finsight-fin13-pg/PG_VERSION 2>&1

printf '\n===== POSTGRES SERVER IDENTITY =====\n'
psql 'postgresql://sandeepeh@localhost:55439/postgres' \
  -X -v ON_ERROR_STOP=1 \
  -c 'SELECT version();' \
  -c 'SHOW data_directory;' \
  -c 'SHOW port;'

printf '\n===== TEMP DIRECTORY AND PROJECT CONFIG =====\n'
grep -n -E 'finsight-fin13-pg|55439|DATABASE_URL' .env .env.example 2>/dev/null

===== DATA DIRECTORY STATUS =====
drwx------@ 12 sandeeph  wheel  384 Oct 23 00:14 /private/tmp/finsight-fin13-pg
total 160
```

```
drwx------@  7 sandeeph  wheel    224 Oct 23 00:14 .
drwx------@ 12 sandeeph  wheel    384 Oct 23 00:14 ..
-rw-------@  1 sandeeph  wheel   8192 Oct 21 13:14 1262
-rw-------@  1 sandeeph  wheel  16384 Oct 21 13:15 2671
-rw-------@  1 sandeeph  wheel  16384 Oct 21 13:15 2672
-rw-------@  1 sandeeph  wheel   8192 Oct 22 16:27 pg_control
-rw-------@  1 sandeeph  wheel  28724 Oct 21 13:14 pg_internal.init
ls: /private/tmp/finsight-fin13-pg/PG_VERSION: No such file or directory

===== POSTGRES SERVER IDENTITY =====
psql: error: connection to server at "localhost" (::1), port 55439 failed: FATAL:  could not open
file "global/pg_filenode.map": No such file or directory

===== TEMP DIRECTORY AND PROJECT CONFIG =====
.env.example:1:# Local development placeholders only. Replace DATABASE_URL locally; never
commit it.
.env.example:2:DATABASE_URL=postgresql://USER:PASSWORD@127.0.0.1:5432/finsight
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
_URL=postgresql://USER:PASSWORD@127.0.0.1:5432/finsight
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % printf '\n===== PROJECT
DATABASE SCRIPTS =====\n'
find . -maxdepth 4 -type f \

-iname' * postgres * ' -o -iname' * database * ' -o -iname' * db * setup * ' -o -iname' * migra
\
  -not -path './.venv/*' \
  -not -path './node_modules/*' | sort | head -100

printf '\n===== POSTGRES BACKUP FILES =====\n'
find /private/tmp "$HOME/Documents/NVDIA_Play_Book" \
  -maxdepth 4 -type f \
  -name' *.dump' -o -name' *.backup' -o -name' *.sql' -o -name' *.sql.gz' \
  -print 2>/dev/null | head -100

printf '\n===== POSTGRES CLUSTER CONFIG FILES =====\n'
ls -la /private/tmp/finsight-fin13-pg
find /private/tmp/finsight-fin13-pg -maxdepth 2 \
  -type f
-name'postgresql.conf' -o -name'pg_hba.conf' -o -name'postmaster.opts' -o -name'postmaster.
\
  -print

===== PROJECT DATABASE SCRIPTS =====
./Dcos/FinSight_AI_Database_Schema_and_ERD_Reference.pdf
```

```
./apps/api/app/database.py
./docs/architecture/adr/0002-postgresql-source-of-truth.md
./docs/architecture/adr/0004-migrations-and-environment-configuration.md
./htmlcov/z_4cf260f6e38b59ee_database_py.html

===== POSTGRES BACKUP FILES =====
/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/Database/seed_reference_data.s
ql
/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/Database/schema (1).sql
/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/Database/schema_optional_tradin
g.sql

===== POSTGRES CLUSTER CONFIG FILES =====
total 16
drwx------@ 12 sandeeph  wheel   384 Oct 23 00:14 .
drwxrwxrwt  35 root      wheel  1120 Oct 23 10:41 ..
drwx------@  8 sandeeph  wheel   256 Oct 21 13:14 base
drwx------@  7 sandeeph  wheel   224 Oct 23 00:14 global
drwx------@  3 sandeeph  wheel    96 Oct 23 00:14 pg_logical
drwx------@  4 sandeeph  wheel   128 Oct 18 22:08 pg_multixact
drwx------@  2 sandeeph  wheel    64 Oct 19 11:25 pg_stat
drwx------@  3 sandeeph  wheel    96 Oct 18 22:08 pg_subtrans
drwx------@  4 sandeeph  wheel   128 Oct 23 00:14 pg_wal
drwx------@  3 sandeeph  wheel    96 Oct 18 22:08 pg_xact
-rw-------@  1 sandeeph  wheel   106 Oct 19 11:25 postmaster.opts
-rw-------@  1 sandeeph  wheel    98 Oct 19 11:25 postmaster.pid
/private/tmp/finsight-fin13-pg/postmaster.pid
/private/tmp/finsight-fin13-pg/postmaster.opts
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % printf '\n===== MIGRATION
FILES =====\n'
find apps/api -maxdepth 5 -type f \
  − name' *.sql' − o − iname' * migration * ' -print | sort

printf '\n===== DATABASE MIGRATION REFERENCES =====\n'
grep -R -n -E 'finsight-fin13-pg|finsight_fin15_test|55439|DATABASE_URL|0005_mfa|0006_mfa'
\
  scripts apps/api/tests docs/architecture/adr \
  --exclude-dir='__pycache__' 2>/dev/null | head -160

printf '\n===== POSTGRES STARTUP OPTIONS =====\n'
cat /private/tmp/finsight-fin13-pg/postmaster.opts
cat /private/tmp/finsight-fin13-pg/postmaster.pid
```

```
printf '\n===== CURRENT ENVIRONMENT DATABASE URL =====\n'
if [ -n "$DATABASE_URL" ]; then
  python -c 'import os, urllib.parse; u=urllib.parse.urlsplit(os.environ["DATABASE_URL"]);
print("host:", u.hostname, "port:", u.port, "database:", u.path.lstrip("/"), "username:",
u.username)'
else
  echo 'DATABASE_URL is not exported in this shell'
fi

===== MIGRATION FILES =====
apps/api/migrations/0001_auth_identity.sql
apps/api/migrations/0002_platform_authorization.sql
apps/api/migrations/0003_auth_audit_and_login_throttling.sql
apps/api/migrations/0004_immutable_auth_audit.sql
apps/api/migrations/0005_mfa_factors.sql
apps/api/migrations/0006_mfa_login_challenges.sql

===== DATABASE MIGRATION REFERENCES =====
scripts/run_live_e2e.py:51:    if not os.environ.get("DATABASE_URL"):
scripts/run_live_e2e.py:52:        raise SystemExit("DATABASE_URL must point to a disposable
PostgreSQL test database")
scripts/package_release.py:54:            "database_migrations": ["0001_auth_identity.sql",
"0002_platform_authorization.sql", "0003_auth_audit_and_login_throttling.sql",
"0004_immutable_auth_audit.sql", "0005_mfa_factors.sql", "0006_mfa_login_challenges.sql"],
apps/api/tests/test_auth_unit.py:164:    monkeypatch.delenv("DATABASE_URL", raising=False)
apps/api/tests/test_auth_unit.py:166:    with pytest.raises(RuntimeError,
match="DATABASE_URL is required"):
apps/api/tests/test_provision_user.py:23:    assert "DATABASE_URL" not in result.stderr

===== POSTGRES STARTUP OPTIONS =====
/opt/homebrew/Cellar/postgresql@17/17.6_1/bin/postgres "-D" "/private/tmp/finsight-fin13-pg"
"-p" "55439"
96429
/private/tmp/finsight-fin13-pg
1792389340
55439
/tmp
localhost
 92097809    196610
ready

===== CURRENT ENVIRONMENT DATABASE URL =====
DATABASE_URL is not exported in this shell
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
```

E_URL is not exported in this shell
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % printf '\n===== DATABASE SETUP DOCUMENTATION =====\n'
sed -n '1,220p' docs/architecture/adr/0002-postgresql-source-of-truth.md
sed -n '1,220p' docs/architecture/adr/0004-migrations-and-environment-configuration.md

printf '\n===== MIGRATION TABLES AND DEPENDENCIES =====\n'
for f in apps/api/migrations/000*.sql; do
  echo
  echo "===== $f ====="
  sed -n '1,100p' "$f"
done

printf '\n===== CHECK FOR OTHER POSTGRES CLUSTERS =====\n'
find /opt/homebrew/var "$HOME/Library/Application Support/Postgres" \
  /private/tmp -maxdepth 3 -type f -name PG_VERSION -print 2>/dev/null

===== DATABASE SETUP DOCUMENTATION =====
ADR-0002: PostgreSQL as structured source of truth

- **Status:** Accepted; current implementation and project baseline
- **Context:** Current login, user and session operations persist to PostgreSQL. The schema reference defines a broader relational target model.
- **Decision:** Use PostgreSQL as authoritative storage for implemented structured identity/session records. Keep the current implementation limited to its `roles`, `users`, and`auth_sessions` migration subset.
- **Alternatives considered:** In-memory/session-cookie-only persistence or a document database. These do not match the persistence baseline and would not provide current cross-request revocation.
- **Consequences:** API availability depends on PostgreSQL for authenticated operations; tests use disposable PostgreSQL. pgvector and the broader target entities are not implemented by this decision's current scope.

ADR-0004: Versioned migrations and environment configuration

- **Status:** Accepted; partially implemented
- **Context:** The supplied schema is a target reference, not an instruction to deploy every table. Runtime environments need different database and origin settings.
- **Decision:** Keep reference schema files immutable; implement schema changes as versioned migration files and load environment-specific values outside application source. Usesafe placeholder templates only.
- **Alternatives considered:** Applying the full target SQL directly or embedding credentials/configuration in code. Both increase data and secret risk.

- **Consequences:** The current identity migration is a one-time SQL script for a new development/test database. A general migration runner, production ownership, and deployed environment are not yet established; production migration strategy needs an owner and forward/rollback plan.

===== MIGRATION TABLES AND DEPENDENCIES =====

===== apps/api/migrations/0001_auth_identity.sql =====

```
-- FIN-13 identity/session subset. Derived from Database/schema (1).sql.
-- Apply only to a new development/test database; this is not the full target schema.
BEGIN;

CREATE TABLE roles (
    role_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    role_key VARCHAR(50) NOT NULL UNIQUE,
    display_name VARCHAR(100) NOT NULL,
    scope VARCHAR(20) NOT NULL CHECK (scope IN ('platform','workspace')),
    description TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE users (
    user_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(320) NOT NULL,
    password_hash TEXT,
    display_name VARCHAR(160) NOT NULL,
    account_status VARCHAR(20) NOT NULL DEFAULT 'active'
        CHECK (account_status IN ('invited','active','locked','disabled','deleted')),
    platform_role_id UUID REFERENCES roles(role_id) ON DELETE SET NULL,
    email_verified_at TIMESTAMPTZ,
    mfa_required BOOLEAN NOT NULL DEFAULT false,
    last_login_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    deleted_at TIMESTAMPTZ
);
CREATE UNIQUE INDEX uq_users_email_lower ON users (lower(email));

CREATE TABLE auth_sessions (
    session_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    refresh_token_hash TEXT UNIQUE,
    ip_address INET,
```

```
    user_agent TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    last_seen_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    expires_at TIMESTAMPTZ NOT NULL,
    revoked_at TIMESTAMPTZ,
    revoke_reason VARCHAR(80)
);
CREATE INDEX ix_auth_sessions_active ON auth_sessions(user_id, expires_at) WHERE
revoked_at IS NULL;

COMMIT;

===== apps/api/migrations/0002_platform_authorization.sql =====
-- FIN-15: only the platform-scoped administrator permission is implemented.
-- Workspace roles and workspace permissions remain unavailable until workspace
membership exists.
BEGIN;

CREATE TABLE permissions (
    permission_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    permission_key VARCHAR(120) NOT NULL UNIQUE,
    description TEXT NOT NULL
);

CREATE TABLE role_permissions (
    role_id UUID NOT NULL REFERENCES roles(role_id) ON DELETE CASCADE,
    permission_id UUID NOT NULL REFERENCES permissions(permission_id) ON DELETE
CASCADE,
    PRIMARY KEY (role_id, permission_id)
);

INSERT INTO roles (role_key, display_name, scope, description)
VALUES ('platform_admin', 'Platform Administrator', 'platform',
        'Administrator privileges for currently implemented platform operations only.')
ON CONFLICT (role_key) DO NOTHING;

DO $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM roles WHERE role_key = 'platform_admin' AND scope =
'platform') THEN
        RAISE EXCEPTION 'Existing platform_admin role has non-platform scope';
    END IF;
END
$$;
```

```
INSERT INTO permissions (permission_key, description)
VALUES ('platform.admin', 'Access operations explicitly assigned platform administrator
permission.')
ON CONFLICT (permission_key) DO NOTHING;

INSERT INTO role_permissions (role_id, permission_id)
SELECT r.role_id, p.permission_id
FROM roles r CROSS JOIN permissions p
WHERE r.role_key = 'platform_admin' AND r.scope = 'platform'
  AND p.permission_key = 'platform.admin'
ON CONFLICT DO NOTHING;

CREATE FUNCTION enforce_platform_role_scope() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
   IF NEW.platform_role_id IS NOT NULL AND NOT EXISTS (
      SELECT 1 FROM roles WHERE role_id = NEW.platform_role_id AND scope = 'platform'
   ) THEN
      RAISE EXCEPTION 'platform_role_id must reference a platform-scoped role'
         USING ERRCODE = '23514';
   END IF;
   RETURN NEW;
END
$$;

CREATE TRIGGER users_platform_role_scope_check
BEFORE INSERT OR UPDATE OF platform_role_id ON users
FOR EACH ROW EXECUTE FUNCTION enforce_platform_role_scope();

CREATE FUNCTION prevent_platform_role_scope_drift() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
   IF OLD.scope = 'platform' AND NEW.scope <> 'platform' AND EXISTS (
      SELECT 1 FROM users WHERE platform_role_id = OLD.role_id
   ) THEN
      RAISE EXCEPTION 'Cannot change scope of an assigned platform role';
   END IF;
   RETURN NEW;
END
$$;

CREATE TRIGGER roles_prevent_scope_drift
BEFORE UPDATE OF scope ON roles
```

```
FOR EACH ROW EXECUTE FUNCTION prevent_platform_role_scope_drift();

COMMIT;

===== apps/api/migrations/0003_auth_audit_and_login_throttling.sql =====
BEGIN;

-- Authentication/security audit trail.
-- workspace_id remains nullable because the current application schema
-- does not yet contain the workspaces table.
CREATE TABLE audit_events (
    audit_event_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id UUID NULL,
    actor_user_id UUID NULL REFERENCES users(user_id) ON DELETE SET NULL,
    action_key VARCHAR(120) NOT NULL,
    resource_type VARCHAR(80) NOT NULL,
    resource_id UUID NULL,
    outcome VARCHAR(20) NOT NULL
        CHECK (outcome IN ('success', 'failure', 'denied')),
    correlation_id UUID NULL,
    details_redacted JSONB NOT NULL DEFAULT '{}'::jsonb,
    occurred_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_audit_events_occurred_at
    ON audit_events (occurred_at DESC);

CREATE INDEX idx_audit_events_actor_occurred
    ON audit_events (actor_user_id, occurred_at DESC);

CREATE INDEX idx_audit_events_action_occurred
    ON audit_events (action_key, occurred_at DESC);

-- Persistent, shared login-failure counters.
-- key_hash must be a keyed digest generated by the application;
-- raw email addresses, IP addresses, passwords and tokens do not belong here.
CREATE TABLE auth_login_throttle (
    scope_type VARCHAR(16) NOT NULL
        CHECK (scope_type IN ('account', 'ip')),
    key_hash CHAR(64) NOT NULL,
    failed_attempts INTEGER NOT NULL DEFAULT 0
        CHECK (failed_attempts >= 0),
    window_started_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    blocked_until TIMESTAMPTZ NULL,
```

```
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (scope_type, key_hash)
);

CREATE INDEX idx_auth_login_throttle_blocked_until
    ON auth_login_throttle (blocked_until)
    WHERE blocked_until IS NOT NULL;

COMMIT;

===== apps/api/migrations/0004_immutable_auth_audit.sql =====
BEGIN;

-- Preserve the actor UUID and prevent hard deletion from mutating history.
ALTER TABLE audit_events
    DROP CONSTRAINT audit_events_actor_user_id_fkey;

ALTER TABLE audit_events
    ADD CONSTRAINT audit_events_actor_user_id_fkey
    FOREIGN KEY (actor_user_id)
    REFERENCES users(user_id)
    ON DELETE RESTRICT;

-- Audit events are append-only. Corrections must be represented by new events.
CREATE FUNCTION reject_audit_events_mutation()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    RAISE EXCEPTION 'audit_events is append-only; % is not permitted', TG_OP
        USING ERRCODE = '55000';
END;
$$;

CREATE TRIGGER audit_events_append_only
BEFORE UPDATE OR DELETE ON audit_events
FOR EACH ROW
EXECUTE FUNCTION reject_audit_events_mutation();

COMMIT;

===== apps/api/migrations/0005_mfa_factors.sql =====
-- FIN-14 MFA factor metadata; secret material remains in the configured secret store.
BEGIN;
```

```
CREATE TABLE mfa_factors (
   mfa_factor_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
   user_id UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
   factor_type VARCHAR(30) NOT NULL CHECK (factor_type IN
('totp','webauthn','recovery_code')),
   label VARCHAR(120),
   secret_ref TEXT,
   public_key_data JSONB,
   factor_status VARCHAR(20) NOT NULL DEFAULT 'pending'
      CHECK (factor_status IN ('pending', 'active', 'revoked')),
   enrolled_at TIMESTAMPTZ,
   last_used_at TIMESTAMPTZ,
   created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX ix_mfa_factors_user_status
   ON mfa_factors(user_id, factor_status);

COMMIT;

===== apps/api/migrations/0006_mfa_login_challenges.sql =====
BEGIN;

CREATE TABLE mfa_login_challenges (
   challenge_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
   user_id UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
   challenge_token_hash TEXT NOT NULL UNIQUE,
   expires_at TIMESTAMPTZ NOT NULL,
   attempt_count INTEGER NOT NULL DEFAULT 0
      CHECK (attempt_count >= 0 AND attempt_count <= 5),
   consumed_at TIMESTAMPTZ,
   created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX ix_mfa_login_challenges_user_expiry
   ON mfa_login_challenges(user_id, expires_at);

CREATE TABLE mfa_totp_replay_guard (
   mfa_factor_id UUID PRIMARY KEY
      REFERENCES mfa_factors(mfa_factor_id) ON DELETE CASCADE,
   last_accepted_step BIGINT NOT NULL
      CHECK (last_accepted_step >= 0),
   updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
```

```
);

COMMIT;

===== CHECK FOR OTHER POSTGRES CLUSTERS =====
/opt/homebrew/var/postgresql@17/PG_VERSION
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % printf '\n===== DEFAULT
POSTGRES SERVER =====\n'
pg_isready -h localhost -p 5432

printf '\n===== POSTGRES CLUSTER VERSION =====\n'
cat /opt/homebrew/var/postgresql@17/PG_VERSION

printf '\n===== DATABASES ON PORT 5432 =====\n'
psql -h localhost -p 5432 -d postgres \
  -X -v ON_ERROR_STOP=1 \
  -c 'SELECT current_user, version();' \
  -c "SELECT datname FROM pg_database WHERE datistemplate = false ORDER BY
datname;"

printf '\n===== TEST DATABASE ON PORT 5432 =====\n'
psql -h localhost -p 5432 -d postgres \
  -X -v ON_ERROR_STOP=1 \
  -c "SELECT datname FROM pg_database WHERE datname = 'finsight_fin15_test';"

===== DEFAULT POSTGRES SERVER =====
localhost:5432 - accepting connections

===== POSTGRES CLUSTER VERSION =====
17

===== DATABASES ON PORT 5432 =====
 current_user |                                                 version
--------------+---------------------------------------------------------------------------------------------------------
--------------
 sandeeph     | PostgreSQL 17.6 (Homebrew) on aarch64-apple-darwin25.0.0, compiled by
Apple clang version 17.0.0 (clang-1700.3.19.1), 64-bit
(1 row)

    datname
---------------
 finsight_test
 postgres
```

```
(2 rows)


===== TEST DATABASE ON PORT 5432 =====
 datname
---------
(0 rows)

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %




(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % printf '\n===== TABLES IN
finsight_test =====\n'
psql -h localhost -p 5432 -U sandeeph -d finsight_test \
  -X -v ON_ERROR_STOP=1 \
  -c "SELECT table_schema, table_name
      FROM information_schema.tables
      WHERE table_type = 'BASE TABLE'
        AND table_schema NOT IN ('pg_catalog', 'information_schema')
      ORDER BY table_schema, table_name;"

printf '\n===== MIGRATION TRACKING TABLES =====\n'
psql -h localhost -p 5432 -U sandeeph -d finsight_test \
  -X -v ON_ERROR_STOP=1 \
  -c "SELECT table_schema, table_name
      FROM information_schema.tables
      WHERE table_name ILIKE '%migration%'
         OR table_name ILIKE '%schema_version%'
      ORDER BY table_schema, table_name;"

printf '\n===== MFA TABLES ALREADY PRESENT =====\n'
psql -h localhost -p 5432 -U sandeeph -d finsight_test \
  -X -v ON_ERROR_STOP=1 \
  -c "SELECT table_name
      FROM information_schema.tables
      WHERE table_schema = 'public'
        AND table_name IN
          ('users', 'roles', 'auth_sessions', 'audit_events',
           'auth_login_throttle', 'mfa_factors',
           'mfa_login_challenges', 'mfa_totp_replay_guard')
      ORDER BY table_name;"
```

```
===== TABLES IN finsight_test =====
 table_schema |    table_name
--------------+------------------
 public       | auth_sessions
 public       | permissions
 public       | role_permissions
 public       | roles
 public       | users
(5 rows)


===== MIGRATION TRACKING TABLES =====
 table_schema | table_name
--------------+------------
(0 rows)


===== MFA TABLES ALREADY PRESENT =====
  table_name
---------------
 auth_sessions
 roles
 users
(3 rows)

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % printf '\n===== DATABASE
OWNERSHIP =====\n'
psql -h localhost -p 5432 -U sandeeph -d postgres \
  -X -v ON_ERROR_STOP=1 \
  -c "SELECT datname, pg_get_userbyid(datdba) AS owner
      FROM pg_database
      WHERE datname IN ('finsight_test', 'postgres')
      ORDER BY datname;"

printf '\n===== EXISTING TEST DATABASE ROW COUNTS =====\n'
psql -h localhost -p 5432 -U sandeeph -d finsight_test \
  -X -v ON_ERROR_STOP=1 \
  -c "SELECT
        (SELECT count(*) FROM users) AS users,
        (SELECT count(*) FROM roles) AS roles,
        (SELECT count(*) FROM auth_sessions) AS sessions;"
```

```
printf '\n===== DATABASE CREATION PRIVILEGE =====\n'
psql -h localhost -p 5432 -U sandeeph -d postgres \
  -X -v ON_ERROR_STOP=1 \
  -c "SELECT current_user,
            rolsuper,
            rolcreatedb
      FROM pg_roles
      WHERE rolname = current_user;"

===== DATABASE OWNERSHIP =====
    datname    |  owner
---------------+----------
 finsight_test | sandeeph
 postgres      | sandeeph
(2 rows)


===== EXISTING TEST DATABASE ROW COUNTS =====
 users | roles | sessions
-------+-------+----------
     0 |     1 |        0
(1 row)


===== DATABASE CREATION PRIVILEGE =====
 current_user | rolsuper | rolcreatedb
--------------+----------+-------------
 sandeeph     | t        | t
(1 row)

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % psql -h localhost -p 5432 -U
sandeeph -d finsight_fin15_test \
  -X -v ON_ERROR_STOP=1 \
  -c 'SELECT current_database(), current_user, version();'
  current_database   | current_user |                                                 version
---------------------+--------------+---------------------------------------------------------------------------------------------------
-------------------------------------
 finsight_fin15_test | sandeeph     | PostgreSQL 17.6 (Homebrew) on
aarch64-apple-darwin25.0.0, compiled by Apple clang version 17.0.0 (clang-1700.3.19.1), 64-bit
(1 row)

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % set -e
```

```
for migration in apps/api/migrations/000{1,2,3,4,5,6}_*.sql; do
  echo "Applying $migration"
  psql -h localhost -p 5432 -U sandeeph \
    -d finsight_fin15_test \
    -X -v ON_ERROR_STOP=1 \
    -f "$migration"
done
Applying apps/api/migrations/0001_auth_identity.sql
BEGIN
CREATE TABLE
CREATE TABLE
CREATE INDEX
CREATE TABLE
CREATE INDEX
COMMIT
Applying apps/api/migrations/0002_platform_authorization.sql
BEGIN
CREATE TABLE
CREATE TABLE
INSERT 0 1
DO
INSERT 0 1
INSERT 0 1
CREATE FUNCTION
CREATE TRIGGER
CREATE FUNCTION
CREATE TRIGGER
COMMIT
Applying apps/api/migrations/0003_auth_audit_and_login_throttling.sql
BEGIN
CREATE TABLE
CREATE INDEX
CREATE INDEX
CREATE INDEX
CREATE TABLE
CREATE INDEX
COMMIT
Applying apps/api/migrations/0004_immutable_auth_audit.sql
BEGIN
ALTER TABLE
ALTER TABLE
CREATE FUNCTION
CREATE TRIGGER
COMMIT
```

```
Applying apps/api/migrations/0005_mfa_factors.sql
BEGIN
CREATE TABLE
CREATE INDEX
COMMIT
Applying apps/api/migrations/0006_mfa_login_challenges.sql
BEGIN
CREATE TABLE
CREATE INDEX
CREATE TABLE
COMMIT
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % psql -h localhost -p 5432 -U
sandeeph -d finsight_fin15_test \
  -X -v ON_ERROR_STOP=1 \
  -c "SELECT table_name
      FROM information_schema.tables
      WHERE table_schema = 'public'
        AND table_type = 'BASE TABLE'
      ORDER BY table_name;"
      table_name
----------------------
 audit_events
 auth_login_throttle
 auth_sessions
 mfa_factors
 mfa_login_challenges
 mfa_totp_replay_guard
 permissions
 role_permissions
 roles
 users
(10 rows)

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % PYTHONPATH=apps/api \
DATABASE_URL='postgresql://sandeeph@localhost:5432/finsight_fin15_test' \
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' \
.venv/bin/python -m pytest apps/api/tests -q
.....................................................................................................................
[100%]
=============================================================================
======== warnings summary
=============================================================================
========
```

```
.venv/lib/python3.14/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-packages/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

.venv/lib/python3.14/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-packages/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is deprecated, use anyio.from_thread.BlockingPortal instead.
    _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

apps/api/tests/test_auth.py: 8 warnings
apps/api/tests/test_auth_unit.py: 3 warnings
apps/api/tests/test_fin14_auth_security.py: 1 warning
apps/api/tests/test_identity_constraints.py: 9 warnings

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-packages/starlette/testclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being deprecated, because the expected behaviour on cookie persistence is ambiguous. Set cookies directly on the client instance instead.
    return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
143 passed, 23 warnings in 5.38s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %



(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % printf '\n===== TEST FIXTURES AND CLIENT SETUP =====\n'
sed -n '1,140p' apps/api/tests/test_auth.py

printf '\n===== ORIGIN AND AUTH HELPERS =====\n'
grep -n -E 'def _check_origin|def get_current_user|def _get_mfa_secret_store|def _auth_correlation_id|ALLOWED_ORIGIN|ORIGIN' \
  apps/api/app/main.py

printf '\n===== EXISTING ORIGIN VALIDATION TESTS =====\n'
grep -R -n -E 'Origin|origin|403|CSRF|correlation_id|X-Correlation' \
  apps/api/tests --include='*.py' | head -100
```

```
===== TEST FIXTURES AND CLIENT SETUP =====
from __future__ import annotations

import secrets
from uuid import uuid4

import pytest
from fastapi.testclient import TestClient
from psycopg.rows import dict_row

from app.auth import hash_password, token_digest, verify_password
from app.throttle import make_throttle_key
from app.database import connect
from app.main import SESSION_COOKIE, app

TEST_PASSWORD = secrets.token_urlsafe(32)


@pytest.fixture
def identity():
    email = f"{uuid4().hex}@example.com"
    encoded = hash_password(TEST_PASSWORD)
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "INSERT INTO users (email, password_hash, display_name, account_status) VALUES
(%s, %s, %s, 'active') RETURNING user_id",
            (email, encoded, "Synthetic Analyst"),
        )
        user_id = cur.fetchone()[0]
    try:
        yield user_id, email, encoded
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                "UPDATE auth_sessions SET revoked_at = COALESCE(revoked_at, now()), "
                "revoke_reason = COALESCE(revoke_reason, 'test_cleanup') "
                "WHERE user_id = %s",
                (user_id,),
            )
            cur.execute(
                "UPDATE users SET account_status = 'deleted', deleted_at =
COALESCE(deleted_at, now()) "
                "WHERE user_id = %s",
                (user_id,),
```

```
        )


@pytest.fixture
def client(monkeypatch):
    monkeypatch.setenv("SESSION_TTL_MINUTES", "60")
    monkeypatch.setenv("COOKIE_SECURE", "false")
    monkeypatch.setenv("WEB_ORIGIN", "http://localhost:3000")
    throttle_secret = "test-only-secret-material-at-least-32-bytes"
    monkeypatch.setenv("AUTH_THROTTLE_SECRET", throttle_secret)
    ip_key = make_throttle_key(
        throttle_secret.encode("utf-8"), "ip", "testclient"
    )

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "DELETE FROM auth_login_throttle WHERE scope_type = 'ip' AND key_hash = %s",
            (ip_key,),
        )

    try:
        with TestClient(app) as test_client:
            yield test_client
    finally:
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                "DELETE FROM auth_login_throttle WHERE scope_type = 'ip' AND key_hash = %s",
                (ip_key,),
            )


def test_password_hash_is_argon2_and_never_plaintext():
    encoded = hash_password(TEST_PASSWORD)
    assert encoded.startswith("argon2id")
    assert encoded != TEST_PASSWORD
    assert verify_password(encoded, TEST_PASSWORD)
    assert not verify_password(encoded, "wrong")


def test_login_creates_protected_hashed_expiring_session(client, identity, caplog, capsys):
    user_id, email, encoded = identity
    response = client.post("/auth/login", json={"email": email, "password": TEST_PASSWORD})
    assert response.status_code == 200
```

```
    assert response.json() == {"user_id": str(user_id), "email": email, "display_name": "Synthetic
Analyst"}
    assert TEST_PASSWORD not in response.text and encoded not in response.text
    cookie = response.cookies.get(SESSION_COOKIE)
    assert cookie
    set_cookie = response.headers["set-cookie"].lower()
    assert "httponly" in set_cookie and "samesite=lax" in set_cookie and "path=/" in set_cookie
    with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
        cur.execute("SELECT refresh_token_hash, expires_at, revoked_at FROM auth_sessions
WHERE user_id = %s", (user_id,))
        stored = cur.fetchone()
    assert stored["refresh_token_hash"] == token_digest(cookie)
    assert stored["refresh_token_hash"] != cookie
    assert stored["expires_at"] is not None and stored["revoked_at"] is None
    protected = client.get("/auth/me", cookies={SESSION_COOKIE: cookie})
    assert protected.status_code == 200 and protected.json()["user_id"] == str(user_id)
    captured = capsys.readouterr()
    assert TEST_PASSWORD not in caplog.text + captured.out + captured.err
    assert cookie not in caplog.text + captured.out + captured.err


def test_invalid_credentials_have_same_generic_response(client, identity):
    _, email, _ = identity
    known = client.post("/auth/login", json={"email": email, "password": "wrong-password"})
    absent = client.post("/auth/login", json={"email": "absent@example.com", "password":
"wrong-password"})
    assert (known.status_code, known.json()) == (absent.status_code, absent.json())
    assert known.status_code == 401 and known.json() == {"detail": "Email or password is
incorrect."}
    assert SESSION_COOKIE not in known.cookies and SESSION_COOKIE not in
absent.cookies


def test_logout_revokes_session_and_replay_fails(client, identity):
    _, email, _ = identity
    login = client.post("/auth/login", json={"email": email, "password": TEST_PASSWORD})
    cookie = login.cookies.get(SESSION_COOKIE)
    assert cookie
    logout = client.post("/auth/logout", cookies={SESSION_COOKIE: cookie})
    assert logout.status_code == 204
    replay = client.get("/auth/me", cookies={SESSION_COOKIE: cookie})
    assert replay.status_code == 401
    with connect() as conn, conn.cursor() as cur:
```

```
        cur.execute("SELECT revoked_at FROM auth_sessions WHERE refresh_token_hash =
%s", (token_digest(cookie),))
        assert cur.fetchone()[0] is not None


def test_logout_without_session_is_idempotent(client):
    response = client.post("/auth/logout")

    assert response.status_code == 204
    assert response.headers["set-cookie"].startswith(f"{SESSION_COOKIE}=")


def test_expired_session_is_rejected(client, identity):
    user_id, email, _ = identity
    login = client.post("/auth/login", json={"email": email, "password": TEST_PASSWORD})
    cookie = login.cookies.get(SESSION_COOKIE)
    assert cookie
    with connect() as conn, conn.cursor() as cur:
        cur.execute("UPDATE auth_sessions SET last_seen_at = now() - interval '61 minutes'
WHERE user_id = %s", (user_id,))

===== ORIGIN AND AUTH HELPERS =====
55:def _get_mfa_secret_store(request: Request) -> SecretStore:
93:web_origin = os.environ.get("WEB_ORIGIN", "http://localhost:3000")
149:def _check_origin(request: Request) -> None:
155:def _auth_correlation_id(request: Request) -> UUID:
177:def get_current_user(

===== EXISTING ORIGIN VALIDATION TESTS =====
apps/api/tests/test_fin14_auth_security.py:213:def
test_auth_correlation_id_is_returned_and_audited(monkeypatch):
apps/api/tests/test_fin14_auth_security.py:220:    correlation_id = uuid4()
apps/api/tests/test_fin14_auth_security.py:235:                headers={"X-Correlation-ID":
str(correlation_id)},
apps/api/tests/test_fin14_auth_security.py:239:        assert
response.headers.get("X-Correlation-ID") == str(correlation_id)
apps/api/tests/test_fin14_auth_security.py:243:                """SELECT correlation_id
apps/api/tests/test_fin14_auth_security.py:254:        assert event[0] == correlation_id
apps/api/tests/test_fin14_auth_security.py:270:def
test_invalid_auth_correlation_id_is_rejected(monkeypatch):
apps/api/tests/test_fin14_auth_security.py:280:            headers={"X-Correlation-ID":
"not-a-uuid"},
apps/api/tests/test_fin14_auth_security.py:284:    assert response.json() == {"detail": "Invalid
X-Correlation-ID"}
```

```
apps/api/tests/test_fin14_auth_security.py:287:def
test_failed_login_returns_the_audited_correlation_id(monkeypatch):
apps/api/tests/test_fin14_auth_security.py:292:    correlation_id = uuid4()
apps/api/tests/test_fin14_auth_security.py:301:            headers={"X-Correlation-ID":
str(correlation_id)},
apps/api/tests/test_fin14_auth_security.py:305:    assert
response.headers.get("X-Correlation-ID") == str(correlation_id)
apps/api/tests/test_fin14_auth_security.py:309:            """SELECT correlation_id
apps/api/tests/test_fin14_auth_security.py:313:                AND correlation_id = %s
apps/api/tests/test_fin14_auth_security.py:315:            (correlation_id,),
apps/api/tests/test_fin14_auth_security.py:320:    assert event[0] == correlation_id
apps/api/tests/test_fin14_auth_security.py:323:def
test_generated_auth_correlation_id_matches_audit_record(monkeypatch):
apps/api/tests/test_fin14_auth_security.py:337:    correlation_id =
response.headers.get("X-Correlation-ID")
apps/api/tests/test_fin14_auth_security.py:338:    assert correlation_id is not None
apps/api/tests/test_fin14_auth_security.py:339:    parsed_correlation_id = UUID(correlation_id)
apps/api/tests/test_fin14_auth_security.py:343:            """SELECT correlation_id
apps/api/tests/test_fin14_auth_security.py:347:                AND correlation_id = %s""",
apps/api/tests/test_fin14_auth_security.py:348:            (parsed_correlation_id,),
apps/api/tests/test_fin14_auth_security.py:353:    assert event[0] == parsed_correlation_id
apps/api/tests/test_fin14_auth_security.py:389:                headers={"X-Correlation-ID":
str(failed_correlation)},
apps/api/tests/test_fin14_auth_security.py:396:                headers={"X-Correlation-ID":
str(blocked_correlation)},
apps/api/tests/test_fin14_auth_security.py:406:                headers={"X-Correlation-ID":
str(retry_correlation)},
apps/api/tests/test_fin14_auth_security.py:410:            assert retry.headers["X-Correlation-ID"]
== str(retry_correlation)
apps/api/tests/test_fin14_auth_security.py:414:                """SELECT correlation_id, outcome
apps/api/tests/test_fin14_auth_security.py:416:                  WHERE correlation_id = ANY(%s)
apps/api/tests/test_identity_constraints.py:86:          #₹ The request_id header is optional, but if
supplied it must be validated and preserved as the correlation_id.
apps/api/tests/test_identity_constraints.py:96:          assert correlated_body["correlation_id"] ==
"client-correlation-123"
apps/api/tests/test_identity_constraints.py:114:          assert body["correlation_id"] ==
body["request_id"]
apps/api/tests/test_identity_constraints.py:121:          assert denied.status_code == 403
apps/api/tests/test_identity_constraints.py:200:          assert next_response.status_code ==
403
apps/api/tests/test_identity_constraints.py:279:          assert next_response.status_code ==
403
apps/api/tests/test_auth.py:149:def test_auth_mutation_rejects_foreign_origin(client):
apps/api/tests/test_auth.py:151:        "/auth/logout", headers={"Origin": "https://attacker.invalid"},
```

```
apps/api/tests/test_auth.py:153:    assert response.status_code == 403
apps/api/tests/test_auth.py:211:        headers={"Origin": "http://localhost:3000"},
apps/api/tests/test_auth.py:272:            headers={"Origin": "http://localhost:3000"},
apps/api/tests/test_auth_unit.py:13:def
test_password_helpers_accept_only_the_original_password():
apps/api/tests/test_auth_unit.py:97:    [({"permissions": ["platform.admin"]}, 200),
({"permissions": []}, 403), ({}, 403)],
apps/api/tests/test_auth_unit.py:114:    if expected_status == 403:
apps/api/tests/test_auth_unit.py:132:    assert response.status_code == 403
apps/api/tests/test_audit_service.py:58:        "correlation_id",
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
    assert "secret_ref" not in factors[0]
    assert "secret" not in factors[0]

    # Revocation succeeds and is reflected in subsequent listings.
    revocation = client.delete(f"/auth/mfa/factors/{factor_id}")
    assert revocation.status_code == 204

    after_revoke = client.get("/auth/mfa/factors")
    assert after_revoke.status_code == 200
    assert after_revoke.json()[0]["factor_status"] == "revoked"


def test_mfa_activation_rejects_invalid_code(client, identity, monkeypatch):
    from app.secret_store import FakeSecretStore

    _, email, _ = identity
    monkeypatch.setattr(
        app.state, "mfa_secret_store", FakeSecretStore(), raising=False
    )

    login = client.post(
        "/auth/login",
        json={"email": email, "password": TEST_PASSWORD},
    )
    assert login.status_code == 200

    enrollment = client.post(
        "/auth/mfa/enroll",
        json={"label": "Invalid-code test"},
    )
    assert enrollment.status_code == 201
```

```
    activation = client.post(
        "/auth/mfa/activate",
        json={
            "factor_id": enrollment.json()["factor_id"],
            "code": "not-a-code",
        },
    )
    assert activation.status_code == 400

    factors = client.get("/auth/mfa/factors")
    assert factors.status_code == 200
    assert factors.json()[0]["factor_status"] == "pending"


def test_mfa_factor_listing_requires_authentication(client):
    response = client.get("/auth/mfa/factors")
    assert response.status_code == 401
PY

(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % PYTHONPATH=apps/api \
DATABASE_URL='postgresql://sandeeph@localhost:5432/finsight_fin15_test' \
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' \
.venv/bin/python -m pytest apps/api/tests/test_auth.py -q
.....................
[100%]
========================================================================
======== warnings summary
========================================================================
========
.venv/lib/python3.14/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is
deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

.venv/lib/python3.14/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is
deprecated, use anyio.from_thread.BlockingPortal instead.
    _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

apps/api/tests/test_auth.py::test_login_creates_protected_hashed_expiring_session
```

```
apps/api/tests/test_auth.py::test_logout_revokes_session_and_replay_fails
apps/api/tests/test_auth.py::test_logout_revokes_session_and_replay_fails
apps/api/tests/test_auth.py::test_expired_session_is_rejected
apps/api/tests/test_auth.py::test_list_active_sessions_shows_current_session
apps/api/tests/test_auth.py::test_terminate_own_session_revokes_it
apps/api/tests/test_auth.py::test_terminate_own_session_revokes_it
apps/api/tests/test_auth.py::test_user_cannot_terminate_another_users_session

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being
deprecated, because the expected behaviour on cookie persistence is ambiguous. Set cookies
directly on the client instance instead.
  return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
23 passed, 10 warnings in 2.59s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % PYTHONPATH=apps/api \
DATABASE_URL='postgresql://sandeeph@localhost:5432/finsight_fin15_test' \
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' \
.venv/bin/python -m pytest apps/api/tests -q

printf '\n===== PATCH WHITESPACE CHECK =====\n'
git diff --check
.....................................................................................................
[100%]
=============================================================================
======== warnings summary
=============================================================================
========
.venv/lib/python3.14/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is
deprecated; install `httpx2` instead.
  from starlette.testclient import TestClient as TestClient  # noqa

.venv/lib/python3.14/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is
deprecated, use anyio.from_thread.BlockingPortal instead.
  _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]
```

```
apps/api/tests/test_auth.py: 8 warnings
apps/api/tests/test_auth_unit.py: 3 warnings
apps/api/tests/test_fin14_auth_security.py: 1 warning
apps/api/tests/test_identity_constraints.py: 9 warnings

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being
deprecated, because the expected behaviour on cookie persistence is ambiguous. Set cookies
directly on the client instance instead.
    return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
148 passed, 23 warnings in 5.69s

===== PATCH WHITESPACE CHECK =====
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % printf '\n===== MFA
FACTOR SCHEMA =====\n'
grep -n -A35 -B5 'CREATE TABLE.*mfa_factors' \
  apps/api/migrations/0005_mfa_factors.sql

printf '\n===== SECRET STORE ERRORS =====\n'
sed -n '1,45p' apps/api/app/secret_store.py

printf '\n===== ROUTE ERROR HANDLING =====\n'
grep -n -A35 -B8 'def delete_mfa_factor' apps/api/app/main.py

===== MFA FACTOR SCHEMA =====
1--- FIN-14 MFA factor metadata; secret material remains in the configured secret store.
2-BEGIN;
3-
4:CREATE TABLE mfa_factors (
5-    mfa_factor_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
6-    user_id UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
7-    factor_type VARCHAR(30) NOT NULL CHECK (factor_type IN
('totp','webauthn','recovery_code')),
8-    label VARCHAR(120),
9-    secret_ref TEXT,
10-    public_key_data JSONB,
11-    factor_status VARCHAR(20) NOT NULL DEFAULT 'pending'
12-        CHECK (factor_status IN ('pending', 'active', 'revoked')),
13-    enrolled_at TIMESTAMPTZ,
14-    last_used_at TIMESTAMPTZ,
15-    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
```

```
16-);
17-
18-CREATE INDEX ix_mfa_factors_user_status
19-    ON mfa_factors(user_id, factor_status);
20-
21-COMMIT;
```

```
===== SECRET STORE ERRORS =====
"""Provider-neutral storage for MFA secrets.

The development adapter is for local development only. Production provider
integration must be selected and implemented separately.
"""

from __future__ import annotations

import base64
import os
import secrets
import stat
from pathlib import Path
from typing import Protocol
from uuid import uuid4

from cryptography.exceptions import InvalidTag
from cryptography.hazmat.primitives.ciphers.aead import AESGCM


class SecretStoreError(RuntimeError):
    """Base error for secret-store operations."""


class SecretNotFoundError(SecretStoreError):
    """The requested secret reference does not exist."""


class SecretStoreConfigurationError(SecretStoreError):
    """The configured secret store is missing or unsafe."""


class SecretStore(Protocol):
    """Interface consumed by MFA application services."""

    def put(self, secret: bytes) -> str: ...
```

```
    def get(self, ref: str) -> bytes: ...
    def delete(self, ref: str) -> None: ...


class FakeSecretStore:
    """In-memory adapter with predictable references for automated tests."""

    def __init__(self) -> None:
        self._values: dict[str, bytes] = {}

===== ROUTE ERROR HANDLING =====
615-    _auth_correlation_id(request)
616-    try:
617-        return list_totp_factors(user_id=user["user_id"])
618-    except psycopg.Error as exc:
619-        raise HTTPException(status_code=503, detail="MFA service unavailable") from exc
620-
621-
622-@app.delete("/auth/mfa/factors/{factor_id}", status_code=204)
623:def delete_mfa_factor(
624-    factor_id: UUID,
625-    request: Request,
626-    user: CurrentUser,
627-) -> Response:
628-    _check_origin(request)
629-    correlation_id = _auth_correlation_id(request)
630-
631-    try:
632-        revoked = revoke_totp_factor(
633-            user_id=user["user_id"],
634-            factor_id=factor_id,
635-        )
636-        if not revoked:
637-            raise HTTPException(status_code=404, detail="MFA factor not found")
638-        return Response(status_code=status.HTTP_204_NO_CONTENT)
639-    except HTTPException:
640-        raise
641-    except psycopg.Error as exc:
642-        raise HTTPException(status_code=503, detail="MFA service unavailable") from exc
643-
644-
645-@app.post("/auth/logout", status_code=204)
646-def logout(
647-    request: Request,
```

```
648-    response: Response,
649-    session_token: Annotated[str | None, Cookie(alias=SESSION_COOKIE)] = None,
650-) -> Response:
651-    _check_origin(request)
652-    correlation_id = _auth_correlation_id(request)
653-    response.headers['X-Correlation-ID'] = str(correlation_id)
654-
655-    try:
656-        with connect() as conn:
657-            actor_user_id = None
658-
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % printf '\n===== ALL
REVOCATION CALL SITES =====\n'
grep -R -n --include='*.py' 'revoke_totp_factor(' apps/api

printf '\n===== MAIN.PY SECRET-STORE IMPORTS =====\n'
grep -n -A20 -B5 'secret_store import' apps/api/app/main.py

printf '\n===== REPOSITORY SECRET-STORE IMPORT =====\n'
grep -n -A5 -B5 'secret_store import' apps/api/app/mfa_repository.py

===== ALL REVOCATION CALL SITES =====
apps/api/app/mfa_repository.py:207:def revoke_totp_factor(*, user_id: UUID, factor_id: UUID) ->
bool:
apps/api/app/main.py:632:        revoked = revoke_totp_factor(
apps/api/tests/test_mfa_repository.py:166:    assert revoke_totp_factor(
apps/api/tests/test_mfa_repository.py:189:    assert revoke_totp_factor(user_id=user_id,
factor_id=factor_id) is True

===== MAIN.PY SECRET-STORE IMPORTS =====
25-from pydantic import BaseModel, EmailStr, Field
26-from psycopg.rows import dict_row
27-
28-from app.auth import hash_password, new_session_token, session_expiry, token_digest,
verify_password
29-from app.database import connect
30:from app.secret_store import (
31-    SecretStore,
32-    SecretStoreConfigurationError,
33-    SecretStoreError,
34-    secret_store_from_environment,
35-)
36-from app.mfa_challenge_repository import (
```

```
37-    MFAChallengeInvalidError,
38-    issue_mfa_challenge,
39-    verify_mfa_challenge,
40-)
41-from app.mfa_repository import (
42-    MFAFactorNotFoundError,
43-    MFAInvalidCodeError,
44-    activate_totp_factor,
45-    create_pending_totp_factor,
46-    list_totp_factors,
47-    revoke_totp_factor,
48-)
49-
50-load_dotenv(Path(__file__).resolve().parents[3] / ".env")

===== REPOSITORY SECRET-STORE IMPORT =====
13-import psycopg
14-from psycopg.rows import dict_row
15-
16-from app.database import connect
17-from app.mfa_service import matching_totp_step
18:from app.secret_store import SecretStore
19-
20-
21-class MFAFactorError(RuntimeError):
22-    """Base class for safe MFA factor operation failures."""
23-
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % git diff --
apps/api/app/mfa_repository.py apps/api/app/main.py
diff --git a/apps/api/app/main.py b/apps/api/app/main.py
index 5e0e618..44be4d2 100644
--- a/apps/api/app/main.py
+++ b/apps/api/app/main.py
@@ -7,7 +7,16 @@ import secrets
 from datetime import UTC, datetime
 from pathlib import Path
 from typing import Annotated, Any
-from uuid import UUID
+from uuid import UUID, uuid4
+
+from app.audit import record_auth_event
+from app.throttle import (
+    is_blocked,
```

```
+    load_throttle_config,
+    make_throttle_key,
+    record_failure,
+    reset_bucket,
+)

 import psycopg
 from dotenv import load_dotenv
@@ -18,17 +27,76 @@ from psycopg.rows import dict_row

 from app.auth import hash_password, new_session_token, session_expiry, token_digest,
verify_password
 from app.database import connect
+from app.secret_store import (
+    SecretStore,
+    SecretStoreConfigurationError,
+    SecretStoreError,
+    secret_store_from_environment,
+)
+from app.mfa_challenge_repository import (
+    MFAChallengeInvalidError,
+    issue_mfa_challenge,
+    verify_mfa_challenge,
+)
+from app.mfa_repository import (
+    MFAFactorNotFoundError,
+    MFAInvalidCodeError,
+    activate_totp_factor,
+    create_pending_totp_factor,
+    list_totp_factors,
+    revoke_totp_factor,
+)

 load_dotenv(Path(__file__).resolve().parents[3] / ".env")

 app = FastAPI(title="FinSight AI API", version="0.2.0")
+
+
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % command -v patch
/usr/bin/patch
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
        secret_ref, factor_status = row
        if factor_status != "revoked":
```

```
            cur.execute(
                """
                UPDATE mfa_factors
                SET factor_status = 'revoked'
                WHERE mfa_factor_id = %s AND user_id = %s
                """,
                (factor_id, user_id),
            )

        cur.execute(
            "DELETE FROM mfa_totp_replay_guard WHERE mfa_factor_id = %s",
            (factor_id,),
        )

    # The database transaction has committed; authentication is blocked now.
    if secret_ref:
        try:
            store.delete(secret_ref)
        except SecretNotFoundError:
            # Already absent: cleanup is complete.
            pass

        # Keep the reference if deletion raised another error so cleanup can
        # be retried by repeating the authenticated DELETE request.
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                """
                UPDATE mfa_factors
                SET secret_ref = NULL
                WHERE mfa_factor_id = %s
                  AND user_id = %s
                  AND factor_type = 'totp'
                  AND factor_status = 'revoked'
                  AND secret_ref = %s
                """,
                (factor_id, user_id, secret_ref),
            )

    return True
'''

if old_import not in source:
    raise SystemExit("STOP: expected secret-store import was not found.")
if source.count(old_function) != 1:
```

```
    raise SystemExit("STOP: expected revocation function was not found exactly once.")

path.write_text(source.replace(old_import, new_import, 1).replace(old_function, new_function,
1))
print("Updated apps/api/app/mfa_repository.py")
PY
Updated apps/api/app/mfa_repository.py
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % git diff --
apps/api/app/mfa_repository.py
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
        secret_ref, factor_status = row
        if factor_status != "revoked":
            cur.execute(
                """
                UPDATE mfa_factors
                SET factor_status = 'revoked'
                WHERE mfa_factor_id = %s AND user_id = %s
                """,
                (factor_id, user_id),
            )

        cur.execute(
            "DELETE FROM mfa_totp_replay_guard WHERE mfa_factor_id = %s",
            (factor_id,),
        )

    # The database transaction has committed; authentication is blocked now.
    if secret_ref:
        try:
            store.delete(secret_ref)
        except SecretNotFoundError:
            # Already absent: cleanup is complete.
            pass

        # Keep the reference if deletion raised another error so cleanup can
        # be retried by repeating the authenticated DELETE request.
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                """
                UPDATE mfa_factors
                SET secret_ref = NULL
                WHERE mfa_factor_id = %s
                  AND user_id = %s
```

```
                AND factor_type = 'totp'
                AND factor_status = 'revoked'
                AND secret_ref = %s
            """,
            (factor_id, user_id, secret_ref),
        )

    return True
'''

if old_import not in source:
    raise SystemExit("STOP: expected secret-store import was not found.")
if source.count(old_function) != 1:
    raise SystemExit("STOP: expected revocation function was not found exactly once.")

path.write_text(source.replace(old_import, new_import, 1).replace(old_function, new_function,
1))
print("Updated apps/api/app/mfa_repository.py")
PY
Updated apps/api/app/mfa_repository.py
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % git diff --
apps/api/app/mfa_repository.py
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % git status --short --
apps/api/app/mfa_repository.py

printf '\n===== UPDATED REVOCATION FUNCTION =====\n'
sed -n '12,22p' apps/api/app/mfa_repository.py
sed -n '205,275p' apps/api/app/mfa_repository.py

printf '\n===== SYNTAX CHECK =====\n'
PYTHONPATH=apps/api .venv/bin/python -m py_compile \
  apps/api/app/mfa_repository.py
?? apps/api/app/mfa_repository.py

===== UPDATED REVOCATION FUNCTION =====

import psycopg
from psycopg.rows import dict_row

from app.database import connect
from app.mfa_service import matching_totp_step
from app.secret_store import SecretNotFoundError, SecretStore
```

```
class MFAFactorError(RuntimeError):
    """Base class for safe MFA factor operation failures."""


def revoke_totp_factor(
    *, user_id: UUID, factor_id: UUID, store: SecretStore
) -> bool:
    """Revoke an owned factor, then delete its secret with retryable cleanup."""
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            """
            SELECT secret_ref, factor_status
            FROM mfa_factors
            WHERE mfa_factor_id = %s
              AND user_id = %s
              AND factor_type = 'totp'
            FOR UPDATE
            """,
            (factor_id, user_id),
        )
        row = cur.fetchone()
        if row is None:
            return False

        secret_ref, factor_status = row
        if factor_status != "revoked":
            cur.execute(
                """
                UPDATE mfa_factors
                SET factor_status = 'revoked'
                WHERE mfa_factor_id = %s AND user_id = %s
                """,
                (factor_id, user_id),
            )

        cur.execute(
            "DELETE FROM mfa_totp_replay_guard WHERE mfa_factor_id = %s",
            (factor_id,),
        )

    # The database transaction has committed; authentication is blocked now.
    if secret_ref:
        try:
            store.delete(secret_ref)
```

```
        except SecretNotFoundError:
            # Already absent: cleanup is complete.
            pass

        # Keep the reference if deletion raised another error so cleanup can
        # be retried by repeating the authenticated DELETE request.
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                """
                UPDATE mfa_factors
                SET secret_ref = NULL
                WHERE mfa_factor_id = %s
                  AND user_id = %s
                  AND factor_type = 'totp'
                  AND factor_status = 'revoked'
                  AND secret_ref = %s
                """,
                (factor_id, user_id, secret_ref),
            )

    return True


def list_totp_factors(*, user_id: UUID) -> list[dict[str, Any]]:
    """List factor metadata only; never return secret references."""
    with connect() as conn, conn.cursor(row_factory=dict_row) as cur:
        cur.execute(
            """
            SELECT mfa_factor_id, label, factor_status, enrolled_at,

===== SYNTAX CHECK =====
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
        raise HTTPException(
            status_code=503, detail="MFA service unavailable"
        ) from exc
    except psycopg.Error as exc:
        raise HTTPException(status_code=503, detail="MFA service unavailable") from exc
'''

# Restrict error-handler replacement to the delete route.
route_start = source.index('@app.delete("/auth/mfa/factors/{factor_id}"')
route_end = source.index('@app.post("/auth/logout"', route_start)
route = source[route_start:route_end]
```

```
if route.count(old_error) != 1:
    raise SystemExit("STOP: expected delete-route error handler was not found exactly once.")

route = route.replace(old_call, new_call, 1).replace(old_error, new_error, 1)
source = source[:route_start] + route + source[route_end:]
path.write_text(source)
print("Updated apps/api/app/main.py")

# Update existing repository tests for the new required store argument.
path = Path("apps/api/tests/test_mfa_repository.py")
source = path.read_text()

old_cross_user = '''    assert revoke_totp_factor(
        user_id=other_user_id,
        factor_id=factor_id,
    ) is False'''

new_cross_user = '''    assert revoke_totp_factor(
        user_id=other_user_id,
        factor_id=factor_id,
        store=store,
    ) is False'''

old_owned = '''    assert revoke_totp_factor(user_id=user_id, factor_id=factor_id) is True'''

new_owned = '''    assert revoke_totp_factor(
        user_id=user_id, factor_id=factor_id, store=store
    ) is True'''

if source.count(old_cross_user) != 1:
    raise SystemExit("STOP: expected cross-user test call was not found exactly once.")
if source.count(old_owned) != 1:
    raise SystemExit("STOP: expected owned-factor test call was not found exactly once.")

source = source.replace(old_cross_user, new_cross_user, 1)
source = source.replace(old_owned, new_owned, 1)
path.write_text(source)
print("Updated apps/api/tests/test_mfa_repository.py")
PY
Updated apps/api/app/main.py
Updated apps/api/tests/test_mfa_repository.py
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % PYTHONPATH=apps/api \
DATABASE_URL='postgresql://sandeeph@localhost:5432/finsight_fin15_test' \
```

```
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' \
.venv/bin/python -m pytest \
  apps/api/tests/test_mfa_repository.py \
  apps/api/tests/test_auth.py -q
.............................
[100%]
============================================================================
======== warnings summary
============================================================================
========
.venv/lib/python3.14/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is
deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

.venv/lib/python3.14/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is
deprecated, use anyio.from_thread.BlockingPortal instead.
    _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

apps/api/tests/test_auth.py::test_login_creates_protected_hashed_expiring_session
apps/api/tests/test_auth.py::test_logout_revokes_session_and_replay_fails
apps/api/tests/test_auth.py::test_logout_revokes_session_and_replay_fails
apps/api/tests/test_auth.py::test_expired_session_is_rejected
apps/api/tests/test_auth.py::test_list_active_sessions_shows_current_session
apps/api/tests/test_auth.py::test_terminate_own_session_revokes_it
apps/api/tests/test_auth.py::test_terminate_own_session_revokes_it
apps/api/tests/test_auth.py::test_user_cannot_terminate_another_users_session

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being
deprecated, because the expected behaviour on cookie persistence is ambiguous. Set cookies
directly on the client instance instead.
    return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
28 passed, 10 warnings in 2.94s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
    secret_ref = "fake://mfa/000001"
```

```
    with pytest.raises(SecretStoreError):
        revoke_totp_factor(
            user_id=user_id,
            factor_id=factor_id,
            store=store,
        )

    # Revocation commits before external secret cleanup is attempted.
    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "SELECT factor_status, secret_ref FROM mfa_factors "
            "WHERE mfa_factor_id = %s",
            (factor_id,),
        )
        factor_status, retained_ref = cur.fetchone()

    assert factor_status == "revoked"
    assert retained_ref == secret_ref

    # Retrying revocation retries deletion and clears the stored reference.
    assert revoke_totp_factor(
        user_id=user_id,
        factor_id=factor_id,
        store=store,
    ) is True

    from app.secret_store import SecretNotFoundError

    with pytest.raises(SecretNotFoundError):
        underlying_store.get(secret_ref)

    with connect() as conn, conn.cursor() as cur:
        cur.execute(
            "SELECT factor_status, secret_ref FROM mfa_factors "
            "WHERE mfa_factor_id = %s",
            (factor_id,),
        )
        factor_status, secret_ref_after_retry = cur.fetchone()

    assert factor_status == "revoked"
    assert secret_ref_after_retry is None
'''
```

```
if "def test_revocation_retries_secret_cleanup_after_store_failure" in source:
    raise SystemExit("STOP: retry test already exists; refusing to append a duplicate.")

path.write_text(source.rstrip() + "\n" + addition)
print("Added secret cleanup and retry tests.")
PY
Added secret cleanup and retry tests.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % PYTHONPATH=apps/api \
DATABASE_URL='postgresql://sandeeph@localhost:5432/finsight_fin15_test' \
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' \
.venv/bin/python -m pytest apps/api/tests/test_mfa_repository.py -q
......
[100%]
6 passed in 0.54s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
        other_cookie = other_login.cookies.get(SESSION_COOKIE)
        assert other_cookie

        response = client.delete(
            f"/auth/mfa/factors/{factor_id}",
            cookies={SESSION_COOKIE: other_cookie},
            headers={"Origin": "http://localhost:3000"},
        )
        assert response.status_code == 404

        # The owner's factor and secret must remain intact.
        with connect() as conn, conn.cursor() as cur:
            cur.execute(
                """
                SELECT factor_status, secret_ref
                FROM mfa_factors
                WHERE mfa_factor_id = %s AND user_id = %s
                """,
                (factor_id, owner_user_id),
            )
            factor_status, secret_ref = cur.fetchone()

        assert factor_status == "pending"
        assert secret_ref
        assert store.get(secret_ref)

    finally:
        with connect() as conn, conn.cursor() as cur:
```

```
        cur.execute(
            """
            UPDATE auth_sessions
            SET revoked_at = COALESCE(revoked_at, now()),
                revoke_reason = COALESCE(revoke_reason, 'test_cleanup')
            WHERE user_id = %s
            """,
            (other_user_id,),
        )
        cur.execute(
            """
            UPDATE users
            SET account_status = 'deleted',
                deleted_at = COALESCE(deleted_at, now())
            WHERE user_id = %s
            """,
            (other_user_id,),
        )
'''

path.write_text(source.rstrip() + "\\n" + test)
print("Added HTTP cross-user MFA revocation test.")
PY
Added HTTP cross-user MFA revocation test.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % PYTHONPATH=apps/api \
DATABASE_URL='postgresql://sandeeph@localhost:5432/finsight_fin15_test' \
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' \
.venv/bin/python -m pytest \
  apps/api/tests/test_mfa_repository.py \
  apps/api/tests/test_auth.py -q

==========================================================================
========== ERRORS
==========================================================================
==========
_____ ERROR
collecting apps/api/tests/test_auth.py
_____
.venv/lib/python3.14/site-packages/_pytest/python.py:507: in importtestmodule
    mod = import_path(
.venv/lib/python3.14/site-packages/_pytest/pathlib.py:587: in import_path
    importlib.import_module(module_name)
/opt/anaconda3/lib/python3.14/importlib/__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
```

```
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
<frozen importlib._bootstrap>:1406: in _gcd_import
   ???
<frozen importlib._bootstrap>:1371: in _find_and_load
   ???
<frozen importlib._bootstrap>:1342: in _find_and_load_unlocked
   ???
<frozen importlib._bootstrap>:938: in _load_unlocked
   ???
.venv/lib/python3.14/site-packages/_pytest/assertion/rewrite.py:188: in exec_module
   source_stat, co = _rewrite_test(fn, self.config)
                     ^^^^^^^^^^^^^^^^^^^
```

```
    response = client.get("/auth/mfa/factors")
    assert response.status_code == 401\n

def test_user_cannot_revoke_another_users_mfa_factor(
    client, identity, monkeypatch
):
    from app.secret_store import FakeSecretStore

    owner_user_id, owner_email, _ = identity
    other_email = f"mfa-other-{uuid4().hex}@example.com"
    other_user_id = None

    with connect() as conn, conn.cursor() as cur:
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % .venv/bin/python - <<'PY'
from pathlib import Path

path = Path("apps/api/tests/test_auth.py")
source = path.read_text()
lines = source.splitlines(keepends=True)

line_number = 579
if len(lines) < line_number:
    raise SystemExit("STOP: File is shorter than the reported line number.")

line = lines[line_number - 1]
if r"\n" not in line:
    raise SystemExit(
        f"STOP: No literal \\\\n found on line {line_number}. "
        "Inspect the file before changing anything."
    )

lines[line_number - 1] = line.replace(r"\n", "", 1)
path.write_text("".join(lines))
print(f"Removed the accidental literal \\\\n from line {line_number}.")
PY
Removed the accidental literal \\n from line 579.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % .venv/bin/python -m
py_compile \
  apps/api/tests/test_auth.py \
  apps/api/tests/test_mfa_repository.py
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % PYTHONPATH=apps/api \
DATABASE_URL='postgresql://sandeeph@localhost:5432/finsight_fin15_test' \
AUTH_THROTTLE_SECRET='integration-test-secret-material-32-bytes' \
.venv/bin/python -m pytest \
```

```
  apps/api/tests/test_mfa_repository.py \
  apps/api/tests/test_auth.py -q
.............................
[100%]
=============================================================================
======== warnings summary
=============================================================================
========
.venv/lib/python3.14/site-packages/fastapi/testclient.py:1

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is
deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

.venv/lib/python3.14/site-packages/starlette/testclient.py:53

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is
deprecated, use anyio.from_thread.BlockingPortal instead.
    _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

apps/api/tests/test_auth.py: 10 warnings

/Users/sandeeph/Documents/NVDIA_Play_Book/FinSight_AI/.venv/lib/python3.14/site-package
s/starlette/testclient.py:454: DeprecationWarning: Setting per-request cookies=<...> is being
deprecated, because the expected behaviour on cookie persistence is ambiguous. Set cookies
directly on the client instance instead.
    return super().request(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
30 passed, 12 warnings in 3.18s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % printf '\n=====
IMPLEMENTATION STATUS =====\n'
tail -n 45 IMPLEMENTATION_STATUS.txt

printf '\n===== SPRINT 1 TEST STATUS =====\n'
tail -n 45 SPRINT_1_TEST_EXECUTION_STATUS.txt

printf '\n===== FUNCTION TEST TRACEABILITY =====\n'
tail -n 45 FUNCTION_TEST_TRACEABILITY.txt

printf '\n===== RELEASE README MFA REFERENCES =====\n'
```

```
grep -in -E 'MFA|multi.factor|secret.store|TOTP|throttl' \
  docs/release/RELEASE_README.md .env.example

===== IMPLEMENTATION STATUS =====
  authentication audit events and PostgreSQL-backed login throttle state.
- Added apps/api/app/audit.py for authentication audit insertion and
  redaction of secret-like fields.
- Added apps/api/app/throttle.py for validated configuration, HMAC-derived
  account/IP keys, persistent failure counting, and temporary blocking.
- Integrated login success/failure/denied audit events and throttle checks
  into the login flow; logout revokes the presented active session and
  records an audit event without storing the session token.
- Updated .env.example, release configuration metadata, and release README
  for the throttle settings and migration.
- Added tests in test_audit_service.py, test_throttle.py, and
  test_fin14_auth_security.py; extended authentication integration coverage.

Verification evidence:
- Focused auth/security suite: 66 passed, 0 failed, 10 warnings.
- Full API suite: 82 passed, 0 failed, 19 warnings.
- git diff --check passed in the recorded verification.
- Local release package included migration 0003 and the new audit/throttle
  modules; archive checksum verification passed.
- Test database: finsight_fin15_test. Synthetic test records were cleaned up
  by the integration tests; do not treat this as production deployment evidence.

Remaining gaps:
- TC_AUTHSEC_001 remains PARTIAL: end-to-end correlation propagation and
  append-only enforcement are not established.
- TC_AUTHSEC_002 remains PARTIAL: the complete login-level lockout-expiry
  and successful-retry flow needs explicit integration coverage.
- TC_AUTHSEC_003 remains BLOCKED: MFA policy/provider and challenge flow are
  not implemented; inspection and termination of other active sessions are
  also absent. Current-session logout does not satisfy cross-session controls.
- FIN-14 must not be marked accepted or VERIFIED COMPLETE until the relevant
  acceptance criteria are implemented, tested, and reconciled.

No source requirements, reference SQL/schema, diagrams, or original test plan
were modified. No commit or push was performed.

FIN-14 — LATEST CORRELATION AND API VERIFICATION (2026-10-21)

- Focused FIN-14 security suite: 7 passed, 0 failed, 3 warnings.
- Full API suite: 86 passed, 0 failed, 19 warnings.
```

- Integration tests verify supplied and generated correlation IDs match audit records.
- CORS permits the X-Correlation-ID request header.
- git diff --check passed.
- Open: database-level append-only audit enforcement; complete login-level throttle expiry and successful retry; MFA policy/provider and challenge enforcement; cross-session inspection and termination.
- Overall FIN-14 status remains PARTIAL. No commit or push performed.

===== SPRINT 1 TEST STATUS =====
the individual acceptance criteria. TC_AUTHZ_001 and TC_AUTHZ_002 remain PARTIAL; TC_AUTHZ_003 is historical PASS only; TC_PRIV_002 is DEFERRED / NOT RUN under the approved platform-only scope; TC_SYS_001 remains NOT RUN for the full planned system acceptance workflow.

Overall FIN-15 / FS-01-02 status: PARTIAL, not accepted or VERIFIED COMPLETE. Workspace/resource authorization remains deferred. FIN-14 is unchanged.

FIN-14 / FS-01-03 — AUTHENTICATION SECURITY EXECUTION ADDENDUM
Execution date: 2026-10-21
Tester/owner: Codex
Environment: dedicated local PostgreSQL test database finsight_fin15_test.

Observed verification:
- Focused authentication/security suites: 66 passed, 0 failed, 10 warnings.
- Full API suite: 82 passed, 0 failed, 19 warnings.
- Local release-package validation included migration 0003, audit/throttle modules, metadata, and checksum verification.
- Warnings were reported as test-client deprecations; no test failures occurred.

Acceptance-case reconciliation:
- TC_AUTHSEC_001: PARTIAL. Login success/failure and logout audit events, plus secret-redaction behavior, have tests. Complete correlation propagation and database-level append-only enforcement have not been established.
- TC_AUTHSEC_002: PARTIAL. Configurable PostgreSQL-backed throttling and repeated-failure blocking are implemented. Throttle behavior/expiry has been tested, but the complete login-level expiry-and-successful-retry flow is not established by the recorded integration tests.
- TC_AUTHSEC_003: BLOCKED. MFA enforcement and listing/terminating other active sessions are not implemented. Existing logout only revokes the session presented to that request.

Overall FIN-14 / FS-01-03 status: PARTIAL, not accepted or VERIFIED COMPLETE. No source requirements, schema reference documents, diagrams, or original test

plan were changed. No commit or push was performed.

FIN-14 — LATEST VERIFICATION ADDENDUM (2026-10-21)

- Focused FIN-14 security suite: 7 passed, 0 failed, 3 warnings.
- Full API suite: 86 passed, 0 failed, 19 warnings.
- Correlation ID propagation is verified for supplied and generated IDs.
- TC_AUTHSEC_001 remains PARTIAL: database-level append-only enforcement is unverified.
- TC_AUTHSEC_002 remains PARTIAL: complete login-level expiry and successful retry are unestablished.
- TC_AUTHSEC_003 remains BLOCKED: MFA and cross-session inspection/termination are not implemented.

===== FUNCTION TEST TRACEABILITY =====
- Traceability validator passed.

These results supersede neither the historical 2026-10-20 run record nor the individual acceptance criteria. TC_AUTHZ_001 and TC_AUTHZ_002 remain PARTIAL; TC_AUTHZ_003 is historical PASS only; TC_PRIV_002 is DEFERRED / NOT RUN under the approved platform-only scope; TC_SYS_001 remains NOT RUN for the full planned system acceptance workflow.

Overall FIN-15 / FS-01-02 status: PARTIAL, not accepted or VERIFIED COMPLETE. Workspace/resource authorization remains deferred. FIN-14 is unchanged.

FIN-14 — AUTHENTICATION SECURITY TRACEABILITY ADDENDUM (2026-10-21)

Story: FIN-14 / FS-01-03 — Protect Accounts and Audit Authentication.

- TC_AUTHSEC_001 — PARTIAL.
  Evidence: apps/api/tests/test_fin14_auth_security.py, apps/api/tests/test_audit_service.py, apps/api/app/audit.py.
  Tested authentication audit events and secret redaction. End-to-end correlation propagation and append-only enforcement remain unverified.

- TC_AUTHSEC_002 — PARTIAL.
  Evidence: apps/api/tests/test_throttle.py, apps/api/tests/test_fin14_auth_security.py, apps/api/app/throttle.py.
  Configuration validation and repeated-failure blocking are tested; complete login-level expiry-and-successful-retry acceptance remains unverified.

- TC_AUTHSEC_003 — BLOCKED.
  Evidence: apps/api/app/main.py and apps/api/tests/test_auth.py.

Current-session logout/revocation is implemented; MFA and inspection/
termination of other active sessions are not implemented.

Latest recorded execution: focused auth/security suite 66 passed;
full API suite 82 passed. Passing test counts do not establish complete
FIN-14 acceptance. Overall story status remains PARTIAL.

FIN-14 — LATEST VERIFICATION ADDENDUM (2026-10-21)

- Focused FIN-14 security suite: 7 passed, 0 failed, 3 warnings.
- Full API suite: 86 passed, 0 failed, 19 warnings.
- Supplied and generated correlation ID propagation is covered by integration tests.
- TC_AUTHSEC_001 remains PARTIAL: append-only enforcement is unverified.
- TC_AUTHSEC_002 remains PARTIAL: complete login-level expiry and successful retry are unestablished.
- TC_AUTHSEC_003 remains BLOCKED: MFA and cross-session controls are not implemented.

===== RELEASE README MFA REFERENCES =====
docs/release/RELEASE_README.md:18:Extract the verified archive into a new, versioned release directory. Install a supported Node.js 22 runtime and Python 3.12 runtime. Configurethe frontend process with `API_BASE_URL=https://<api-origin>` and the backend process with `DATABASE_URL`, `WEB_ORIGIN`, `SESSION_TTL_MINUTES`, `COOKIE_SECURE=true`, `APP_ENV=production`, `AUTH_THROTTLE_SECRET`, `AUTH_LOGIN_MAX_FAILURES`, `AUTH_LOGIN_WINDOW_SECONDS`, and `AUTH_LOGIN_BLOCK_SECONDS`. Generate a unique random `AUTH_THROTTLE_SECRET` of at least 32 bytes and keep it in the deployment secret manager. Rotating this secret changes throttle identifiers and makes existing throttle buckets inaccessible. Values containing credentials belong only in the deployment secret manager. Set the web origin exactly to the browser-facing frontend origin; terminate TLS at an approved proxy/load balancer and forward HTTPS safely.
.env.example:9:AUTH_THROTTLE_SECRET=replace-with-a-random-secret-of-at-least-32-byte
s
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % printf '\n===== MFA SECRET STORE CONFIGURATION =====\n'
grep -n -E 'os\.environ|os\.getenv|environ\.get|SecretStoreConfigurationError|def secret_store_from_environment|class .*SecretStore' \
  apps/api/app/secret_store.py

printf '\n===== MFA CONFIG REFERENCES =====\n'
grep -Rni --exclude-dir=.venv --exclude='*.pyc' \
  -E 'MFA_SECRET|SECRET_STORE|secret_store_from_environment|MFA.*STORE' \
  apps/api/app apps/api/tests .env.example docs/release/RELEASE_README.md

```
printf '\n===== RELEASE PACKAGING REFERENCES =====\n'
grep -n -E 'migrations|secret_store|mfa_|audit\.py|throttle\.py|requirements' \
  scripts/package_release.py

===== MFA SECRET STORE CONFIGURATION =====
21:class SecretStoreError(RuntimeError):
25:class SecretNotFoundError(SecretStoreError):
29:class SecretStoreConfigurationError(SecretStoreError):
33:class SecretStore(Protocol):
41:class FakeSecretStore:
68:class EncryptedFileSecretStore:
75:            raise SecretStoreConfigurationError(
85:            raise SecretStoreConfigurationError(
147:def secret_store_from_environment() -> SecretStore:
150:    app_env = os.environ.get("APP_ENV", "development").strip().lower()
151:    backend = os.environ.get("MFA_SECRET_STORE_BACKEND", "").strip().lower()
154:        raise SecretStoreConfigurationError(
160:            raise SecretStoreConfigurationError(
166:        raise SecretStoreConfigurationError("Unsupported APP_ENV for MFA secret storage")
169:        raise SecretStoreConfigurationError(
173:    encoded_key = os.environ.get("MFA_DEV_ENCRYPTION_KEY", "")
174:    store_path = os.environ.get("MFA_SECRET_STORE_PATH", "").strip()
176:        raise SecretStoreConfigurationError(
183:        raise SecretStoreConfigurationError(
188:        raise SecretStoreConfigurationError(

===== MFA CONFIG REFERENCES =====
apps/api/app/mfa_challenge_repository.py:15:from app.secret_store import SecretStore
apps/api/app/mfa_repository.py:18:from app.secret_store import SecretNotFoundError,
SecretStore
apps/api/app/secret_store.py:147:def secret_store_from_environment() -> SecretStore:
apps/api/app/secret_store.py:151:    backend =
os.environ.get("MFA_SECRET_STORE_BACKEND", "").strip().lower()
apps/api/app/secret_store.py:155:            "No production MFA secret-store provider is
configured"
apps/api/app/secret_store.py:161:                "Tests must explicitly select
MFA_SECRET_STORE_BACKEND=fake"
apps/api/app/secret_store.py:170:            "Development requires
MFA_SECRET_STORE_BACKEND=dev-encrypted"
apps/api/app/secret_store.py:174:    store_path =
os.environ.get("MFA_SECRET_STORE_PATH", "").strip()
apps/api/app/main.py.backup-20261023-mfa-routes:30:from app.secret_store import (
apps/api/app/main.py.backup-20261023-mfa-routes:34:    secret_store_from_environment,
```

apps/api/app/main.py.backup-20261023-mfa-routes:47:def _get_mfa_secret_store(request: Request) -> SecretStore:
apps/api/app/main.py.backup-20261023-mfa-routes:49:   store = getattr(request.app.state, "mfa_secret_store", None)
apps/api/app/main.py.backup-20261023-mfa-routes:51:   store = secret_store_from_environment()
apps/api/app/main.py.backup-20261023-mfa-routes:52:   request.app.state.mfa_secret_store = store
apps/api/app/main.py.backup-20261023-mfa-routes:371: _get_mfa_secret_store(request)
apps/api/app/main.py.backup-20261023-mfa-routes:420:   store = _get_mfa_secret_store(request)
apps/api/app/mfa_service.py:13:from app.secret_store import SecretStore
apps/api/app/main.py:30:from app.secret_store import (
apps/api/app/main.py:34:   secret_store_from_environment,
apps/api/app/main.py:55:def _get_mfa_secret_store(request: Request) -> SecretStore:
apps/api/app/main.py:57:   store = getattr(request.app.state, "mfa_secret_store", None)
apps/api/app/main.py:59:   store = secret_store_from_environment()
apps/api/app/main.py:60:   request.app.state.mfa_secret_store = store
apps/api/app/main.py:389:   _get_mfa_secret_store(request)
apps/api/app/main.py:438:   store = _get_mfa_secret_store(request)
apps/api/app/main.py:532:   store = _get_mfa_secret_store(request)
apps/api/app/main.py:594:   store=_get_mfa_secret_store(request),
apps/api/app/main.py:635:   store=_get_mfa_secret_store(request),
apps/api/tests/test_mfa_service.py:9:from app.secret_store import FakeSecretStore
apps/api/tests/test_auth.py:309:   from app.secret_store import FakeSecretStore
apps/api/tests/test_auth.py:314:   monkeypatch.setattr(app.state, "mfa_secret_store", store, raising=False)
apps/api/tests/test_auth.py:482:   from app.secret_store import FakeSecretStore
apps/api/tests/test_auth.py:487:   monkeypatch.setattr(app.state, "mfa_secret_store", store, raising=False)
apps/api/tests/test_auth.py:544:   from app.secret_store import FakeSecretStore
apps/api/tests/test_auth.py:548:   app.state, "mfa_secret_store", FakeSecretStore(), raising=False
apps/api/tests/test_auth.py:584:   from app.secret_store import FakeSecretStore
apps/api/tests/test_auth.py:603:   monkeypatch.setattr(app.state, "mfa_secret_store", store, raising=False)
apps/api/tests/test_secret_store.py:8:from app.secret_store import (
apps/api/tests/test_secret_store.py:14:   secret_store_from_environment,
apps/api/tests/test_secret_store.py:71: monkeypatch.setenv("MFA_SECRET_STORE_BACKEND", "dev-encrypted")
apps/api/tests/test_secret_store.py:73:   monkeypatch.setenv("MFA_SECRET_STORE_PATH", "/tmp/should-not-be-used")
apps/api/tests/test_secret_store.py:76:   secret_store_from_environment()

```
apps/api/tests/test_secret_store.py:81:
monkeypatch.delenv("MFA_SECRET_STORE_BACKEND", raising=False)
apps/api/tests/test_secret_store.py:84:        secret_store_from_environment()
apps/api/tests/test_secret_store.py:86:
monkeypatch.setenv("MFA_SECRET_STORE_BACKEND", "fake")
apps/api/tests/test_secret_store.py:87:    assert isinstance(secret_store_from_environment(),
FakeSecretStore)
apps/api/tests/test_secret_store.py:92:
monkeypatch.setenv("MFA_SECRET_STORE_BACKEND", "dev-encrypted")
apps/api/tests/test_secret_store.py:94:    monkeypatch.setenv("MFA_SECRET_STORE_PATH",
str(tmp_path / "private-mfa"))
apps/api/tests/test_secret_store.py:97:        secret_store_from_environment()
apps/api/tests/test_mfa_repository.py:16:from app.secret_store import FakeSecretStore
apps/api/tests/test_mfa_repository.py:194:    from app.secret_store import SecretNotFoundError
apps/api/tests/test_mfa_repository.py:221:    from app.secret_store import SecretStoreError
apps/api/tests/test_mfa_repository.py:272:    from app.secret_store import SecretNotFoundError
apps/api/tests/test_mfa_challenge_repository.py:15:from app.secret_store import
FakeSecretStore

===== RELEASE PACKAGING REFERENCES =====
45:        shutil.copytree(ROOT / "apps/api/migrations", backend / "migrations")
46:        shutil.copy2(ROOT / "apps/api/requirements.lock", backend / "requirements.lock")
54:            "database_migrations": ["0001_auth_identity.sql", "0002_platform_authorization.sql",
"0003_auth_audit_and_login_throttling.sql", "0004_immutable_auth_audit.sql",
"0005_mfa_factors.sql", "0006_mfa_login_challenges.sql"],
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % sed -n '140,205p'
apps/api/app/secret_store.py

printf '\n===== ENVIRONMENT EXAMPLE =====\n'
cat .env.example

printf '\n===== RELEASE README =====\n'
sed -n '1,100p' docs/release/RELEASE_README.md

printf '\n===== PACKAGE CONTENTS LOGIC =====\n'
sed -n '1,110p' scripts/package_release.py
        path = self._path_for(ref)
        try:
            path.unlink()
        except FileNotFoundError as exc:
            raise SecretNotFoundError("Secret reference not found") from exc
```

```
def secret_store_from_environment() -> SecretStore:
    """Select an explicitly permitted adapter; never fall back silently."""

    app_env = os.environ.get("APP_ENV", "development").strip().lower()
    backend = os.environ.get("MFA_SECRET_STORE_BACKEND", "").strip().lower()

    if app_env == "production":
        raise SecretStoreConfigurationError(
            "No production MFA secret-store provider is configured"
        )

    if app_env == "test":
        if backend != "fake":
            raise SecretStoreConfigurationError(
                "Tests must explicitly select MFA_SECRET_STORE_BACKEND=fake"
            )
        return FakeSecretStore()

    if app_env != "development":
        raise SecretStoreConfigurationError("Unsupported APP_ENV for MFA secret storage")

    if backend != "dev-encrypted":
        raise SecretStoreConfigurationError(
            "Development requires MFA_SECRET_STORE_BACKEND=dev-encrypted"
        )

    encoded_key = os.environ.get("MFA_DEV_ENCRYPTION_KEY", "")
    store_path = os.environ.get("MFA_SECRET_STORE_PATH", "").strip()
    if not encoded_key or not store_path:
        raise SecretStoreConfigurationError(
            "Development secret storage requires an encryption key and storage path"
        )

    try:
        key = base64.b64decode(encoded_key, validate=True)
    except (ValueError, base64.binascii.Error) as exc:
        raise SecretStoreConfigurationError(
            "Development encryption key must be valid Base64"
        ) from exc

    if len(key) != 32:
        raise SecretStoreConfigurationError(
            "Development encryption key must decode to exactly 32 bytes"
        )
```

```
    return EncryptedFileSecretStore(Path(store_path), key)

===== ENVIRONMENT EXAMPLE =====
# Local development placeholders only. Replace DATABASE_URL locally; never commit it.
DATABASE_URL=postgresql://USER:PASSWORD@127.0.0.1:5432/finsight
WEB_ORIGIN=http://localhost:3000
SESSION_TTL_MINUTES=60
COOKIE_SECURE=false

# Generate a unique secret locally; do not commit the actual secret.
# Example: python -c 'import secrets; print(secrets.token_urlsafe(48))'
AUTH_THROTTLE_SECRET=replace-with-a-random-secret-of-at-least-32-bytes
AUTH_LOGIN_MAX_FAILURES=5
AUTH_LOGIN_WINDOW_SECONDS=900
AUTH_LOGIN_BLOCK_SECONDS=900

===== RELEASE README =====
# FinSight AI release bundle

This bundle is built by `.github/workflows/ci.yml` after the frontend build, backend
dependency/migration/tests, and repository hygiene jobs pass. It is an application package, not
a deployment. The repository has no approved hosting target or production environment
configured.

## Identify and verify a release

CI archives are named `finsight-<version>-<12-character-commit-prefix>.tar.gz`. A local bundle
without a source commit is named by version and records a null commit SHA; do not promote
such a worktree package as a committed release. `RELEASE-METADATA.json` records the
version, full Git commit SHA when available, application entry points, and included migration
names. A sibling `.build-info.json` records the UTC packaging timestamp and archive checksum;
the `.sha256` file contains the checksum independently. Verify before extracting:

```sh
sha256sum --check finsight-<version>-<12-character-commit-prefix>.tar.gz.sha256
tar -tzf finsight-<version>-<12-character-commit-prefix>.tar.gz
```

On macOS, use `shasum -a 256` and compare its output with the `.sha256` file.

Configure and start
```

Extract the verified archive into a new, versioned release directory. Install a supported Node.js 22 runtime and Python 3.12 runtime. Configure the frontend process with `API_BASE_URL=https://<api-origin>` and the backend process with `DATABASE_URL`, `WEB_ORIGIN`, `SESSION_TTL_MINUTES`, `COOKIE_SECURE=true`, `APP_ENV=production`, `AUTH_THROTTLE_SECRET`,`AUTH_LOGIN_MAX_FAILURES`, `AUTH_LOGIN_WINDOW_SECONDS`, and `AUTH_LOGIN_BLOCK_SECONDS`. Generate a unique random `AUTH_THROTTLE_SECRET` of at least 32 bytes and keep it in the deployment secret manager. Rotating this secret changes throttle identifiers and makes existing throttle buckets inaccessible. Values containing credentials belong only in the deployment secret manager. Set the web origin exactly to the browser-facing frontend origin; terminate TLS at an approved proxy/load balancer and forward HTTPS safely.

For the frontend, from the extracted bundle root:

```sh
cd frontend
HOSTNAME=0.0.0.0 PORT=3000 node server.js
```

For the backend, from the extracted bundle root, create a virtual environment and install the bundled pinned runtime requirements:

```sh
cd backend
python3.12 -m venv .venv
. .venv/bin/activate
python -m pip install -r requirements.lock
python -m uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Run database migrations as a separate, explicitly approved deployment step with a migration-owner role. The bundle includes the current versioned migration under `backend/migrations`; it does not include or apply the supplied target schema wholesale. Use a separate least-privileged application database role at runtime.

Promotion and rollback

Retain each verified archive, checksum, metadata, migration result, smoke-test report, and deployment record. Promote the same versioned bundle through environments; configure `API_BASE_URL` and backend settings per environment at runtime. Record which artifact version and full commit SHA is active.

To roll back application code, stop the affected processes and point the deployment supervisor to the previous known-good versioned release directory by its metadata version andcommit

SHA. Verify its checksum before reuse, start it with the appropriate environment configuration, then run the deployment's readiness and authentication smoke checks. Keep the database at its current migrated version unless the migration has a reviewed, tested, explicitly approved down migration. Do not restore an older application against a schemait cannot safely read.

Database migration policy for safe application rollback:

1. Prefer additive expand migrations that preserve existing columns/records and remain compatible with both old and new application versions.
2. Deploy code that can operate with both schema shapes before any contract/removal migration.
3. Delay destructive or incompatible schema changes to a separately reviewed maintenance window after backup and restore validation; document a forward-fix or recovery strategy and owner.
4. Never run the optional trading schema in Release 1. Never automatically reverse a migration as part of application rollback.

Deployment prerequisite

Before deployment is automated, the deployment owner must choose the hosting target and staging/production topology, configure protected environments and secret references, establish TLS and database network policy, define migration ownership, backups/RTO/RPO, monitoring/readiness, and approval gates. No deployment target or credentials are configured by this repository task. No deployment has been performed.

===== PACKAGE CONTENTS LOGIC =====
#!/usr/bin/env python3
"""Build a versioned FinSight application bundle from locked/build outputs."""

from __future__ import annotations

import argparse
import hashlib
import json
import os
import re
import shutil
import tarfile
import tempfile
from datetime import UTC, datetime
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
VERSION_PATTERN = re.compile(r"\A[0-9]+\.[0-9]+\.[0-9]+(?:[-+][0-9A-Za-z.-]+)?\Z")

```
SHA_PATTERN = re.compile(r"\A[0-9a-f]{40}\Z")


def package(frontend_dir: Path, version: str, commit_sha: str | None, output_dir: Path) ->
tuple[Path, Path, Path]:
    if not VERSION_PATTERN.fullmatch(version):
        raise ValueError("version must be a numeric semantic version, optionally with a
prerelease/build suffix")
    if commit_sha and not SHA_PATTERN.fullmatch(commit_sha):
        raise ValueError("commit SHA must be a full 40-character lowercase hexadecimal SHA")
    if not frontend_dir.is_dir() or not (frontend_dir / "server.js").is_file():
        raise FileNotFoundError("frontend standalone build is missing server.js")

    web_static = ROOT / "apps/web/.next/static"
    runtime_config = ROOT / "apps/web/app/api/runtime-config/route.js"
    if not web_static.is_dir() or not runtime_config.is_file():
        raise FileNotFoundError("build the frontend before packaging")

    output_dir.mkdir(parents=True, exist_ok=True)
    artifact_base = f"finsight-{version}" + (f"-{commit_sha[:12]}" if commit_sha else "")
    with tempfile.TemporaryDirectory(prefix="finsight-release-") as temp_dir:
        payload = Path(temp_dir) / "finsight"
        frontend = payload / "frontend"
        shutil.copytree(frontend_dir, frontend, symlinks=True)
        shutil.copytree(web_static, frontend / ".next/static", symlinks=True)

        backend = payload / "backend"
        shutil.copytree(ROOT / "apps/api/app", backend / "app",
ignore=shutil.ignore_patterns("__pycache__", "*.pyc"))
        shutil.copytree(ROOT / "apps/api/migrations", backend / "migrations")
        shutil.copy2(ROOT / "apps/api/requirements.lock", backend / "requirements.lock")

        metadata = {
            "artifact": artifact_base,
            "version": version,
            "commit_sha": commit_sha,
            "frontend": {"runtime": "Node.js 22", "entrypoint": "node frontend/server.js"},
            "backend": {"runtime": "Python 3.12", "entrypoint": "python -m uvicorn app.main:app"},
            "database_migrations": ["0001_auth_identity.sql", "0002_platform_authorization.sql",
"0003_auth_audit_and_login_throttling.sql", "0004_immutable_auth_audit.sql",
"0005_mfa_factors.sql", "0006_mfa_login_challenges.sql"],
            "configuration": "Runtime values are supplied through environment variables; see
RELEASE_README.md.",
        }
```

```
        (payload / "RELEASE-METADATA.json").write_text(json.dumps(metadata, indent=2) + "\n",
encoding="utf-8")
        release_readme = ROOT / "docs/release/RELEASE_README.md"
        shutil.copy2(release_readme, payload / "RELEASE_README.md")

        artifact_path = output_dir / f"{artifact_base}.tar.gz"
        def normalized_member(member: tarfile.TarInfo) -> tarfile.TarInfo:
            member.mtime = 0
            member.uid = 0
            member.gid = 0
            member.uname = ""
            member.gname = ""
            return member

        with artifact_path.open("wb") as raw:
            import gzip

            with gzip.GzipFile(fileobj=raw, mode="wb", mtime=0, filename="") as compressed:
                with tarfile.open(fileobj=compressed, mode="w", format=tarfile.PAX_FORMAT) as
archive:
                    for item in sorted(payload.rglob("*")):
                        archive.add(
                            item,
                            arcname=(Path("finsight") / item.relative_to(payload)).as_posix(),
                            recursive=False,
                            filter=normalized_member,
                        )

    digest_builder = hashlib.sha256()
    with artifact_path.open("rb") as stream:
        for chunk in iter(lambda: stream.read(1024 * 1024), b""):
            digest_builder.update(chunk)
    digest = digest_builder.hexdigest()
    checksum_path = output_dir / f"{artifact_path.name}.sha256"
    checksum_path.write_text(f"{digest}  {artifact_path.name}\n", encoding="ascii")
    build_info_path = output_dir / f"{artifact_path.name}.build-info.json"
    build_info = {
        **metadata,
        "built_at_utc": datetime.now(UTC).replace(microsecond=0).isoformat(),
        "sha256": digest,
    }
    build_info_path.write_text(json.dumps(build_info, indent=2) + "\n", encoding="utf-8")
    return artifact_path, checksum_path, build_info_path
```

```
def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--frontend-dir", type=Path, required=True, help="Next standalone
output directory")
    parser.add_argument("--version", required=True)
    parser.add_argument("--commit-sha", default=os.environ.get("GITHUB_SHA"))
    parser.add_argument("--output-dir", type=Path, default=ROOT / "dist")
    args = parser.parse_args()
    artifact, checksum, build_info = package(args.frontend_dir, args.version, args.commit_sha,
args.output_dir)
    print(f"Created {artifact}")
    print(f"Created {checksum}")
    print(f"Created {build_info}")
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % >....
- Focused MFA repository and authentication tests: 30 passed, 0 failed.
- Covered behaviors include factor revocation, retryable secret cleanup,
  secret-reference clearing after successful cleanup, and cross-user revocation
  rejection.
- Warnings are test-client/dependency deprecations; no tests failed.
- This is local test evidence only, not production or release-deployment evidence.
- Overall FIN-14 remains PARTIAL; full acceptance criteria are not all verified.
- No commit or push performed.
"""
    ,
    "FUNCTION_TEST_TRACEABILITY.txt": """

FIN-14 / MFA — TRACEABILITY CHECKPOINT (2026-10-23)

- MFA factor repository and authentication integration coverage was extended.
- Secret deletion failure preserves the reference for retry; a successful retry
  deletes the secret and clears the reference.
- Cross-user factor revocation is rejected without changing the owner's factor
  or secret.
- Latest full API run: 150 passed, 0 failed, 25 warnings.
- TC_AUTHSEC_001 and TC_AUTHSEC_002 remain PARTIAL until all outstanding
  acceptance criteria are reconciled with explicit test evidence.
- TC_AUTHSEC_003 is no longer accurately described as having no MFA
  implementation: MFA enrollment, factor listing/revocation, and login challenge
  components now exist and have test coverage. However, the overall criterion
  remains PARTIAL/BLOCKED for acceptance until the full MFA policy, production
  secret-store provider, and cross-session inspection/termination requirements
  are verified.
- Overall FIN-14 remains PARTIAL, not accepted or VERIFIED COMPLETE.
```

```
- No commit or push performed.
""",
}

for filename, addition in updates.items():
    path = Path(filename)
    if not path.is_file():
        raise SystemExit(f"STOP: Expected status file missing: {filename}")
    text = path.read_text()
    marker = "VERIFICATION CHECKPOINT (2026-10-23)" if filename ==
"IMPLEMENTATION_STATUS.txt" else (
        "TEST EXECUTION ADDENDUM (2026-10-23)" if filename ==
"SPRINT_1_TEST_EXECUTION_STATUS.txt"
        else "TRACEABILITY CHECKPOINT (2026-10-23)"
    )
    if marker in text:
        raise SystemExit(f"STOP: Checkpoint already exists in {filename}; no files changed.")
    updates[filename] = text.rstrip() + "\n" + addition

# Validate all files first; write only after every precondition succeeds.
for filename, content in updates.items():
    Path(filename).write_text(content)

print("Appended 2026-10-23 verification checkpoints to 3 status files.")
PY
Appended 2026-10-23 verification checkpoints to 3 status files.
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cat >>
docs/release/RELEASE_README.md <<'EOF'

## MFA production readiness limitation

The current MFA secret-store selector deliberately rejects `APP_ENV=production`
because no production secret-store provider is configured. The fake backend is
for tests only, and the encrypted file backend is development-only. Do not
enable or rely on MFA in a production deployment until an approved production
secret-store provider is implemented, configured, and tested. A passing local
API test suite does not establish production MFA readiness. This limitation
must be resolved before claiming production-ready MFA support.
EOF
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % git diff --check

tail -n 24 IMPLEMENTATION_STATUS.txt
tail -n 22 SPRINT_1_TEST_EXECUTION_STATUS.txt
tail -n 24 FUNCTION_TEST_TRACEABILITY.txt
```

tail -n 12 docs/release/RELEASE_README.md

git status --short
- Full API suite: 86 passed, 0 failed, 19 warnings.
- Integration tests verify supplied and generated correlation IDs match audit records.
- CORS permits the X-Correlation-ID request header.
- git diff --check passed.
- Open: database-level append-only audit enforcement; complete login-level throttle expiry and successful retry; MFA policy/provider and challenge enforcement; cross-session inspection and termination.
- Overall FIN-14 status remains PARTIAL. No commit or push performed.

FIN-14 / MFA — VERIFICATION CHECKPOINT (2026-10-23)

- Full API test suite: 150 passed, 0 failed, 25 warnings in 6.19 seconds.
- Focused MFA repository and authentication tests: 30 passed, 0 failed.
- MFA revocation marks the factor revoked and removes replay-guard records.
- Secret cleanup is retryable after a secret-store deletion error; successful cleanup clears the persisted secret reference.
- HTTP cross-user MFA factor revocation is covered by an integration test.
- git diff --check passed at this checkpoint.
- MFA storage supports a fake test backend and an encrypted development file backend. Production currently has no configured MFA secret-store provider; production selection fails closed.
- These test results do not establish production deployment readiness or complete FIN-14 acceptance. Outstanding audit, throttling, and session-control criteria must still be reconciled against their acceptance tests.
- No commit or push performed.

- Focused FIN-14 security suite: 7 passed, 0 failed, 3 warnings.
- Full API suite: 86 passed, 0 failed, 19 warnings.
- Correlation ID propagation is verified for supplied and generated IDs.
- TC_AUTHSEC_001 remains PARTIAL: database-level append-only enforcement is unverified.
- TC_AUTHSEC_002 remains PARTIAL: complete login-level expiry and successful retry are unestablished.
- TC_AUTHSEC_003 remains BLOCKED: MFA and cross-session inspection/termination are not implemented.

FIN-14 / MFA — TEST EXECUTION ADDENDUM (2026-10-23)

Environment: local dedicated PostgreSQL test database finsight_fin15_test.

- Full API suite: 150 passed, 0 failed, 25 warnings in 6.19 seconds.
- Focused MFA repository and authentication tests: 30 passed, 0 failed.
- Covered behaviors include factor revocation, retryable secret cleanup, secret-reference clearing after successful cleanup, and cross-user revocation rejection.
- Warnings are test-client/dependency deprecations; no tests failed.
- This is local test evidence only, not production or release-deployment evidence.
- Overall FIN-14 remains PARTIAL; full acceptance criteria are not all verified.
- No commit or push performed.
- Supplied and generated correlation ID propagation is covered by integration tests.
- TC_AUTHSEC_001 remains PARTIAL: append-only enforcement is unverified.
- TC_AUTHSEC_002 remains PARTIAL: complete login-level expiry and successful retry are unestablished.
- TC_AUTHSEC_003 remains BLOCKED: MFA and cross-session controls are not implemented.

FIN-14 / MFA — TRACEABILITY CHECKPOINT (2026-10-23)

- MFA factor repository and authentication integration coverage was extended.
- Secret deletion failure preserves the reference for retry; a successful retry deletes the secret and clears the reference.
- Cross-user factor revocation is rejected without changing the owner's factor or secret.
- Latest full API run: 150 passed, 0 failed, 25 warnings.
- TC_AUTHSEC_001 and TC_AUTHSEC_002 remain PARTIAL until all outstanding acceptance criteria are reconciled with explicit test evidence.
- TC_AUTHSEC_003 is no longer accurately described as having no MFA implementation: MFA enrollment, factor listing/revocation, and login challenge components now exist and have test coverage. However, the overall criterion remains PARTIAL/BLOCKED for acceptance until the full MFA policy, production secret-store provider, and cross-session inspection/termination requirements are verified.
- Overall FIN-14 remains PARTIAL, not accepted or VERIFIED COMPLETE.
- No commit or push performed.

Before deployment is automated, the deployment owner must choose the hosting target and staging/production topology, configure protected environments and secret references, establish TLS and database network policy, define migration ownership, backups/RTO/RPO, monitoring/readiness, and approval gates. No deployment target or credentials are configured by this repository task. No deployment has been performed.

MFA production readiness limitation

The current MFA secret-store selector deliberately rejects `APP_ENV=production`
because no production secret-store provider is configured. The fake backend is
for tests only, and the encrypted file backend is development-only. Do not
enable or rely on MFA in a production deployment until an approved production
secret-store provider is implemented, configured, and tested. A passing local
API test suite does not establish production MFA readiness. This limitation
must be resolved before claiming production-ready MFA support.
 M .env.example
 M FUNCTION_TEST_TRACEABILITY.txt
 M IMPLEMENTATION_STATUS.txt
 M SPRINT_1_TEST_EXECUTION_STATUS.txt
 M apps/api/app/main.py
 M apps/api/requirements.lock
 M apps/api/requirements.txt
 M apps/api/tests/test_auth.py
 M conversation.txt
 M docs/release/RELEASE_README.md
 M scripts/package_release.py
?? apps/api/app/audit.py
?? apps/api/app/main.py.backup-20261022-080642
?? apps/api/app/main.py.backup-20261023-mfa-routes
?? apps/api/app/mfa_challenge_repository.py
?? apps/api/app/mfa_repository.py
?? apps/api/app/mfa_service.py
?? apps/api/app/secret_store.py
?? apps/api/app/throttle.py
?? apps/api/app/totp.py
?? apps/api/migrations/0003_auth_audit_and_login_throttling.sql
?? apps/api/migrations/0004_immutable_auth_audit.sql
?? apps/api/migrations/0005_mfa_factors.sql
?? apps/api/migrations/0006_mfa_login_challenges.sql
?? apps/api/tests/test_audit_immutability.py
?? apps/api/tests/test_audit_service.py
?? apps/api/tests/test_fin14_auth_security.py
?? apps/api/tests/test_mfa_challenge_repository.py
?? apps/api/tests/test_mfa_repository.py
?? apps/api/tests/test_mfa_schema.py
?? apps/api/tests/test_mfa_service.py
?? apps/api/tests/test_secret_store.py
?? apps/api/tests/test_throttle.py
?? apps/api/tests/test_totp.py
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % printf '\n===== PACKAGING
TESTS =====\n'

```
grep -Rni --exclude-dir=.venv --exclude='*.pyc' \
  -E 'package_release|RELEASE-METADATA|database_migrations|sha256' \
  apps/api/tests tests scripts .github 2>/dev/null | head -n 100

printf '\n===== PACKAGING SCRIPT DIFF =====\n'
git diff -- scripts/package_release.py

printf '\n===== RELEASE README DIFF =====\n'
git diff -- docs/release/RELEASE_README.md

printf '\n===== ENV EXAMPLE DIFF =====\n'
git diff -- .env.example

===== PACKAGING TESTS =====
apps/api/tests/test_mfa_challenge_repository.py:96:    assert stored_hash ==
hashlib.sha256(token.encode()).hexdigest()
scripts/package_release.py:54:            "database_migrations": ["0001_auth_identity.sql",
"0002_platform_authorization.sql", "0003_auth_audit_and_login_throttling.sql",
"0004_immutable_auth_audit.sql", "0005_mfa_factors.sql", "0006_mfa_login_challenges.sql"],
scripts/package_release.py:57:        (payload /
"RELEASE-METADATA.json").write_text(json.dumps(metadata, indent=2) + "\n",
encoding="utf-8")
scripts/package_release.py:83:    digest_builder = hashlib.sha256()
scripts/package_release.py:88:    checksum_path = output_dir / f"{artifact_path.name}.sha256"
scripts/package_release.py:94:        "sha256": digest,
.github/workflows/ci.yml:348:          python3 scripts/package_release.py \
.github/workflows/ci.yml:353:          (cd dist && sha256sum --check "finsight-VERSION −
{GITHUB_SHA:0:12}.tar.gz.sha256")
.github/workflows/ci.yml:360:            dist/*.sha256

===== PACKAGING SCRIPT DIFF =====
diff --git a/scripts/package_release.py b/scripts/package_release.py
index c31476c..f3ccea3 100644
--- a/scripts/package_release.py
+++ b/scripts/package_release.py
@@ -51,7 +51,7 @@ def package(frontend_dir: Path, version: str, commit_sha: str | None,
output_dir
            "commit_sha": commit_sha,
            "frontend": {"runtime": "Node.js 22", "entrypoint": "node frontend/server.js"},
            "backend": {"runtime": "Python 3.12", "entrypoint": "python -m uvicorn app.main:app"},
-           "database_migrations": ["0001_auth_identity.sql", "0002_platform_authorization.sql"],
+           "database_migrations": ["0001_auth_identity.sql", "0002_platform_authorization.sql",
"0003_auth_audit_and_login_throttling.sql", "0004_immutable_auth_audit.sql",
"0005_mfa_factors.sql", "0006_mfa_login_challenges.sql"],
```

```
            "configuration": "Runtime values are supplied through environment variables; see
RELEASE_README.md.",
        }
        (payload / "RELEASE-METADATA.json").write_text(json.dumps(metadata, indent=2) +
"\n", encoding="utf-8")

===== RELEASE README DIFF =====
diff --git a/docs/release/RELEASE_README.md b/docs/release/RELEASE_README.md
index 2e38486..b36875e 100644
--- a/docs/release/RELEASE_README.md
+++ b/docs/release/RELEASE_README.md
@@ -15,7 +15,7 @@ On macOS, use `shasum -a 256` and compare its output with the
`.sha256` file.

 ## Configure and start

-Extract the verified archive into a new, versioned release directory. Install a supported Node.js
22 runtime and Python 3.12 runtime. Configure the frontend process with
`API_BASE_URL=https://<api-origin>` and the backend process with `DATABASE_URL`,
`WEB_ORIGIN`, `SESSION_TTL_MINUTES`, `COOKIE_SECURE=true`, and
`APP_ENV=production`. Values containing credentials belong only in the deployment secret
manager. Set the web origin exactly to the browser-facing frontend origin; terminate TLS at an
approved proxy/load balancer and forward HTTPS safely.
+Extract the verified archive into a new, versioned release directory. Install a supported Node.js
22 runtime and Python 3.12 runtime. Configure the frontend process with
`API_BASE_URL=https://<api-origin>` and the backend process with `DATABASE_URL`,
`WEB_ORIGIN`, `SESSION_TTL_MINUTES`, `COOKIE_SECURE=true`,
`APP_ENV=production`, `AUTH_THROTTLE_SECRET`, `AUTH_LOGIN_MAX_FAILURES`,
`AUTH_LOGIN_WINDOW_SECONDS`, and `AUTH_LOGIN_BLOCK_SECONDS`. Generate a
unique random `AUTH_THROTTLE_SECRET` of at least 32 bytes and keep it in the
deployment secret manager. Rotating this secret changes throttle identifiers and makes existing
throttle buckets inaccessible. Values containing credentials belong only in the deployment secret
manager. Set the web origin exactly to the browser-facing frontend origin; terminate TLS at an
approved proxy/load balancer and forward HTTPS safely.

 For the frontend, from the extracted bundle root:

@@ -52,3 +52,13 @@ Database migration policy for safe application rollback:
 ## Deployment prerequisite

 Before deployment is automated, the deployment owner must choose the hosting target and
staging/production topology, configure protected environments and secret references, establish
TLS and database network policy, define migration ownership, backups/RTO/RPO,
```

```
monitoring/readiness, and approval gates. No deployment target or credentials are configured
by this repository task. No deployment has been performed.
+
+## MFA production readiness limitation
+
+The current MFA secret-store selector deliberately rejects `APP_ENV=production`
+because no production secret-store provider is configured. The fake backend is
+for tests only, and the encrypted file backend is development-only. Do not
+enable or rely on MFA in a production deployment until an approved production
+secret-store provider is implemented, configured, and tested. A passing local
+API test suite does not establish production MFA readiness. This limitation
+must be resolved before claiming production-ready MFA support.

===== ENV EXAMPLE DIFF =====
diff --git a/.env.example b/.env.example
index fc40fc7..338b7d4 100644
--- a/.env.example
+++ b/.env.example
@@ -3,3 +3,10 @@
DATABASE_URL=postgresql://USER:PASSWORD@127.0.0.1:5432/finsight
 WEB_ORIGIN=http://localhost:3000
 SESSION_TTL_MINUTES=60
 COOKIE_SECURE=false
+
+# Generate a unique secret locally; do not commit the actual secret.
+# Example: python -c 'import secrets; print(secrets.token_urlsafe(48))'
+AUTH_THROTTLE_SECRET=replace-with-a-random-secret-of-at-least-32-bytes
+AUTH_LOGIN_MAX_FAILURES=5
+AUTH_LOGIN_WINDOW_SECONDS=900
+AUTH_LOGIN_BLOCK_SECONDS=900
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % cat >> .env.example <<'EOF'

# MFA secret storage — DEVELOPMENT ONLY.
# Generate a fresh 32-byte key; never commit the generated key.
# Example: python -c 'import base64,secrets;
print(base64.b64encode(secrets.token_bytes(32)).decode())'
# MFA_SECRET_STORE_BACKEND=dev-encrypted
# MFA_DEV_ENCRYPTION_KEY=<base64-encoded-32-byte-key>
# MFA_SECRET_STORE_PATH=<private-local-path-outside-the-repository>
#
# Production MFA is not supported by the current secret-store selector.
# Do not use the development encrypted-file backend in production.
EOF
```

```
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % .venv/bin/python -m
py_compile scripts/package_release.py

printf '\n===== FRONTEND BUILD ARTIFACTS =====\n'
test -f apps/web/.next/standalone/server.js \
  && echo "Standalone frontend server: present" \
  || echo "Standalone frontend server: missing"

test -d apps/web/.next/static \
  && echo "Frontend static assets: present" \
  || echo "Frontend static assets: missing"

test -f apps/web/app/api/runtime-config/route.js \
  && echo "Runtime config route: present" \
  || echo "Runtime config route: missing"

printf '\n===== WHITESPACE CHECK =====\n'
git diff --check

===== FRONTEND BUILD ARTIFACTS =====
Standalone frontend server: present
Frontend static assets: present
Runtime config route: present

===== WHITESPACE CHECK =====
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %


Commit SHA: None
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro finsight-package-check % cd
"$HOME/Documents/NVDIA_Play_Book/FinSight_AI"

git diff --check
git status --short

printf '\n===== CHANGE SUMMARY =====\n'
git diff --stat

printf '\n===== UNTRACKED FILES =====\n'
git ls-files --others --exclude-standard
 M .env.example
 M FUNCTION_TEST_TRACEABILITY.txt
 M IMPLEMENTATION_STATUS.txt
 M SPRINT_1_TEST_EXECUTION_STATUS.txt
```

```
 M apps/api/app/main.py
 M apps/api/requirements.lock
 M apps/api/requirements.txt
 M apps/api/tests/test_auth.py
 M conversation.txt
 M docs/release/RELEASE_README.md
 M scripts/package_release.py
?? apps/api/app/audit.py
?? apps/api/app/main.py.backup-20261022-080642
?? apps/api/app/main.py.backup-20261023-mfa-routes
?? apps/api/app/mfa_challenge_repository.py
?? apps/api/app/mfa_repository.py
?? apps/api/app/mfa_service.py
?? apps/api/app/secret_store.py
?? apps/api/app/throttle.py
?? apps/api/app/totp.py
?? apps/api/migrations/0003_auth_audit_and_login_throttling.sql
?? apps/api/migrations/0004_immutable_auth_audit.sql
?? apps/api/migrations/0005_mfa_factors.sql
?? apps/api/migrations/0006_mfa_login_challenges.sql
?? apps/api/tests/test_audit_immutability.py
?? apps/api/tests/test_audit_service.py
?? apps/api/tests/test_fin14_auth_security.py
?? apps/api/tests/test_mfa_challenge_repository.py
?? apps/api/tests/test_mfa_repository.py
?? apps/api/tests/test_mfa_schema.py
?? apps/api/tests/test_mfa_service.py
?? apps/api/tests/test_secret_store.py
?? apps/api/tests/test_throttle.py
?? apps/api/tests/test_totp.py

===== CHANGE SUMMARY =====
 .env.example                    |  17 ++++
 FUNCTION_TEST_TRACEABILITY.txt  |  55 ++++++++++
 IMPLEMENTATION_STATUS.txt       |  70 ++++++++++++++
 SPRINT_1_TEST_EXECUTION_STATUS.txt |  59 ++++++++++-
 apps/api/app/main.py            | 704
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++--------
 apps/api/requirements.lock      |   5 +-
 apps/api/requirements.txt       |   1 +
 apps/api/tests/test_auth.py     | 547
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
++++++++++++++++++++++++++++++++-
```

```
 conversation.txt                  |  30 ++++++
 docs/release/RELEASE_README.md    |  12 ++-
 scripts/package_release.py        |   2 +-
 11 files changed, 1450 insertions(+), 52 deletions(-)

===== UNTRACKED FILES =====
apps/api/app/audit.py
apps/api/app/main.py.backup-20261022-080642
apps/api/app/main.py.backup-20261023-mfa-routes
apps/api/app/mfa_challenge_repository.py
apps/api/app/mfa_repository.py
apps/api/app/mfa_service.py
apps/api/app/secret_store.py
apps/api/app/throttle.py
apps/api/app/totp.py
apps/api/migrations/0003_auth_audit_and_login_throttling.sql
apps/api/migrations/0004_immutable_auth_audit.sql
apps/api/migrations/0005_mfa_factors.sql
apps/api/migrations/0006_mfa_login_challenges.sql
apps/api/tests/test_audit_immutability.py
apps/api/tests/test_audit_service.py
apps/api/tests/test_fin14_auth_security.py
apps/api/tests/test_mfa_challenge_repository.py
apps/api/tests/test_mfa_repository.py
apps/api/tests/test_mfa_schema.py
apps/api/tests/test_mfa_service.py
apps/api/tests/test_secret_store.py
apps/api/tests/test_throttle.py
apps/api/tests/test_totp.py
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro NVDIA_Play_Book % curl --max-time 30
-sS -o /tmp/finsight-home.html \
  -w 'HTTP status: %{http_code}\n' \
  http://127.0.0.1:3000/
HTTP status: 200
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro NVDIA_Play_Book % cd
"$HOME/Documents/NVDIA_Play_Book/FinSight_AI"

printf '\n===== RUNTIME CONFIG ROUTE =====\n'
find apps/web/app -path '*runtime-config*' -type f -print

printf '\n===== RUNTIME CONFIG RESPONSE =====\n'
curl --max-time 10 -sS -i http://127.0.0.1:3000/api/runtime-config
```

```
printf '\n===== BACKEND OPENAPI =====\n'
curl --max-time 5 -sS -o /dev/null \
  -w 'Backend HTTP status: %{http_code}\n' \
  http://127.0.0.1:8000/openapi.json

===== RUNTIME CONFIG ROUTE =====
apps/web/app/api/runtime-config/route.js

===== RUNTIME CONFIG RESPONSE =====
HTTP/1.1 503 Service Unavailable
vary: rsc, next-router-state-tree, next-router-prefetch, next-router-segment-prefetch
content-type: application/json
Date: Mon, 26 Oct 2026 05:59:12 GMT
Connection: keep-alive
Keep-Alive: timeout=5
Transfer-Encoding: chunked

{"detail":"API_BASE_URL is not configured."}
===== BACKEND OPENAPI =====
Backend HTTP status: 200
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI % grep -RniE --include='*.js'
--include='*.jsx' --include='*.ts' --include='*.tsx' \
  'runtime-config|API_BASE_URL|NEXT_PUBLIC_API|BACKEND_URL' \
  apps/web/app apps/web/src apps/web/lib 2>/dev/null | head -n 100
apps/web/app/page.js:24:    fetch("/api/runtime-config")
apps/web/app/api/runtime-config/route.js:6:  const configured = process.env.API_BASE_URL;
apps/web/app/api/runtime-config/route.js:8:    return NextResponse.json({ detail:
"API_BASE_URL is not configured." }, { status: 503 });
apps/web/app/api/runtime-config/route.js:19:    return NextResponse.json({ detail:
"API_BASE_URL is invalid." }, { status: 500 });
(.venv) (base) sandeeph@Sandeeps-MacBook-Pro FinSight_AI %
```